

awdec 算法笔记

算法与理论整理

awdec

2026-08-11

目录

前言	xxi
第一部分 数据结构	1
1 平衡树	3
1.1 Treap	3
1.1.1 插入:	3
1.1.2 删除:	4
1.2 FHQ Treap	6
1.2.1 分裂:	6
1.2.2 合并:	7
1.3 Splay	9
1.3.1 伸展:	9
1.4 替罪羊树	12
1.4.1 重构:	12
1.4.2 插入:	12
1.4.3 删除:	12
1.4.4 前驱/后继:	12
1.5 隐式序列平衡树	15
1.5.1 区间加与区间翻转	15
1.5.2 区间移动	15
1.5.3 区间插入	16
1.5.4 线性建树	16
1.6 边界约定	16
1.7 比较	16
2 线段树	17
2.1 zkw 线段树	17
2.2 猫树	18
2.3 李超线段树	20
2.3.1 询问:	20
2.4 可持久化线段树	22
2.4.1 动态开点线段树:	22

2.4.2	标记永久化:	23
2.4.3	主席树:	23
2.4.4	区间修改:	25
2.4.5	Trick:	25
2.5	吉司机线段树	25
2.6	线段树合并	25
2.7	线段树分裂	26
2.8	线段树优化建图	26
2.9	线段树分治	28
2.10	线段树上二分	29
3	树状数组	31
3.1	线性建树	31
3.2	树状数组倍增	31
3.3	树状数组维护区间加、区间和:	32
3.4	维护不可差分信息	32
3.4.1	单点修改:	32
3.4.2	区间询问:	33
4	树套树	35
4.1	树状数组套DS	35
4.1.1	树状数组套树状数组	35
4.1.2	树状数组套vector	36
4.1.3	树状数组套动态开点线段树	37
4.2	线段树套DS	39
4.2.1	线段树套动态开点线段树	39
4.2.2	线段树套平衡树	39
4.3	分块套 DS	40
5	并查集	41
5.1	启发式合并	41
5.2	可撤销并查集	42
5.3	带权并查集	42
5.4	扩展域并查集	44
5.5	可持久化并查集	44
5.6	赋值并查集	45
5.7	倍增并查集	47

6	堆	49
6.1	对顶堆	49
6.2	可并堆	49
7	动态树	51
7.1	Link-Cut-Tree	51
7.1.1	access(x):	51
7.1.2	isroot(x):	52
7.1.3	make_root(x):	52
7.1.4	find_root(x):	52
7.1.5	connected(x,y):	53
7.1.6	split(x,y):	53
7.1.7	link(x,y):	53
7.1.8	cut(x,y):	53
7.1.9	LCT 中的Splay:	54
7.1.10	LCT 的替代品:	55
7.2	全局平衡二叉树	55
8	根号树	57
8.1	Sqrt Tree	57
8.2	vEB	61
8.2.1	插入	61
8.2.2	删除	62
8.2.3	后继/前驱	62
8.2.4	最大值/最小值	62
9	莫队	69
9.1	离线莫队	69
9.1.1	普通莫队	69
9.1.2	回滚莫队	70
9.1.3	带修莫队	72
9.1.4	树上莫队	74
9.1.5	二次离线莫队	75
9.2	在线莫队	76
9.2.1	在线普通莫队	76
9.2.2	在线带修莫队	77

9.3	莫队卡常	77
9.3.1	计算变化量	77
9.3.2	下标连续访问	77
10	树链剖分	79
10.1	dfs 序	79
10.1.1	性质:	79
10.2	重链剖分	79
10.2.1	定义	79
10.2.2	性质	79
10.3	长链剖分	80
10.3.1	定义	80
10.3.2	性质	80
10.3.3	应用	81
10.3.4	长链剖分优化dp	82
10.4	“启发式”剖分	82
10.4.1	”猫猫虫“剖分	82
11	点分治	83
11.1	点分治	83
11.2	动态点分治(点分树)	84
12	dsu on tree	87
12.1	树上启发式合并:	87
12.2	dsu on tree:	88
12.3	与点分治相比	89
13	01 Trie	91
13.1	普通 01-Trie	91
13.2	可持久化01-Trie	91
13.3	压缩 01-Trie	92
13.4	压位 01-Trie	94
13.4.1	插入/删除	95
13.4.2	前驱/后继	95
13.4.3	最大值/最小值	95
13.5	动态开点 · 压位01-Trie	98

14 线性基	99
14.1 构建:	99
14.1.1 高斯消元:	99
14.1.2 贪心法:	100
14.2 性质:	100
14.2.1 0 是否可表示	101
14.2.2 最小值:	101
14.2.3 最大值:	101
14.2.4 第 k 小/大	101
14.3 解的构造	102
14.3.1 set 维护	102
14.3.2 bitset 维护	103
14.4 线性基的合并	104
14.4.1 子空间和 / 线性基合并	104
14.4.2 线性基的交	104
14.5 不同的线性基构造	105
14.5.1 高位线性基	105
14.5.2 低位线性基	105
14.5.3 任意主元顺序	106
14.5.4 最简线性基 / 规范线性基	106
14.6 区间线性基	106
14.6.1 线段树维护	106
14.6.2 ST 表维护	107
14.6.3 猫树维护	107
14.6.4 前缀线性基	107
14.6.5 分块维护	108
14.6.6 实现对比	108
14.7 处理线性基的不可删	110
14.7.1 线段树直接维护	111
14.7.2 线段树分治	111
15 cdq 分治	113
15.1 时间复杂度	113
15.2 重复点与等号	114
16 整体二分	115
16.1 扩展	116

17	珂朵莉树	117
17.1	init	117
17.2	split	117
17.3	assign	117
17.4	perform	118
17.5	时间复杂度分析	118
17.5.1	区间推平时	118
17.5.2	数据随机时	118
18	bitset	119
19	小波树	127
19.1	小波树	127
19.1.1	优化 1 - 离散化	128
19.1.2	优化 2 - 位图	128
19.2	小波矩阵	128
19.3	小波矩阵优化可持久化01-Trie	130
20	K-D Tree	131
第二部分 字符串		137
21	字符串处理	139
21.1	append	139
21.2	substr	139
21.3	查找、替换	139
21.4	字符串-整数转换	140
21.5	字符分类函数	141
21.6	字符转换函数	141
22	Border	143
22.1	周期和循环节:	143
22.2	Border vs 周期:	143
22.3	s 所有前缀的 Border 的求法:	143
23	Kmp	145
23.1	字符串匹配:	145
23.2	扩展 Kmp (Z 函数):	145

23.3 Kmp 自动机 / Border 树(失配树):	146
24 Hash	147
24.1 单哈:	147
24.2 自然溢出:	147
24.3 双哈:	147
25 Manacher	149
26 Trie	151
27 ACAM	153
27.1 Trie 图优化	154
28 PAM	157
28.1 实现约定	157
28.2 本质不同回文子串:	159
28.3 Palindrome Series:	159
29 SAM	163
29.1 Trie 图:	164
29.2 fail 树:	164
29.2.1 endpos 与出现次数:	165
29.3 广义 SAM	166
30 SA	167
30.1 height 数组:	167
第三部分 数学	169
数论	171
31 质数	173
31.1 哥德巴赫猜想	173
31.2 波利尼亚克猜想	173
31.3 质数距离:	173
31.4 素性测试	173
31.4.1 试除法:	173
31.4.2 Miller-Rabin	173

31.5	质因数分解	174
31.5.1	试除法	174
31.5.2	Pollard-Rho	175
31.6	质数筛:	176
31.6.1	埃氏筛	176
31.6.2	欧拉筛	176
31.6.3	min_25 筛	176
31.6.4	Meissel-Lehmer	177
32	初等数论函数	179
32.1	积性函数	179
32.2	常见积性函数	179
32.3	欧拉筛求积性函数	179
32.4	欧拉函数	180
32.4.1	公式	180
32.4.2	欧拉定理	180
32.4.3	费马小定理	180
32.4.4	扩展欧拉定理	180
32.4.5	欧拉降幂	180
33	Gcd	181
33.1	一般的 gcd 求法	181
33.1.1	辗转相除法(欧几里得算法)	181
33.1.2	更相减损术 + Stein	181
33.2	基于值域的 gcd 求法	182
33.2.1	分解质因数	182
33.2.2	“根号分治”	182
33.3	gcd 的势能均摊	184
33.3.1	连续求 gcd 的均摊	184
33.3.2	区间 gcd 的均摊	184
34	取模运算	185
34.1	阶:	185
34.1.1	性质	185
34.2	逆元:	185
34.2.1	求解	185
34.2.2	预处理 n 个数的逆元	185

34.2.3 $O(1)$ 在线逆元	186
35 同余	189
35.1 扩展欧几里得算法	189
35.1.1 求解二元一次方程整数解	190
35.1.2 求解线性同余方程	190
35.2 欧拉定理	191
35.3 扩展欧拉定理	191
35.3.1 欧拉降幂	191
35.4 卢卡斯定理	192
35.5 扩展卢卡斯定理	192
35.6 BSGS	195
35.7 扩展 BSGS	196
35.7.1 离散对数	197
35.7.2 模数为质数的 N 次剩余	197
35.7.3 一般意义下的 N 次剩余	198
35.8 中国剩余定理	198
35.8.1 Garner 算法	199
35.9 扩展中国剩余定理	200
35.10 同余最短路:	202
35.10.1 转圈法求解同余最短路问题:	202
36 Dirichlet 前缀和	203
37 筛法	205
37.1 整除分块	205
37.1.1 多维整除分块	205
37.1.2 二次整除分块	205
37.2 杜教筛	206
37.3 $\min_25$ 筛	207
组合数学	209
38 组合数列	211
38.1 组合数	211
38.2 卡特兰数	211
38.3 斯特林数	212
38.3.1 第一类斯特林数	212

38.3.2	第二类斯特林数	212
38.4	斐波那契数列	213
38.4.1	通项公式:	213
38.4.2	倍增:	214
38.4.3	矩阵递推:	214
38.4.4	性质	214
38.4.5	模意义下的周期性	214
38.5	错位排列	215
38.5.1	指数生成函数	215
38.6	贝尔数	215
38.6.1	贝尔三角形	216
38.6.2	指数生成函数	216
38.7	欧拉数	216
38.7.1	递推式:	216
38.8	分拆数	216
38.8.1	性质:	217
38.8.2	五边形数定理:	217
39	反演	219
39.1	二项式反演	219
39.2	欧拉反演	219
39.3	莫比乌斯反演	220
39.4	子集反演	220
39.5	单位根反演	220
39.6	斯特林反演	221
第四部分	图论	223
40	最短路	225
40.1	Dijkstra	225
40.2	Bellman-Ford	226
40.2.1	spfa	226
40.2.2	判负环	227
40.3	Floyd	228
40.3.1	传递闭包	228
40.3.2	最小环:	228

40.4 Johnson 全源最短路	229
40.5 不同最短路算法的比较	230
40.6 0-1 BFS	231
40.7 差分约束	231
40.7.1 最小解	231
40.7.2 最大解:	232
40.7.3 最后	232
40.7.4 expand 1.	232
40.7.5 expand 2.	232
40.7.6 expand 3.	232
40.8 最短路图/最短路树	232
41 最小生成树	235
41.1 Kruskal	235
41.2 Prim	235
41.2.1 朴素维护	235
41.2.2 堆优化	236
41.2.3 其他数据结构优化	236
41.3 Boruvka	236
41.4 次小生成树	237
41.5 瓶颈路	238
41.6 Kruskal 重构树	238
41.7 最小树形图	238
41.7.1 朱刘算法	238
41.7.2 左偏树优化。	239
42 二分图	241
42.1 判定	241
42.1.1 图 G 是 2- 着色的 $\Leftrightarrow$ 图 G 是二分图。	241
42.1.2 图 G 中不存在奇环 $\Leftrightarrow$ 图 G 是二分图。	241
42.1.3 扩展域并查集判二分图	242
42.2 应用	242
42.2.1 1. 二分图最大匹配	242
42.2.2 2. 二分图最小点覆盖	243
42.2.3 3.二分图最大独立集	243
42.2.4 4.有向无环图最小路径覆盖	244
42.2.5 5.有向无环图最小路径重复点覆盖	244

42.2.6 6. 二分图最大权匹配	245
43 Tarjan	247
43.1 有向图 dfs 树	247
43.2 无向图 dfs 树	247
43.2.1 环	247
43.3 强连通分量	248
43.3.1 定义:	248
43.3.2 Tarjan:	248
43.3.3 性质:	248
43.3.4 杂谈:	249
43.4 边双连通分量	249
43.4.1 定义:	249
43.4.2 Tarjan:	249
43.4.3 性质:	250
43.5 点双连通分量	250
43.5.1 定义:	250
43.5.2 Tarjan:	250
43.5.3 性质:	251
43.6 2-sat	252
43.6.1 求解:	252
43.7 圆方树	252
43.7.1 仙人掌	252
43.7.2 广义圆方树	252
44 环计数问题	253
44.1 普通环计数	253
44.2 三元环计数	254
44.3 四元环计数	254
44.4 五元环计数	255
44.5 固定长度 $X \geq 6$ 的环计数	256
44.6 有向图三元环计数	256
45 欧拉路	257
45.1 判定:	257
45.2 求解:	258
45.2.1 有向图欧拉路径(开路):	258

45.2.2 有向图欧拉回路:	258
45.2.3 无向图欧拉路径(开路):	258
45.2.4 无向图欧拉回路:	258
第五部分 多项式	261
46 基础-多项式全家桶	263
46.1 FFT	263
46.2 NTT	263
46.3 多项式乘法	264
46.4 任意模数多项式乘法	265
46.5 多项式乘法逆	266
46.6 多项式开根	266
46.7 多项式除法	267
46.8 多项式 $\ln$	268
46.9 多项式 $\exp$	268
46.10 多项式幂	269
46.11 拉格朗日插值	270
第六部分 计算几何	273
47 计算几何全家桶	275
47.1 补充	288
47.1.1 三维凸包	288
47.1.2 动态凸包	289
47.1.3 最小圆覆盖	290
47.1.4 自适应辛普森积分	290
48 计算几何基础概念	293
48.1 浮点数与精度问题	293
48.1.1 特殊值:	293
48.2 点	293
48.2.1 向量:	293
48.2.2 点积:	293
48.2.3 叉积:	294
48.2.4 to-left 测试	294

48.2.5	向量逆时针旋转	295
48.3	线段	295
48.3.1	判断点是否在线段上:	295
48.3.2	判断两条线段是否相交:	295
48.4	直线	295
48.4.1	求点 A 到直线的距离:	295
48.4.2	求点 A 在直线上的投影点B:	296
48.4.3	两直线交点:	296
48.5	多边形	296
48.5.1	多边形的面积:	296
48.5.2	多边形的方向:	297
48.5.3	判断点是否在多边形内	297
49	极角序	301
49.1	极坐标系	301
49.2	极角排序	301
49.2.1	排序范围小于 π	301
49.2.2	完整一周的排序	301
50	凸包	303
50.1	凸包基础	303
50.1.1	定义	303
50.1.2	求凸包	304
50.2	凸包进阶	306
50.2.1	凸包二分	306
50.2.2	动态凸包(不删)	307
51	闵可夫斯基和	309
51.1	求两个凸多边形的闵可夫斯基和	309
51.2	求两个凸多边形的最大、最小距离	310
52	半平面交	311
52.1	概念	311
52.2	做法	311
52.2.1	法一:朴素算法	311
52.2.2	法二:增量法	312
52.2.3	法三:排序增量法	312
52.2.4	法四:分治法	314

53	旋转卡壳	315
53.1	输入约定	315
53.2	旋转卡壳求直径	315
53.3	算法流程	316
53.4	应用	316
53.4.1	两个凸多边形间的最大、最小距离	316
53.4.2	凸多边形的最小面积、最小周长外接矩形	317
54	扫描线	319
54.1	扫描线	319
54.2	轴对齐矩形面积并	319
54.3	判断线段集中是否存在交点	319
54.3.1	一般位置下的简化算法	319
54.3.2	同横坐标与退化情况	320
54.4	三角形面积并	321
54.4.1	法一:按横坐标分条积分	321
54.4.2	法二:构造并集轮廓	321
55	圆	323
55.1	圆基础	323
55.2	圆的关系问题	323
55.2.1	点与圆的关系	323
55.2.2	直线与圆的关系	323
55.2.3	圆与圆的关系	324
55.3	交集问题	324
55.3.1	直线与圆的交点	324
55.3.2	圆与圆交点	324
55.3.3	圆与多边形面积交	325
55.3.4	多个圆的面积并	325
55.4	切线问题	326
55.4.1	过圆外一点圆的切线	326
55.4.2	两圆的公切线	326
55.5	圆的反演	328
55.6	最小圆覆盖	328

56 基础三维几何	331
56.1 点	331
56.1.1 点积	331
56.1.2 叉积	331
56.1.3 三重积	331
56.2 线段	332
56.3 直线	332
56.4 平面	332
56.4.1 三点式	332
56.4.2 点法式	333
56.5 球	333
56.5.1 球截面	333
56.5.2 球面弧	333
56.5.3 球冠	334
56.5.4 球面三角形	334
57 杂项	337
57.1 皮克定理	337
57.2 平面最近点对	337
57.2.1 法一:经典分治算法	337
57.2.2 法二:扫描线	337
57.2.3 法三:随机化算法	338
57.3 辛普森积分	338
57.3.1 普通辛普森积分	338
57.3.2 自适应辛普森积分	339
57.4 仿射变换	339
57.4.1 应用	340
第七部分 动态规划	341
58 背包 dp	343
58.1 01 背包	343
58.2 完全背包	343
58.3 多重背包	343
58.3.1 二进制优化	343
58.3.2 单调队列优化	344

58.4 分组背包	344
58.5 多维背包	345
58.6 混合背包	345
58.7 可行性背包	345
58.7.1 bitset 优化	345
58.8 限和背包的二进制优化	345
58.9 背包求方案数	346
58.9.1 卷积优化	346
58.9.2 可撤销性	347
58.10 前后缀背包	348
58.11 树上背包	348
58.11.1 有依赖的背包	349
58.12 可行性完全背包的最少物品数	349
58.12.1 倍增 NTT	349
 第八部分 杂项	 351
59 区间变换	353
59.0.1 选最少的点覆盖所有区间	353
59.0.2 选最多的区间互不相交	353
59.0.3 区间分成组内区间无交的最少组数	353
59.0.4 选最少的区间覆盖整段区间	353
59.0.5 区间求并	354
60 树同构	355
60.1 有根树同构	355
60.2 无根树同构	355
60.3 AHU:	355
60.3.1 朴素做法:	356
60.3.2 优化:	356
60.4 树哈希:	357
61 哈希	359
61.1 哈希表:	359
61.1.1 开放寻址法:	359
61.1.2 拉链法:	359

61.2	多项式模数哈希:	361
61.2.1	优点:	361
61.2.2	缺点:	361
61.3	自然溢出:	361
61.3.1	优点:	361
61.3.2	缺点:	361
61.4	值域与哈希冲突	362
61.5	多项式哈希冲突概率:	362
61.6	双模数多项式哈希:	363
61.7	集合哈希:	363
61.7.1	哈希做法:	363
61.7.2	随机化	364
61.7.3	多项式哈希做法:	364
61.8	sum hash	364
61.9	xor hash	365
61.9.1	xor hash 应用:	365
61.9.2	xor hash 再扩展:	366
62	优秀的编码习惯(卡常)	367
62.1	STL 常数	367
62.2	数组下标访问	369
62.3	快速读入	369
62.4	快速取模	371
62.5	小常数数据结构	372
62.6	枚举变量的上下界	372
62.7	位运算	372
62.8	局部变量	372
62.9	lambda 表达式	373
63	范数	375
64	一些有用的库函数	377
64.1	GCC、Clang 等编译器提供的非 cpp 标准函数	377
64.2	C++20 引入了<bit> 头文件	377
64.3	mt19937	378
64.4	nth-element	378

前言

本书由 awdec 的算法笔记博客自动排版生成,面向 ACM-ICPC/ICPC 与同类算法竞赛训练。全书继续以网站中的 Markdown 为唯一内容源;代码、公式与图示均按纸质阅读重新排版。

电子版保留目录、书签与内部链接;打印时建议启用双面打印,并按实际装订方式保留内侧装订边距。

- 版本: 2026-08
- 构建日期: 2026-08-11
- 收录章节: 64

第一部分

数据结构

1 平衡树

算法竞赛常用平衡树有 Treap、Fhq-Treap、Splay、替罪羊树。

实际上还有 WBLT,但是感觉 WBLT 能干的上面总的找到能干的,算了,体量不要太大,后续再说。

1.1 Treap

Treap 是二叉查找树和二叉堆的结合,即:树上一个节点维护两个值,第一关键字维护BST 的性质,第二关键字维护 heap 的性质。在本文中,称第一关键字为键值(Key),第二关键字为权值(Val),且本文中Treap 均默认维护大根堆。

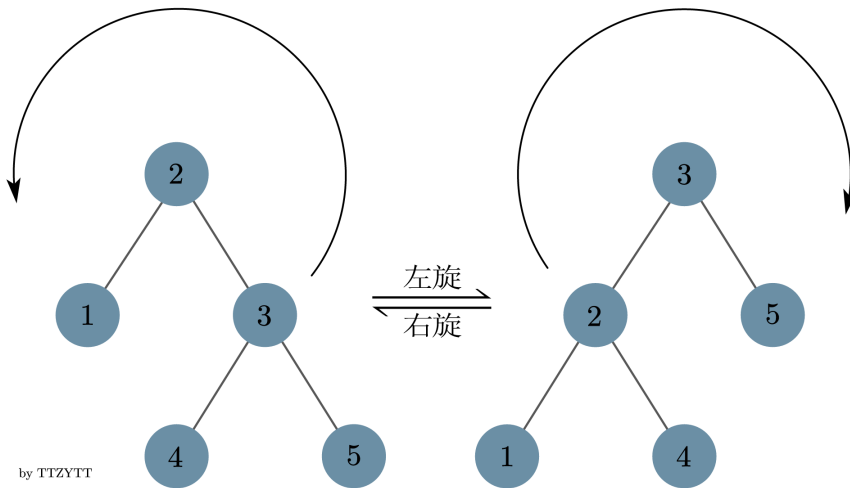
Treap 通过固定每个节点的 Val,使得 BST 能根据 heap 的性质在增删改查时动态调整形态。若各节点的 Val 独立随机生成, Treap 的形态等价于随机二叉搜索树,因此树高和单次操作的期望复杂度为 $O(\log n)$,并且在常见随机模型下以高概率保持对数高度;这不是最坏情况保证,极端情况下树高仍可能达到 $O(n)$ 。

因为 Treap 只通过 Val 维护树高,所以增删改查操作不受影响,与朴素 BST 操作完全一致。只是需要在增删等会影响 BST 形态的操作时,同时维护 Val 的heap 性质即可。

1.1.1 插入:

假设插入元素 v 在 Treap 上新建了一个节点 p ,新建节点 p 的Val 是随机产生的,可能并不满足 $Val(p) \leq Val(fa(p))$,那么就需要修正Treap 的形态。

根据 p 是 $fa(p)$ 的左儿子还是右儿子,可将修正操作分为左旋和右旋。



上图中不满足 heap 性质需要旋转的点分别为 3 (左旋) 和 2 (右旋) 只有上图中涉及的点会在旋转中被影响到, 若其中部分点不存在, 当作空节点同样处理即可。

具体而言, 当前节点为 p , p 左儿子为 ls , p 右儿子为 rs 。对于右旋, 令 q 为 ls 的右儿子, ls 成为根, p 成为 ls 的右儿子, 因此 q 成为 p 的左儿子; 对于左旋, 令 q 为 rs 的左儿子, rs 成为根, p 成为 rs 的左儿子, 因此 q 成为 p 的右儿子。

将 p 调整至父节点后, 可能会继续和新的父节点不满足 heap 的性质, 所以递归下去继续旋转即可。旋转次数与树高同阶, 因此期望为 $O(\log n)$, 最坏为 $O(n)$ 。

1.1.2 删除:

假设删除元素 v 使得 Treap 上节点 p 被删除, 若 p 不存在左右儿子, 则 p 可以直接删除。若 p 只存在左儿子或右儿子, 则可以将 p 替换成左儿子或右儿子即可。若 p 既存在左儿子也存在右儿子, 通常有两种解决办法: 第一种, 惰性删除, 即只是将节点上的 cnt 标记为 0, 在后续操作中特殊处理。第二种, 将 p 旋转至叶子后直接删去。旋转次数的期望复杂度为 $O(\log n)$, 最坏为 $O(n)$ 。向左儿子旋转还是向右儿子旋转由 heap 的性质决定: 应把 Val 更大的儿子旋转到父节点的位置。

```

struct node {
    int l, r;
    int key, val;
    int cnt, sz;
};
struct Treap {
    node tr[N];
    int root, idx;
    void push_up(int p) { tr[p].sz = tr[ls(p)].sz + tr[rs(p)].sz + tr[p].cnt; }
    int make_node(int key) {
        tr[++idx] = {0, 0, key, rand(), 1, 1};
        return idx;
    }
}

```

```

void zig(int &p) {
    int q = ls(p);
    ls(p) = rs(q);
    rs(q) = p;
    p = q;
    push_up(rs(p)), push_up(p);
}
void zag(int &p) {
    int q = rs(p);
    rs(p) = ls(q);
    ls(q) = p;
    p = q;
    push_up(ls(p)), push_up(p);
}
void build() {
    root = idx = 0;
    tr[0] = {0, 0, 0, 0, 0, 0};
}
void insert(int &p, int key) {
    if (!p)
        p = make_node(key);
    else if (tr[p].key == key)
        tr[p].cnt++;
    else if (key < tr[p].key) {
        insert(ls(p), key);
        if (tr[ls(p)].val > tr[p].val)
            zig(p);
    } else if (key > tr[p].key) {
        insert(rs(p), key);
        if (tr[rs(p)].val > tr[p].val)
            zag(p);
    }
    if (p)
        push_up(p);
}
bool remove(int &p, int key) {
    if (!p)
        return false;
    bool removed = false;
    if (tr[p].key == key) {
        removed = true;
        if (tr[p].cnt > 1) {
            tr[p].cnt--;
        } else if (ls(p) || rs(p)) {
            if (!rs(p) || tr[ls(p)].val > tr[rs(p)].val) {
                zig(p);
                remove(rs(p), key);
            } else {
                zag(p);
                remove(ls(p), key);
            }
        } else
            p = 0;
    } else if (key < tr[p].key) {
        removed = remove(ls(p), key);
    } else {
        removed = remove(rs(p), key);
    }
    if (p)
        push_up(p);
    return removed;
}
int get_rank_by_key(int p, int key) {
    if (!p)
        return 0;
    if (key == tr[p].key)
        return tr[ls(p)].sz;
    if (key < tr[p].key)
        return get_rank_by_key(ls(p), key);
    return tr[ls(p)].sz + tr[p].cnt + get_rank_by_key(rs(p), key);
}
std::optional<int> get_key_by_rank(int p, int rank) {

```

```

    if (!p || rank <= 0 || rank > tr[p].sz)
        return std::nullopt;
    if (tr[ls(p)].sz >= rank)
        return get_key_by_rank(ls(p), rank);
    if (tr[ls(p)].sz + tr[p].cnt >= rank)
        return tr[p].key;
    return get_key_by_rank(rs(p), rank - tr[ls(p)].sz - tr[p].cnt);
}
std::optional<int> get_prev(int p, int key) {
    std::optional<int> result;
    while (p) {
        if (tr[p].key < key) {
            result = tr[p].key;
            p = rs(p);
        } else {
            p = ls(p);
        }
    }
    return result;
}
std::optional<int> get_next(int p, int key) {
    std::optional<int> result;
    while (p) {
        if (tr[p].key > key) {
            result = tr[p].key;
            p = ls(p);
        } else {
            p = rs(p);
        }
    }
    return result;
}
}
} treap;

```

1.2 FHQ Treap

通常情况下, Treap 指的是有旋 Treap,即通过旋转操作维护 heap 性质。但是实际上,还存在另一种维护 heap 性质的方式:分裂、合并。一般称之为 Fhq-Treap 或无旋 Treap。

1.2.1 分裂:

分裂操作有两种:按键值分裂,按排名分裂。

1.2.1.1 键值分裂:

对于一个参数 v ,把 Treap 分裂成两部分: $tree_1$ 满足所有节点键值 $\leq v$, $tree_2$ 满足所有节点键值 $> v$ 。返回 $tree_1$ 的根和 $tree_2$ 的根。

具体实现,从 Treap 根节点开始,若当前节点键值 $Key > v$,则当前节点和右子树都属于 $tree_2$,同时左子树中可能仍然存在键值 $> v$ 的节点,所以要递归下去分裂并把递归返回的 $tree_2$ 作为当前节点的左子树;若当前节点键值 $Key \leq v$,则当前节点和左子树都属于 $tree_1$,同时右子树中可能仍然存在键值 $\leq v$ 的节点,所以要递归

下去分裂并把递归返回的 $tree_1$ 作为当前节点的右子树。

1.2.1.2 排名定位分裂：

下面的值域 Treap 将相同键值聚合在一个节点中, $cnt[p]$ 表示重复次数, $sz[p]$ 按元素个数计数。因此一个节点可能同时覆盖一段排名, 不能把落在 $cnt[p]$ 中部的排名直接解释成“节点之前”和“节点之后”。

本文的 $split_around_rank(p, rank)$ 定义为: 在 $1 \leq rank \leq sz[p]$ 时, 返回键值更小的树、排名区间覆盖 $rank$ 的完整节点、键值更大的树。若 $rank$ 落在某个节点的重复计数内部, 该节点的全部 cnt 个元素都放在中间部分; 它不是“严格取前 $rank - 1$ 个元素”的分裂。若确实需要按元素个数切成前 k 个和其余元素, 就必须拆分该节点的 cnt , 并在重新合并时恢复相同键值只占一个节点的不变量。隐式序列 Treap 通常令每个节点的 $cnt=1$, 不存在这个歧义。

实际分裂中, 当然可能出现不存在满足条件的 $tree$, 对应部分返回空节点即可。注意分裂时要修改分裂后, 节点的父子关系。

1.2.2 合并：

合并接受两个参数: $root_1, root_2$, 要求 $root_1$ 为根的 $tree_1$ 中所有节点的键值 $\leq root_2$ 为根的 $tree_2$ 中的所有节点的键值, 并且两棵输入树本身都已经满足 heap 性质。合并时根据两个根的 Val 决定新根, 再递归合并相邻的两棵子树。若 Val 独立随机, $split$ 和 $merge$ 的期望复杂度为 $O(\log n)$, 最坏复杂度仍为 $O(n)$ 。

Fhq-Treap 要保证操作结束后还是一整棵树, 也就是每一次通过分裂操作实现别的操作后, 都要通过合并操作把树合并回去。

围绕 Fhq-Treap 的分裂、合并操作, 增删改查操作和朴素 BST 有所不同, 存在更加简便的做法: 本质是将要操作元素单独分裂出来, 然后操作即可。

通过分裂和合并实现查询时常数较大; 不改变树形的查询也可以像朴素 BST 一样沿路径完成。后一种写法的期望复杂度为 $O(\log n)$, 但同样不具有最坏 $O(\log n)$ 保证。

```
struct node {
    int ls, rs, key, val, cnt, sz;
};
struct fhq_treap {
    node tr[N];
    int idx, root;
    int make_node(int key) {
        tr[++idx] = {0, 0, key, -rand(), 1, 1};
        return idx;
    }
    void push_up(int p) { tr[p].sz = tr[ls(p)].sz + tr[rs(p)].sz + tr[p].cnt; }
    // equal_to_left 为 true 时分成 <= key 和 > key,
    // 否则分成 < key 和 >= key, 避免对键值做减一运算。
    pair<int, int> split_by_key(int p, int key, bool equal_to_left) {
```

```

    if (!p)
        return {0, 0};
    if (tr[p].key < key || (equal_to_left && tr[p].key == key)) {
        auto temp = split_by_key(rs(p), key, equal_to_left);
        rs(p) = temp.first;
        push_up(p);
        return {p, temp.second};
    } else {
        auto temp = split_by_key(ls(p), key, equal_to_left);
        ls(p) = temp.second;
        push_up(p);
        return {temp.first, p};
    }
}

tuple<int, int, int> split_around_rank(int p, int rank) {
    if (!p)
        return {0, 0, 0};
    if (rank <= tr[ls(p)].sz) {
        int l, mid, r;
        tie(l, mid, r) = split_around_rank(ls(p), rank);
        ls(p) = r;
        push_up(p);
        return {l, mid, p};
    } else if (rank <= tr[ls(p)].sz + tr[p].cnt) {
        int l = ls(p), r = rs(p);
        ls(p) = rs(p) = 0;
        push_up(p);
        return {l, p, r};
    } else {
        int l, mid, r;
        tie(l, mid, r) =
            split_around_rank(rs(p), rank - tr[ls(p)].sz - tr[p].cnt);
        rs(p) = l;
        push_up(p);
        return {p, mid, r};
    }
}

int merge(int u, int v) {
    if (!u && !v)
        return 0;
    if (u && !v)
        return u;
    if (!u && v)
        return v;
    if (tr[u].val < tr[v].val) {
        rs(u) = merge(rs(u), v);
        push_up(u);
        return u;
    } else {
        ls(v) = merge(u, ls(v));
        push_up(v);
        return v;
    }
}

void insert(int key) {
    auto temp = split_by_key(root, key, true);
    auto l = split_by_key(temp.first, key, false);
    int now = 0;
    if (!l.second) {
        now = make_node(key);
    } else {
        tr[l.second].cnt++;
        push_up(l.second);
    }
    int L = merge(l.first, !l.second ? now : l.second);
    root = merge(L, temp.second);
}

bool remove(int key) {
    auto temp = split_by_key(root, key, true);
    auto l = split_by_key(temp.first, key, false);
    bool removed = (l.second != 0);
    if (tr[l.second].cnt > 1) {

```

```

        tr[l.second].cnt--;
        push_up(l.second);
        l.first = merge(l.first, l.second);
    }
    root = merge(l.first, temp.second);
    return removed;
}
int get_rank_by_key(int p, int key) {
    auto temp = split_by_key(p, key, false);
    int res = tr[temp.first].sz + 1;
    root = merge(temp.first, temp.second);
    return res;
}
std::optional<int> get_key_by_rank(int p, int rank) {
    if (rank <= 0 || rank > tr[p].sz)
        return std::nullopt;
    while (p) {
        if (rank <= tr[ls(p)].sz) {
            p = ls(p);
        } else if (rank <= tr[ls(p)].sz + tr[p].cnt) {
            return tr[p].key;
        } else {
            rank -= tr[ls(p)].sz + tr[p].cnt;
            p = rs(p);
        }
    }
    return std::nullopt;
}
std::optional<int> get_pre(int key) {
    auto temp = split_by_key(root, key, false);
    auto res = get_key_by_rank(temp.first, tr[temp.first].sz);
    root = merge(temp.first, temp.second);
    return res;
}
std::optional<int> get_suf(int key) {
    auto temp = split_by_key(root, key, true);
    auto res = get_key_by_rank(temp.second, 1);
    root = merge(temp.first, temp.second);
    return res;
}
} tree;

```

1.3 Splay

1.3.1 伸展：

Splay 通过伸展操作，不断将某个节点旋转到根节点，即任意操作后得到的节点，都要转到根。能够在均摊 $O(\log n)$ 的时间内完成增删改查。

因为 Splay 的伸展操作，需要考虑的情况过于繁多（主要是多，单独并不难考虑），所以为了简化问题，本文将略过这个具体过程，将其视作一个伸展操作的封装即可。

具体而言， $\text{splay}(x, y)$ 即表示把 x 旋转成 y 的儿子。要求 y 是 x 的祖先，否则不会执行。因为一般情况下，根节点没有父节点，而按照 $\text{splay}(x, y)$ 的定义，如果想把 x 转到根，根不能没有父亲，所以 Splay 一般特殊定义根的父亲为 0。

Splay 上的增删改查都基于 $\text{splay}(x, y)$ 实现。具体而言，先目标元素的前驱转到根，再把目标元素的后继转到前驱的右儿子，此时，目标的位置就是后继的左儿

子。这里和Fhq-Treap 相同,都是将目标元素表示成一棵子树。

```

struct node {
    int s[2];
    int key, fa, cnt, sz;
};
struct Splay {
    node tr[N];
    int root, idx;
    void push_up(int p) { tr[p].sz = tr[ls(p)].sz + tr[rs(p)].sz + tr[p].cnt; }
    void rotate(int x) {
        int y = tr[x].fa, z = tr[y].fa;
        // push_down(y); 如果需要 push_down 的话
        // push_down(x);
        bool w = (rs(y) == x);
        if (z)
            tr[z].s[rs(z) == y] = x;
        tr[x].fa = z;
        int child = tr[x].s[w ^ 1];
        tr[y].s[w] = child;
        if (child)
            tr[child].fa = y;
        tr[x].s[w ^ 1] = y, tr[y].fa = x;
        push_up(y), push_up(x);
    }
    void splay(int x, int k) {
        while (tr[x].fa != k) {
            int y = tr[x].fa, z = tr[y].fa;
            if (z != k) {
                if ((rs(y) == x) ^ (rs(z) == y))
                    rotate(x);
                else
                    rotate(y);
            }
            rotate(x);
        }
        if (!k)
            root = x;
    }
    int make_node(int key, int fa) {
        tr[++idx] = {0, 0, key, fa, 1, 1};
        return idx;
    }
    void init() {
        for (int i = 1; i <= idx; i++)
            tr[i] = {0, 0, 0, 0, 0, 0};
        idx = root = 0;
        tr[++idx] = {0, 2, -inf, 0, 1, 2};
        tr[++idx] = {0, 0, inf, 1, 1, 1};
        root = 1;
    }
    int get_pre(int key, int y = 0) {
        int x = root, res = 0;
        while (x) {
            if (tr[x].key < key) {
                if (!res || tr[res].key < tr[x].key)
                    res = x;
                x = rs(x);
            } else {
                x = ls(x);
            }
        }
        // 因为初始化插入了 -inf 所以前驱一定存在
        splay(res, y);
        return res;
    }
    int get_suf(int key, int y = 0) {
        int x = root, res = 0;
        while (x) {
            if (tr[x].key > key) {
                if (!res || tr[res].key > tr[x].key)
                    res = x;
            }
        }
    }
}

```



```

        x = ls(x);
    } else {
        x = rs(x);
    }
}
// 因为初始化插入了 inf 所以后继一定存在
splay(res, y);
return res;
}
void insert(int key) {
    auto pre = get_pre(key);
    auto suf = get_suf(key, pre);
    auto &now = ls(suf);
    if (now) {
        tr[now].cnt++;
    } else {
        now = make_node(key, suf);
    }
    splay(now, 0);
}
bool remove(int key) {
    auto pre = get_pre(key);
    auto suf = get_suf(key, pre);
    auto &now = ls(suf);

    if (!now || tr[now].key != key)
        return false;

    if (tr[now].cnt > 1) {
        tr[now].cnt--;
        push_up(now);
        splay(now, 0);
    } else {
        tr[now].fa = 0;
        now = 0;
        push_up(suf);
        push_up(pre);
    }
    return true;
}
int get_rank_by_key(int key) {
    // 统计比 key 小的数量, 注意 -inf
    get_pre(key);
    return tr[root].cnt + tr[ls(root)].sz;
}
std::optional<int> get_key_by_rank(int rank) {
    // 根的 sz 还包含 -inf 和 inf 两个内部哨兵。
    if (rank <= 0 || rank > tr[root].sz - 2)
        return std::nullopt;
    int x = root;
    rank++; // 需要加上 -inf 的贡献
    while (x) {
        if (tr[ls(x)].sz >= rank) {
            x = ls(x);
        } else if (tr[ls(x)].sz + tr[x].cnt >= rank) {
            splay(x, 0);
            return tr[root].key;
        } else {
            rank -= tr[ls(x)].sz + tr[x].cnt;
            x = rs(x);
        }
    }
    return std::nullopt;
}
std::optional<int> predecessor(int key) {
    int p = get_pre(key);
    if (tr[p].key == -inf)
        return std::nullopt;
    return tr[p].key;
}
std::optional<int> successor(int key) {
    int p = get_suf(key);

```

```

    if (tr[p].key == inf)
        return std::nullopt;
    return tr[p].key;
}
} tree;

```

Splay 采用迭代实现,一方面是为了方便在操作后进行 splay 操作,另一方面是抵消常数(实际这部分影响比较小)。

1.4 替罪羊树

替罪羊树通过引入一个平衡因子 α ,表示当子节点的子树大小超过当前节点的子树大小 $\times \alpha$ 时将子节点的子树重构的方式,保证树高始终在 $O(\log n)$ 。一般 α 设为 0.7 或 0.8。

1.4.1 重构:

重构分为两个步骤:按中序遍历展开成序列;二分建树。返回重构后的根节点。

1.4.2 插入:

插入部分和朴素 BST 一致,区别在于递归返回时要判断子节点的子树是否需要重构。

1.4.3 删除:

替罪羊树可以实现普通的物理删除,但删除后必须继续维护子树大小并在失衡时重构;“不能普通删除”并不是数据结构本身的限制。下面的模板为了简化实现采用惰性删除, cnt 为 0 表示该键当前不存在。删除前仍须检查键是否存在,避免重复删除使 cnt 变成负数。

下面的实现不再插入 $-INF, +INF$ 实体哨兵,避免它们参与有效元素的子树最值;只有 0 号空节点使用 $\text{maxn}=-INF, \text{minn}=INF$ 作为 max、min 的幺元。

1.4.4 前驱/后继:

因为替罪羊树采用惰性删除,所以查询前驱/后继时,经过的键值不能直接递归。在朴素BST中查询 v 的前驱时,若当前的键值 $< v$,则递归右子树查询。因为朴素 BST 中节点上的键值是一定存在的,所以可以向更大的右节点递归。但是在替罪

羊树中,当前节点的键值可能不存在,此时不能向右递归,因为可能左子树中还可能
有前驱。因此最坏情况下需要遍历整棵BST。时间复杂度: $O(n)$ 。

如果要直接递归求前驱/后继,为每个点再维护一个子树 min, max 即可。

另一个简单的实现是, rank 和 K-th 不受惰性删除影响,所以,可以通过 rank
和 K-th 查询前驱/后继。

实践中,虽然通过 rank 和 K-th 查询前驱/后继需要操作两次,但是因为不需要
维护 min, max, 所以常数差不多,可能前者还更快一点。

```
struct node {
    int ls, rs, key, cnt, sz, s;
    int maxn, minn;
};
struct Tzy_tree {
    node tr[N];
    int root, idx;
    int sec[N];
    double alpha = 0.7;
    bool need_rebuild(int p) {
        return alpha * tr[p].s <= (double)max(tr[ls(p)].s, tr[rs(p)].s);
    }

    void push_up(int p) {
        tr[p].s = tr[ls(p)].s + tr[rs(p)].s + 1;
        tr[p].sz = tr[ls(p)].sz + tr[rs(p)].sz + tr[p].cnt;
        if (!tr[p].cnt) {
            tr[p].maxn = -inf;
            tr[p].minn = inf;
        } else {
            tr[p].maxn = tr[p].minn = tr[p].key;
        }

        tr[p].maxn = max({tr[p].maxn, tr[ls(p)].maxn, tr[rs(p)].maxn});
        tr[p].minn = min({tr[p].minn, tr[ls(p)].minn, tr[rs(p)].minn});
    }

    void Flatten(int &id, int p) {
        if (!p)
            return;
        Flatten(id, ls(p));
        if (tr[p].cnt)
            sec[++id] = p;
        Flatten(id, rs(p));
    }

    int Rebuild(int l, int r) {
        if (l > r)
            return 0;
        int mid = l + r >> 1;
        ls(sec[mid]) = Rebuild(l, mid - 1);
        rs(sec[mid]) = Rebuild(mid + 1, r);
        push_up(sec[mid]);
        return sec[mid];
    }

    void Re(int &p) {
        int id = 0;
        Flatten(id, p);
        p = Rebuild(1, id);
    }

    int make_node(int key) {
        tr[++idx] = {0, 0, key, 1, 1, 1, key, key};
        return idx;
    }

    void init() {
        for (int i = 0; i <= idx; i++)
            tr[i] = {0, 0, 0, 0, 0, 0, -inf, inf};
        idx = root = 0;
    }
}
```

```

void insert(int &p, int key) {
    if (!p) {
        p = make_node(key);
        return;
    }
    if (key < tr[p].key) {
        insert(ls(p), key);
    } else if (key == tr[p].key) {
        tr[p].cnt++;
    } else {
        insert(rs(p), key);
    }
    push_up(p);
    if (need_rebuild(p))
        Re(p);
}

bool remove(int p, int key) {
    if (!p)
        return false;

    bool removed;
    if (key < tr[p].key) {
        removed = remove(ls(p), key);
    } else if (key == tr[p].key) {
        if (!tr[p].cnt)
            return false;
        tr[p].cnt--;
        removed = true;
    } else {
        removed = remove(rs(p), key);
    }
    if (removed)
        push_up(p);
    return removed;
}

int get_rank_by_key(int p, int key) {
    if (!p)
        return 0;
    if (tr[p].key < key)
        return tr[p].cnt + tr[ls(p)].sz + get_rank_by_key(rs(p), key);
    return get_rank_by_key(ls(p), key);
}

std::optional<int> get_key_by_rank(int p, int rank) {
    if (!p || rank <= 0 || rank > tr[p].sz)
        return std::nullopt;
    if (tr[ls(p)].sz >= rank)
        return get_key_by_rank(ls(p), rank);
    if (tr[ls(p)].sz + tr[p].cnt >= rank)
        return tr[p].key;
    return get_key_by_rank(rs(p), rank - tr[ls(p)].sz - tr[p].cnt);
}

std::optional<int> get_pre(int key) {
    int x = root, res = 0;
    bool found = false;
    while (x) {
        if (tr[x].key < key) {
            if (tr[x].cnt) {
                if (!found || res < tr[x].key)
                    res = tr[x].key, found = true;
            } else if (tr[ls(x)].sz && (!found || res < tr[ls(x)].maxn)) {
                res = tr[ls(x)].maxn, found = true;
            }
            x = rs(x);
        } else {
            x = ls(x);
        }
    }
    if (!found)
        return std::nullopt;
    return res;
}

std::optional<int> get_suf(int key) {

```

```

int x = root, res = 0;
bool found = false;
while (x) {
    if (tr[x].key > key) {
        if (tr[x].cnt) {
            if (!found || tr[x].key < res)
                res = tr[x].key, found = true;
        } else if (tr[rs(x)].sz && (!found || tr[rs(x)].minn < res)) {
            res = tr[rs(x)].minn, found = true;
        }
        x = ls(x);
    } else {
        x = rs(x);
    }
}
if (!found)
    return std::nullopt;
return res;
}
} tree;

```

1.5 隐式序列平衡树

值域平衡树用键值决定中序顺序；隐式序列平衡树不把固定的原下标存成 BST 键。节点的当前位置由中序顺序和子树大小动态确定，即当前排名是它的“隐式键”。插入、删除、移动或翻转都会改变后续节点的排名，因此固定原下标无法维护这些操作。

FHQ Treap 通常按“前 k 个元素”和“其余元素”分裂，Splay 通常把第 $l-1$ 个与第 $r+1$ 个节点伸展成祖孙关系，从而把区间 $[l, r]$ 暴露成一棵子树。序列中的每个元素通常单独占一个节点，即 $\text{cnt}=1$ ；节点可以保存元素值，但元素值不参与树形的 BST 比较。访问子节点前还必须下传区间懒标记。

1.5.1 区间加与区间翻转

先按当前排名把 $[l, r]$ 分离成一棵子树，再给该子树根增加加法或翻转懒标记，最后重新合并。FHQ Treap 的期望复杂度为 $O(\log n)$ ，Splay 的均摊复杂度为 $O(\log n)$ 。

1.5.2 区间移动

先分离并删除待移动区间，再按“删除后的序列下标”分裂目标位置，最后把区间子树拼接进去。必须明确目标位置是在删除前还是删除后编号，避免区间位于目标位置之前时产生下标偏移。复杂度与常数分裂、合并相同。

1.5.3 区间插入

在位置 p 后插入长度为 k 的序列时,先把原树分成前 p 个元素和其余元素,再把新序列的树拼在中间。逐个插入新节点的期望复杂度为 $O(k \log n)$;若先在线性时间内建好新序列对应的树,则总期望复杂度为 $O(k + \log n)$ 。

1.5.4 线性建树

FHQ Treap 的中序顺序已经由输入序列确定。为每个节点独立生成随机 Val 后,应使用单调栈按笛卡尔树方式连接父子关系,并在建树后按后序计算 sz,总时间为 $O(n)$ 。不能先随意二分建树、随机填写 Val,再期待后续一次 merge 修复内部 heap 性质;merge 的前提正是两棵输入子树已经分别满足 heap 性质。

Splay 不依赖随机优先级,可以按序列中点递归建立近似平衡的初始树,并自底向上维护信息,建树时间为 $O(n)$ 。

1.6 边界约定

上述模板的第 k 小、前驱和后继接口使用 `std::optional<int>` 表示无解,需要 C++17 和 `<optional>`。Splay 内部仍使用 $-INF, +INF$ 两个哨兵简化伸展操作,但哨兵不是合法输入值,公开查询通过 `std::nullopt` 隐藏哨兵。删除接口在目标不存在时返回 `false`,不得修改 0 号节点或已有子树信息。

1.7 比较

种类	常数的常见表现	区间翻转	持久化
Treap	通常较小	×	√
FHQ Treap	取决于 split/merge 次数	√	√
Splay	通常较大	√	×
替罪羊树	取决于重构频率	×	×

WBLT 时间常数可能还要更小一些,再说吧。

表中的常数只是在特定编译器、内存布局、随机数生成器和操作分布下的经验描述,不是普遍性能排序;实际选择应以目标环境中的基准测试为准。

2 线段树

2.1 zkw 线段树

普通线段树是一棵二叉树，zkw 线段树是一棵满二叉树，同样采用堆式建树。因为其满二叉树的性质，使得它能容易地获取叶子节点编号，也可以通过位运算简单地获取区间。具有代码短，常数小的优点。

了解即可，实用价值有，但是不大。

下面以区间加、区间和为例。接口统一使用从 0 开始的半开区间 $[l, r)$ 。与递归线段树一样，迭代线段树也可以使用懒标记：修改完整覆盖节点时，必须让 `sum` 增加“增量乘区间长度”，查询或继续向下访问前再把祖先标记下传。

数组固定树形也不意味着不能持久化；可以持久化底层数组或记录每个版本的修改。只是竞赛中路径复制通常配合显式左右儿子的递归实现更方便。

区间加、区间和参考实现

```
struct zkw_segment_tree {
    int size, height;
    vector<long long> sum, tag;
    vector<int> length;

    void apply(int p, long long value) {
        sum[p] += value * length[p];
        if (p < size)
            tag[p] += value;
    }

    void push_up(int p) {
        // tag[p] 永久保留在 p 上时也不能丢掉它的贡献。
        sum[p] = sum[p << 1] + sum[p << 1 | 1] + tag[p] * length[p];
    }

    void push_down(int p) {
        if (!tag[p])
            return;
        apply(p << 1, tag[p]);
        apply(p << 1 | 1, tag[p]);
        tag[p] = 0;
    }

    void push_path(int p) {
        for (int level = height; level >= 1; level--)
            push_down(p >> level);
    }

    void init(const vector<long long> &a) {
        size = 1;
        height = 0;
        while (size < (int)a.size()) {
            size <<= 1;
            height++;
        }

        sum.assign(size << 1, 0);
        tag.assign(size, 0);
        length.assign(size << 1, 1);
    }
};
```

```

length[0] = 0;
for (int p = size - 1; p; p--)
    length[p] = length[p << 1] + length[p << 1 | 1];
for (int i = 0; i < (int)a.size(); i++)
    sum[size + i] = a[i];
for (int p = size - 1; p; p--)
    push_up(p);
}

void range_add(int l, int r, long long value) {
    if (l >= r)
        return;
    int left = l + size, right = r + size;
    int left_leaf = left, right_leaf = right - 1;
    push_path(left_leaf);
    push_path(right_leaf);

    while (left < right) {
        if (left & 1)
            apply(left++, value);
        if (right & 1)
            apply(--right, value);
        left >>= 1;
        right >>= 1;
    }

    for (int p = left_leaf >> 1; p; p >>= 1)
        push_up(p);
    for (int p = right_leaf >> 1; p; p >>= 1)
        push_up(p);
}

long long range_sum(int l, int r) {
    if (l >= r)
        return 0;
    int left = l + size, right = r + size;
    push_path(left);
    push_path(right - 1);

    long long left_sum = 0, right_sum = 0;
    while (left < right) {
        if (left & 1)
            left_sum += sum[left++];
        if (right & 1)
            right_sum += sum[--right];
        left >>= 1;
        right >>= 1;
    }
    return left_sum + right_sum;
}

void point_add(int position, long long value) {
    range_add(position, position + 1, value);
}

long long point_query(int position) {
    return range_sum(position, position + 1);
}
};

```

建树为 $O(n)$, 区间修改和区间查询均为 $O(\log n)$, 空间复杂度为 $O(n)$ 。

2.2 猫树

名字由来似乎是首次在国内 OI 届引入此数据结构的选手的网名。

猫树通常指 Disjoint Sparse Table, 用于静态区间询问。它只要求合并运算满足结合律, 不要求幂等性; 预处理时间和空间均为 $O(n \log n)$, 非空区间查询为 $O(1)$ 。

对于两个叶子节点 l, r , 设它们的 LCA 表示区间 $[L, R]$, 中点为 mid 。当 $l < r$ 时, 查询区间恰好按顺序分成 $[l, mid]$ 与 $[mid + 1, r]$, 两部分拼接后得到 $[l, r]$ 。

若每一层都维护块内的后缀积和前缀积, 那么查询 $[l, r]$ 时, 只需找到 l, r 的最高不同位所在层, 把从 l 到块中点的后缀积与从中点到 r 的前缀积合并。对于非交换运算, 顺序必须是 $op(suffix[l], prefix[r])$, 不能颠倒。

对于 LCA, 因为 zkw 是满二叉树, 所以 l, r 的 LCA 容易发现就是它们节点编号的二进制表达下的 Lcp (类似 01-Trie)。使用位运算容易得到 $LCA = l \gg (\log[l \wedge r] + 1)$ 。

```
template <class T, class Op>
struct disjoint_sparse_table {
    int n, levels;
    vector<T> value;
    vector<vector<T>> table;
    Op op;

    // a 使用 0 下标。
    void init(const vector<T> &a, Op operation = Op()) {
        value = a;
        op = operation;
        n = (int)a.size();
        levels = 0;
        while ((1 << levels) < n)
            levels++;
        table.assign(levels, vector<T>(n));

        for (int k = 0; k < levels; k++) {
            int half = 1 << k;
            int block_length = half << 1;
            for (int left = 0; left < n; left += block_length) {
                int middle = min(left + half, n);
                int right = min(left + block_length, n);

                if (left < middle) {
                    table[k][middle - 1] = value[middle - 1];
                    for (int i = middle - 2; i >= left; i--)
                        table[k][i] = op(value[i], table[k][i + 1]);
                }
                if (middle < right) {
                    table[k][middle] = value[middle];
                    for (int i = middle + 1; i < right; i++)
                        table[k][i] = op(table[k][i - 1], value[i]);
                }
            }
        }
    }

    // 查询 0 下标闭区间 [left, right], 要求 left <= right.
    T query(int left, int right) const {
        if (left == right)
            return value[left];
        int level = 31 - __builtin_clz((unsigned)(left ^ right));
        return op(table[level][left], table[level][right]);
    }
};
```

经典的重叠 Sparse Table 依赖幂等性, 而猫树与 Sqrt Tree 都能维护任意结合运算。它们的功能不存在严格包含关系, 空间常数也取决于布局、是否补齐到二次幂

以及实现方式,不能笼统断言“至少是ST 表四倍”或“Sqrt Tree 功能严格更强”。

2.3 李超线段树

李超线段树用于动态插入直线或限定横坐标范围的线段,并查询给定横坐标处的最优直线。下文维护最大值;若纵坐标相同,规定编号更小的直线更优。

下面的实现维护离散横坐标数组 x 。树上区间 $[l, r]$ 表示 $x[l]$ 到 $x[r]$,所有插入端点和查询横坐标都必须先映射为数组下标。节点的 $id == 0$ 明确表示“当前节点没有直线”,不会访问 $line[0]$,因此区间插入时允许出现已经创建、但尚未保存主导直线的祖先节点。

2.3.1 询问:

查询时沿根到叶子的路径,把途中所有非空主导直线按同一个比较规则合并即可,时间复杂度为 $O(\log C)$,其中 C 是离散横坐标数量。

整数直线、离散横坐标参考实现

```
using i128 = __int128_t;

struct li_chao_tree {
    struct line_type {
        long long slope, intercept;
        i128 value(long long x) const {
            return (i128)slope * x + intercept;
        }
    };
    struct node_type {
        int left = 0, right = 0, id = 0;
    };

    vector<long long> x;
    vector<line_type> line{line_type{}}; // 下标 0 不存放直线
    vector<node_type> tree{node_type{}}; // 下标 0 表示空节点
    int root = 0;

    void init(vector<long long> coordinates) {
        sort(coordinates.begin(), coordinates.end());
        coordinates.erase(unique(coordinates.begin(), coordinates.end()), coordinates.end());
        assert(!coordinates.empty());
        x = move(coordinates);
        line.assign(1, line_type{});
        tree.assign(1, node_type{});
        root = 0;
    }

    int add_line(long long slope, long long intercept) {
        line.push_back({slope, intercept});
        return (int)line.size() - 1;
    }

    int new_node() {
        tree.push_back({});
        return (int)tree.size() - 1;
    }

    bool better(int first, int second, int position) const {
```

```

    if (!first)
        return false;
    if (!second)
        return true;
    i128 first_value = line[first].value(x[position]);
    i128 second_value = line[second].value(x[position]);
    if (first_value != second_value)
        return first_value > second_value;
    return first < second;
}

int insert_line(int p, int left, int right, int id) {
    if (!p)
        p = new_node();
    if (!tree[p].id) {
        tree[p].id = id;
        return p;
    }

    int middle = (left + right) >> 1;
    if (better(id, tree[p].id, middle))
        swap(id, tree[p].id);
    if (left == right)
        return p;

    if (better(id, tree[p].id, left))
        tree[p].left = insert_line(tree[p].left, left, middle, id);
    else if (better(id, tree[p].id, right))
        tree[p].right = insert_line(tree[p].right, middle + 1, right, id);
    return p;
}

void insert_line(int id) {
    root = insert_line(root, 0, (int)x.size() - 1, id);
}

int insert_segment(int p, int left, int right,
                  int query_left, int query_right, int id) {
    if (query_right < left || right < query_left)
        return p;
    if (!p)
        p = new_node();
    if (query_left <= left && right <= query_right)
        return insert_line(p, left, right, id);

    int middle = (left + right) >> 1;
    tree[p].left = insert_segment(tree[p].left, left, middle,
                                query_left, query_right, id);
    tree[p].right = insert_segment(tree[p].right, middle + 1, right,
                                query_left, query_right, id);
    return p;
}

void insert_segment(int left, int right, int id) {
    root = insert_segment(root, 0, (int)x.size() - 1, left, right, id);
}

int query(int p, int left, int right, int position) const {
    if (!p)
        return 0;
    int answer = tree[p].id;
    if (left == right)
        return answer;

    int middle = (left + right) >> 1;
    int child_answer;
    if (position <= middle)
        child_answer = query(tree[p].left, left, middle, position);
    else
        child_answer = query(tree[p].right, middle + 1, right, position);
    if (better(child_answer, answer, position))
        answer = child_answer;
}

```

```

    return answer;
}

// 返回 0 表示该横坐标没有任何已插入线段。
int query(int position) const {
    return query(root, 0, (int)x.size() - 1, position);
}
};

```

整条直线插入调用 `insert_line(id)`, 为 $O(\log C)$; 限定区间的线段会被分解到 $O(\log C)$ 个规范节点, 插入为 $O(\log^2 C)$ 。实现使用 `__int128` 比较整数斜率、截距和横坐标, 避免 `long long` 乘法先溢出。若数据是浮点数, 则相等判断、近交点误差和无穷值协议必须按题目精度要求重新设计, 不能直接套用整数比较。

2.4 可持久化线段树

2.4.1 动态开点线段树:

因为是可持久化线段树的前置知识(严格需要), 略提一下。

一般线段树建树时, 将所有区间都在节点上表示了出来。但是事实上, 对于一个节点 p 的区间 $[l, r]$, 如果整个操作流程中都未涉及, 那么这个节点及其所在的子树是否存在, 是不会影响结果的正确性的。动态开点线段树应运而生。

即: 只有在访问到某个区间时, 才创建对应的节点编号。如果只是将朴素线段树, 以单点加、区间和为例替换成动态开点线段树, 只需在函数体中, 把节点编号改为引用, 并把节点的左右儿子显式地存储在节点的结构体中, 而不是沿用堆式建树的隐式儿子表示。当当前递归区间的节点不存在时, 将其赋予新的编号即可。询问时若遇到空节点则表示没有内容, 返回空即可。

```

void update(int &p, int l, int r, int x, int v) {
    int mid = l + r >> 1;
    if (!p)
        p = ++idx;
    if (l == r) {
        tr[p].sum += v;
        return;
    }
    if (x <= mid)
        update(ls(p), l, mid, x, v);
    else
        update(rs(p), mid + 1, r, x, v);
    push_up(p);
}

int query(int p, int l, int r, int x, int y) {
    int mid = l + r >> 1, res = 0;
    if (!p)
        return 0;
    if (x <= l && r <= y) {
        return tr[p].sum;
    }
    if (x <= mid)
        res += query(ls(p), l, mid, x, y);
    if (y > mid)
        res += query(rs(p), mid + 1, r, x, y);
    return res;
}

```

```
}

```

因为动态开点只涉及到需要的节点,所以可以支持维护序列长度很大的情况,每一次操作最多涉及线段树上 $O(\log n)$ 个节点。

因为动态开点每次最劣会创建访问到的区间数量的节点数,所以动态开点的空间复杂度是 $O(m \log n)$ 的。不过如果是一般线段树中使用动态开点替换,那么不会每一次的节点都是未涉及过的,所以空间复杂度还会是和堆式建树一致为 $O(n)$ 。

2.4.2 标记永久化:

因为是可持久化线段树的前置知识(不严格需要),略提一下。

一个区间在线段树上的规范分解只包含 $O(\log n)$ 个完整覆盖节点;可能与它相交的全部后代确实可达 $O(n)$,但标准修改不会继续遍历已经完整覆盖的节点。标记永久化把修改保留在这 $O(\log n)$ 个规范节点上,查询向下时累计祖先标记,而不是把标记下传到所有后代。

标记永久化的实现选择不下传,而是在访问路径上累计祖先标记。与懒标记相比,对于不能直接累计的标记,通常更难处理。

可持久化数据结构意味可以维护历史版本,即第 i 次操作的结果是建立在第 $i - 1$ 次操作结果的基础上的。

因为线段树是自上而下的数据结构,对于一个确定的子树,其维护的信息是确定的。所以若节点 p 和节点 q 的儿子维护信息相同,那么可以将它们的儿子节点设为同一个。所以,对于可持久化线段树,对于第 i 次操作中,未被修改,也就是和第 $i - 1$ 次操作信息一致的区间,用同一个节点表示即可。对于被修改的节点,可以创建一个旧版本的拷贝,再在这个新的节点上进行修改即可。

2.4.3 主席树:

可持久化权值(值域)线段树一般被称作主席树。下面维护前缀出现次数: $\text{root}[i]$ 表示前 i 个元素的频率版本, $\text{kth_in_subarray}(\text{left}, \text{right}, k)$ 返回子数组 $[\text{left}, \text{right}]$ 的第 k 小离散值。它不是“查询某个位置的点值”。

```

struct node {
    int left = 0, right = 0, sum = 0;
} tr[N << 5];

int idx, root[N];
int version_count, compressed_size;

int clone_node(int previous) {
    int current = ++idx;
    tr[current] = tr[previous]; // 叶子和内部节点的旧信息全部复制
    return current;
}

int update(int previous, int left, int right, int position) {
    int current = clone_node(previous);
    tr[current].sum++;
    if (left == right)
        return current;

    int middle = (left + right) >> 1;
    if (position <= middle)
        tr[current].left = update(tr[previous].left, left, middle, position);
    else
        tr[current].right = update(tr[previous].right, middle + 1, right, position);
    return current;
}

// rank[i] 是第 i 个元素在 [1, value_count] 中的离散排名。
void build_prefix_versions(int n, int value_count, const vector<int> &rank) {
    idx = 0;
    version_count = n;
    compressed_size = value_count;
    tr[0] = {};
    root[0] = 0;
    for (int i = 1; i <= n; i++)
        root[i] = update(root[i - 1], 1, value_count, rank[i]);
}

int kth(int left_root, int right_root,
        int left, int right, int k) {
    if (left == right)
        return left;

    int middle = (left + right) >> 1;
    int left_count = tr[tr[right_root].left].sum
        - tr[tr[left_root].left].sum;
    if (k <= left_count)
        return kth(tr[left_root].left, tr[right_root].left,
            left, middle, k);
    return kth(tr[left_root].right, tr[right_root].right,
        middle + 1, right, k - left_count);
}

// 返回 -1 表示参数越界; 否则返回离散后的值域下标。
int kth_in_subarray(int left, int right, int k) {
    if (left < 1 || right > version_count || left > right || k <= 0)
        return -1;
    int count = tr[root[right]].sum - tr[root[left - 1]].sum;
    if (k > count)
        return -1;
    return kth(root[left - 1], root[right], 1, compressed_size, k);
}

```

每个新版本只复制根到一个叶子的 $O(\log V)$ 个节点, 其中 V 是离散值域大小; 构建 n 个前缀版本的时间和空间均为 $O(n \log V)$, 单次区间第 k 小查询为 $O(\log V)$ 。

2.4.4 区间修改:

若第 i 次操作是在第 $i - 1$ 次操作的基础上,进行区间修改,以区间加为例,那么在后续访问第 i 次操作的版本时,若使用懒标记,且直接将节点上的懒标记 `push_down`,常数略大,容易爆空间。使用标记永久化,会有更好的效果。

2.4.5 Trick:

若第 i 个版本,不止进行一次操作。一个简单的解决办法就是,直接迭代操作次数个版本,然后记录下第 i 个版本对应的最后一个版本是哪个。另一个解决办法先让 $root[i] = root[i - 1]$,然后创建一个临时根,用这个临时根迭代 $root[i]$ 操作次数次即可(每一次迭代都用临时根替换 $root[i]$)。

2.5 吉司机线段树

历史最值线段树,经典的“势能线段树”,暂略。

2.6 线段树合并

线段树合并就是将两棵线段树上表示相同区间的信息合并起来,所以时间复杂度显然与节点数相关,所以线段树合并通常针对动态开点线段树,只合并需要用到的点。

实现不难理解,对于合并 $tree_1, tree_2$,若当前 $tree_1, tree_2$ 其中一个左儿子不存在,另一个左儿子存在,那么直接将存在的那一个左儿子作为合并后的左儿子即可;若 $tree_1, tree_2$ 左儿子都存在,那么递归合并;若 $tree_1, tree_2$ 左儿子都不存在,不进行合并,结束递归。右儿子同理。容易发现,最终合并的节点都是叶子节点,然后通过 `push_up` 更新祖先节点即可。

以区间和为例:

```
int merge(int p, int q, int l, int r) {
    int mid = l + r >> 1;
    if (!p)
        return q;
    if (!q)
        return p;
    if (l == r) {
        tr[p].sum += tr[q].sum;
        return p;
    }
    ls(p) = merge(ls(p), ls(q), l, mid);
    rs(p) = merge(rs(p), rs(q), mid + 1, r);
    push_up(p);
    return p;
}
```

上述代码是破坏式合并:调用后必须丢弃根 q , 不能再从其他版本或容器访问它。只有在这种前提下, 并且每个被吞并的节点至多参与一次重叠递归时, 才能把一系列合并的总复杂度按初始节点数 S 摊还为 $O(S)$; 若初始每棵权值线段树合计创建了 $O(n \log V)$ 个节点, 就得到 $O(n \log V)$ 。若需要保留两棵输入树、支持持久化, 或让同一棵树反复参与合并, 上述一次性收费不成立, 复杂度必须按每次实际访问的节点重新计算。

线段树合并从应用角度而言, 通常用于合并权值线段树, 并且更多地应用在树上, 用于合并子树信息。

2.7 线段树分裂

线段树分裂是线段树合并的逆过程。若节点只保存一个不可逆聚合值, 例如只保存两个集合合并后的最大值, 就不能仅凭这个聚合值反推出两部分; 通常需要保留子树、叶子信息或修改历史。这不等于“最大值操作不能撤销”: 若记录修改前的值或维护修改栈, 最大值同样可以回滚。

以权值线段树统计数字出现次数为例: 保留前 k 小的数字, 将其余数字分裂到一棵新的线段树上。

```
void split(int p, int &q, int k) {
    if (!p)
        return;
    if (!q)
        q = ++idx;
    if (k > tr[ls(p)].sum)
        split(rs(p), rs(q), k - tr[ls(p)].sum);
    else
        swap(rs(p), rs(q));
    if (k < tr[ls(p)].sum)
        split(ls(p), ls(q), k);
    tr[q].sum = tr[p].sum - k;
    tr[p].sum = k;
}
```

线段树分裂的时间复杂度容易发现是单次 $O(\log n)$ 的, 因为只会向一侧递归。

2.8 线段树优化建图

线段树优化建图用于将一个点和一个区间内的所有点连边的问题, 例如将 u 和 $i \in [l, r]$ 的所有 i 连边, 或将 $i \in [l, r]$ 的所有 i 和 u 连边, 后求最短路。若直接建边, 边数是 $O(n^2)$ 的。

根据任意一个区间 $[l, r]$ 在线段树上一定可以拆分成 $O(\log n)$ 个区间的性质。那么若用一个点代替一个区间, 那么对于任意一个区间 $[l, r]$, 都可以拆分成 $O(\log n)$ 个节点。

若 u 连向了 $[l, r]$, 而最后要求的还是单点的最短路。因为 $[l, r]$ 表示 $i \in [l, r]$ 的所有 i , 所以 $[l, r]$ 代表的节点要连向 $i \in [l, r]$ 的所有 i , 但是仍然不能直接连, 所以所有节点在线段树上向儿子节点连权值为 0 的边。

若 $[l, r]$ 连向了 u , 那么相当于 $i \in [l, r]$ 的所有 i 连向了 u , 和上文同理, 线段树上所有点要向父节点连权值为 0 的边。

但是容易发现, 这不能在同一棵线段树上进行。所以建两棵线段树, 一棵向儿子节点连边, 一棵向父亲节点连边。两棵树表示同一个原点的叶子必须双向连接权值为 0 的边, 使 $\text{id1}[i]$ 与 $\text{id2}[i]$ 成为原点 i 的两个等价入口。示例中的普通边 $u \rightarrow v$ 同时补齐四种叶子入口组合, 不能把其中一条重复写成 $\text{id2}[u] \rightarrow \text{id2}[v]$ 。

以最短路为例: $u \xrightarrow{w} [l, r]$, $u \xrightarrow{w} v$, $[l, r] \xrightarrow{w} u$, 求 s 为起点的最短路。

```
struct segment {
    int idx;
    int id1[N], id2[N];
    void build1(int t, int l, int r) {
        idx = max(idx, t);
        int mid = l + r >> 1;
        if (l == r) {
            id1[l] = t;
            return;
        }
        add(t, t << 1, 0);
        add(t, t << 1 | 1, 0);
        build1(t << 1, l, mid);
        build1(t << 1 | 1, mid + 1, r);
    }
    void build2(int t, int l, int r) {
        int mid = l + r >> 1;
        if (l == r) {
            id2[l] = t + idx;
            return;
        }
        add((t << 1) + idx, t + idx, 0);
        add((t << 1 | 1) + idx, t + idx, 0);
        build2(t << 1, l, mid);
        build2(t << 1 | 1, mid + 1, r);
    }
    void link1(int t, int l, int r, int x, int y, int u, int w) {
        int mid = l + r >> 1;
        if (x <= l && r <= y) {
            add(id2[u], t, w);
            return;
        }
        if (x <= mid)
            link1(t << 1, l, mid, x, y, u, w);
        if (y > mid)
            link1(t << 1 | 1, mid + 1, r, x, y, u, w);
    }
    void link2(int t, int l, int r, int x, int y, int u, int w) {
        int mid = l + r >> 1;
        if (x <= l && r <= y) {
            add(t + idx, id1[u], w);
            return;
        }
        if (x <= mid)
            link2(t << 1, l, mid, x, y, u, w);
        if (y > mid)
            link2(t << 1 | 1, mid + 1, r, x, y, u, w);
    }
} tree;
signed main() {
    int n, q, s;
```

```

cin >> n >> q >> s;
tree.build1(1, 1, n);
tree.build2(1, 1, n);
int i;
For(i, 1, n) add(tree.id1[i], tree.id2[i], 0),
    add(tree.id2[i], tree.id1[i], 0);
while (q--) {
    int op;
    cin >> op;
    if (op == 1) {
        int u, v, w;
        cin >> u >> v >> w;
        add(tree.id1[u], tree.id1[v], w);
        add(tree.id2[u], tree.id2[v], w);
        add(tree.id1[u], tree.id2[v], w);
        add(tree.id2[u], tree.id1[v], w);
    }
    if (op == 2) {
        int u, l, r, w;
        cin >> u >> l >> r >> w;
        tree.link1(1, 1, n, l, r, u, w);
    }
    if (op == 3) {
        int u, l, r, w;
        cin >> u >> l >> r >> w;
        tree.link2(1, 1, n, l, r, u, w);
    }
}
return 0;
}

```

线段树优化建图本质上只是一个建图的工具,结合到具体题目,还是需要一定的图论知识。

2.9 线段树分治

线段树分治一定程度上类似线段树优化建图,本质还是利用线段树的性质:一个区间 $[l, r]$ 可以在线段树上被完整地拆分成 $O(\log n)$ 个节点。线段树分治往往应用于操作按时间点出现消失,即某一个操作只会在某一段连续时间内存在。

对于这一段时间,可以拆分成 $O(\log n)$ 段线段树上的连续区间。对于整个操作时间轴,视作线段树的根节点。递归左子树表示处理 $[l, mid]$ 时间内的所有操作。

把一段时间 $[l, r]$ 的操作放在 $O(\log n)$ 个节点上,当递归进入这个节点时执行操作,退出时回滚到进入前的状态。要求的是状态能够高效恢复;最大值也可以通过记录旧值或修改栈撤销。递归到叶子节点时,当前时刻有效的全部操作均已执行,此时必须实际回答该时刻的询问。

一个线段树分治的一个经典应用是和可撤销并查集的结合使用,用来维护点的连通性。

```

struct sege {
    int n;
    vector<int> tr[N << 2];
    function<void(int)> answer_query;
    void update(int t, int l, int r, int x, int y, int id) {
        int mid = l + r >> 1;
        if (x <= l && r <= y) {
            tr[t].pb(id);
            return;
        }
        if (x <= mid)
            update(t << 1, l, mid, x, y, id);
        if (y > mid)
            update(t << 1 | 1, mid + 1, r, x, y, id);
    }
    void solve(int t, int l, int r) {
        int his = dsu.History();
        for (auto u : tr[t]) {
            dsu.merge(edge[u].x, edge[u].y);
        }
        if (l == r) {
            answer_query(l); // 根据题意回答第 l 个时刻的询问
        } else {
            int mid = l + r >> 1;
            solve(t << 1, l, mid);
            solve(t << 1 | 1, mid + 1, r);
        }
        while (dsu.History() != his)
            dsu.roll(); // 可撤销并查集部分
    }
} tree;

```

2.10 线段树上二分

对于一个单调的区间询问,例如正权区间内第一个前缀和大于 x 的位置。如果是一个 $[1, n]$ 上的全局询问,不难得到直接在线段树左右递归即可,时间复杂度 $O(\log n)$ 。一个经典应用是可持久化权值线段树(主席树)上求 k -th。

如果询问不是全局的,同样地,将询问区间拆分成线段树上的 $O(\log n)$ 个区间,答案一定落在某一个区间内。一个很 naive 的做法是,直接把这 $O(\log n)$ 个节点离线下来,然后扫一遍找到答案所在区间,然后对该节点进行一个和全局询问类似的子树递归即可。

时间复杂度: $O(\log n)$ 。

3 树状数组

3.1 线性建树

树上共有 n 个节点,父节点比子节点编号大,树状数组上 x 的父亲是 $x + \text{lowbit}(x)$,所以从小到大枚举节点,用当前节点更新父亲的值即可。

```
void init(int n, vector<int> &a) {
    this->n = n;
    for (int i = 1; i <= n; i++) {
        tr[i] += a[i];
        int j = i + lowbit(i);
        if (j <= n)
            tr[j] += tr[i];
    }
}
```

还有一个更简单的 $O(n)$ 建树,即根据树状数组管辖区间定义, tr_x 表示 $[x - \text{lowbit}(x) + 1, x]$ 的区间,所以预处理前缀和数组 sum , $tr_x = sum_x - sum_{x - \text{lowbit}(x)}$ 即可。

```
void init(int n, vector<int> &sum) {
    this->n = n;
    for (int i = 1; i <= n; i++) {
        tr[i] += sum[i] - sum[i - lowbit(i)];
    }
}
```

3.2 树状数组倍增

对于 $a_i \geq 0$,求最小的 x 满足 $\sum_{i=1}^x a_i > y$ 。

从最高位开始,若这一位的管辖区间的 sum 加上已经累计的 res 后超过了 y ,则考虑下一位,反之累计到 res 中。最后累计过的所有位的或就是满足题意的最小 x 。

空间复杂度: $O(n)$ 。单次时间复杂度: $O(\log n)$ 。只能维护前缀。

```
int ask(int v) {
    int now = 0;
    for (int i = __lg(n); i >= 0; i--) {
        int cur = now + (1 << i);
        if (cur > n)
            continue;
        if (tr[cur] <= v) {
            now = cur;
            v -= tr[now];
        }
    }
    return now + 1; // 无解返回 n + 1
}
```

3.3 树状数组维护区间加、区间和：

区间和可差分为前缀和。

令 a' 表示 a 的一阶差分数组,那么 a 的一阶前缀和等于 a' 的二阶前缀和。

交换求和次序可得: $\sum_{j=1}^i \sum_{k=1}^j a'_k = \sum_{k=1}^i a'_k \times (i - k + 1) = (i + 1) \times \sum_{k=1}^i a'_k - \sum_{k=1}^i k \times a'_k$ 。

所以可以转换成在两个差分数组上的单点加和区间和,使用两个树状数组维护即可。

与线段树相比具有常数小、代码短的优势。洛谷【线段树1】上述实现效率和码量都是递归线段树 $\frac{1}{2}$ 。

3.4 维护不可差分信息

以区间 $\max$ 为例。

3.4.1 单点修改：

将 a_x 修改成 v 后,若直接对 x 的所有祖先节点取 $\max$,是错误的。因为将 a_x 修改成 v ,若 $\max$ 就是原本的 a_x ,那么 a_x 不存在了,理论上 x 的所有祖先节点都应先变成次大值,然后再更新,但是因为信息的不可差分性,导致了这个次大无法维护。

解决办法是,从后代更新到祖先,那么当更新到祖先时,后代就一定正确的,所以每次将当前节点修改为 v 后,再用该节点的所有儿子节点更新该节点即可。

不难得到, x 的所有儿子节点为: $x - 2^k, 2^k < \text{lowbit}(x)$ 。时间复杂度: $O(\log^2 n)$ 。

```
void update(int x, int v) {
    a[x] = v;
    for (; x <= n; x += lowbit(x)) {
        tr[x] = a[x];
        for (int y = 1; y < lowbit(x); y <= 1)
            tr[x] = max(tr[x], tr[x - y]);
    }
}
```

3.4.2 区间询问：

3.4.2.1 不可差分信息：

和树状数组倍增类似,假设询问区间 $[l, r]$,从 r 不断 $-lowbit(r)$ 直到继续减会小于 l 。而后 $r - 1$,继续迭代。时间复杂度: $O(\log^2 n)$ 。

```
int query(int l, int r) {
    int res = -inf;
    while (r >= l) {
        res = max(res, a[r]);
        --r;
        for (; r - lowbit(r) >= l; r -= lowbit(r)) res = max(res, tr[r]);
    }
    return res;
}
```

3.4.2.2 可差分信息：

```
int query(int l, int r)
{
    l--;
    int res = 0;
    for (; r > l; r -= lowbit(r))
        res += tr[r];
    for (; l > r; l -= lowbit(l))
        res -= tr[l];
    return res;
}
```

时间复杂度: $O(\log n)$, 常数远比原本做两次前缀询问后差分小。

4 树套树

在算法竞赛中,我们有时需要维护多维度信息。在这种时候,可以使用树套树来维护。树套树简单来说就是树形数据结构上的每一个节点都维护了一个树形结构,而不是简单的几个信息。

为避免混淆,下文约定 n 为初始点数(同时假设外层下标已经压缩到 $[1, n]$), q 为初始化后的操作数, V 为内层坐标或权值的值域大小。未特别说明时,“修改、询问复杂度”均指单次操作;处理全部操作的时间还要乘以相应的操作次数。

一般而言,是线段树的每一个节点维护了另一个数据结构。若每个初始点被存入外层线段树中所有包含它的位置,则所有初始点在内层结构中的副本总数为 $O(n \log n)$ 。这只能用于估计静态建树的基础规模;修改引入的新权值还可能使空间与 q 、 V 有关,不能据此直接断定整棵树套树的时空复杂度。

往往外层线段树只是用于维护内层数据结构,信息还是由内层数据结构维护。

树套树的询问,同样依托于线段树的核心性质:一个区间 $[l, r]$ 在线段树上是 $O(\log n)$ 个节点,所以树套树的询问相当于将 $O(\log n)$ 个内层数据结构的信息询问后进行合并。若内层结构单次询问为 $O(w_q)$,则一次外层区间询问为 $O(w_q \log n)$ 。

对于单点修改操作,只会涉及到线段树上 $O(\log n)$ 个节点,所以树套树的修改相当于对 $O(\log n)$ 个内层数据结构进行修改。若内层结构单次修改为 $O(w_u)$,则一次外层单点修改为 $O(w_u \log n)$ 。注意不要只修改叶子然后用 `push_up` 维护祖先节点,那样依赖于节点信息的重构显然时间复杂度会爆炸。

至此,树套树与仅使用内层数据结构维护信息相比,不仅支持了区间询问,还支持了修改操作。

4.1 树状数组套DS

4.1.1 树状数组套树状数组

实际上就是所谓二维树状数组。

设外层坐标范围为 $[1, n]$, 内层坐标范围为 $[1, V]$ 。单点修改和二维矩形前缀询问的单次时间复杂度均为 $O(\log n \log V)$; 用单点修改加入 n 个初始点需要 $O(n \log n \log V)$, 处理 q 次操作需要 $O(q \log n \log V)$ 。

使用稠密二维数组时,空间复杂度为 $O(nV)$ 。

```

struct BBIT
{
    int tr[N][N], n, m;
    int lowbit(int x)
    {
        return x & -x;
    }
    void update(int x, int y, int v)
    {
        for (; x <= n; x += lowbit(x))
            for (int i = y; i <= m; i += lowbit(i))
                tr[x][i] += v;
    }
    int query(int x, int y)
    {
        int res = 0;
        for (; x; x -= lowbit(x))
            for (int i = y; i; i -= lowbit(i))
                res += tr[x][i];
        return res;
    }
};

```

4.1.2 树状数组套vector

需要离线收集 n 个初始点以及后续修改可能出现的所有 (x, y) 。候选修改点至多有 $n + q$ 个；每个候选点的 y 坐标会被注册到 $O(\log n)$ 个外层树状数组节点中。

因此，所有内层 vector 共保存 $O((n + q) \log n)$ 个离散化坐标，空间复杂度也是 $O((n + q) \log n)$ 。对各个 vector 排序、去重的预处理时间上界为 $O((n + q) \log n \log(n + q))$ 。

前缀询问时，对于内层 vector 再维护一个数据结构，这里可以直接选择再套一个树状数组。

一次修改或前缀询问需要访问 $O(\log n)$ 个外层节点，并在内层离散坐标上执行 $O(\log(n + q))$ 的操作，因此单次时间复杂度为 $O(\log n \log(n + q))$ ，处理 q 次操作的总时间为 $O(q \log n \log(n + q))$ 。

实现上常数较小。

```

struct BitVec
{
    vector<vector<int>> tr, vec;
    vector<int> cnt;
    int n;
    int lowbit(int x)
    {
        return x & -x;
    }
    void init(vector<pii> &a)
    {
        n = a[0].first;
        for (auto [u, v] : a)
        {
            n = max(n, u);
        }
        tr.resize(n + 1);
        vec.resize(n + 1);
        cnt.assign(n + 1, 0);
        for (auto [u, v] : a)

```

```

        for (; u <= n; u += lowbit(u))
            cnt[u]++;
    for (int i = 1; i <= n; i++)
        vec[i].reserve(cnt[i]);
    for (auto [u, v] : a)
        for (; u <= n; u += lowbit(u))
            vec[u].push_back(v);
    for (int i = 1; i <= n; i++)
    {
        sort(vec[i].begin(), vec[i].end());
        vec[i].erase(unique(vec[i].begin(), vec[i].end()), vec[i].end());
        tr[i].assign(vec[i].size() + 1, 0);
    }
}

void update(int x, int y, int v)
{
    for (; x <= n; x += lowbit(x))
    {
        int now = lower_bound(vec[x].begin(), vec[x].end(), y) - vec[x].begin() + 1;
        for (int i = now; i <= vec[x].size(); i += lowbit(i))
            tr[x][i] += v;
    }
}

int query(int x, int y)
{
    int res = 0;
    for (; x; x -= lowbit(x))
    {
        int now = upper_bound(vec[x].begin(), vec[x].end(), y) - vec[x].begin();
        for (int i = now; i; i -= lowbit(i))
            res += tr[x][i];
    }
    return res;
}
};

```

4.1.3 树状数组套动态开点线段树

和套 vector 类似地,只是内层使用动态开点线段树在线维护。

但是实际实现上,不用生硬地给每一个树状数组节点单独开一个动态开点线段树结构。

可以给树状数组节点一个 *root* 数组,每个节点分配一个根,这样就可以使用同一个动态开点线段树结构。

求解区间第 k 大时,对于涉及到的 $O(\log n)$ 个树状数组节点,可以同步维护左右子树走向。这时可以发现,这和“主席树”的结构就很像了(虽然两者之间没有什么实际关系),所以被俗称“带修主席树”。

注:实际上在特化维护带修区间第 k 大时,还有一些常数的技巧,这里不做展开。

```

struct BitSeg
{
    struct node
    {
        int sum, ls, rs;
    } tr[N * 400];
    int root[N];
    int n, m, idx;
    void push_up(int p)

```

```

{
    tr[p].sum = tr[ls(p)].sum + tr[rs(p)].sum;
}
void update(int &p, int l, int r, int x, int v)
{
    int mid = l + r >> 1;
    if (!p)
        p = ++idx;
    if (l == r)
    {
        tr[p].sum += v;
        return;
    }
    if (x <= mid)
        update(ls(p), l, mid, x, v);
    else
        update(rs(p), mid + 1, r, x, v);
    push_up(p);
}
int lowbit(int x)
{
    return x & -x;
}
void init(int n, int m)
{
    this->n = n;
    this->m = m;
    for (int i = 1; i <= idx; i++)
        tr[i] = {0, 0, 0};
    idx = 0;
    for (int i = 1; i <= n; i++)
        root[i] = 0;
}
void add(int x, int v)
{
    for (; x <= n; x += lowbit(x))
    {
        update(root[x], 0, m, v, 1);
    }
}
void del(int x, int v)
{
    for (; x <= n; x += lowbit(x))
    {
        update(root[x], 0, m, v, -1);
    }
}
int query(int l, int r, int k)
{
    static int al[40], ar[40];

    int cntl = 0, cntr = 0;
    for (int x = l - 1; x; x -= lowbit(x))
        al[++cntl] = root[x];
    for (int x = r; x; x -= lowbit(x))
        ar[++cntr] = root[x];
    int L = 0, R = m;
    while (L != R)
    {
        int mid = L + R >> 1;
        int cnt = 0;
        for (int i = 1; i <= cntr; i++)
        {
            cnt += tr[ls(ar[i])].sum;
        }
        for (int i = 1; i <= cntl; i++)
        {
            cnt -= tr[ls(al[i])].sum;
        }
        if (k <= cnt)
        {
            for (int i = 1; i <= cntl; i++)

```

```

        al[i] = ls(al[i]);
        for (int i = 1; i <= cntr; i++)
            ar[i] = ls(ar[i]);
        R = mid;
    }
    else
    {
        k -= cnt;
        for (int i = 1; i <= cntl; i++)
            al[i] = rs(al[i]);
        for (int i = 1; i <= cntr; i++)
            ar[i] = rs(ar[i]);
        L = mid + 1;
    }
}
return L;
}
};

```

内层动态线段树的深度为 $O(\log V)$ 。加入 n 个初始点需要 $O(n \log n \log V)$ 的时间；单次修改或区间第 k 小询问均为 $O(\log n \log V)$ ，处理 q 次操作的总时间为 $O(q \log n \log V)$ 。若后续修改不断引入新的权值路径，动态结点个数的上界为 $O((n + q) \log n \log V)$ 。

4.2 线段树套DS

4.2.1 线段树套动态开点线段树

通常是线段树套权值线段树。

外层线段树上，每一个节点维护一棵动态开点线段树。

加入 n 个初始点的建树时间为 $O(n \log n \log V)$ 。一次单点修改或区间权值询问需要访问 $O(\log n)$ 个外层节点，并在深度为 $O(\log V)$ 的内层线段树中操作，因此单次时间为 $O(\log n \log V)$ ，处理 q 次操作的总时间为 $O(q \log n \log V)$ 。不回收动态结点时，空间上界为 $O((n + q) \log n \log V)$ 。

4.2.2 线段树套平衡树

外层线段树上，每一个节点维护了一棵平衡树。

固定维护 n 个点时，若逐点插入建树，初始化时间为 $O(n \log^2 n)$ ，空间复杂度为 $O(n \log n)$ 。一次单点修改或区间询问为 $O(\log^2 n)$ ，处理 q 次操作的总时间为 $O(q \log^2 n)$ 。平衡树的高度由其中保存的元素数决定，因此这里的复杂度不直接依赖数值域 V ；若操作会增加点数而不只是替换权值，应将公式中的 n 换成操作过程中的最大活动点数，最坏可达 $n + q$ 。

4.3 分块套 DS

稍带一下。

形如「第二分块」就是一种分块套并查集。

5 并查集

并查集是一片森林,一般的并查集通过路径压缩/按秩合并实现,将两者结合的均摊时间复杂度为 $O(n\alpha(n))$ 可近似认为 $O(n)$ (远比 $O(n \log n)$ 小)。

5.1 启发式合并

一般实现的并查集,通过维护每个节点的父节点实现快速锁定根节点。但是这并不能在任一时刻正确维护所有节点所在树的根节点,即:无法 $O(1)$ 的判断连通性。

考虑“真正意义上”的启发式合并,初始每个节点视作一个单独的集合,对每个节点维护一个根节点,每次合并两个节点 u, v 时,把所在集合 $size$ 更小的那个节点所在的集合暴力地合并到另一个集合中去,并维护每个节点的根节点。做到了任一时刻,都维护了每个节点的根节点,所以只要 $O(1)$ 即可判断两个节点是否连通,能够在任一时刻都正确地维护每个节点的根节点。

因为最终只会合并成一个集合,总均摊时间复杂度 $O(n \log n)$ (和询问次数不强相关)。

常数相对较大。

采用链式前向星可适当减小常数。

```
struct DSU
{
    int root[N];
    list<int> p[N];
    void init(int n)
    {
        for (int i = 1; i <= n; i++)
            root[i] = i, p[i].clear(), p[i].emplace_back(i);
    }
    bool same(int x, int y) { return root[x] == root[y]; }
    void merge(int x, int y)
    {
        if (same(x, y))
            return;
        x = root[x], y = root[y];
        if (p[x].size() > p[y].size())
        {
            swap(x, y);
        }
        for (auto u : p[x])
        {
            root[u] = y;
            p[y].push_back(u);
        }
        p[x].clear();
    }
} dsu;
```

5.2 可撤销并查集

因为路径压缩不止在合并时,在判断连通性时,也会改变树的形态,并且改变树的形态是时间复杂度的保证,所以路径压缩不可撤销。与之相比,按秩合并的合并操作是可撤销的。因为按秩合并只需要在合并时把一棵树的根接到另一棵树上即可,在找根节点时,暴力跳父节点。

因为撤销操作也是有顺序的,即按照合并的顺序逆序撤销,所以把合并操作放在栈中,每次撤销栈顶操作即可。合并撤销不影响子树结果,所以记录下是哪个节点接到哪个节点上即可(u 成为 v 的儿子节点)。

```
void init(int n) {
    for (int i = 1; i <= n; i++)
        fa[i] = i, sz[i] = 1;
    while (history.size())
        history.pop();
}

int find(int x) {
    while (x != fa[x])
        x = fa[x];
    return x;
}

bool same(int u, int v) { return find(u) == find(v); }
void merge(int u, int v) {
    if (same(u, v)) {
        history.push(-1);
        return;
    }
    u = find(u), v = find(v);
    if (sz[u] < sz[v])
        swap(u, v);
    history.push(v);
    sz[u] += sz[v];
    fa[v] = u;
}

int History() { return history.size(); }
void roll() {
    if (history.empty())
        return;
    auto t = history.top();
    history.pop();
    if (t == -1)
        return;
    sz[fa[t]] -= sz[t];
    fa[t] = t;
}
```

5.3 带权并查集

带权并查集维护节点到父节点的相对关系。以模 3 的食物链为例,设每个节点具有势值 $P[x]$,以下统一规定

$$weight[x] = (P[x] - P[fa[x]]) \bmod 3,$$

即 $weight[x]$ 表示从儿子 x 指向父亲 $fa[x]$ 的关系,根节点的 $weight$ 恒为 0。

取值 0, 1, 2 分别表示同类、前者吃后者、前者被后者吃。反向关系需要取相反数并对 3 取模。

路径压缩前, 若 x 的父亲为 p , 则 x 到根的关系是 x 到 p 与 p 到根的关系之和。因此压缩后有

$$weight[x] \leftarrow weight[x] + weight[p] \pmod{3}.$$

约定 $merge(x, y, w)$ 加入限制 $(P[x] - P[y]) \bmod 3 = w$ 。执行 $find$ 后, 记 fx, fy 分别为 x, y 的根, 此时 $weight[x]$ 和 $weight[y]$ 已经分别表示 $x \rightarrow fx$ 、 $y \rightarrow fy$ 的关系。若把 fx 接到 fy , 则

$$weight[fx] = w - weight[x] + weight[y] \pmod{3}.$$

若按集合大小需要反向连接, 把 fy 接到 fx , 边的方向也要反转, 因此

$$weight[fy] = -weight[fx] = -w + weight[x] - weight[y] \pmod{3}.$$

当 $fx = fy$ 时不能再连接根节点, 而应检查已有关系是否满足 $weight[x] - weight[y] == w \pmod{3}$; 不满足说明新限制与已有信息矛盾。下面的 $merge$ 以返回值表示限制是否一致, 并使用按大小合并控制树高。

```
int normalize(int value) {
    value %= 3;
    return value < 0 ? value + 3 : value;
}

void init(int n) {
    for (int i = 1; i <= n; i++)
        fa[i] = i, sz[i] = 1, weight[i] = 0;
}

int find(int x) {
    if (fa[x] == x)
        return x;

    int parent = fa[x];
    fa[x] = find(parent);
    weight[x] = normalize(weight[x] + weight[parent]);
    return fa[x];
}

bool same(int x, int y) {
    return find(x) == find(y);
}

// 返回 false 表示新限制与已有关系矛盾。
bool merge(int x, int y, int w) {
    int fx = find(x);
    int fy = find(y);
    w = normalize(w);

    if (fx == fy)
        return normalize(weight[x] - weight[y]) == w;
```

```

// 若连接  $fx \rightarrow fy$ , 这条边应具有权值。
int fx_to_fy = normalize(w - weight[x] + weight[y]);

if (sz[fx] <= sz[fy]) {
    fa[fx] = fy;
    weight[fx] = fx_to_fy;
    sz[fy] += sz[fx];
} else {
    fa[fy] = fx;
    weight[fy] = normalize(-fx_to_fy);
    sz[fx] += sz[fy];
}
return true;
}

```

在路径压缩和按大小合并下, q 次合并或查询的总时间复杂度为 $O((n + q)\alpha(n))$, 空间复杂度为 $O(n)$ 。

5.4 扩展域并查集

也有说法叫“种类并查集”的。在带权并查集中, 通过点权表示节点的“种类”, 通过边权的累计维护点权。在扩展域并查集中, 通过对于每个点不同身份都维护一个“分身”节点来实现身份之间的判断。

同样以食物链为例: $A \rightarrow B, B \rightarrow C, C \rightarrow A$ 。对于一个节点 x , 它会存在三种节点, x 同类、 x 吃、吃 x 。也就是把一个点拆成了三个点, 为了方便, 一般直接用 $x, x + n, x + 2n$ 表示。

判断 x, y 的关系时, 只要看 x 是和 $y, y + n, y + 2n$ 中的哪一个在同一个集合中即可(不用关心, $x + n, x + 2n$ 因为若维护合并时是正确的, 那么三种点都是正确的, 选择其中之一即可)。

合并 x, y 时, 根据合并的关系合并 $x, x + n, x + 2n$ 和 $y, y + n, y + 2n$ 即可。例如 x 吃 y , 那么就把 $(x, y + 2n), (x + n, y), (x + 2n, y + n)$ 合并。

具体实现和朴素并查集一致, 是合并和判断的节点有区别。

5.5 可持久化并查集

支持历史版本的并查集。本质上就是把并查集的数组修改用单点修改、单点询问的可持久化线段树(可持久化数组)替代了。

这里不使用路径压缩: 一次 `find` 可能修改路径上的多个父指针, 持久化每个修改都要产生新的线段树节点, 而且反复访问旧版本时不能沿用通常的均摊分析。改用按大小合并后, 并查集树高为 $O(\log n)$, `find` 只读取父指针, 不改变历史版本。

设并查集有 n 个节点, 共产生 q 次版本操作。可持久化线段树单次读取或修改

父亲、集合大小的时间均为 $O(\log n)$ ；`find` 最多经过 $O(\log n)$ 个父节点，因此单次 `find`、连通性查询或合并通常为 $O(\log^2 n)$ 。复制一个历史版本只需复制两棵线段树的根指针，为 $O(1)$ 。

初始建树为 $O(n)$ ，全部操作的时间复杂度为 $O(n + q \log^2 n)$ 。若其中有 u 次操作实际修改了并查集，则空间复杂度更精确地写作 $O(n + u \log n + q)$ ；由于 $u \leq q$ ，可简写为 $O(n + q \log n)$ 。

不过同样的问题，在支持离线的前提下，可以用可撤销并查集做到线性空间。具体而言，就是需要访问历史版本时，把当前版本编号指向历史版本编号，那么每个版本都只会指向一个比它小的编号，那么建出来的一定是一棵树，在这个树上递归执行合并操作，递归退出时撤销即可。

```
p_tree fa, sz; // 使用可持久化线段树维护
int n;
void init(int n) {
    this->n = n;
    vector<int> a(n + 1), b(n + 1);
    for(int i = 1; i <= n; i++) a[i] = i, b[i] = 1;
    fa.build(fa.root[0], 1, n, a);
    sz.build(sz.root[0], 1, n, b);
}
int find(int k, int x) {
    int fa;
    while ((fa = this->fa.query(this->fa.root[k], 1, n, x)) != x) {
        x = fa;
    }
    return fa;
}
bool same(int k, int x, int y) { return find(k, x) == find(k, y); }
void merge(int p, int q, int x, int y) {
    if (same(p, x, y)) {
        copy(q, p);
        return;
    }
    x = find(p, x), y = find(p, y);
    int sz_x = sz.query(sz.root[p], 1, n, x),
        sz_y = sz.query(sz.root[p], 1, n, y);
    if (sz_x > sz_y) {
        swap(x, y);
        swap(sz_x, sz_y);
    }
    fa.update(fa.root[p], fa.root[q], 1, n, x, y);
    sz.update(sz.root[p], sz.root[q], 1, n, y, sz_x + sz_y);
}
void copy(int now, int cur) {
    fa.root[now] = fa.root[cur];
    sz.root[now] = sz.root[cur];
}
```

5.6 赋值并查集

维护把集合中所有 x 修改成 y ，查询初始 x 当前是什么值。

设预处理值域为 $[1, V]$ ，初始序列长度为 n ，修改与查询合计 q 次。这里的 V 必须覆盖初始值和所有操作中可能出现的值；若值域很大，可以先对这 $n + q$ 个值离散化，此时 $V = O(n + q)$ 。

初始化并查集数组并扫描初始序列需要 $O(V+n)$ 时间。按大小合并并使用路径压缩后,每次修改或查询的均摊时间为 $O(\alpha(V))$,总时间复杂度为 $O(V+n+q\alpha(V))$; 并查集本身的空间复杂度为 $O(V)$,若还保留初始序列则为 $O(V+n)$ 。

```
struct merge_dsu
{
    int fa[N], sz[N], val[N], belong[N];
    bool exist[N];
    void init(int value_limit)
    {
        for (int i = 1; i <= value_limit; i++)
        {
            fa[i] = i;
            sz[i] = 1;
            val[i] = i;
        }
    }
    void init(vector<int> &a)
    {
        for (auto u : a)
        {
            exist[u] = 1;
            belong[u] = u;
        }
    }
    int find(int x)
    {
        return fa[x] == x ? x : fa[x] = find(fa[x]);
    }
    int query(int initial_value)
    {
        return val[find(initial_value)];
    }
    int merge(int x, int y, int z)
    {
        x = find(x), y = find(y);
        if (sz[x] > sz[y])
            swap(x, y);
        fa[x] = y;
        sz[y] += sz[x];
        val[y] = z;
        return y;
    }
    void modify(int x, int y)
    {
        if (x == y)
            return;
        if (!belong[x])
            return;
        int rx = find(belong[x]);
        if (!belong[y])
        {
            belong[y] = rx;
            belong[x] = 0;
            val[rx] = y;
        }
        else
        {
            int ry = find(belong[y]);
            int root = merge(rx, ry, y);
            belong[y] = root;
            belong[x] = 0;
        }
    }
};
```

5.7 倍增并查集

倍增并查集用于专门解决区间等价问题。

形如存在若干个限制 $a_x = a_y, x \in [l_1, r_1], y \in [l_2, r_2], r_1 - l_1 = r_2 - l_2$ 。

最后维护出所有等价类。

具体而言,令区间长度为 $L = r_1 - l_1 + 1$,取 $k = \lfloor \log_2 L \rfloor$ 。将两个区间的限制拆成前缀块 $[l_1, l_1 + 2^k - 1]$ 、 $[l_2, l_2 + 2^k - 1]$ 和后缀块 $[r_1 - 2^k + 1, r_1]$ 、 $[r_2 - 2^k + 1, r_2]$,每条限制只需在第 k 层执行至多两次合并。

对每个 $0 \leq k \leq \lfloor \log_2 n \rfloor$,维护所有合法起点 $1 \leq x \leq n - 2^k + 1$ 。第 k 层的并查集节点 (k, x) 表示从 x 开始、长度为 2^k 的区间。

处理完 q 条区间等价限制后,从高层向低层传递等价关系,直到第 0 层得到单点的等价类。所有层共有

$$\sum_{k=0}^{\lfloor \log_2 n \rfloor} (n - 2^k + 1) = O(n \log n)$$

个合法区间节点,向下传递共执行 $O(n \log n)$ 次并查集合并。

令 $M = O(n \log n)$ 为所有层的节点数。使用普通并查集时,总时间复杂度为

$$O((n \log n + q)\alpha(M)).$$

保留全部层需要 $O(n \log n)$ 空间。若将 q 条限制按层离线分组,并从高层向低层处理,则并查集只需保留相邻两层,工作空间为 $O(n)$;计入尚未处理的限制后,总空间为 $O(n + q)$ 。

使用普通并查集维护即可。

6 堆

6.1 对顶堆

对顶堆可以维护动态集合的全局第 k 大。重复值按多个元素计算, 查询时必须保证 $1 \leq k \leq n$ 。

维护两个堆:

- 小根堆 q : 恰好保存前 k 大的元素;
- 大根堆 Q : 保存其余元素。

始终保持 $q.size() == k$, 且 Q 中的最大值不大于 q 中的最小值。 q 堆顶就是第 k 大。

当 k 变小时, 将 q 的堆顶不断移入 Q ; 当 k 变大时, 将 Q 的堆顶不断移入 q , 直到 q 的大小等于新的 k 。

插入时, 若新元素不大于当前第 k 大, 直接放入 Q ; 否则先放入 q , 再把 q 的最小值移入 Q , 以保持 q 的大小不变。

时间复杂度: $O(n \log n + \sum |k_i - k_{i-1}| \log n)$, 常数小。

单次插入为 $O(\log n)$, 将 k 从 k_1 调整到 k_2 为 $O(|k_1 - k_2| \log n)$, 查询为 $O(1)$, 空间为 $O(n)$ 。

6.2 可并堆

与朴素二叉堆相比, 可以在 $O(\log n)$ 的时间合并两个堆, 而不是 $O(n)$ 。

采用启发式合并可以总 $O(n \log^2 n)$ 的合并, 采用随机合并类似。

一般使用的可并堆为配对堆、左偏树。其中配对堆为均摊时间复杂度无法可持久化, 左偏树支持可持久化。可持久化可并堆一般仅用于求 k 短路, 根据持久化的堆不同, 时间复杂度略有不同。

虽然 STL 中仅支持了优先队列 `priority_queue` (二叉堆), 但是 `pb_ds` 中扩展了可并堆, 以及一些其它堆, 默认使用配对堆。虽然不同堆的时间效率不同, 但是综合效率配对堆已足够优秀。

tag	push	pop	modify	erase	join
pair- ing_heap_tag	$O(1)$	均摊 $O(\log n)$	均摊 $O(\log n)$	均摊 $O(\log n)$	$O(1)$
bi- nary_heap_tag	均摊 $O(\log n)$	均摊 $O(\log n)$	$O(n)$	$O(n)$	$O(n)$
bino- mial_heap_tag	均摊 $O(1)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
rc_bino- mial_heap_tag	$O(1)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
thin_heap_tag	$O(1)$	均摊 $O(\log n)$	均摊 $O(1)$	均摊 $O(1)$	$O(n)$

```
#include <algorithm>
#include <cstdio>
#include <ext/pb_ds/priority_queue.hpp>
#include <iostream>
using namespace __gnu_pbds;

// #define pair_heap __gnu_pbds::priority_queue<int, greater<int>, pairing_heap_tag> 小根堆,
// greater<int> 需要 namespace std
#define pair_heap __gnu_pbds::priority_queue<int> // 大根堆
pair_heap q1;
pair_heap q2;
pair_heap::point_iterator id; // 一个迭代器

int main() {
    id = q1.push(1);
    // 堆中元素 : [1];
    for (int i = 2; i <= 5; i++) q1.push(i);
    // 堆中元素 : [1, 2, 3, 4, 5];
    std::cout << q1.top() << std::endl;
    // 输出结果 : 5;
    q1.pop();
    // 堆中元素 : [1, 2, 3, 4];
    id = q1.push(10);
    // 堆中元素 : [1, 2, 3, 4, 10];
    q1.modify(id, 1);
    // 堆中元素 : [1, 1, 2, 3, 4];
    std::cout << q1.top() << std::endl;
    // 输出结果 : 4;
    q1.pop();
    // 堆中元素 : [1, 1, 2, 3];
    id = q1.push(7);
    // 堆中元素 : [1, 1, 2, 3, 7];
    q1.erase(id);
    // 堆中元素 : [1, 1, 2, 3];
    q2.push(1), q2.push(3), q2.push(5);
    // q1中元素 : [1, 1, 2, 3], q2中元素 : [1, 3, 5];
    q2.join(q1);
    // q1中无元素, q2中元素 : [1, 1, 1, 2, 3, 3, 5];
}
```

注: 迭代器可被记录, 在 push 元素时, 将其迭代器用数组存起来, 可在后续访问。

7 动态树

动态树是用于解决“动态”树上问题的数据结构,即维护森林。常见的动态树有三种: Link-Cut-Tree, Euler Tour Tree, Top Tree。

其中:

- Link-Cut-Tree 用于解决路径问题。
- Euler-Tour-Tree 用于解决子树问题。
- Top-Tree 用于解决路径、子树问题。

由于 ETT 和 Top-Tree 的编码难度,一般仅使用 LCT。

7.1 Link-Cut-Tree

简单来说, LCT 就是用序列 Splay 维护树链,与树链剖分不同,因为 LCT 支持修改树形态,所以是任意链剖分,不是轻重链剖分。

将一条链视作一个序列,将链上节点的深度作为 Splay 的 Key。

一般而言,将每一条链的 Splay 视作一个节点后,形成的 LCT 视作结构树, Splay 视作节点树。

LCT 直接维护的对象是动态森林,即每个连通块都是一棵树。对于含环的一般无向图,普通 LCT 不能把所有边都作为树边存入,但可以用 LCT 维护图的一棵动态生成森林,再额外维护非树边及题目所需的信息;因此不能简单概括为“不能维护无向图”。

7.1.1 access(x):

access 是 LCT 的核心操作,其直接目的是构造一条从 x 到当前 LCT $root$ 的链,其中 x 是链上深度最深的节点。

对于当前 x 所在的 Splay,将 x 转到根后,右子树是链上深度比 x 大的节点,按照 Access 的定义,应把右子树从 x 上断开。而后,把 x 在 LCT 上的父节点 y ,在其所在的 Splay 中转到根,把 x 替换 y 的右儿子即可。以此迭代,直到 LCT 的根节点。

```
int access(int x) {
    int y = 0, z = x;
    for (; x; y = x, x = tree.tr[x].fa) {
        tree.splay(x);
        tree.rs(x) = y;
        tree.push_up(x);
    }
    tree.splay(z);
    return y;
}
```

7.1.2 isroot(x):

LCT 节点树的 Splay 与常规 Splay 略有区别,因为 LCT 有多棵 Splay,所以将节点转到 Splay 根时,是转到其所在节点树的根。但是节点树 Splay 内根节点要储存在结构树上的父节点。所以节点树 Splay 还需要一个判断是否是节点树根的函数。

```
bool isroot(int p) {
    return ls(tr[p].fa) != p && rs(tr[p].fa) != p;
}
```

7.1.3 make_root(x):

make_root 是将 x 作为 LCT 的根。一个作用是,维护 (x, y) 的路径信息时,先将 x 作为根,后 access(y) 打通 (x, y) 的路径,就形成了一条以 x, y 为两端点的链。

具体实现,只要 access(x),此时 x 是到 LCT root 链上的另一个端点,对其所在 Splay 做全局翻转操作即可把 x 变成 LCT root。

```
void make_root(int x) {
    access(x);
    tree.rev(x);
}
```

LCT 核心操作,只有上述 access(x) 和 make_root(x) 两个,还有一些扩展操作。

7.1.4 find_root(x):

LCT 是一片森林, find_root 是找到 x 所在树的根。具体而言, access(x) 后, root 是深度最小的节点,也就是 Splay 中左链的端点,一直在 Splay 上向左儿子走即可。

```
int find_root(int x) {
    access(x);
    while (true) {
        tree.push_down(x);
        if (!tree.ls(x))
            break;
        x = tree.ls(x);
    }
    tree.splay(x);
}
```

```

    return x;
}

```

7.1.5 connected(x,y):

两个节点属于同一棵树,当且仅当它们的结构树根相同。该查询会改变首选路径组成的Splay,但不会改变表示森林中的边。

```

bool connected(int x, int y) {
    if (x == y)
        return true;
    int root_x = find_root(x);
    int root_y = find_root(y);
    return root_x == root_y;
}

```

7.1.6 split(x,y):

split 就是前文中提到的,将 (x, y) 路径构造到一条链上, x, y 是链的两端点。

```

void split(int x, int y) {
    make_root(x);
    access(y);
}

```

7.1.7 link(x,y):

link(x, y) 的前提是 x, y 当前不连通,而不只是“二者之间没有直接边”。如果它们已经通过其他路径连通,再加入 (x, y) 就会产生环,超出普通 LCT 直接维护森林的范围。安全接口应在内部完成连通性检查,并用返回值说明是否成功加边。

```

bool link(int x, int y) {
    make_root(x);
    if (find_root(y) == x)
        return false;
    tree.tr[x].fa = y;
    return true;
}

```

7.1.8 cut(x,y):

删除边前先执行 `make_root(x); access(y);`。此时若 (x, y) 是表示森林中的直接边,则在以 y 为根的辅助树中必须满足 `ls(y) == x && rs(x) == 0`: x 是 y 的左儿子,并且二者之间没有其他路径节点。检查方向不能写反。

断边时应清空 `ls(y)` 和 `fa[x]`,随后调用 `push_up(y)` 更新路径聚合信息。即使调用者保证边存在,也不能省略这次更新。下面的接口在边不存在时返回 `false`,且不改变表示森林。

```

bool cut(int x, int y) {
    make_root(x);
    access(y);

    if (tree.ls(y) != x || tree.rs(x) != 0)
        return false;

    tree.ls(y) = 0;
    tree.tr[x].fa = 0;
    tree.push_up(y);
    return true;
}

```

7.1.9 LCT 中的Splay:

如下:

```

struct node {
    int s[2];
    int v, fa;
    int sum;
    bool rev;
};

struct Splay {
    node tr[N];
    vector<int> q;
    // a 使用 1 下标; 每个节点作为孤立点初始化。
    void init(int n, const vector<int> &a) {
        q.clear();
        q.reserve(n + 1);
        for (int i = 0; i <= n; i++)
            tr[i] = {};
        for (int i = 1; i <= n; i++) {
            tr[i].sum = tr[i].v = a[i];
        }
    }

    void rev(int p) {
        if (!p)
            return;
        swap(ls(p), rs(p));
        tr[p].rev ^= 1;
    }

    void push_up(int p) { // 以具体 push_up 维护信息为准
        tr[p].sum = (tr[ls(p)].sum ^ tr[rs(p)].sum ^ tr[p].v);
    }

    void push_down(int p) {
        if (p && tr[p].rev) {
            rev(ls(p));
            rev(rs(p));
            tr[p].rev = 0;
        }
    }

    bool isroot(int p) { return ls(tr[p].fa) != p && rs(tr[p].fa) != p; }

    void rotate(int x) {
        int y = tr[x].fa, z = tr[y].fa;
        bool w = (rs(y) == x);
        if (!isroot(y))
            tr[z].s[rs(z) == y] = x;
        tr[x].fa = z;
        int child = tr[x].s[w ^ 1];
        tr[y].s[w] = child;
        if (child)
            tr[child].fa = y;
        tr[x].s[w ^ 1] = y, tr[y].fa = x;
        push_up(y), push_up(x);
    }

    void splay(int x) {
        int r = x;
        q.clear();
    }
}

```

```

q.push_back(r);
while (!isroot(r)) {
    q.push_back(tr[r].fa);
    r = tr[r].fa;
}
int len = q.size();
len--;
while (len >= 0)
    push_down(q[len--]);
while (!isroot(x)) {
    int y = tr[x].fa, z = tr[y].fa;
    if (!isroot(y)) {
        if ((rs(y) == x) ^ (rs(z) == y))
            rotate(x);
        else
            rotate(y);
    }
    rotate(x);
}
}
};

```

7.1.10 LCT 的替代品:

在允许离线的情況下，LCT 有許多替代品。

7.1.10.1 樹鏈剖分:

因為動態樹維護的是森林，所以在操作結束後，若建立一個虛根把所有樹接到這個虛根上，那麼會形成一棵樹。

若操作過程中只涉及加邊，而不涉及刪邊（也就是對於最後的樹而言，樹的形態是確定的），那麼對於詢問中的路徑問題，可以轉化成離線後的樹上路徑問題，使用樹鏈剖分（或根據實際詢問內容，選擇合適的方式維護即可）維護即可。

但是時間複雜度是 $O(n \log^2 n)$ 的，不過樹剖小常數 $O(\log^2 n)$ 多數情況下不比 LCT 的 $O(\log n)$ 劣。

7.1.10.2 線段樹分治+可撤銷并查集:

若詢問內容和连通性有關，或是并查集能夠維護的子樹信息。那麼可以將操作過程中，對某一條邊的link, cut 視作在操作序列時間軸上的一段存在後消失的邊。使用線段樹分治+可撤銷并查集維護即可。

但是時間複雜度是 $O(n \log^2 n)$ 的，並且這裡的常數並不小。

7.2 全局平衡二叉樹

感覺主要用於處理動態 DP 問題，數據結構上不是很有不可替代性，暫略。

8 根号树

根号叉树。

8.1 Sqrt Tree

Sqrt Tree 可以 $O(n \log \log n)$ 预处理, $O(1)$ 区间询问。

考虑序列分块, 将原序列分成 $\sqrt{n}$ 块, 每块长 $\sqrt{n}$, 每一块内维护前后缀, 若询问区间不在同一块内, 将首尾散块前后缀拼接, 中间整块整取, 时间复杂度 $O(\sqrt{n})$, 若再预处理 $B[i][j]$ 表示第 i 块到第 j 的区间和, 预处理时间复杂度 $O(n)$ 。询问时中间的整块就可以 $O(1)$ 的得到结果, 时间复杂度 $O(1)$ 。若询问区间在同一块内, 无法表示, 暴力维护时间复杂度: $O(\sqrt{n})$ 。

不妨将询问区间在同一块内的情况继续递归下去, 若每次询问区间不在同一块内, 都是 $O(1)$ 可以解决的, 反之就递归下去, 直到区间长度为 1, $O(1)$ 直接解决。

因为每次递归分块长度都是 $\sqrt{n}$ (n 是当前序列长度), 所以次递归序列长度都缩小至原来的 $\sqrt{n}$, 递归 $O(\log \log n)$ 次后即会到 1, 所以此时询问复杂度已降至 $O(\log \log n)$ 。空间复杂度也为 $O(n \log \log n)$ 。

考虑将这个递归的过程建成一棵树, 每个节点都有 $\sqrt{n}$ 个子节点 (除了最后一块可能不足 $\sqrt{n}$)。

那么递归的过程就是自下而上找到第一层存在完全包含于询问区间的块, 这个过程可以二分, 此时时间复杂度降至 $O(\log \log \log n)$ 。

通过调整块长还可继续优化, 假设每一层块长都为 2 的整数幂次 (自下而上递减)。令当前层块长为 2^k , 序列编号从 0 开始。若 l, r 位于同一个块中, 则它们除末尾 k 位以外的高位完全相同; 反过来, l, r 的最高不同位若不低于第 k 位, 它们就一定属于不同的块。令 $d = l \oplus r$, 当 $l \neq r$ 时, 最高不同位为

$$h = \lfloor \log_2 d \rfloor,$$

也就是唯一满足 $2^h \leq d < 2^{h+1}$ 的 h 。根据 h 映射到预先选定的层, 就能 $O(1)$ 找到可直接拼接前缀、整块和后缀的层。当 $l = r$ 时 $d = 0$, 对 0 取对数没有定义, 必须直接返回单点值。下面的代码使用 1 下标, 因此实际计算的是 $(1 - 1) \wedge (r - 1)$ 。

关于块长 k 的选定, 要保证树高, 不能任意选取。事实上可以发现, 若 k 每次只减小 1, 那么树高为 $O(\log n)$, 此时 Sqrt Tree 就是猫树。具体而言, 先将原序列扩展

至 2 的整数幂长度后,第一次分块,块长选取 2 的 2 的整数幂次次(即 2^{2^k})而后即可按 $\sqrt{n}$ 分块。树高 $O(\log \log n)$ 。最后一层的块长可以到 2^1 ,也可以到 2^0 ,但是都要特判等于块长的情况。(询问区间长度等于 2^1 或 2^0 直接做就行)。

实际维护的时候,不用显式建树,只要维护若干层的分块即可。

```
struct SqrtTree
{
    // op 必须满足结合律;代码始终按区间从左到右的顺序合并,
    // 因此不要求 op 满足交换律或幂等性。
    static inline int op(int x, int y)
    {
        return max(x, y);
    }

    struct Layer
    {
        int n = 0;
        int k = 0;
        int S = 0;
        int block_cnt = 0;
        bool top_layer = false;

        vector<int> pre, suf;
        vector<int> between;

        inline int get_between(int lb, int rb) const
        {
            if (top_layer)
            {
                return between[lb * block_cnt + rb];
            }
            else
            {
                int g = lb >> k;
                int li = lb & (S - 1);
                int ri = rb & (S - 1);

                return between[g * S * S + li * S + ri];
            }
        }
    }

    void build(int _k, int _n, const vector<int> &a, bool _top_layer)
    {
        k = _k;
        n = _n;
        S = 1 << k;
        top_layer = _top_layer;
        block_cnt = (n + S - 1) >> k;

        pre.assign(n + 1, 0);
        suf.assign(n + 1, 0);

        vector<int> whole(block_cnt);

        for (int b = 0; b < block_cnt; b++)
        {
            int L = (b << k) + 1;
            int R = min(n, (b + 1) << k);

            pre[L] = a[L];
            for (int i = L + 1; i <= R; i++)
                pre[i] = SqrtTree::op(pre[i - 1], a[i]);

            suf[R] = a[R];
            for (int i = R - 1; i >= L; i--)
                suf[i] = SqrtTree::op(a[i], suf[i + 1]);

            whole[b] = pre[R];
        }
    }
}
```



```

    if (top_layer)
    {
        between.assign(block_cnt * block_cnt, 0);

        for (int i = 0; i < block_cnt; i++)
        {
            int cur = whole[i];
            between[i * block_cnt + i] = cur;

            for (int j = i + 1; j < block_cnt; j++)
            {
                cur = SqrtTree::op(cur, whole[j]);
                between[i * block_cnt + j] = cur;
            }
        }
    }
    else
    {
        int group_cnt = (block_cnt + S - 1) >> k;

        between.assign(group_cnt * S * S, 0);

        for (int g = 0; g < group_cnt; g++)
        {
            int start = g * S;
            int end = min(block_cnt - 1, start + S - 1);

            for (int i = start; i <= end; i++)
            {
                int li = i - start;
                int cur = whole[i];

                between[g * S * S + li * S + li] = cur;

                for (int j = i + 1; j <= end; j++)
                {
                    cur = SqrtTree::op(cur, whole[j]);

                    int lj = j - start;
                    between[g * S * S + li * S + lj] = cur;
                }
            }
        }
    }
}

inline int query(int l, int r) const
{
    int bl = (l - 1) >> k;
    int br = (r - 1) >> k;

    int ans = suf[l];

    if (bl + 1 <= br - 1)
        ans = SqrtTree::op(ans, get_between(bl + 1, br - 1));

    ans = SqrtTree::op(ans, pre[r]);

    return ans;
}

};

int orig_n = 0;
int n = 0;

vector<int> a;
vector<Layer> layer;
vector<short> which;

static inline int highest_bit(int x)
{

```

```

    return x == 0 ? 0 : 31 - __builtin_clz((unsigned)x);
}

void build(const vector<int> &src, int _n)
{
    // src 使用 1 下标,合法元素为 src[1..n]。
    assert(_n >= 1 && _n < (int)src.size());
    orig_n = _n;
    layer.clear();
    which.clear();

    n = 1;
    while (n < orig_n)
        n <<= 1;

    a.assign(n + 1, 0);

    for (int i = 1; i <= orig_n; i++)
        a[i] = src[i];

    // 补到 2 的幂。正常询问不会访问补出来的位置。
    // 对本题 max 来说,补什么都不影响合法询问。
    for (int i = orig_n + 1; i <= n; i++)
        a[i] = src[orig_n];

    if (n <= 2)
        return;

    int lgN = __lg(n);

    // top_k 是不超过 log2(n) 的最大 2 的幂。
    // 例如 n = 2^17, 则 top_k = 16。
    int top_k = 1;
    while ((top_k << 1) <= lgN)
        top_k <<= 1;

    vector<int> ks;
    for (int k = top_k; k >= 1; k >>= 1)
        ks.push_back(k);

    layer.reserve(ks.size());

    for (int i = 0; i < (int)ks.size(); i++)
    {
        layer.emplace_back();
        layer.back().build(ks[i], n, a, i == 0);
    }

    which.assign(n, 0);

    for (int mask = 1; mask < n; mask++)
    {
        int hb = highest_bit(mask);

        int id = 0;
        while (id + 1 < (int)ks.size() && ks[id] > hb)
            id++;

        which[mask] = (short)id;
    }
}

inline int query(int l, int r) const
{
    if (l == r)
        return a[l];

    if (r == l + 1)
        return op(a[l], a[r]);

    int mask = (l - 1) ^ (r - 1);

```

```

    int id = which[mask];
    return layer[id].query(l, r);
}
};

```

可以发现, Sqrt Tree 维护信息的本质是预处理首尾前后缀和中间整块信息。合并运算必须满足结合律;若运算不满足交换律,查询时必须严格按“左侧后缀、中间整块、右侧前缀”的顺序合并。Sqrt Tree 与猫树(Disjoint Sparse Table)都能处理任意结合运算,不要求幂等性,二者不存在功能上的严格包含关系;经典的重叠 Sparse Table 则通常要求运算幂等。它们的主要差别在预处理层数、空间布局 and 实现常数,不能笼统写成 Sqrt Tree > 猫树 > ST 表。

上面的参考实现是静态结构,只提供 build 和 query,不支持修改。某些 Sqrt Tree 变体可以通过重建受影响的块实现单点修改或区间赋值,但复杂度取决于具体分层、重建范围和标记设计,不能把某一变体的修改复杂度直接当作当前模板的能力。

8.2 vEB

Sqrt Tree 是序列上的根号树, vEB 是对值域做根号树。

vEB 功能上与压位 Trie 相同,用于维护插入、删除、前驱、后继、最大值、最小值。

vEB 上每个节点将值域分成 $O(\sqrt{V})$ 块,并缓存当前集合的最小值和最大值以便 $O(1)$ 查询。标准递归表示会把最小值单独保存在当前节点、不再放入子树;最大值也是当前节点的缓存,但除集合只有一个元素外,它对应的值仍保存在某个子树中。

每个节点上这 $O(\sqrt{V})$ 份子节点,对应 $O(\sqrt{V})$ 个 vEB 维护 $O(\sqrt{V})$ 份值域。再用一个 vEB 维护哪些子节点存在。

值域根号 $O(\log \log V)$ 次到 $O(1)$,所以树高 $O(\log \log V)$ 。

空间复杂度: $T(V) = (\sqrt{V} + 1) \times T(\sqrt{V}) + O(\sqrt{V}) = O(V)$ 。

8.2.1 插入

如果当前位置为空,直接令最小值和最大值都等于 v 。

若 v 小于当前最小值,交换 v 与最小值:新的最小值保留在当前节点,旧的最小值继续向对应子节点递归插入。

随后把 v 插入对应子节点;若该子节点原先为空,还要把它的编号插入 summary。若 v 大于当前最大值,插入完成后直接更新最大值,不与旧最大值交

换:若此前只有一个元素,旧最大值就是仍保存在当前节点的最小值;否则旧最大值本来就已经存在于子树中。重复插入同一个值时不做任何操作。

时间复杂度: $O(\log \log V)$ 。

8.2.2 删除

如果当前节点表示的值域大小 ≤ 2 ,特判处理。

如果删除的值为最大值/最小值,需要重新计算子树中的最大值/最小值,找到最大/最小的存在值的子树,返回起最大/最小值即可。

反之,找到 v 所在的子树,递归删除。并且,如果删除 v 后,某个子树空了(如果子树会空,那么说明只有一个元素,只会进行一次操作),还要把这个子节点在维护子节点vEB 中删除(这是一个子问题)。

时间复杂度: $O(\log \log V)$ 。

8.2.3 后继/前驱

如果和查询的 v 同一个子树中有解,则递归子树。

反之,找到高位/低位第一个存在值的子树,相当于在维护子节点的那个 vEB 中查询后继/前驱,这也是一个递归的过程。

找到那个第一个存在值的子树后,直接返回它维护的最小值/最大值即可。

时间复杂度: $O(\log \log V)$ 。

8.2.4 最大值/最小值

全局的最大最小值是在根节点直接维护的,直接获取即可。

时间复杂度: $O(1)$ 。

```
struct vEB
{
    struct VEB
    {
        static constexpr int BASE_BITS = 6;

        int bits;
        int low_bits, high_bits;
        int mn, mx;

        unsigned long long mask;

        VEB *summary;
        VEB **child;

        VEB(int b = 20)
            : bits(b),
              low_bits(0),
              high_bits(0),

```

```

        mn(-1),
        mx(-1),
        mask(0),
        summary(nullptr),
        child(nullptr)
    {
        if (bits > BASE_BITS)
        {
            low_bits = bits >> 1;
            high_bits = bits - low_bits;
        }
    }

    bool is_base() const
    {
        return bits <= BASE_BITS;
    }

    bool empty() const
    {
        return mn == -1;
    }

    int high(int x) const
    {
        return x >> low_bits;
    }

    int low(int x) const
    {
        return x & ((1 << low_bits) - 1);
    }

    int idx(int h, int l) const
    {
        return (h << low_bits) | l;
    }

    int get_min() const
    {
        return mn;
    }

    int get_max() const
    {
        return mx;
    }

    void pull_base()
    {
        if (mask == 0)
        {
            mn = mx = -1;
        }
        else
        {
            mn = __builtin_ctzll(mask);
            mx = 63 - __builtin_clzll(mask);
        }
    }

    void ensure_child_array()
    {
        if (child == nullptr)
        {
            child = new VEB *[1 << high_bits]();
        }
    }

    VEB *ensure_child(int h)
    {
        ensure_child_array();
    }

```

```

    if (child[h] == nullptr)
    {
        child[h] = new VEB(low_bits);
    }
    return child[h];
}

VEB *get_child(int h) const
{
    return child == nullptr ? nullptr : child[h];
}

VEB *ensure_summary()
{
    if (summary == nullptr)
    {
        summary = new VEB(high_bits);
    }
    return summary;
}

bool contains(int x) const
{
    if (mn == -1)
        return false;
    if (x == mn || x == mx)
        return true;
    if (x < mn || x > mx)
        return false;

    if (is_base())
    {
        return (mask >> x) & 1ULL;
    }

    int h = high(x);
    int l = low(x);
    VEB *c = get_child(h);
    return c != nullptr && c->contains(l);
}

void insert(int x)
{
    if (is_base())
    {
        mask |= (1ULL << x);
        pull_base();
        return;
    }

    if (mn == -1)
    {
        mn = mx = x;
        return;
    }

    if (x == mn || x == mx)
        return;

    if (x < mn)
    {
        int t = x;
        x = mn;
        mn = t;
    }

    int h = high(x);
    int l = low(x);

    VEB *c = ensure_child(h);

    if (c->empty())

```

```

    {
        ensure_summary()->insert(h);
    }

    c->insert(1);

    if (x > mx)
    {
        mx = x;
    }
}

void erase(int x)
{
    if (is_base())
    {
        mask &= ~(1ULL << x);
        pull_base();
        return;
    }

    if (mn == -1 || x < mn || x > mx)
        return;

    if (x != mn && x != mx)
    {
        int h0 = high(x);
        int l0 = low(x);
        VEB *c0 = get_child(h0);
        if (c0 == nullptr || !c0->contains(l0))
            return;
    }

    if (mn == mx)
    {
        mn = mx = -1;
        return;
    }

    if (x == mn)
    {
        int first_cluster = summary->get_min();
        VEB *c = child[first_cluster];
        int new_low = c->get_min();

        x = idx(first_cluster, new_low);
        mn = x;
    }

    int h = high(x);
    int l = low(x);

    VEB *c = child[h];
    c->erase(1);

    if (c->empty())
    {
        summary->erase(h);

        if (x == mx)
        {
            int last_cluster = summary->get_max();

            if (last_cluster == -1)
            {
                mx = mn;
            }
            else
            {
                mx = idx(last_cluster, child[last_cluster]->get_max());
            }
        }
    }
}

```

```

    }
    else if (x == mx)
    {
        mx = idx(h, c->get_max());
    }
}

int prev(int x) const
{
    if (mn == -1)
        return -1;

    if (is_base())
    {
        if (x <= 0)
            return -1;

        unsigned long long m;
        if (x >= 64)
        {
            m = mask;
        }
        else
        {
            m = mask & ((1ULL << x) - 1ULL);
        }

        if (m == 0)
            return -1;
        return 63 - __builtin_clzll(m);
    }

    if (x <= mn)
        return -1;
    if (x > mx)
        return mx;

    int h = high(x);
    int l = low(x);

    VEB *c = get_child(h);
    if (c != nullptr)
    {
        int p = c->prev(l);
        if (p != -1)
        {
            return idx(h, p);
        }
    }

    int pc = summary == nullptr ? -1 : summary->prev(h);

    if (pc == -1)
    {
        return mn;
    }

    return idx(pc, child[pc]->get_max());
}

int next(int x) const
{
    if (mn == -1)
        return -1;

    if (is_base())
    {
        if (x < 0)
            return get_min();
        if (x >= 63)
            return -1;
    }

```



```

        unsigned long long m = mask & (~0ULL << (x + 1));

        if (m == 0)
            return -1;
        return __builtin_ctzll(m);
    }

    if (x < mn)
        return mn;
    if (x >= mx)
        return -1;

    int h = high(x);
    int l = low(x);

    VEB *c = get_child(h);
    if (c != nullptr)
    {
        int s = c->next(l);
        if (s != -1)
        {
            return idx(h, s);
        }
    }

    int sc = summary == nullptr ? -1 : summary->next(h);

    if (sc == -1)
    {
        return mx;
    }

    return idx(sc, child[sc]->get_min());
}
};

static constexpr int UNIVERSE_BITS = 20;
static constexpr int UNIVERSE_SIZE = 1 << UNIVERSE_BITS;

VEB root;

veb() : root(UNIVERSE_BITS) {}

static bool in_domain(int x)
{
    return 0 <= x && x < UNIVERSE_SIZE;
}

void insert(int x)
{
    assert(in_domain(x));
    root.insert(x);
}

void erase(int x)
{
    assert(in_domain(x));
    root.erase(x);
}

bool contains(int x) const
{
    return in_domain(x) && root.contains(x);
}

static std::optional<int> to_optional(int value)
{
    if (value == -1)
        return std::nullopt;
    return value;
}

```

```
std::optional<int> get_min() const
{
    return to_optional(root.get_min());
}

std::optional<int> get_max() const
{
    return to_optional(root.get_max());
}

std::optional<int> get_prev(int x) const
{
    assert(in_domain(x));
    return to_optional(root.prev(x));
}

std::optional<int> get_next(int x) const
{
    assert(in_domain(x));
    return to_optional(root.next(x));
}
};
```

这份实现的合法值域是 $[0, 2^{20})$, 并按集合语义忽略重复值。内部用 -1 表示空节点, 公开的最小值、最大值、前驱和后继接口返回 `std::optional<int>`; `std::nullopt` 表示不存在, 因此合法值 0 不再与“无解”混淆。使用代码时需要包含 `<optional>`。

9 莫队

默认的莫队时间复杂度分析仅考虑指针移动的次数,具体问题中,还需要考虑指针移动时维护信息的时间复杂度,和处理询问答案时的时间复杂度,因此具体实现时,可以根据这个均衡块长。

若无特殊说明,本文时间复杂度分析均为指针移动次数分析。

9.1 离线莫队

正经莫队。

9.1.1 普通莫队

普通莫队是一个基于分块的离线解决静态区间询问问题的算法。

设询问区间为 $[l_i, r_i]$ 。对原序列索引分块。将询问区间离线后以 l_i 所在块的编号为第一关键字, r_i 为第二关键字升序。

9.1.1.1 算法流程:

对于每个询问 $[l_i, r_i]$ 的答案,由 $[l_1, r_1]$ 扩展而来,每次用while 循环暴力移动 l, r 指针。

此时,升序后的 $[l_i, r_i]$,同一块内的 l_i 对应的询问 $[l_i, r_i]$ 视作一个整体,共有 $\frac{n}{t}$ 个块。在一个块内的 r 最多移动 n 次(因为同一块内的 r 升序,最多从 1 移动到 n 。在同一个块内 l 每次扩展最多移动 t 次。同时,从一个块的右端点询问的 l 向下一个块的左端点的 l 扩展时, l 最多移动 $2t$, r 最多移动 n 。

9.1.1.2 时间复杂度分析:

综上:处理完 m 个询问, l, r 指针的总移动次数为: $O(\frac{n^2}{t} + mt)$ 。

当 $t = \frac{n}{\sqrt{m}}$ 时,时间复杂度最优为: $O(n\sqrt{m})$ 。

认为 n, m 同阶,所以直接取 $\sqrt{n}$ 。

9.1.1.3 优化：

奇偶化排序:对奇数块内的询问的 r 进行升序,对偶数块内的询问的 r 进行降序。

这是很自然的,因为按原本的排序方式,每次进入一个新块时, r 都会从最大值降到最小值,再又升到最大值。这当然是不优的。

```
struct Query {
    int l, r, id;
    bool operator<(const Query &t) const {
        return (kind[l] ^ kind[t.l]) ? kind[l] < kind[t.l]
            : ((kind[l] & 1) ? r < t.r : r > t.r);
    }
};
void add(int x) {
    /*
     * 将 a[x] 加入答案
     */
}
void del(int x) {
    /*
     * 将 a[x] 删去
     */
}
void solve() {
    for (int i = 1; i <= n; i++)
        kind[i] = (i - 1) / B;
    sort(q + 1, q + 1 + m);
    int l = 1, r = 0;
    for (int i = 1; i <= m; i++) {
        while (q[i].l < l) add(--l);
        while (q[i].r > r) add(++r);
        while (q[i].l > l) del(l++);
        while (q[i].r < r) del(r--);
        /*
         * ans[q[i].id] = ...
         */
    }
}
```

9.1.2 回滚莫队

回滚莫队用于 add 容易维护, del 不易维护的问题,可以将 del 操作转换成 add 操作。

9.1.2.1 算法流程：

在普通莫队中,处理 l_i 位于同一块内的询问时:

- 若 r_i 也在 l_i 的块内,询问区间长度 $\leq t$,直接暴力处理。
- 若 r_i 不在 l_i 的块内,按 r_i 升序 add,对于相同的 r_i 的不同 l_i ,因为 l_i 在同一块内,所以直接重新从 l_i 所在块的右边界 add 过去。

这样实现时,不同块的 l_i 的询问之间就独立了,进入下一个块时,要清空维护的信息。

左端点在块内回滚时,最无脑的做法就是开一个 history 栈,把所有数组和修改的位置和值存下来,通过清空栈回退。

9.1.2.2 时间复杂度分析:

处理过程中,右端点移动 $O(n)$,有 $O(\frac{n}{B})$,总 $O(\frac{n^2}{B})$,左端点均摊每个询问移动 $O(B)$,总 $O(mB)$ 。

```
struct Query {
    int l, r, id;
    bool operator<(const Query &t) const {
        if (kind[l] != kind[t.l])
            return kind[l] < kind[t.l];
        return r < t.r;
    }
} q[N];

void solve() {
    for (int i = 1; i <= n; i++)
        kind[i] = (i - 1) / B + 1;

    sort(q + 1, q + 1 + m);

    int i = 1, l, r, res = 0;
    auto add = [&](int x){
        };
    while (i <= m) {
        int j = i;
        while (j <= m && kind[q[i].l] == kind[q[j].l])
            j++;
        int right = min(n, kind[q[i].l] * B);
        while (i < j && q[i].r <= right) {
            res = 0;
            for (int k = q[i].l; k <= q[i].r; k++)
                add(k);
            /*
             * ans[q[i].id] = ...;
             */
            // 清空信息
            i++;
        }
        l = right + 1, r = right;
        while (i < j) {
            while (r < q[i].r)
                add(++r);
            /*
             * 拷贝旧信息
             */
            while (l > q[i].l)
                add(--l);
            /*
             * ans[q[i].id] = ...;
             */
            while (l < right + 1){
                l++; // 回滚
            }
            /*
             * 回滚到旧信息
             */
            i++;
        }
        // 清空信息
    }
}
```

9.1.2.3 只减不删的回滚莫队

如果信息是 del 容易维护, add 不容易维护,那么可以每次,左端点从块的左端点向右移动,右端点从 n 向块的右端点移动。

全局信息从 $[1, n]$ 不断移动左端点得到。

9.1.3 带修莫队

9.1.3.1 算法流程:

带修莫队与普通莫队相比,多了一维时间轴表示每修改一次,时间增加 1。

把询问表示成 (l, r, t) 的形式,其中 t 表示这次询问位于第 t 次修改和第 $t + 1$ 次修改之间。

将三元组按照某种规则排序后考虑指针移动次数。

令 L, R 分别表示 l, r 所在的块的编号,全 6 种排序的移动次数表

排序关键字	L 移动次数	R 移动次数	T 移动次数	总移动次数
L, R, T	$O(QB + n)$	$O(QB + nG)$	$O(CG^2)$	$O(QB + nG + CG^2)$
R, L, T	$O(QB + nG)$	$O(QB + n)$	$O(CG^2)$	$O(QB + nG + CG^2)$
L, T, R	$O(QB + n)$	$O(QB + nCG)$	$O(CG)$	$O(QB + nCG)$
R, T, L	$O(QB + nCG)$	$O(QB + n)$	$O(CG)$	$O(QB + nCG)$
T, L, R	$O(QB + nC)$	$O(QB + nCG)$	$O(C)$	$O(QB + nCG)$
T, R, L	$O(QB + nCG)$	$O(QB + nC)$	$O(C)$	$O(QB + nCG)$

把 $G = \frac{n}{B}$ 代进去:

排序关键字	总移动次数
L, R, T	$O\left(QB + \frac{n^2}{B} + \frac{Cn^2}{B^2}\right)$
R, L, T	$O\left(QB + \frac{n^2}{B} + \frac{Cn^2}{B^2}\right)$
L, T, R	$O\left(QB + \frac{Cn^2}{B}\right)$
R, T, L	$O\left(QB + \frac{Cn^2}{B}\right)$

排序关键字	总移动次数
T, L, R	$O\left(QB + nC + \frac{Cn^2}{B}\right)$, 通常写作 $O\left(QB + \frac{Cn^2}{B}\right)$
T, R, L	$O\left(QB + nC + \frac{Cn^2}{B}\right)$, 通常写作 $O\left(QB + \frac{Cn^2}{B}\right)$

9.1.3.2 时间复杂度分析:

对于所有排序规则分析取最小值, 只有 $(L, R, T), (R, L, T)$ 是 $O(n^{5/3})$, 剩下的均会出现 $O(\frac{cn^2}{B})$, 最优为 $O(n^2)$ 。

认为序列大小 n , 询问次数 m , 修改次数 t 同阶, 块长取 $n^{\frac{2}{3}}$ 时, 时间复杂度最优, 为 $O(n^{\frac{5}{3}})$ 。

9.1.3.3 优化:

和普通莫队奇偶排序类似地, 带修莫队多一维, 进行蛇形排序。

```

struct Query {
    int l, r, t, id;
    bool operator<(const Query &x) const
    {
        if (kind[l] != kind[x.l])
            return l < x.l;
        if (kind[r] != kind[x.r])
            return (kind[l] & 1) ? r < x.r : r > x.r;
        return (kind[r] & 1) ? t < x.t : t > x.t;
    }
};

void solve() {
    for (int i = 1; i <= n; i++)
        kind[i] = (i - 1) / B;

    // m1 是询问数, m2 是已经读入的修改数。
    m1 = m2 = 0;
    for (int i = 1; i <= m; i++) {
        char op;
        cin >> op;
        if (op == 'Q') {
            int l, r;
            cin >> l >> r;
            ++m1;
            // 当前询问发生在前 m2 次修改之后, 答案编号为 m1。
            q[m1] = {l, r, m2, m1};
        } else if (op == 'C') {
            int x, v;
            cin >> x >> v;
            c[++m2] = {x, v};
        }
    }

    sort(q + 1, q + 1 + m1);
    auto add = [&](int x){
        };
    auto del = [&](int x){

```

```

};
int l = 1, r = 0, t = 0;
for (int i = 1; i <= m1; i++) {
    while (q[i].l < 1) add(a[--l]);
    while (q[i].r > r) add(a[++r]);
    while (q[i].l > 1) del(a[l++]);
    while (q[i].r < r) del(a[r--]);
    while (t < q[i].t)
    {
        t++;
        auto &[now, cur] = c[t];
        if (now >= 1 && now <= r)
        {
            del(a[now]);
            add(cur);
        }
        swap(a[now], cur);
    }
    while (t > q[i].t)
    {
        auto &[now, cur] = c[t];
        if (now >= 1 && now <= r)
        {
            del(a[now]);
            add(cur);
        }
        swap(a[now], cur);
        t--;
    }
    /*
    ans[q[i].id] = ...;
    */
}
}

```

9.1.4 树上莫队

树上莫队就是莫队上树,询问树上路径信息,因为莫队通过 add 和 del 维护信息,所以在括号序上跑莫队即可。括号序会通过一次 add 和一次 del 将树上两点的 LCA 的祖先信息去掉。

如果是询问子树信息,那么通常通过 dfs 序转换成连续的区间处理,后续同其他莫队。

9.1.4.1 算法流程:

通过「欧拉序」将树上的一条路径转换成序列上一段连续的区间。

这一部分本质是「欧拉序」的技巧。

对于路径 (x, y) , 令 $tin[x] \leq tin[y]$, 令 $z = LCA(x, y)$ 。- 若 $x = z$, 则路径上的点在 $[tin[x], tin[y]]$ 上出现奇数次。- 若 $x \neq z$, 则除 z 外路径上的点在 $[tout[x], tin[y]]$ 上出现奇数次, 此时要给这组询问特殊加一个 z 单点的指针扩展。

指针移动时, 维护变化元素的奇偶次数, 奇相当于加入, 偶相当于删去。

注: 不难发现, 树上莫队维护时, 必定需要支持 add 和 del, 所以不能处理「树上回滚莫队」。

9.1.4.2 时间复杂度分析:

与普通莫队一致。

9.1.5 二次离线莫队

9.1.5.1 算法流程:

普通莫队进行指针移动时,如果 add 和 del 操作的时间复杂度不为 $O(1)$,那么时间复杂度会变劣,如果这一部分贡献可以通过前缀和差分的方式离线计算,那么就可以去掉这一部分的时间复杂度。

具体而言,处理每一个询问的指针移动时, l, r 只会向某一侧单调移动且不会出现 $l > r$ 的情况。

以 $[l, r]$ 移动到 $[l, r']$, $r' > r$ 为例,每次 add 的贡献为 $f(a[x], l, x - 1)$, $x \in [r + 1, r']$ 。

如果 f 可差分,那么 $f(a[x], l, x - 1) = g(a[x], x - 1) - g(a[x], l - 1)$ 。

r 指针移动过程中总贡献为 $\sum_{x=r+1}^{r'} (g(a[x], x - 1) - g(a[x], l - 1))$ 。

继续拆成两部分: $\sum_{x=r+1}^{r'} g(a[x], x - 1) - \sum_{x=r+1}^{r'} g(a[x], l - 1)$ 。

第一部分,令 $h(x) = g(a[x], x - 1)$,可以写成 $h(r') - h(r)$,预处理 $h(x)$ 即可。

第二部分,以一个区间形式「二次离线」(目的是做到线性空间),需要被计算的 $g(a[x], l - 1)$ 数量规模同原莫队指针移动次数规模 $O(n\sqrt{m})$ 。

回滚莫队二次离线同理。

剩下三种指针移动情况类似,具体贡献范围下标略有区别。

注意最后要将询问的贡献按莫队排序后的顺序前缀累计。

9.1.5.2 时间复杂度分析:

第二部分的贡献仍然计算 $O(n\sqrt{m})$ 次,剩下部分的瓶颈为计算前缀信息的时间复杂度,需要结合具体问题分析。

```
auto addEvent = [&](int p, int L, int R, int id, int k)
{
    event[p].push_back({L, R, id, k});
};

for (int i = 1; i <= m; i++)
{
    int q1 = q[i].l, qr = q[i].r, id = q[i].id;
    if (q1 < 1)
    {
        int L = q1;
        int R = 1 - 1;
```

```

    addEvent(r, L, R, id, 1);
    ans[id] -= sum2(L, R);
    l = ql;
}

if (qr > r)
{
    int L = r + 1;
    int R = qr;
    ans[id] += sum1(L, R);
    addEvent(l - 1, L, R, id, -1);
    r = qr;
}

if (ql > l)
{
    int L = l;
    int R = ql - 1;
    addEvent(r, L, R, id, -1);
    ans[id] += sum2(L, R);
    l = ql;
}

if (qr < r)
{
    int L = qr + 1;
    int R = r;
    ans[id] -= sum1(L, R);
    addEvent(l - 1, L, R, id, 1);
    r = qr;
}
}

```

9.2 在线莫队

和“莫队”实际没有什么关系,随着时代发展,普及的技巧而已。

9.2.1 在线普通莫队

对原序列预处理 $res_{i,j}$ 表示第 i 块到第 j 块的结果。

询问 $[l, r]$ 时, L, R 分别表示 l 右侧的一个块, r 左侧第一个块,那么从 $res_{L,R}$ 向两侧扩展即可。如果是 del,就从 l, r 左右的块向内 del。

卡常优化:从离 l, r 近的一侧的块开始扩展,可以小一半指针移动常数。

时间复杂度: $O((\frac{n}{B})^2 + mB)$,具体还要取决于求解 $res_{i,j}$ 所需要的时间复杂度。

以区间不同数的个数为例,那么就是 $O(\frac{n}{B} \times n + mB)$, B 取 $\frac{n}{\sqrt{m}}$ 时,最优为 $O(n\sqrt{m})$ 。

空间复杂度: $O((\frac{n}{B})^2 \times n)$,如果每个 $res_{i,j}$ 单独维护一个信息状态是这样。

此时, B 取 $n^{\frac{2}{3}}$ 最优,时空复杂度均为 $O(n^{\frac{5}{3}})$ 。

如果信息可以差分,以区间不同数的个数为例,维护数的个数,可以调用 $[1, j] -$

$[1, i - 1]$ 的信息状态得到 $res_{i,j}$ 的信息状态。那么空间复杂度: $O(\frac{n^2}{B})$, 此时 B 取 $\sqrt{n}$ 最优, 时空复杂度均为 $O(n\sqrt{n})$ 。

注: 也可以说这是「在线的回滚莫队」, 因为它可以处理仅 add 或仅 del。

9.2.2 在线带修莫队

在在线普通莫队的基础上, 为 $res_{i,j}$ 支持修改, 每次修改涉及 $O((\frac{n}{B})^2)$ 个状态。

所以时间复杂度加上 $O(m(\frac{n}{B})^2)$, 其他部分不变。

B 取 $n^{\frac{2}{3}}$ 时最优, 时空复杂度均为: $O(n^{\frac{5}{3}})$ 。

9.3 莫队卡常

9.3.1 计算变化量

莫队移动指针时, 考虑对答案的影响, 可以不重新计算, 只考虑变化量。

9.3.2 下标连续访问

莫队指针移动在序列上是连续的, 如果问题涉及位置上的值, 使用桶维护值域的时候, 可以采用特殊的离散化方式, 将原序列上连续的一段尽可能连续。或者直接吧统计方式放在序列的位置上, 因为位置的指针移动本身就是连续的。

10 树链剖分

10.1 dfs 序

对一棵树 dfs 时,记录遍历的次序,即:时间戳 dfn 。

10.1.1 性质:

子树在 dfs 序上是连续的一段。

因此可以将树上的子树问题转换成 dfs 序上的区间问题。

10.2 重链剖分

10.2.1 定义

- 重儿子:子树大小最大的儿子(如果有多个,选择任意一个)
- 重边:和重儿子相连的边
- 轻儿子:除重儿子外的所有儿子
- 轻边:除重边外的所有边
- 重链:由连续的重边形成的链

10.2.2 性质

树上两点间路径仅经过不超过 $O(\log n)$ 条重链

简单证明:每走一条轻边子树大小至少减小一半,那么从祖先走向后代最多走 $O(\log n)$ 条轻边。而经过的边,不是轻边就是重边,因为连续的重边是一段重链,所以重链的数量和轻边的数量相当,同样不超过 $O(\log n)$ 。两点路径本质上是两点 LCA 到两点的两段路径,所以树上两点间路径同样经过不超过 $O(\log n)$ 条重链。

dfs 时,优先遍历重儿子,可以保证重链中的节点 dfs 序连续,且由祖先向后代递增。

那么树上一条路径可以对应到 dfs 序上的 $O(\log n)$ 个区间。

注:钦定遍历顺序的 dfs 序仍然是 dfs 序,那么仍然满足子树的 dfs 序是连续的一段。所以重链剖分可以同时维护子树和路径。

```

void dfs1(int x, int y) {
    dep[x] = dep[y] + 1, fa[x] = y, sz[x] = 1;
    son[x] = 0;
    for (auto v : p[x]) {
        if (v == y)
            continue;
        dfs1(v, x);
        sz[x] += sz[v];
        if (sz[son[x]] < sz[v])
            son[x] = v;
    }
}

void dfs2(int x, int y) {
    dfn[x] = ++idx, sec[idx] = a[x], top[x] = y;
    if (!son[x])
        return;
    dfs2(son[x], y);
    for (auto v : p[x]) {
        if (v == fa[x] || v == son[x])
            continue;
        dfs2(v, v);
    }
}

void get_path(int x, int y) {
    while (top[x] != top[y]) {
        if (dep[top[x]] < dep[top[y]])
            swap(x, y);
        /*
            dfn[top[x]] -> dfn[x] 表示当前重链
        */
        x = fa[top[x]];
    }
    if (dep[x] > dep[y])
        swap(x, y);
    /*
        dfn[x] -> dfn[y] 剩余部分
    */
}

```

10.3 长链剖分

10.3.1 定义

和重链剖分类似,修改重儿子的定义:子树中深度最深的节点的深度最大的儿子(如果有多个,选择任意一个)。

10.3.2 性质

树上任意两点间路径仅经过不超过 $O(\sqrt{n})$ 条轻边

简单证明:若经过轻边 (u, v) , 令 u 的重链长度为 L , 则 v 的重链长度 $\leq L - 1$, 因为 $1 + 2 + \dots + L = \frac{L(L+1)}{2} \leq n$ 所以 $L \leq \sqrt{n}$, 那么重链长度减小的次数不超过 $O(\sqrt{n})$, 经过轻边的数量也不超过 $O(\sqrt{n})$ 。

对于树上两点 x, y 若 x 为 y 的祖先, 那么 x 所在的重链长度 $\geq dep_y - dep_x$ 。

简单证明:若 $dep_y - dep_x$ 超过 x 所在重链长度,那么 x, y 路径会成为新的重链的一部分。

10.3.3 应用

树上 k 级祖先

对于每条重链链顶节点,预处理向上/向下重链长度的信息,容易发现预处理的规模等于重链的总长度 $O(n)$ 。

预处理每个节点的 2^i 级祖先,询问时先跳 $2^{\lfloor \log_2 k \rfloor}$ 级祖先,那么 $k - 2^{\lfloor \log_2 k \rfloor} < 2^{\lfloor \log_2 k \rfloor}$,直接调用链顶的不超过重链长度的预处理信息即可。

单次询问 $O(1)$ 。

```
void dfs1(int x) {
    dep[x] = dep[f[x][0]] + 1;
    h[x] = dep[x];
    son[x] = 0;
    for (int i = 1; i < 19; i++)
        f[x][i] = f[f[x][i - 1]][i - 1];
    for (int i = d[x]; i < d[x + 1]; i++) {
        int u = g[i];
        dfs1(u);
        if (h[u] > h[son[x]])
            son[x] = u;
        h[x] = max(h[x], h[u]);
    }
}

void dfs2(int x, int y) {
    dfn[x] = ++idx;
    down[idx] = x;
    up[idx] = y;
    if (son[x]) {
        top[son[x]] = top[x];
        dfs2(son[x], f[y][0]);
    }
    for (int i = d[x]; i < d[x + 1]; i++) {
        int u = g[i];
        if (u == son[x])
            continue;
        top[u] = u;
        dfs2(u, u);
    }
}

int query(int x, int k) {
    if (!k)
        return x;
    int now = __lg(k);
    x = f[x][now];
    k -= (1 << now) + dep[x] - dep[top[x]];
    x = top[x];
    if (k >= 0)
        return up[dfn[x] + k];
    return down[dfn[x] - k];
}

dfs1(root);
top[root] = root;
dfs2(root, root);
```

10.3.4 长链剖分优化dp

长链剖分主题常用于优化状态和深度有关的 dp, 此处略。

10.4 “启发式”剖分

还有一些可以结合题目的“自定义”的剖分方式。

10.4.1 ”猫猫虫“剖分

和重链剖分的区别仅在于修改节点的编号。

传统重链剖分直接按 dfs 序编号, ”猫猫虫“剖分按以下顺序为节点编号: - 递归重儿子 - 为所有轻儿子编号 - 递归轻儿子

(所以其实笔者私以为这个从剖分的角度就是重链剖分, 其核心是维护节点的编号, 而非剖分)

其节点编号具有的性质如下: - 一条重链, 除了顶端节点编号可能不连续, 其余节点编号连续 - 一棵子树, 除了根节点编号可能不连续, 其余节点编号连续 - 一个点的所有邻接点最多有三段连续区间, 即父亲、重儿子、所有轻儿子

虽然和传统重链剖分略有差距, 但是仍然可以维护传统重链剖分能维护的信息。还额外支持维护任一节点的所有儿子的信息。

时间复杂度分析和重链剖分一致。

11 点分治

点分治用于解决树上路径问题。

11.1 点分治

树上的路径可以分为两部分：- 经过重心 - 不经过重心

对于经过重心的路径，以重心为根，dfs 处理出所有路径信息，在重心处合并。

对于不经过重心的路径，递归子树处理。

删除一个规模为 s 的连通块的重心后，每个剩余连通块的大小都不超过 $s/2$ ，因此点分治的递归层数为 $O(\log n)$ 。

在同一递归深度，各连通块互不相交。求子树大小、寻找重心以及 DFS 收集路径信息时，这一层访问的节点总数为 $O(n)$ ；跨越全部递归层后，点分治结构本身的总扫描量为 $O(n \log n)$ 。

若每访问一个节点会产生 $O(1)$ 条待处理信息，并对题目所用数据结构执行一次代价为 $F(n)$ 的加入、查询或撤销操作，则总时间复杂度为 $O(n \log n F(n))$ ；当这些操作均为 $O(1)$ 时，就是 $O(n \log n)$ 。若某层采用排序、卷积等整批处理，应按每个连通块的批处理成本分别求和，不能用一个未定义的“总合并成本”笼统代替这部分分析。

```
int get_size(int x, int fa) {
    if (vis[x])
        return 0;
    int res = 1;
    for (auto u : p[x]) {
        if (u == fa)
            continue;
        res += get_size(u, x);
    }
    return res;
}

int get_wc(int x, int fa, int tot, int &wc) {
    if (vis[x])
        return 0;
    int sum = 1, maxs = 0, t;
    for (auto u : p[x]) {
        if (u == fa)
            continue;
        t = get_wc(u, x, tot, wc);
        maxs = max(maxs, t);
        sum += t;
    }
    maxs = max(maxs, tot - sum);
    if (maxs <= tot / 2)
        wc = x;
    return sum;
}
```

```

void dfs0(int x, int y) { // 添加 x 子树信息
    if (vis[x])
        return;

    for (auto u : p[x]) {
        if (u == y)
            continue;
        dfs0(u, x);
    }
}

void dfs1(int x, int y) { // 删除 x 子树信息
    if (vis[x])
        return;

    for (auto u : p[x]) {
        if (u == y)
            continue;
        dfs1(u, x);
    }
}

void dfs2(int x, int y) { // 将 x 子树路径和已有信息合并, 计算贡献 & 计算子树中节点到重心这条路径的答案
    if (vis[x])
        return;

    for (auto u : p[x]) {
        if (u == y)
            continue;
        dfs2(u, x);
    }
}

void calc(int x) {
    if (vis[x])
        return;
    get_wc(x, 0, get_size(x, 0), x);
    vis[x] = 1;
    // 特殊讨论重心单点的答案
    for (auto u : p[x]) {
        dfs2(u, x);
        dfs0(u, x);
    }

    for (auto u : p[x]) {
        dfs1(u, x);
    }

    for (auto u : p[x])
        calc(u);
}

```

11.2 动态点分治(点分树)

考虑强制在线地询问一个点作为端点的路径信息。

若每一次都做一遍点分治, 发现分治过程中递归的重心是相同的。

那么把递归层更深的重心视作当前递归层重心的儿子, 形成的树, 即为点分树。

而每一个点作为端点合并路径, 只会在其在点分树上的祖先处合并, 点分治递归层数为 $O(\log n)$, 那么同样地, 点分树的树高也为 $O(\log n)$ 。

结合具体题目, 维护点分树上每个点的子树信息即可。

```

int get_size(int x, int fa) {
    if (vis[x])
        return 0;
    int res = 1;
    for (auto u : p[x]) {
        if (u == fa)
            continue;
        res += get_size(u, x);
    }
    return res;
}

int get_wc(int x, int fa, int tot, int &wc) {
    if (vis[x])
        return 0;
    int sum = 1, maxs = 0, t;
    for (auto u : p[x]) {
        if (u == fa)
            continue;
        t = get_wc(u, x, tot, wc);
        maxs = max(maxs, t);
        sum += t;
    }
    maxs = max(maxs, tot - sum);
    if (maxs <= tot / 2)
        wc = x;
    return sum;
}

int fa[N];

void calc(int x, int y) { // 只需要保留重心递归部分
    if (vis[x])
        return;
    get_wc(x, 0, get_size(x, 0), x);
    fa[x] = y;
    vis[x] = 1;
    for (auto u : p[x])
        calc(u, x);
}

```


12 dsu on tree

实际情况中,下面的树上启发式合并和 dsu on tree 的概念可能是混用,以具体情境为准。

12.1 树上启发式合并:

(注意此处的树上启发式合并和 dsu on tree 的区别)

点分治的内核是为每一条路径选定一个划分点进行划分,通过快速合并划分点两侧的路径计算答案。

事实上,对于树上路径,有一个“天然的划分点”——路径两端点的 LCA。

将 LCA 作为路径的划分点,那么每条路径一定会被考虑且只考虑一次。

在树上进行 dfs 时,遍历到的每个点的不同儿子的子树的节点之间的 LCA 就是这个点。

而对于遍历到的当前节点,依次遍历儿子的子树时,从一个儿子回溯到当前节点,开始遍历下一个儿子后,对于新遍历的儿子的子树内的信息就可以和已经遍历过的儿子的子树信息进行合并。

用数据结构维护信息,因为只能是当前节点的后代节点信息进行合并,而 dfs 时,可能会遍历过当前节点的非后代节点。所以需要对每个节点单独开一个「数据结构」进行统计。

同时,将后代信息合并到当前节点是不断合并集合状态的过程。因为总状态数对应总节点数,所以合并的。采用启发式合并,合并两个「数据结构」的信息时,将信息少的那个「数据结构」的元素暴力取出插入到另一个「数据结构」中。

总时间复杂度: $O(n \log n T(n))$, 其中 $T(n)$ 表示「数据结构」插入的时间复杂度。

```
void merge(/*x 的数据结构*/, /*儿子 u 子树的数据结构*/) {
    if (/*x 的数据结构.size*/ < /*儿子 u 子树的数据结构.size*/)
        swap(/*x 的数据结构*/, /*儿子 u 子树的数据结构*/);

    for (auto u : /*儿子 u 子树的数据结构*/) {
        // 合并 x 的数据结构的信息 和 儿子 u 子树的数据结构 的信息,计算贡献
    }
    for (auto u : /*儿子 u 子树的数据结构*/) {
        // 插入 x 的数据结构
    }

    // 清空 儿子 u 子树的数据结构
}

void dfs(int x, int y) {
    for (auto u : p[x]) {
        if (u == y)
```

```

        continue;
    dfs(u, x);
    merge(/*x 的数据结构*/, /*儿子 u 子树的数据结构*/);
}
// 计算 x 的贡献
// 添加 x 的信息
}

```

12.2 dsu on tree:

(注意此处的 dsu on tree 和树上启发式合并的区别)

dsu on tree 和树上启发式合并有的地方会混为一谈。

dsu on tree 维护的方式还是和上述做法相同,本质就是枚举分界点,合并两端路径作为答案路径。和树上启发式合并一样,是将路径在LCA 处合并。

dsu on tree 的时间复杂度分析是基于轻重链剖分的。

简单来说, dsu on tree 就是在从儿子子树回溯时,保留重儿子的信息,清空轻儿子的信息。同时,使重儿子是最后一个被遍历的。递归完儿子后,重儿子信息保留,轻儿子的信息暴力统计一遍(暴力统计的过程可以另写一个递归,也可以利用子树在dfn 序上是连续的一段去遍历)。

时间复杂度简单说明:

树上任意节点到根节点的轻边数量不超过 $O(\log n)$ 条。

设根到该节点轻边数量为 x 条,该节点的子树大小为 y 。显然轻边连接的子节点的子树大小小于父亲子树的一半,则 $y < \frac{n}{2^x}$, $\because n > 2^x \therefore x < \log n$ 。

如果一个节点是其父亲的重儿子,则任意节点到根的路径上所有重边连接的父节点在计算答案时必定不会遍历到这个节点,所以一个节点被遍历的次数等于它到根节点路径上的轻边数 +1,所以一个节点被遍历到的次数就是 $O(\log n)$ 的。

所以 dsu on tree 遍历所有节点的总时间复杂度就是 $O(n \log n)$ 的。

时间复杂度: $O(n \log n T(n))$, 其中 $T(n)$ 表示维护信息的时间复杂度。

```

void dfs(int x, int y) { // 维护重儿子
    fa[x] = y;
    sz[x] = 1;
    for (auto u : p[x]) {
        if (u == y)
            continue;
        dfs(u, x);
        sz[x] += sz[u];
        if (sz[son[x]] < sz[u])
            son[x] = u;
    }
}

void dfs1(int x) { // 将 x 子树的信息和已有信息合并,计算贡献
    for (auto u : p[x]) {
        if (u == fa[x])
            continue;
        dfs1(u);
    }
}

```

```

    }
}
void dfs2(int x) { // 添加 x 子树的信息
    for (auto u : p[x]) {
        if (u == fa[x])
            continue;
        dfs2(u);
    }
}
void dfs0(int x, bool y) {
    for (auto u : p[x]) { // 优先递归轻儿子
        if (u == fa[x] || u == son[x])
            continue;
        dfs0(u, 0);
    }
    if (son[x]) {
        dfs0(son[x], 1);
    }

    // 暴力遍历轻子树
    for (auto u : p[x]) {
        if (u == fa[x] || u == son[x])
            continue;
        dfs1(u);
        dfs2(u);
    }
    if (y) {
        // 回溯时,若要保留则添加 x 的信息
    }
    if (!y) {
        // 回溯时,若不保留则清空信息
    }
}
dfs(1, 0);
dfs0(1, 1);

```

12.3 与点分治相比

dsu on tree 从 x 向上回溯时,重子树的信息不能逐个重新更新,只能在信息可以统一增量时维护,而点分治没有这个限制,会重新遍历到每一个节点。

比如树上路径,从 x 向 y 回溯时,相当于给 x 的子树所有节点都加了一条长度为 (x, y) 的边。

13 01 Trie

13.1 普通 01-Trie

01-Trie 与 Trie 结构相同,区别在于维护的字符集是 $\{0,1\}$ 。

本文模板约定键为非负 `int`,由调用者传入最高处理位 $x \in [0,30]$;查询前 Trie 必须非空。设处理位数为 B 、值域上界为 V ,则单次插入和查询均为 $O(B) = O(\log V)$,插入 n 个数的空间复杂度为 $O(nB) = O(n \log V)$ 。

```
struct Trie01 {
    int root = 0, idx = 0;
    int son[(N << 5) + 1][2]{};
    void init() {
        for (int i = 0; i <= idx; i++)
            memset(son[i], 0, sizeof son[i]);
        root = idx = 0;
    }
    void insert(int &p, int v, int x) {
        if (!p) {
            assert(idx < (N << 5));
            p = ++idx;
            son[p][0] = son[p][1] = 0;
        }
        if (x < 0) {
            return;
        }
        int now = (v >> x) & 1;
        insert(son[p][now], v, x - 1);
    }
    int query_xor_max(int p, int v, int x) {
        assert(p);
        if (x < 0) {
            return 0;
        }
        int now = (v >> x) & 1;
        if (son[p][now ^ 1])
            return query_xor_max(son[p][now ^ 1], v, x - 1) | (1 << x);
        return query_xor_max(son[p][now], v, x - 1);
    }
};
```

13.2 可持久化01-Trie

一般可持久化 Trie 仅用于 01-Trie,通过差分可维护区间二进制信息。

每次插入会复制一条长度为 B 的路径,因此构建 n 个版本需要 $O(nB) = O(n \log V)$ 个节点和 $O(n)$ 个版本根。这与普通 01-Trie 的渐进空间量级相同,但并非“不需要额外空间”。

模板约定 `root[i]` 表示插入前 i 个数后的版本。查询区间 $[l,r]$ 时传入 `p = root[l - 1]`、`q = root[r]`,并且必须保证 $1 \leq l \leq r$ 。

```

struct PTrie01 {
    int root[N + 1]{}, idx = 0;
    int sz[(N << 5) + 1]{}, son[(N << 5) + 1][2]{};
    void init(int n) {
        assert(0 <= n && n <= N);
        for (int i = 0; i <= idx; i++){
            memset(son[i], 0, sizeof son[i]);
            sz[i] = 0;
        }
        for (int i = 0; i <= n; i++)
            root[i] = 0;
        idx = 0;
    }
    void insert(int p, int &q, int v, int x) {
        if (!q) {
            assert(idx < (N << 5));
            q = ++idx;
            son[q][0] = son[q][1] = 0;
            sz[q] = 0;
        }
        sz[q] = sz[p];
        sz[q]++;
        if (x < 0)
            return;
        int now = (v >> x) & 1;
        son[q][now ^ 1] = son[p][now ^ 1];
        insert(son[p][now], son[q][now], v, x - 1);
    }
    int query_xor_max(int p, int q, int v, int x) {
        assert(sz[q] > sz[p]);
        if (x < 0) {
            return 0;
        }
        int now = (v >> x) & 1;
        if (sz[son[q][now ^ 1]] - sz[son[p][now ^ 1]] > 0)
            return query_xor_max(son[p][now ^ 1], son[q][now ^ 1], v, x - 1) |
                (1 << x);
        return query_xor_max(son[p][now], son[q][now], v, x - 1);
    }
};

```

13.3 压缩 01-Trie

Patricia Trie

容易发现, 01-Trie 在插入时, 只有当同时拥有左右儿子时, 才会分叉。

不分叉时, 单链的递归路径是唯一的, 那么就可以尝试把它压缩起来。

引理: 对于一棵除了叶子, 其他节点儿子数都是 2 的树而言, 非叶子数 = 叶子数 - 1。

设当前不同元素的数量为 m , 则叶子数为 m , 非空树的节点总数恰为 $2m - 1$ 。重复元素由叶子计数维护, 不会新建节点。

在时间复杂度 $O(B)$ 不变的前提下, 将 01-Trie 的空间复杂度优化至 $O(m)$ 。下面的静态节点池不回收删除的节点, 因此 N 必须按整个操作序列中可能创建的节点总数预留, 不能只按某一时刻的集合大小估算。

dep 表示当前子树可以继续分叉的位置上界 (不含)。对非负 int 的第 0 ~ 30

位建树时,首次调用为 `insert(root, 31, v)`,删除时对应调用 `remove(root, 31, v)`。

且因为保证了非叶子的左右儿子一定是存在的,所以查询比 01-Trie 更加方便。

```
struct node
{
    int son[2], dep, val;
    int cnt;
};

struct PatriciaTrie
{
    node tr[(N << 1) + 1]{};
    int root = 0, idx = 0;
    void init()
    {
        for (int i = 0; i <= idx; i++)
            tr[i] = {};
        root = idx = 0;
    }
    void insert(int &p, int dep, int v)
    {
        assert(v >= 0 && 0 <= dep && dep <= 31);
        if (!p)
        {
            assert(idx + 1 <= (N << 1));
            p = ++idx;
            tr[p] = {{0, 0}, -1, v, 1};
            return;
        }
        int diff = v ^ tr[p].val;
        int mask1 = (1ll << dep) - 1;
        int mask2 = (tr[p].dep == -1) ? ~0 : ~((1ll << (tr[p].dep + 1)) - 1);
        diff &= (mask1 & mask2);
        if (diff)
        {
            assert(idx + 2 <= (N << 1));
            int high = __lg(diff), old = p;
            int now = ++idx, leaf = ++idx;
            tr[now] = {{0, 0}, high, tr[old].val, 0};
            tr[leaf] = {{0, 0}, -1, v, 1};
            tr[now].son[(tr[old].val >> high) & 1] = old;
            tr[now].son[(v >> high) & 1] = leaf;

            p = now;
            return;
        }
        if (tr[p].dep == -1)
        {
            tr[p].cnt++;
            return;
        }
        insert(tr[p].son[(v >> tr[p].dep) & 1], tr[p].dep, v);
    }
    bool remove(int &p, int dep, int v)
    {
        assert(v >= 0 && 0 <= dep && dep <= 31);
        if (!p)
            return 0;

        int diff = v ^ tr[p].val;
        int mask1 = (1ll << dep) - 1;
        int mask2 = (tr[p].dep == -1) ? ~0 : ~((1ll << (tr[p].dep + 1)) - 1);
        if (diff & mask1 & mask2)
            return 0;
        if (tr[p].dep == -1)
        {
            if (tr[p].val != v)
```

```

        return 0;
    assert(tr[p].cnt > 0);
    if (--tr[p].cnt == 0)
        p = 0;
    return 1;
}

int now = (v >> tr[p].dep) & 1;
if (!remove(tr[p].son[now], tr[p].dep, v))
    return 0;
if (!tr[p].son[now])
{
    p = tr[p].son[now ^ 1];
}
else if (tr[p].val == v)
{
    tr[p].val = tr[tr[p].son[0]].val;
}
return 1;
}
int query_max(int v)
{
    assert(v >= 0 && root);
    int now = root;
    while (tr[now].dep != -1)
    {
        int cur = (v >> tr[now].dep) & 1;
        now = tr[now].son[cur ^ 1];
    }
    return v ^ tr[now].val;
}
};

```

Patricia Trie 同样可以持久化,但是时空复杂度与可持久化 01-Trie 一致,所以不作展开。

13.4 压位 01-Trie

压位 01-Trie 本质是一棵 w 叉树(通常 $w = 64$,即计算机 unsigned long long 的位数)。

每个节点用一个 unsigned long long 压缩儿子节点状态,如果 $mask$ 的第 i 为 1 表示第 i 个儿子存在。这样建出来的树高是 $O(\log_{64} V)$,其中 V 表示的值域大小。

通过位运算计算访问相应的节点。

因为压位 01-Trie 只存储「是否存在儿子」的信息,所以不显示建树,直接将每一层的节点拍平成数组。

默认维护值域 $[0, V]$ 上的不可重集,值 0 是合法元素。所有可能无答案的查询都返回 `std::optional<int>`, `std::nullopt` 表示空集或不存在满足条件的元素。空间复杂度为 $O(\frac{V}{w})$;如果维护可重集,需要增加计数数组或哈希表。

代码使用 `std::optional`,需要 C++17 或更高标准及 `<optional>`。`insert/erase` 返回集合是否实际发生变化,越界、重复插入和删除不存在元素均返回 `false`。

`contains` 的时间复杂度为 $O(1)$, 插入、删除、最值、前驱和后继的时间复杂度均为 $O(\log_w V)$ 。

13.4.1 插入/删除

遍历每一层, 更新对应节点的儿子节点状态即可。

13.4.2 前驱/后继

先自下而上, 向前/后找到第一个可转向的层(也就是子树存在叶子的节点), 再向下找到那个点。

13.4.3 最大值/最小值

其实可以直接转换成判断值域的边界是否存在后查前驱后继。

自上而下, 找到最高/最低位 1 即可。

```
using u64 = unsigned long long;

struct FastTrie
{
    int V = -1;
    vector<vector<u64>> tr;
    vector<int> nbits;
    void init(int V)
    {
        assert(0 <= V && V < INT_MAX);
        this->V = V;
        tr.clear();
        nbits.clear();
        int idx = 0;
        int now = V + 1;
        while (1)
        {
            idx++;
            int cur = (int)((1LL * now + 63) >> 6);
            if (cur == 1)
                break;
            now = cur;
        }
        tr.reserve(idx);
        nbits.reserve(idx);
        now = V + 1;
        while (1)
        {
            nbits.emplace_back(now);
            int cur = (int)((1LL * now + 63) >> 6);
            tr.emplace_back(cur, 0ULL);
            if (cur == 1)
                break;
            now = cur;
        }
    }
};

int lowbit_pos(u64 x)
{
    return __builtin_ctzll(x);
}
```

```

int highbit_pos(u64 x)
{
    return 63 - __builtin_clzll(x);
}

u64 prefix_mask(int b)
{
    return b == 63 ? ~0ULL : ((1ULL << (b + 1)) - 1);
}

bool empty()
{
    return tr.empty() || tr.back()[0] == 0;
}

bool in_range(int x)
{
    return !tr.empty() && 0 <= x && x <= V;
}

int next_in_dep(int dep, int pos)
{
    if (pos >= nbits[dep])
        return -1;
    int w = pos >> 6;
    int b = pos & 63;
    u64 cur = tr[dep][w] & (~0ULL << b);
    if (cur)
    {
        int res = (w << 6) + lowbit_pos(cur);
        return res < nbits[dep] ? res : -1;
    }
    if (dep + 1 >= (int)tr.size())
        return -1;
    int nw = next_in_dep(dep + 1, w + 1);
    if (nw == -1)
        return -1;
    int res = (nw << 6) + lowbit_pos(tr[dep][nw]);
    return res < nbits[dep] ? res : -1;
}

int prev_in_dep(int dep, int pos)
{
    if (pos < 0)
        return -1;
    if (pos >= nbits[dep])
        pos = nbits[dep] - 1;
    int w = pos >> 6;
    int b = pos & 63;
    u64 cur = tr[dep][w] & prefix_mask(b);
    if (cur)
    {
        int res = (w << 6) + highbit_pos(cur);
        return res < nbits[dep] ? res : -1;
    }
    if (dep + 1 >= (int)tr.size())
        return -1;
    int pw = prev_in_dep(dep + 1, w - 1);
    if (pw == -1)
        return -1;
    int res = (pw << 6) + highbit_pos(tr[dep][pw]);
    return res < nbits[dep] ? res : -1;
}

bool contains(int x)
{
    if (!in_range(x))
        return false;
    int w = x >> 6;
    int b = x & 63;
    return (tr[0][w] >> b) & 1ULL;
}

bool insert(int x)

```

```

{
    if (!in_range(x))
        return false;
    int w0 = x >> 6;
    int b0 = x & 63;
    u64 m0 = 1ULL << b0;
    if (tr[0][w0] & m0)
        return false;
    int id = x;
    for (int dep = 0; dep < (int)tr.size(); dep++)
    {
        int w = id >> 6;
        int b = id & 63;
        u64 m = 1ULL << b;
        u64 old = tr[dep][w];
        tr[dep][w] |= m;
        if (old)
            break;
        id = w;
    }
    return true;
}

bool erase(int x)
{
    if (!in_range(x))
        return false;
    int w0 = x >> 6;
    int b0 = x & 63;
    u64 m0 = 1ULL << b0;
    if (!(tr[0][w0] & m0))
        return false;
    int id = x;
    for (int dep = 0; dep < (int)tr.size(); dep++)
    {
        int w = id >> 6;
        int b = id & 63;
        u64 m = 1ULL << b;
        tr[dep][w] &= ~m;
        if (tr[dep][w])
            break;
        id = w;
    }
    return true;
}

std::optional<int> get_min()
{
    if (empty())
        return std::nullopt;
    int top = tr.size() - 1;
    int id = lowbit_pos(tr[top][0]);
    for (int dep = top - 1; dep >= 0; dep--)
    {
        id = (id << 6) + lowbit_pos(tr[dep][id]);
    }
    return id;
}

std::optional<int> get_max()
{
    if (empty())
        return std::nullopt;
    int top = tr.size() - 1;
    int id = highbit_pos(tr[top][0]);
    for (int dep = top - 1; dep >= 0; dep--)
    {
        id = (id << 6) + highbit_pos(tr[dep][id]);
    }
    return id;
}

std::optional<int> get_prev(int x)
{
    if (x <= 0 || empty())
        return std::nullopt;

```

```

    int p = min(x - 1, V);
    int res = prev_in_dep(0, p);
    if (res == -1)
        return std::nullopt;
    return res;
}
std::optional<int> get_next(int x)
{
    if (x >= V || empty())
        return std::nullopt;
    int p = max(0, x + 1);
    int res = next_in_dep(0, p);
    if (res == -1)
        return std::nullopt;
    return res;
}
};

```

13.5 动态开点 · 压位01-Trie

如果值域达到 10^8 甚至更大,那么静态数组就不能支持了。

一个办法是直接恢复到显式地把树建出来(因为 01-Trie 本来就是动态开点的,所以其实就是恢复到了原本的写法),但是这样要存儿子节点的编号。

如果直接存,空间复杂度为 $O(nw \log_w V)$,使用 `map` 维护儿子,时空复杂度均多乘一个 $O(\log w)$,优势变小。

可以用于集合大小不大,但是询问次数爆炸、值域爆炸,并且强制在线的情况。

本节只记录思路,暂未给出可复制的实现。实现时仍需统一约定值域为 $[0, V]$,对所有公开接口进行边界检查,并按整个操作序列中创建的节点总数检查节点映射容量。

// 规划中:暂无可复制实现。

14 线性基

注:线性基应当属于线性代数范畴,但是其在算法竞赛中通常用于维护异或,所以将其放在数据结构中。

算法竞赛中的线性基一般只考虑异或线性基。

线性基简单来讲就是一个整数集合的异或意义下的极大线性无关组。

具体而言,对于给定集合 A , 任意整数 $c = \oplus_{i=1}^n a_i, a_i \in A$, 若都存在 $c = \oplus_{i=1}^m b_i, b_i \in B$, 且 $|B|$ 最小, 则 B 称作 A 的线性基。

从另一个角度来看,线性基将原集合的 2^n 的状态数缩小至 2^{rank} , 其中 n 是集合大小, rank 是线性基的秩。

在具体实现上,将集合中的任一元素,视作一个 $1 \times m$ 的元素为 **01** 的向量;将原集合视作一个 $n \times m$ 的 **01** 矩阵。

假设第一列是最高位。

14.1 构建:

14.1.1 高斯消元:

从**最高位**开始,对该矩形进行高斯消元,化简成最简型矩阵。

去掉全零行,最终形成的 $m' \times m$ 的矩阵即为原集合的线性基。

任一能通过选择原矩阵若干行求异或和得到的向量,都能通过选择最终矩阵的若干行异或得到。

使用 `bitset` 维护,时间复杂度: $O(\frac{m^2n}{w})$ 。

```
void build(int n, vector<u64> &t) {
    a.resize(n);
    num.resize(64);
    this->n = n;
    for (int i = 1; i <= n; i++) {
        a[i - 1] = t[i];
    }
    int now = 63;
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            if (a[j][now]) {
                swap(a[j], a[i]);
                break;
            }
        }
        if (!a[i][now]) {
            now--;
            if (now < 0)
                break;
            i--;
        }
    }
}
```

```

    } else {
        num[i] = now;
        for (int j = 0; j < n; j++) {
            if (j == i)
                continue;
            if (a[j][now])
                a[j] ^= a[i];
        }
        now--;
        if (now < 0)
            break;
    }
}
}

```

14.1.2 贪心法：

按 $1 \sim n$ 的顺序将每一个元素加入线性基,也就是在前 $i - 1$ 个元素构建的线性基的基础上维护新的线性基。

实际上和高斯消元本质相同,加入 x 时,从最高位到最低位枚举 x 二进制中的 1,找到第一个没有被占用的位, x 占用这一位。过程中如果某一位已经被占据,则 x 异或上这一位对应的行。

时间复杂度: $O(nw)$ 。

与高斯消元相比的优势有: - 在线 - 支持回退 - 空间复杂度: $O(w)$

```

void insert(u64 v) {
    for (int i = 63; i >= 0; i--) {
        if ((v >> i) & 1) {
            if (!a[i]) {
                a[i] = v;
                break;
            } else {
                v ^= a[i];
            }
        }
    }
}

```

虽然一般情况下,线性基会从最高位开始消元,但实际上,因为秩的大小不一定为 n ,那么消元后的自由元取决于对哪些位进行了消元。

求子集异或和第 k 大时,需要从最高位开始消元,此时自由元是最高的若干位。但若期望自由元是最低的若干位,那么应当从最低位开始消元。若每一位带有额外权值,应当按位的权值顺序消元。

14.2 性质：

以下均讨论原序列的子集异或,并将相同的异或结果只计一次。

14.2.1 0 是否可表示

如果允许选空集,那么空集的异或和始终是 0,无需另外判定。

如果要求选择至少一个原序列元素,那么异或和能为 0 当且仅当输入向量线性相关,也就是 $\text{rank} < n$;输入中含有 0 是其中一种情况。增量插入时,某个原向量被当前基约化为 0,或批量高斯消元后出现全零行,都说明输入线性相关。实现中应单独记录这个标志;去掉全零行后的线性基本身仍然是线性无关的。

下面关于“最后一行”、“所有行异或”和“下标二进制位对应基向量”的结论,都要求线性基已化为高位主元的行最简形:每个主元位只在它所在的基向量中为 1,且非零行按主元从高到低排列。前面从高位开始的完整高斯消元可得到这种规范基;朴素贪心 insert 通常只得到行阶梯形,查询前还需进一步消元。

14.2.2 最小值:

若允许空集,最小值始终是 0。若要求子集非空,输入线性相关时最小值也是 0;输入线性无关时,在上述规范基中,最后一个非零行才是最小的正异或值。对普通行阶梯基不能直接使用“最后一行”这个结论。

14.2.3 最大值:

在上述规范基中,所有非零行的异或和是最大值。对普通行阶梯基,应从高位主元到低位主元扫描,只在 $\text{res} \oplus \text{basis}[i] > \text{res}$ 时更新 res;直接异或所有行不一定正确。

14.2.4 第 k 小/大

设线性基的秩为 r ,则共有 2^r 个不同的可表示值。先将规范基的非零行改按主元从低到高记为 $b_0, b_1, \dots, b_{r-1}$ 。对于 $0 \leq t < 2^r$,定义

$$F(t) = \bigoplus_{\substack{0 \leq i < r \\ \text{bit}_i(t)=1}} b_i$$

此时 $F(0), F(1), \dots, F(2^r - 1)$ 恰好按数值从小到大排列。因此,若用从 1 开始的 k 且把 0 算在答案中,第 k 小是 $F(k - 1)$ 。若只允许非空子集,输入线性相关时仍包含 0;输入线性无关时需排除 $F(0)$,此时第 k 小是 $F(k)$ 。第 k 大应先按当前约定确定答案总数 M ,再转换为第 $M - k + 1$ 小。

“把下标的二进制 1 位直接对应到基向量”只对上述规范基和排列顺序成立;普通行阶梯基必须先化为行最简形。规范化需要 $O(w^2)$ 时间,之后单次查询需要 $O(r)$ 时间。计算 2^n 和查询下标时还需使用足够宽的无符号整数,并避免对整型位宽执行移位。

下面的查询代码沿用“非零行按主元从高到低排列,全零行位于末尾”的存储约定,其中 `query_min` 和 `query_kth` 按“子集必须非空”处理 0。

```
u64 query_max() {
    u64 res = 0;
    for (int i = 0; i < n; i++)
        res ^= a[i].to_ullong();
    return res;
}
u64 query_min() { return a[n - 1].to_ullong(); }
u64 query_kth(u64 k) {
    u64 cnt = 0;
    bool flag = 1;
    for (int i = 0; i < n; i++) {
        if (a[i].count())
            cnt++;
        else
            flag = 0;
    }
    u64 res = 0;
    int m = n - 1;
    for (int i = n - 1; i >= 0; i--)
        if (a[i].count()) {
            m = i;
            break;
        }
    if (!flag) {
        k--;
    }
    if (!k)
        return 0;
    for (int i = 0; i < n; i++) {
        if ((k >> i) & 1)
            res ^= a[m - i].to_ullong();
    }
    return res;
}
```

14.3 解的构造

对于线性基找出的解,要还原出其在原集合中由哪些数组成。

14.3.1 set 维护

对于线性基中的每一个行向量,直接用一个 `set` 维护其由原序列中哪些下标异或得到。设最终线性基的秩为 r ,则 $r \leq w$ 。

```

void insert(int x, u64 v) {
    set<int> now;
    now.insert(x);
    for (int i = 63; i >= 0; i--) {
        if ((v >> i) & 1) {
            if (!a[i]) {
                a[i] = v;
                pos[i] = now;
                break;
            } else {
                v ^= a[i];
                for (auto u : pos[i]) {
                    if (now.count(u))
                        now.erase(u);
                    else
                        now.insert(u);
                }
            }
        }
    }
}

```

这份增量实现只会把“使秩增加”的输入下标保存到 `pos`。被当前线性基约化为 0 的输入不会写入任何 `pos[i]`, 因此所有已保存的来源下标都属于至多 r 个独立输入, 单个 `pos[i]` 的大小为 $O(r)$, 不会在该不变量下增长到 $O(n)$ 。

内层循环计算 `now` 与 `pos[i]` 的集合对称差。一次 `count`、`erase` 或 `insert` 的代价是 $O(\log(r+1))$, 但处理整个 `pos[i]` 需要

$$O(|pos[i]| \log(r+1)) = O(r \log(r+1)),$$

而不是 $O(\log w)$ 。插入一个数需扫描 w 个二进制位, 并最多与 r 个基向量做对称差, 所以单次复杂度为

$$O(w + r^2 \log(r+1)).$$

处理 n 个输入的总时间复杂度为

$$O(n(w + r^2 \log(r+1))) \subseteq O(nw^2 \log(w+1)).$$

所有 `pos` 中保存的来源下标总数为 $O(r^2)$, 计入 w 个 `set` 对象的空间后为 $O(w + r^2)$ 。若改用不保持上述不变量的一般高斯消元来保存任意原始行组合, 单个来源集合才可能增长到 $O(n)$ 。

14.3.2 bitset 维护

因为每个行向量由且仅有它前面的行向量异或得到, 而行向量插入线性基后不会被覆盖, 所以可以用一个 `bitset` 存储每一个行向量由当前哪些行向量异或得到, 同时存下每个行向量插入线性基时初始对应的值。

还原元素时,把那些行向量的 `bitset` 异或起来,再把最终需要的行向量的初始值异或得到最后的解。

时间复杂度: $O(nw)$ 。

```
void insert(int x, u64 v) {
    bitset<64> now;
    for (int i = 63; i >= 0; i--) {
        if ((v >> i) & 1) {
            if (!a[i]) {
                a[i] = v;
                mp[i] = x;
                pos[i] = now;
                pos[i].set(i);
                break;
            } else {
                v ^= a[i];
                now ^= pos[i];
            }
        }
    }
}
```

14.4 线性基的合并

线性基可以用来求两个子空间的交,或者将两个子空间合并为它们的子空间和。

14.4.1 子空间和 / 线性基合并

令 $U = \text{span}(A)$, $V = \text{span}(B)$ 。集合并 $U \cup V$ 通常对异或不封闭,因此通常不是子空间;只有当 $U \subseteq V$ 或 $V \subseteq U$ 时, $U \cup V$ 才是子空间。

将 A 和 B 的基向量全部插入同一个线性基,得到的是子空间和

$$U + V = \{u \oplus v \mid u \in U, v \in V\} = \text{span}(A \cup B),$$

而不是集合并 $U \cup V$ 。

时间复杂度: $O(w^2)$ 。

14.4.2 线性基的交

构建增广矩阵:

$$\left[\begin{array}{c|c} A & 0 \\ B & B \end{array} \right]$$

其中 A, B 是要求交的线性基。

对于增广矩阵,进行高斯消元,最后得到行最简型矩阵。

系数矩阵中的零行对应的右侧行向量就是 $\text{span}(A) \cap \text{span}(B)$ 。

时间复杂度： $O(w^2)$ 。

```
LinearBasis intersection_basis(LinearBasis &A, LinearBasis &B)
{
    struct Node
    {
        ull x, tag;
    };

    Node q[w + 1];
    for (int i = 0; i <= w; i++)
        q[i] = {0, 0};

    auto insert_aug = [&](ull x, ull tag, LinearBasis *ans)
    {
        for (int i = w; i >= 0; i--)
        {
            if (((x >> i) & 1) == 0)
                continue;

            if (!q[i].x)
            {
                q[i] = {x, tag};
                return;
            }

            x ^= q[i].x;
            tag ^= q[i].tag;
        }

        if (ans && tag)
            ans->insert(tag);
    };

    for (ull x : A.vectors())
    {
        insert_aug(x, 0, nullptr);
    }

    LinearBasis ans;
    for (ull x : B.vectors())
    {
        insert_aug(x, x, &ans);
    }

    return ans;
}
```

14.5 不同的线性基构造

14.5.1 高位线性基

优先保证高位消元成主元。

14.5.2 低位线性基

与高位线性基相对, 优先消低位。

因为整数的大小比较从高位开始,低位线性基不能直接套用高位线性基的最大值贪心,但这不代表它无法回答与大小相关的查询。可以把其至多 w 个基向量重新插入高位线性基,或者从高位开始重新消元,在 $O(w^2)$ 时间内得到高位基或规范基,之后可以在 $O(w)$ 时间内查询最大值。

因此,低位线性基只是不适合直接进行高位贪心,而不是丢失了原线性空间的信息。

14.5.3 任意主元顺序

更一般地,如果题目有相应的约束要求,可以按照任意顺序消主元。

14.5.4 最简线性基 / 规范线性基

前面朴素维护的高位/低位线性基,本质是行阶梯型的线性基,对于不同的插入顺序,得到的线性基也可能不同。

但是最简线性基唯一。

主要用于线性基去重 / 判断本质相同时使用。

本质是用高斯消元消成的行最简型。

可以由任一线性基通过高斯消元得到,时间复杂度: $O(w^2)$ 。

也可以直接在插入时维护行最简型。

14.6 区间线性基

14.6.1 线段树维护

线段树上每个节点维护一个线性基, `push_up` 合并两个线性基。

预处理时间复杂度: $O(nw^2)$ 。

查询合并 $O(\log n)$ 个线性基,时间复杂度: $O(\log nw^2)$ 。

空间复杂度: $O(nw)$ 。

支持修改,时间复杂度: $O(q \log nw^2)$ 。

14.6.2 ST 表维护

每次需要合并两个线性基,预处理时间复杂度: $O(n \log nw^2)$ 。

空间复杂度: $O(n \log nw)$ 。

查询合并两个线性基,时间复杂度: $O(qw^2)$ 。

相交猫树没有时空优势,唯一优势是好写。

14.6.3 猫树维护

因为猫树是向线性基中加入元素,所以单次操作是 $O(w)$ 的。

猫树共有 $O(n \log n)$ 次插入,预处理时间复杂度: $O(n \log nw)$ 。

查询合并两个线性基,时间复杂度: $O(qw^2)$ 。

共存 $O(n \log n)$ 个线性基,空间复杂度: $O(n \log nw)$ 。

14.6.4 前缀线性基

前缀的本质不同线性基只有 $O(w)$ 个。

维护 $[1, i]$ 的 $O(w)$ 个本质不同线性基,记录每个线性基的最后一个左端点的位置。

查询 $[l, r]$ 的线性基的时候,只要 $[1, r]$ 的找到第一个 $\geq l$ 的线性基即可。

预处理时间复杂度: $O(nw^2)$ 。

询问时间复杂度: $O(\log w)$,一般 $O(w)$ 即可。

空间复杂度: $O(nw^2)$ 。

14.6.4.1 优化

实际上,可以发现如果新插入的元素改变了线性基,在不化成行最简型的情况下,只会改变一个行向量。

那么每一个行向量尽可能用最靠后的元素。

处理 $[l, r]$ 的线性基时,就是在 $[1, r]$ 的线性基中保留 $\geq l$ 的行向量。

如果是高位线性基,那么需要替换能够替换的最高位。否则得到的可能是一个任意主元线性基。

时间复杂度: $O(nw + qw)$ 。

空间复杂度: $O(nw)$ 。

14.6.5 分块维护

14.6.5.1 序列分块

维护 $O(\frac{n}{B})$ 个整块线性基,查询时,合并 $O(\frac{n}{B})$ 个线性基和加入 $O(B)$ 个元素。

时间复杂度: $O(\frac{qn}{B}w^2 + qBw)$ 。

空间复杂度: $O(\frac{n}{B}w)$ 。

n, q 视作同阶,最优时间复杂度: $O(n\sqrt{nw}^{\frac{3}{2}})$ 。

此时,空间复杂度为: $O(\sqrt{nw})$ 。

时间复杂度较劣,优势是空间优秀。

14.6.5.2 在线莫队

预处理 $O(\frac{n}{B} \times \frac{n}{B})$ 个线性基。

询问时,加入 $O(B)$ 个元素。

时间复杂度: $O(qBw + qw^2 + \frac{n^2}{B^2}w^2)$ 。

空间复杂度: $O(\frac{n^2}{B^2}w)$ 。

n, q 视作同阶,块长取 $O((nw)^{\frac{1}{3}})$ 时,时间复杂度最优: $O((nw)^{\frac{4}{3}} + nw^2)$ 。

此时,空间复杂度为: $O(n^{\frac{4}{3}}w^{\frac{1}{3}})$ 。

14.6.6 实现对比

下表中, n 表示序列长度, q 表示询问数, w 表示值域二进制位数, B 表示块长。

实现	预处理时间	单次询问时间	q 次询问总时间	空间复杂度	优点	缺点 / 适用场景
线段树维护	$O(nw^2)$	$O(\log nw^2)$	$O(q \log nw^2)$	$O(nw)$	通用性最好,支持单点修改,空间只多一个线段树常数。	静态询问下时间不占优,每次查询要合并 $O(\log n)$ 个线性基。
ST 表维护	$O(n \log nw^2)$	$O(w^2)$	$O(qw^2)$	$O(n \log nw)$	思路 and 实现简单,两个重叠区间的线性基合并即可。	只支持静态序列;相比猫树,预处理更慢、空间相同、询问不更优。
猫树维护	$O(n \log nw)$	$O(w^2)$	$O(qw^2)$	$O(n \log nw)$	静态区间询问较适合,预处理只需要不断插入元素,比 ST 表少一个 w 。	空间较大;不支持修改;询问仍要合并两个线性基。
前缀线性基	$O(nw^2)$	$O(\log w)$ 或 $O(w)$	$O(q \log w)$ 或 $O(qw)$	$O(nw^2)$	查询很快,利用前缀本质不同线性基只有 $O(w)$ 个。	空间较大,实现需要维护每个前缀的多个本质不同线性基;只适合静态序列。

实现	预处理时间	单次询问时间	q 次询问总时间	空间复杂度	优点	缺点 / 适用场景
前缀线性基优化	$O(nw)$	$O(w)$	$O(qw)$	$O(nw)$	静态区间询问通常最优秀, 时空都接近最优, 实现好后常数也小。	需要维护“每个主元尽量靠后”的位置约束; 不如线段树通用, 不能直接支持修改。
序列分块	$O(nw)$	$O(\frac{n}{B}w^2 + Bw)$	$O(\frac{qn}{B}w^2 + qBw)$	$O(\frac{n}{B}w)$	空间最省, 块内暴力加入元素即可; 需要时可以重构单块支持修改。	查询较慢, 块长需要调参; 当 n, q 同阶时最优仍为 $O(n\sqrt{nw}^{\frac{3}{2}})$ 。
在线莫队	$O(\frac{n^2}{B^2}w^2)$	$O(Bw + w^2)$	$O(qBw + qw^2 + \frac{n^2}{B^2}w^2)$	$O(\frac{n^2}{B^2}w)$	查询不用删除元素, 适合静态且询问较多的场景; 可在线回答询问。	预处理和空间压力较大, 块长需要平衡; 当 n, q 同阶时取 $B = O((nw)^{\frac{1}{3}})$ 才较优。

14.7 处理线性基的不可删

线性基容易维护加入一个元素, 但是不容易维护删除一个元素。

14.7.1 线段树直接维护

开一棵大小是 $O(q)$ 的线段树维护即可。

时间复杂度: $O(q \log qw^2)$ 。

14.7.2 线段树分治

设时间轴上有 q 个操作,共得到 m 个元素的有效时间区间。每个时间区间需要拆到线段树的 $O(\log q)$ 个节点中,因此线段树上共存储 $O(m \log q)$ 个事件副本。

插入线性基时,普通行阶梯型实现只会修改至多一个主元位置,每次成功修改只需记录 $O(1)$ 个撤销信息;若始终维护行最简形,一次插入可能修改 $O(w)$ 个位置,需要记录对应的旧值。

说明操作是可以暴力撤销的。

普通线性基的一次插入需要 $O(w)$ 时间,因此使用线段树分治和撤销栈处理所有事件副本的时间复杂度为

$$O(mw \log q).$$

若在至多 q 个叶子处各执行一次 $O(w)$ 的线性基查询,则算法的总时间复杂度为 $O((m \log q + q)w)$ 。

普通行阶梯型实现的空间复杂度为

$$O(m \log q + q + w),$$

其中 $O(m \log q)$ 是事件副本, $O(q)$ 是线段树节点, $O(w)$ 是当前线性基和撤销信息。通常 $m = O(q)$, 此时总时间复杂度为 $O(qw \log q)$, 空间复杂度为 $O(q \log q + w)$ 。若维护行最简形,当前 DFS 路径上的撤销记录最坏还可能需 $O(w^2)$ 空间。

15 cdq 分治

cdq 分治是用于解决二维偏序的分治算法(因为 N 维偏序可以通过排序钦定第一维的顺序,所以 N 维偏序等价于求新序列的 $N-1$ 维偏序)。

对于序列 (a, b) 的二维偏序 $(a_i, b_i) \preceq (a_j, b_j)$, 分为三部分统计: - $[l, mid], i, j \in [l, mid]$, 递归左区间 - $[mid + 1, r], i, j \in [mid + 1, r]$, 递归右区间 - $i \in [l, mid], j \in [mid + 1, r]$, 遍历左/右区间, 使用数据结构维护另一个区间的贡献

```
void cdq(int l, int r) {
    if (l >= r) {
        return;
    }
    int mid = l + r >> 1;
    cdq(l, mid);
    cdq(mid + 1, r);

    int now = 1;
    for (int i = mid + 1; i <= r; i++) {
        while (now <= mid && a[now].a <= a[i].a) {
            // 第一维 已升序, 维护 第二维 的贡献
            now++;
        }
        // 计算 i 的答案
    }
    for (int i = l; i < now; i++){
        // 删除 [l, now] 的贡献
    }

    // 归并 第一维 升序
    int cnt = 0;
    now = mid + 1;
    for (int i = l; i <= mid; i++) {
        while (now <= r && a[now].a <= a[i].a) {
            A[++cnt] = a[now];
            now++;
        }
        A[++cnt] = a[i];
    }
    while (now <= r) {
        A[++cnt] = a[now];
        now++;
    }
    for (int i = 1; i <= cnt; i++)
        a[l - 1 + i] = A[i];
}
```

15.1 时间复杂度

复杂度应按具体维数和每一层实际执行的操作分别计算。一次 CDQ 分治有 $O(\log n)$ 层;若同一层内所有元素只参与常数扫描,则这一部分的总代价为 $O(n \log n)$ 。若扫描过程中还要对数据结构进行单次 $O(\log n)$ 的修改或查询,则总代价为 $O(n \log^2 n)$ 。

例如,三维偏序通常先按第一维排序,用 CDQ 分治处理第二维的先后关系,并

用树状数组维护第三维。CDQ 每层共处理 $O(n)$ 个元素,每个元素在树状数组上的操作为 $O(\log n)$,共有 $O(\log n)$ 层,因此总时间复杂度为 $O(n \log^2 n)$ 。对于常见的固定 N 维偏序实现,每增加一个需要递归分治或由数据结构维护的维度,通常再增加一个对数因子,因而写作 $O(n \log^{N-1} n)$;

实际竞赛中还应结合常数判断实现是否可行:维数升高后,多层分治和数据结构操作的常数较大,理论上优于 $O(n^2)$ 的实现未必在给定数据范围内更快。

15.2 重复点与等号

处理偏序前必须先明确各维使用严格小于还是小于等于。对于各维均采用“ $\leq$ ”的非严格偏序,完全相同的点会互相产生贡献;若它们被直接分到CDQ 的两侧,可能只统计到其中一个方向,使相同点得到不一致的答案。常用做法是先将完全相同的点合并为一个结点并记录出现次数 cnt ,修改数据结构时加入 cnt 。设从其他点组得到的贡献为 ans ,若答案不计自身,则该组中每个原点的答案为 $\text{ans} + \text{cnt} - 1$;若计入自身,则为 $\text{ans} + \text{cnt}$ 。

若某一维要求严格小于,或相等坐标之间不应产生贡献,则同坐标事件需要成组处理:先完成这一组的查询,再统一执行修改,避免组内元素相互贡献。排序规则、归并时使用的 $<$ 或 $\leq$,以及数据结构查询的边界必须采用同一套等号约定。

16 整体二分

整体二分是一种优秀的离线算法。对于可二分答案的问题,我们可以采用整体二分的方法处理多组询问。整体二分的思想非常巧妙,能够替代很多复杂的数据结构,代码也十分好写,是考场上的一大利器。

整体二分一般用于多次询问的问题。并且每次的询问都可以用二分答案的方式求解。

整体二分简单来说就是直接对于所有询问整体上二分一个答案。然后把询问按照是大于还是小于等于当前二分的答案划分到两个集合里去就行了。最后到二分答案的边界时,属于当前集合的询问的答案就是这个边界。

优化时间复杂度的关键不是笼统地说“多组询问共用一次 `check`”,而是同一递归层的所有事件只被整体扫描一次。设:

- E 是事件总数,包括询问以及参与判定的修改、加入等事件;
- V 是离散化后的候选答案数量,或整数答案区间 $[L, R]$ 的长度;
- $F(N)$ 是处理一个事件时,对题目所用数据结构进行一次加入、撤销或查询的时间复杂度。

在递归的同一深度,各节点持有的事件集合互不相交,因此这一层扫描、划分的事件总数为 $O(E)$,而不是每个询问各做一次完整 `check`。答案值域每次减半,共有 $O(\log V)$ 层,所以单纯扫描和稳定划分的总成本为 $O(E \log V)$ 。

若每个事件还需要 $O(F(N))$ 的数据结构操作,则总时间复杂度为

$$O(E \log V F(N)).$$

例如底层使用树状数组时,单次操作为 $O(\log N)$,整体二分部分通常为 $O(E \log V \log N)$;若事件可以 $O(1)$ 维护,则为 $O(E \log V)$ 。排序、离散化以及每层额外的清空或回滚成本应另外计入;事件数、答案值域和单次维护代价必须使用不同变量,不能混在一个含义不明的表达式中。

若递归划分时复用缓冲区,存放事件通常需要 $O(E)$ 空间,递归栈为 $O(\log V)$;题目数据结构自身的空间另计。

```

void solve(int ql, int qr, int L, int R, int al, int ar) {
    if (ql > qr){
        for (int i = al; i <= ar; i++) // 加上 [L, R] 的贡献
            return;
    }
    if (al > ar)
        return;
    if (L == R) {
        for (int i = ql; i <= qr; i++)
            ans[Q[i].id] = L;
        for (int i = al; i <= ar; i++) // 加上 [L, R] 的贡献
            return;
    }
    int mid = L + R >> 1, cnt1 = 0, cnt2 = 0, ret1 = 0, ret2 = 0;

    for (int i = al; i <= ar; i++) {
        if (/* a[i] 信息 <= mid */) {
            update(a[i].id, 1);
        }
        if (/* a[i] 信息 <= mid */) {
            a1[++ret1] = a[i];
        } else {
            a2[++ret2] = a[i];
        }
    }
    for (int i = ql; i <= qr; i++) {
        if (/* Q[i] 的答案在 [L, mid] */) {
            Q1[++cnt1] = Q[i];
        } else {
            Q2[++cnt2] = Q[i];
        }
    }

    for (int i = al; i <= ar; i++)
        if (/* a[i] 信息 <= mid */)
            update(a[i].id, -1); // 把 [L, mid] 的贡献删去

    for (int i = 1; i <= cnt1; i++)
        Q[ql + i - 1] = Q1[i];
    for (int i = 1; i <= cnt2; i++)
        Q[ql + i - 1 + cnt1] = Q2[i];
    for (int i = 1; i <= ret1; i++)
        a[al + i - 1] = a1[i];
    for (int i = 1; i <= ret2; i++)
        a[al + i - 1 + ret1] = a2[i];

    solve(ql, ql + cnt1 - 1, L, mid, al, al + ret1 - 1);
    // 保证递归 [mid+1, R] 时 [L, mid] 的贡献都算了一遍
    solve(ql + cnt1, qr, mid + 1, R, al + ret1, ar);
}

```

16.1 扩展

有一种情况是,如果询问中同时带修,那么部分情况下,整体二分需要能够把修改的贡献在询问中减掉,然后递归。

例如带修的区间第 k 大。

17 珂朵莉树

名字来自于 CF896C 的题图。

本质是用数据结构维护区间连续段。

出于实际使用情况考虑,省略链表维护的介绍。

以下实现中, $mp[x]$ 表示从位置 x 开始的区间值。 mp 的第一个键是左端点 L , 最后一个键是 $R + 1$ 处的右哨兵;右哨兵不属于实际区间,不能被删除。所有区间操作都必须满足 $L \leq l \leq r \leq R$ 。

17.1 init

```
void init(int l, int r) {
    assert(l <= r && r < INT_MAX);
    mp.clear();
    mp[l] = mp[r + 1] = - 1;
}
```

17.2 split

```
auto split(int x) {
    assert(!mp.empty());
    assert(mp.begin()->first <= x && x <= mp.rbegin()->first);

    auto it = mp.lower_bound(x);
    if (it != mp.end() && it->first == x) {
        return it;
    }
    return mp.emplace_hint(it, x, prev(it)->second);
}
```

17.3 assign

区间推平,一般是区间赋值。

```
void assign(int l, int r, int v) {
    assert(!mp.empty());
    assert(mp.begin()->first <= l && l <= r && r < mp.rbegin()->first);

    int sentinel = mp.rbegin()->first;
    auto itr = split(r + 1);
    auto itl = split(l);
    mp.erase(itl, itr);
    auto it = mp.emplace_hint(itr, l, v);

    if (it != mp.begin() && prev(it)->second == it->second) {
        auto pre = prev(it);
        mp.erase(it);
        it = pre;
    }
}
```

```

    auto nxt = next(it);
    if (nxt != mp.end() && nxt->first != sentinel && nxt->second == it->second) {
        mp.erase(nxt);
    }
}

```

17.4 perform

遍历区间 $[l, r]$, 在遍历过程中进行具体操作。

```

void perform(int l, int r, int v) {
    assert(!mp.empty());
    assert(mp.begin()->first <= l && l <= r && r < mp.rbegin()->first);

    auto itr = split(r + 1);
    auto it = split(l);
    while (it != itr) {
        // 具体操作
        it = next(it);
    }
}

```

17.5 时间复杂度分析

17.5.1 区间推平时

若每次操作后, 都会进行一次区间推平操作, 即将区间合并成一个相同的连续段。

每个区间只会被遍历一次, 每次最多增加一个区间, 最多增加到 n 个区间。遍历次数和区间数量的变化成线性相关, 那么均摊时间复杂度为: $O(m \log n)$ 。

17.5.2 数据随机时

CF896C 证明了这种情况的时间复杂度为 $O(m \log \log n)$ 。

18 bitset

手写 bitset

```
#include <bits/stdc++.h>
using namespace std;

struct FastBitset
{
    using ull = unsigned long long;
    static constexpr int W = 64;

    int n; // bit 数量
    int m; // ull 块数量
    vector<ull> a;

    FastBitset(int _n = 0, bool val = false)
    {
        init(_n, val);
    }

    void init(int _n, bool val = false)
    {
        n = _n;
        m = (n + W - 1) >> 6;
        a.assign(m, val ? ~0ULL : 0ULL);
        trim();
    }

    ull last_mask() const
    {
        int r = n & 63;
        if (r == 0)
            return ~0ULL;
        return (1ULL << r) - 1;
    }

    void trim()
    {
        if (m && (n & 63))
        {
            a.back() &= last_mask();
        }
    }

    // 单点操作
    void set(int p)
    {
        a[p >> 6] |= 1ULL << (p & 63);
    }

    void reset(int p)
    {
        a[p >> 6] &= ~(1ULL << (p & 63));
    }

    void flip(int p)
    {
        a[p >> 6] ^= 1ULL << (p & 63);
    }

    bool test(int p) const
    {
        return (a[p >> 6] >> (p & 63)) & 1ULL;
    }

    bool operator[](int p) const
```

```

{
    return test(p);
}

// 整体操作
void set_all()
{
    fill(a.begin(), a.end(), ~0ULL);
    trim();
}

void reset_all()
{
    fill(a.begin(), a.end(), 0ULL);
}

void flip_all()
{
    for (auto &x : a)
        x = ~x;
    trim();
}

int count() const
{
    int res = 0;
    for (ull x : a)
        res += __builtin_popcountll(x);
    return res;
}

bool any() const
{
    for (ull x : a)
    {
        if (x)
            return true;
    }
    return false;
}

bool none() const
{
    return !any();
}

// 找从 pos 开始第一个 1,找不到返回 -1
int find_first1(int pos = 0) const
{
    if (pos >= n)
        return -1;

    int id = pos >> 6;
    int off = pos & 63;

    ull x = a[id] & (~0ULL << off);
    while (true)
    {
        if (x)
        {
            int p = (id << 6) + __builtin_ctzll(x);
            return p < n ? p : -1;
        }
        ++id;
        if (id >= m)
            break;
        x = a[id];
    }
    return -1;
}

// 找从 pos 开始第一个 0,找不到返回 -1

```

```

int find_first0(int pos = 0) const
{
    if (pos >= n)
        return -1;

    int id = pos >> 6;
    int off = pos & 63;

    ull x = (~a[id]) & (~0ULL << off);
    if (id == m - 1)
        x &= last_mask();

    while (true)
    {
        if (x)
        {
            int p = (id << 6) + __builtin_ctzll(x);
            return p < n ? p : -1;
        }
        ++id;
        if (id >= m)
            break;
        x = ~a[id];
        if (id == m - 1)
            x &= last_mask();
    }
    return -1;
}

// 位运算,要求长度相同
FastBitset &operator&=(const FastBitset &b)
{
    assert(n == b.n);
    for (int i = 0; i < m; i++)
        a[i] &= b.a[i];
    return *this;
}

FastBitset &operator|=(const FastBitset &b)
{
    assert(n == b.n);
    for (int i = 0; i < m; i++)
        a[i] |= b.a[i];
    return *this;
}

FastBitset &operator^=(const FastBitset &b)
{
    assert(n == b.n);
    for (int i = 0; i < m; i++)
        a[i] ^= b.a[i];
    return *this;
}

friend FastBitset operator&(FastBitset x, const FastBitset &y)
{
    x &= y;
    return x;
}

friend FastBitset operator|(FastBitset x, const FastBitset &y)
{
    x |= y;
    return x;
}

friend FastBitset operator^(FastBitset x, const FastBitset &y)
{
    x ^= y;
    return x;
}

```

```

FastBitset operator~() const
{
    FastBitset res = *this;
    res.flip_all();
    return res;
}

// 令当前 bitset = src << k
void assign_shift_left(const FastBitset &src, int k)
{
    assert(n == src.n);

    if (k <= 0)
    {
        if (this != &src)
            a = src.a;
        return;
    }

    if (k >= n)
    {
        reset_all();
        return;
    }

    int ws = k >> 6;
    int bs = k & 63;

    // 同源情况: this == &src, 需要按原地左移处理, 从大块往小块更新
    if (this == &src)
    {
        if (ws)
        {
            for (int i = m - 1; i >= 0; i--)
            {
                a[i] = (i >= ws ? a[i - ws] : 0ULL);
            }
        }

        if (bs)
        {
            for (int i = m - 1; i >= 1; i--)
            {
                a[i] = (a[i] << bs) | (a[i - 1] >> (64 - bs));
            }
            a[0] <<= bs;
        }

        trim();
        return;
    }

    // 非同源情况: 直接从 src 计算到当前对象
    for (int i = m - 1; i >= 0; i--)
    {
        ull val = 0;

        int from = i - ws;

        if (from >= 0)
        {
            val |= src.a[from] << bs;

            if (bs && from - 1 >= 0)
            {
                val |= src.a[from - 1] >> (64 - bs);
            }
        }

        a[i] = val;
    }
}

```



```

    trim();
}

// 令当前 bitset = src >> k
void assign_shift_right(const FastBitset &src, int k)
{
    assert(n == src.n);

    if (k <= 0)
    {
        if (this != &src)
            a = src.a;
        return;
    }

    if (k >= n)
    {
        reset_all();
        return;
    }

    int ws = k >> 6;
    int bs = k & 63;

    // 同源情况: this == &src, 需要按原地右移处理, 从低块往高块更新
    if (this == &src)
    {
        if (ws)
        {
            for (int i = 0; i < m; i++)
            {
                a[i] = (i + ws < m ? a[i + ws] : 0ULL);
            }
        }

        if (bs)
        {
            for (int i = 0; i + 1 < m; i++)
            {
                a[i] = (a[i] >> bs) | (a[i + 1] << (64 - bs));
            }
            a[m - 1] >>= bs;
        }

        trim();
        return;
    }

    // 非同源情况: 直接从 src 计算到当前对象
    for (int i = 0; i < m; i++)
    {
        ull val = 0;

        int from = i + ws;

        if (from < m)
        {
            val |= src.a[from] >> bs;

            if (bs && from + 1 < m)
            {
                val |= src.a[from + 1] << (64 - bs);
            }
        }

        a[i] = val;
    }

    trim();
}

FastBitset &operator<<=(int k)

```

```

{
    assign_shift_left(*this, k);
    return *this;
}

FastBitset &operator>>=(int k)
{
    assign_shift_right(*this, k);
    return *this;
}

friend FastBitset operator<<(FastBitset x, int k)
{
    x <<= k;
    return x;
}

friend FastBitset operator>>(FastBitset x, int k)
{
    x >>= k;
    return x;
}

bool operator==(const FastBitset &b) const
{
    return n == b.n && a == b.a;
}

bool operator!=(const FastBitset &b) const
{
    return !(*this == b);
}

void assign_and(const FastBitset &x, const FastBitset &y)
{
    assert(n == x.n && n == y.n);
    for (int i = 0; i < m; i++)
        a[i] = x.a[i] & y.a[i];
}

void assign_or(const FastBitset &x, const FastBitset &y)
{
    assert(n == x.n && n == y.n);
    for (int i = 0; i < m; i++)
        a[i] = x.a[i] | y.a[i];
}

void assign_xor(const FastBitset &x, const FastBitset &y)
{
    assert(n == x.n && n == y.n);
    for (int i = 0; i < m; i++)
        a[i] = x.a[i] ^ y.a[i];
}

int count(const FastBitset &b) const
{
    int res = 0;
    for (int i = 0; i < m; i++)
        res += __builtin_popcountll(a[i] & b.a[i]);
    return res;
}

FastBitset slice(int l, int r) const {
    int len = r - l + 1;
    FastBitset res(len);

    int base = l >> 6;    // 原 bitset 中的起始块
    int off = l & 63;     // 起点在块内的偏移

    for (int i = 0; i < res.m; i++) {
        int j = base + i;

        if (off == 0) {
            res.a[i] = a[j];

```

```
    } else {  
        res.a[i] = a[j] >> off;  
  
        if (j + 1 < m) {  
            res.a[i] |= a[j + 1] << (64 - off);  
        }  
    }  
}  
  
res.trim();  
return res;  
}  
};
```


19 小波树

19.1 小波树

小波树把整数集合按照值域左右分治递归划分到叶子,查询时根据左右子树的叶子数量递归左右子树。

对于区间查询,再借助一个前缀和,求子树中有多少个落在区间中的点。

空间复杂度: $O(n \log V)$, 单次询问时间复杂度: $O(\log V)$ 。

```

struct node
{
    int pos, ls, rs;
};

struct waveTree
{
    node tr[N << 5];
    int sec[N << 5], tmp1[N], tmp2[N];
    int root, idx, pos;

    int build(int L, int R, int l, int r, vector<int> &a)
    {
        if (l > r)
            return 0;

        int now = ++idx;
        ls(now) = rs(now) = 0;
        tr[now].pos = pos;

        if (L == R)
            return now;

        int mid = (L + R) >> 1;

        int cur = 0;
        for (int i = l; i <= r; i++)
        {
            cur += (a[i] <= mid);
            sec[pos + i - l] = cur;
        }
        pos += r - l + 1;

        int cnt1 = 0, cnt2 = 0;
        for (int i = l; i <= r; i++)
        {
            if (a[i] <= mid)
                tmp1[cnt1++] = a[i];
            else
                tmp2[cnt2++] = a[i];
        }
        for (int i = 0; i < cnt1; i++)
            a[l + i] = tmp1[i];
        for (int i = 0; i < cnt2; i++)
            a[l + cnt1 + i] = tmp2[i];
        if (cnt1)
            ls(now) = build(L, mid, l, l + cnt1 - 1, a);
        if (cnt2)
            rs(now) = build(mid + 1, R, l + cnt1, r, a);
        return now;
    }
}

```

```

int kth(int p, int L, int R, int l, int r, int k)
{
    if (L == R)
        return L;

    int mid = (L + R) >> 1;
    auto sum = [&](int x) -> int
    {
        if (x <= 0)
            return 0;
        return sec[tr[p].pos + x - 1];
    };
    int al = sum(l - 1), ar = sum(r);
    int cnt = ar - al;
    if (k <= cnt)
        return kth(ls(p), L, mid, al + 1, ar, k);
    return kth(rs(p), mid + 1, R, l - al, r - ar, k - cnt);
}

void init()
{
    root = idx = pos = 0;
}
};

```

实际上不难发现,小波树递归建树的过程,和逐步将元素插入 01-Trie 的过程是类似的。

而整个分治的过程,又和整体二分的过程,又是类似的。

朴素实现的小波树,与主席树效率差别不大。

19.1.1 优化 1 - 离散化

建小波树前将值域离散化到 $O(n)$,那么叶子覆盖值域 $O(n)$,分析同线段树,只需要 $O(n)$ 的节点。

19.1.2 优化 2 - 位图

经过优化 1 之后,因为还需要维护一个前缀和,所以空间还是 $O(n \log V)$ 。

注意到这个前缀和原数组的每一个只需要维护 0/1,那么可以使用 bitset 维护,将每 64 位压缩成一个更短的数组,询问时,对于不足 64 位的部分,用一次位运算 $O(1)$ 计算 1 的数量。

结合以上两个优化,空间复杂度降为 $O(\frac{n \log n}{w})$ 。

19.2 小波矩阵

小波树是静态的,考虑不显式地把树建出来,对于每一层节点的位图,合并成整个。

这样,小波树就变成了 $O(\log V)$ 层的 $O(n)$ 的位图。

在构建和访问每一层的位图时,因为内存连续,时间效率常数更小。

且因为不需要存储儿子节点信息,只需要通过位图信息递归,因此空间复杂度得到一个机器位数的分母,为 $O(\frac{n \log V}{w})$ 。

查询操作略有变化,原理不变。

```
struct BitVector
{
    vector<u64> block;
    vector<int> sum;
    BitVector(int n)
    {
        block.assign((n >> 6) + 3, 0);
        sum.assign((n >> 6) + 4, 0);
    }
    void set(int x)
    {
        block[x >> 6] |= (1ull << (x & 63));
    }
    void build()
    {
        for (unsigned i = 0; i + 1 < sum.size(); i++)
        {
            sum[i + 1] = sum[i] + __builtin_popcountll(block[i]);
        }
    }
    int pre1(int i) const
    {
        int b = i >> 6;
        int offset = i & 63;
        u64 mask = offset == 63 ? ~0ULL : (1ULL << (offset + 1)) - 1;
        return sum[b] + __builtin_popcountll(block[b] & mask);
    }
    int pre0(int i) const
    {
        return i - pre1(i);
    }
};

struct waveMatrix
{
    int m;
    vector<BitVector> c;
    vector<int> zeros;
    void build(int n, vector<int> &a)
    {
        m = 0;
        for (int i = 1; i <= n; i++)
            m = max(m, a[i] ? __lg(a[i]) : 0);
        c.assign(m + 1, BitVector(n));
        zeros.assign(m + 1, 0);
        vector<int> cur(n + 1), tmp0(n), tmp1(n);
        for (int i = 1; i <= n; i++)
        {
            cur[i] = a[i];
        }
        for (int i = m; i >= 0; i--)
        {
            int cnt0 = 0, cnt1 = 0;
            for (int j = 1; j <= n; j++)
            {
                if ((cur[j] >> i) & 1)
                {
                    c[i].set(j);
                    tmp1[cnt1++] = cur[j];
                }
                else
                {
                    tmp0[cnt0++] = cur[j];
                }
            }
        }
    }
};
```

```

        }
    }
    c[i].build();
    zeros[i] = cnt0;
    for (int j = 0; j < cnt0; j++)
    {
        cur[j + 1] = tmp0[j];
    }
    for (int j = 0; j < cnt1; j++)
    {
        cur[cnt0 + j + 1] = tmp1[j];
    }
}
}
int kth(int l, int r, int k)
{
    int res = 0;
    for (int i = m; i >= 0; i--)
    {
        int cnt = c[i].pre0(r) - c[i].pre0(l - 1);
        if (k <= cnt)
        {
            l = c[i].pre0(l - 1) + 1;
            r = c[i].pre0(r);
        }
        else
        {
            res |= (1 << i);
            k -= cnt;
            l = zeros[i] + c[i].pre1(l - 1) + 1;
            r = zeros[i] + c[i].pre1(r);
        }
    }
    return res;
}
};

```

19.3 小波矩阵优化可持久化01-Trie

既然小波树本质和 01-Trie 几乎相同,那么小波矩阵同样可以用来优化可持久化01-Trie。(或者说小波矩阵实际上本质与可持久化 01-Trie 相同)

// 小波矩阵的实现就是通过子树 θ 的数量判断迭代方向

20 K-D Tree

K-D Tree 用于处理高维点集信息。

K-D Tree 每次选择一个坐标轴,并以该维的中位数为基准把点集分到左右子树。一个节点维护的子树对应一个轴对齐包围盒,查询时可用包围盒判定整棵子树与查询区域相交、包含或分离。

经典的正交范围报告结论需要明确前提:维数 k 是固定常数,树按中位数等方式保持平衡,并且查询区域是轴对齐的长方体。在这种模型下,报告查询的最坏时间通常写为

$$O\left(n^{1-\frac{1}{k}} + \text{output}\right),$$

其中 **output** 是实际输出的点数。若节点保存区间和等聚合信息,完全覆盖时可以对整棵子树取值,不必逐点输出,但 $O(n^{1-1/k})$ 的访问界仍依赖上述平衡、固定维数和轴对齐条件。对任意分割策略、已经失衡的动态KD-tree 或其他类型的查询,不能无条件套用该界;安全的最坏上界可能退化为 $O(n + \text{output})$,聚合查询则可能退化为 $O(n)$ 。

不带重构的在线插入只有在插入顺序可视为随机排列,并且点坐标、分割轴和相等键处理满足相应独立或一般位置假设时,才能把树高 $O(\log n)$ 作为期望结论。它不是最坏情况保证;对抗性插入顺序可以构造出 $O(n)$ 高度。因此需要重构来获得不依赖输入顺序的平衡性:

- 替罪羊树式重构:插入后若左右子树大小失衡,就重构该子树。平衡系数 α 常取 0.75;具体插入均摊复杂度还取决于一次重构的实现成本。
- 二进制分组重构:维护若干大小为 2^i 的静态平衡KD-tree,像二进制进位一样合并重建,常见实现的插入均摊复杂度为 $O(\log^2 n)$ 。对固定 $k \geq 2$,若每个大小为 2^i 的块都满足经典正交查询界,则所有块的非输出查询成本之和为

$$\sum_i O\left((2^i)^{1-\frac{1}{k}}\right) = O\left(n^{1-\frac{1}{k}}\right).$$

二维时这才特化为 $\sum_i O(\sqrt{2^i}) = O(\sqrt{n})$;更高维必须使用指数 $1 - 1/k$ 。各块实际报告的点数之和仍为总 **output**。如果维数随 n 增长,或各静态块不满足平衡构建前提,上述求和结论也不能直接使用。

K-D Tree 的实际查询效率很依赖数据分布、维数、包围盒质量和剪枝效果,竞赛中通常还需要结合题目进行常数优化。

```

struct KDTree // 同时支持 2D/3D
{
    struct node
    {
        int ls = 0, rs = 0, sz = 0, sum = 0, val = 0;
        int d[3];
        int mn[3], mx[3];
        int dim = 0;

        node()
        {
            for (int i = 0; i < 3; i++)
            {
                d[i] = 0;
                mn[i] = INF;
                mx[i] = -INF;
            }
        }
    } tr[N];

    int root, idx;
    int sec[N], cnt;
    int k;

    void init(int _k)
    {
        k = _k;
        root = idx = cnt = 0;
    }

    int make_node(int a[], int z)
    {
        ++idx;
        tr[idx] = node();
        tr[idx].ls = tr[idx].rs = 0;
        tr[idx].sz = 1;
        tr[idx].sum = tr[idx].val = z;
        tr[idx].dim = 0;
        for (int i = 0; i < k; i++)
        {
            tr[idx].d[i] = a[i];
            tr[idx].mn[i] = a[i];
            tr[idx].mx[i] = a[i];
        }
        return idx;
    }

    void push_up(int p)
    {
        auto &now = tr[p];
        now.sz = 1;
        now.sum = now.val;
        for (int i = 0; i < k; i++)
            now.mn[i] = now.mx[i] = now.d[i];

        if (ls(p))
        {
            auto &L = tr[ls(p)];
            now.sz += L.sz;
            now.sum += L.sum;
            for (int i = 0; i < k; i++)
            {
                now.mn[i] = min(now.mn[i], L.mn[i]);
                now.mx[i] = max(now.mx[i], L.mx[i]);
            }
        }
        if (rs(p))
        {
            auto &R = tr[rs(p)];
            now.sz += R.sz;
            now.sum += R.sum;
            for (int i = 0; i < k; i++)

```

```

        {
            now.mn[i] = min(now.mn[i], R.mn[i]);
            now.mx[i] = max(now.mx[i], R.mx[i]);
        }
    }
}

bool cmp(int a, int b, int d)
{
    return tr[a].d[d] < tr[b].d[d];
}

int build(int l, int r)
{
    if (l > r)
        return 0;

    int mn[3], mx[3];
    for (int i = 0; i < k; i++)
        mn[i] = mx[i] = tr[sec[l]].d[i];

    for (int i = l + 1; i <= r; i++)
    {
        auto now = sec[i];
        for (int j = 0; j < k; j++)
        {
            mn[j] = min(mn[j], tr[now].d[j]);
            mx[j] = max(mx[j], tr[now].d[j]);
        }
    }

    int d = 0;
    for (int i = 1; i < k; i++)
        if (mx[i] - mn[i] > mx[d] - mn[d])
            d = i;

    int mid = l + r >> 1;
    nth_element(sec + l, sec + mid, sec + r + 1,
        [&](int a, int b)
        {
            return cmp(a, b, d);
        });

    int now = sec[mid];
    tr[now].dim = d;
    ls(now) = build(l, mid - 1);
    rs(now) = build(mid + 1, r);
    push_up(now);
    return now;
}

void flatten(int p)
{
    if (!p)
        return;
    flatten(ls(p));
    sec[++cnt] = p;
    flatten(rs(p));
    ls(p) = rs(p) = 0;
}

void rebuild(int &p)
{
    cnt = 0;
    flatten(p);
    p = build(1, cnt);
}

bool bad(int p)
{
    return max(tr[ls(p)].sz, tr[rs(p)].sz) * 4 > tr[p].sz * 3;
}

```

```

bool flag;
int rebuild_father, rebuild_side = -1;

void insertNode(int &p, int q, int father, int side)
{
    if (!p)
    {
        p = q;
        return;
    }

    if (cmp(p, q, tr[p].dim))
        insertNode(rs(p), q, p, 1);
    else
        insertNode(ls(p), q, p, 0);

    push_up(p);

    if (bad(p))
    {
        flag = 1;
        rebuild_father = father;
        rebuild_side = side;
    }
}

void insert(int a[], int z)
{
    int p = make_node(a, z);
    flag = 0;
    rebuild_father = 0;
    rebuild_side = -1;
    insertNode(root, p, 0, -1);

    if (flag)
    {
        if (!rebuild_father)
        {
            rebuild(root);
        }
        else
        {
            if (!rebuild_side)
                rebuild(ls(rebuild_father));
            else
                rebuild(rs(rebuild_father));
        }
    }
}

void insert(int x, int y, int z)
{
    int a[3] = {x, y, 0};
    insert(a, z);
}

void insert(int x, int y, int z, int w)
{
    int a[3] = {x, y, z};
    insert(a, w);
}

bool out(int p, int l[], int r[])
{
    for (int i = 0; i < k; i++)
        if (r[i] < tr[p].mn[i] || tr[p].mx[i] < l[i])
            return 1;
    return 0;
}

bool in(int p, int l[], int r[])

```

```

{
    for (int i = 0; i < k; i++)
        if (l[i] > tr[p].mn[i] || tr[p].mx[i] > r[i])
            return 0;
    return 1;
}

bool point_in(int p, int l[], int r[])
{
    for (int i = 0; i < k; i++)
        if (l[i] > tr[p].d[i] || tr[p].d[i] > r[i])
            return 0;
    return 1;
}

int query(int p, int l[], int r[])
{
    if (!p)
        return 0;

    auto &now = tr[p];

    if (out(p, l, r))
        return 0;
    if (in(p, l, r))
        return now.sum;

    int res = 0;
    if (point_in(p, l, r))
        res += now.val;
    res += query(ls(p), l, r);
    res += query(rs(p), l, r);
    return res;
}

int query(int p, int x, int y, int X, int Y)
{
    int l[3] = {x, y, 0};
    int r[3] = {X, Y, 0};
    return query(p, l, r);
}

int query(int p, int x, int y, int z, int X, int Y, int Z)
{
    int l[3] = {x, y, z};
    int r[3] = {X, Y, Z};
    return query(p, l, r);
}
};

```


第二部分

字符串

21 字符串处理

21.1 append

1. 追加 string

```
string s1 = "Hello";
string s2 = " World";
s1.append(s2); // 追加s2的全部内容
cout << s1; // 输出: Hello World
```

2. 追加 string 子串

```
string s1 = "Hello";
string s2 = "123456789";
s1.append(s2, 2, 4); // 从s2的第2个位置(字符'3')开始,截取4个字符(3,4,5,6)
cout << s1; // 输出: Hello3456
```

3. 追加 n 个相同的字符

```
string s1 = "XXX";
s1.append(5, '!'); // 追加5个'!'
cout << s1; // 输出: XXX!!!!
```

4. 追加迭代器范围的字符

```
string s1 = "abc";
vector<char> v = {'d', 'e', 'f', 'g'};
s1.append(v.begin(), v.end()-1); // 追加 v 中从 begin() 到 end() - 1 的字符(d, e, f)
cout << s1; // 输出: abcdef
```

21.2 substr

`substr(pos, count)` 从下标 `pos` 开始提取至多 `count` 个字符。若省略 `count`, 或 `count` 超过剩余长度, 则提取到字符串末尾。 `pos == s.size()` 是合法的, 此时返回空串; 只有 `pos > s.size()` 时才会抛出 `std::out_of_range`。

```
string s = "2026-02-05";
string year = s.substr(0, 4); // "2026"
string month = s.substr(5, 2); // "02"
string day = s.substr(8); // "05" (到末尾)
```

21.3 查找、替换

`find` 和 `rfind` 在未找到目标时返回 `std::string::npos`, 使用返回位置前必须先进行判断。 `starts_with`、`ends_with` 从 C++20 开始提供, `contains` 从 C++23 开始提供。

```
string s = "C++11 is good, C++11 is powerful";

// C++98查找
size_t pos = s.find("11");           // 3 (首次出现的位置)
size_t rpos = s.rfind("11");         // 18 (最后一次出现的位置)

// C++20易用查找
bool is_prefix = s.starts_with("C++"); // true
bool is_suffix = s.ends_with("powerful"); // true

// C++23 contains
bool has_11 = s.contains("11");      // true

// 替换
if (pos != string::npos)
    s.replace(pos, 2, "23");          // 把第一个"11"替换为"23"
cout << s << endl;                  // "C++23 is good, C++11 is powerful"
```

21.4 字符串-整数转换

函数	作用
stoi()/stol()/stoll()	字符串转 int/long/long long
stof()/stod()/stold()	字符串转 float/double/long double
to_string()	数值转字符串(支持 int、float、double 等)

stoi 的完整形式为 `stoi(str, pos, base)`: `base` 指定进制,可以是 2 ~ 36,也可以取 0 让函数根据前缀自动判断进制; `pos` 的类型为 `size_t*`,非空时会写入第一个未参与转换的字符下标。

```
string bin = "1010";    // 二进制
string oct = "12";      // 八进制
string hex1 = "0xFF";   // 十六进制(带0x)
string hex2 = "a3";     // 十六进制(小写,无0x)
string base36 = "z";    // 36进制, z表示35 (a=10, ..., z=35)

int b = stoi(bin, nullptr, 2);    // 二进制转int → 10
int o = stoi(oct, nullptr, 8);    // 八进制转int → 10
int h1 = stoi(hex1, nullptr, 16); // 十六进制转int → 255
int h2 = stoi(hex2, nullptr, 16); // 十六进制转int → 163
int b36 = stoi(base36, nullptr, 36); // 36进制转int → 35

cout << b << " " << o << " " << h1 << " " << h2 << " " << b36;
// 输出: 10 10 255 163 35
```

例如, `stoi("123abc", &pos, 10)` 返回 123,并令 `pos == 3`。若要求整个字符串都是合法整数,可在转换后检查 `pos == str.size()`。

stoi 等字符串转数值函数不会用返回值表示失败,需要处理以下异常:

- 没有任何字符可以转换时,抛出 `std::invalid_argument`。
- 转换结果超出目标类型范围时,抛出 `std::out_of_range`。

安全转换示例

```
string str = "123abc";
size_t pos = 0;
try {
    int value = stoi(str, &pos, 10);
    if (pos != str.size()) {
        // str 中还包含未转换的字符
    }
} catch (const invalid_argument &) {
    // 没有可转换的整数
} catch (const out_of_range &) {
    // 结果超出 int 范围
}
```

21.5 字符分类函数

函数	作用
isupper()	判断是否是大写字母
islower()	判断是否是小写字母
isalpha()	判断是否是字母
isdigit()	判断是否是数字
isalnum()	判断是否是字母或数字
isspace()	判断是否是空白字符
ispunct()	判断是否是标点符号
isxdigit()	判断是否是十六进制数字

21.6 字符转换函数

函数	作用
toupper()	小写转大写
tolower()	大写转小写

上述 <cctype> 函数的参数必须是 EOF,或能够表示为 unsigned char 的值。若 char 默认为有符号类型,直接传入值为负数的 char 会产生未定义行为,因此应先转换为 unsigned char; toupper 和 tolower 返回 int,需要时再转换回 char。

安全用法

```
char ch = s[i];
unsigned char uch = static_cast<unsigned char>(ch);

if (isalpha(uch)) {
    ch = static_cast<char>(toupper(uch));
}
```

22 Border

Border 指:字符串的公共前后缀。

根据情境的不同, Border 可以指字符串、字符串的长度,也可以指最长的字符串、最长的字符串长度, Border 可以是其本身,也可以不含其本身。具体含义需要结合场景分析。

通常情况下, Border 均指真 Border,即不含其本身。

22.1 周期和循环节:

对于字符串 s ,若存在一个正整数 p 满足 $\forall i \in (p, |s|], s_i = s_{i-p}$,则称 p 是 s 的一个周期。

若 $p \mid |s|$,则称 p 是 s 的一个循环节。

22.2 Border vs 周期:

p 是 s 的周期 $\Leftrightarrow |s| - p$ 是 s 的 Border。

所以求 Border 和求周期等价,求最小周期即为求最大 Border。

22.3 s 所有前缀的 Border 的求法:

Border 具有传递性: s 的 Border 的 Border 还是 s 的 Border。

所以 pre_i 的 Border 为 pre_{i-1} 的某一个 Border 加一。

```
void init(string &a) { // 出于方便,考虑 border 的字符串一般下标从 1 开始
    int j = 0, n = a.size() - 1;
    for (int i = 2; i <= n; i++) {
        while (j && a[i] != a[j + 1])
            j = border[j];
        if (a[i] == a[j + 1])
            j++;
        border[i] = j;
    }
}
```

i, j 指针的增量均为 $O(n)$,所以时间复杂度为: $O(n)$ 。

23 Kmp

23.1 字符串匹配:

设文本串为 a , 模式串为 b 。

当匹配到 a_i 和 b_j 时, 若 $a_i \neq b_j$, 那么 b 重新开始匹配, 匹配到 a_i 的即为 $b_{border_{j-1}+1}$ 。

$a_i \neq b_j$ 时被称为失配, 所以 border 也被称作失配指针(fail 指针)。

```
void kmp(string &a, string &b) { // a 是文本串 b 是模式串, 在 a 上找 b
    int j = 0, n = a.size() - 1, m = b.size() - 1;
    for (int i = 1; i <= n; i++) {
        while (j && a[i] != b[j + 1])
            j = border[j];
        if (a[i] == b[j + 1])
            j++;
        if (j == m) {
            // 匹配成功
            j = border[j];
        }
    }
}
```

时间复杂度和求 Border 类似, $O(|a| + |b|)$ 。

23.2 扩展 Kmp (Z 函数):

对于一个字符串 s , 它的 Z 函数定义为 $z(i)$ 表示 s 和 suf_i 的最长公共前缀 (LCP)。

对于 i , 称区间 $[i, i + z(i) - 1]$ 是 i 的匹配段, 也可以叫 Z-box。

算法的过程中, 维护右端点最靠右的匹配段。记作 $[l, r]$ 。根据定义, $s[l, r]$ 和 s 一样前缀相同。在计算 $z(i)$ 时保证 $l \leq i$, 初始时 $l = r = 0$ 。

在计算 $z(i)$ 的过程中:

- 如果 $i \leq r$, 那么根据 $[l, r]$ 的定义有 $s[i, r] = s[i - l, r - l]$, 因此 $z(i) \geq \min(z(i - l), r - i + 1)$
 - 若 $z(i - l) < r - i + 1$, 则 $z(i) = z(i - l)$ 。
 - 否则 $z(i - l) \geq r - i + 1$, 这时令 $z(i) = r - i + 1$, 然后暴力枚举下一位匹配。
- 如果 $i > r$, 那么直接从 s_i 开始暴力匹配。
- 求出 $z(i)$ 后, 如果 $i + z(i) - 1 > r$, 更新 $[l, r]$ 。

时间复杂度分析： r 指针只会增加 $O(n)$ 次， $z(i)$ 的暴力扩展次数和 r 的变化次数相同，所以时间复杂度 $O(n)$ 。

```
void init(string &s) { // 下标从 1 开始
    int n = s.size() - 1;
    for (int i = 2, l = 1, r = 1; i <= n; i++) {
        if (i <= r && z[1 + i - 1] < r - i + 1) {
            z[i] = z[1 + i - 1];
        } else {
            z[i] = max(0, r - i + 1);
            while (i + z[i] <= n && s[1 + z[i]] == s[i + z[i]])
                ++z[i];
        }
        if (i + z[i] - 1 > r)
            l = i, r = i + z[i] - 1;
    }
    z[1] = n;
}
```

更一般地，利用 b 的 Z 函数可用于求 b 和 a 的每一个后缀的 LCP。

```
void exkmp(string &a, string &b) {
    int n = a.size() - 1, m = b.size() - 1;
    for (int i = 2, l = 1, r = 1; i <= n; i++) {
        if (i <= r && z[1 + i - 1] < r - i + 1) {
            Z[i] = z[1 + i - 1];
        } else {
            Z[i] = max(0, r - i + 1);
            while (1 + Z[i] <= m && i + Z[i] <= n && b[1 + Z[i]] == a[i + Z[i]])
                ++Z[i];
        }
        if (i + Z[i] - 1 > r)
            l = i, r = i + Z[i] - 1;
    }
    Z[1] = 0;
    while (1 + Z[1] <= m && 1 + Z[1] <= n && b[1 + Z[1]] == a[1 + Z[1]])
        ++Z[1];
}
```

23.3 Kmp 自动机 / Border 树(失配树):

对于一个字符串 s ，它的 Border 树共有 $n + 1$ 个点： $0, 1, \dots, n$ 。点 i 的父节点 $border_i$ 表示前缀 pre_i 的最长真 Border 长度。节点 0 表示空前缀，只作为树根和无 Border 时的哨兵，不计入非空真 Border。

- 前缀 pre_i 的所有真 Border：从节点 $border_i$ 到根的链，其中不计节点 0。
- 对于 $x > 0$ ，以长度 x 为真 Border 的前缀：节点 x 的所有真后代，不包括节点 x 自身。
- 对于 $p, q > 0$ ，两个前缀的最长公共真Border： $LCA(border_p, border_q)$ ；结果为 0 表示不存在非空公共真Border。

也可以认为是只有一个串的 AC 自动机。

24 Hash

此处 Hash 指字符串哈希,只用来判断两个字符串是否相同。

字符串哈希,一般使用多项式哈希,将字符串 s 映射到整数 $\sum s_i \times base^i$ 上。

24.1 单哈:

将字符串 s 映射到整数 $\sum s_i \times base^i \bmod p$ 上。

在把不同字符串的哈希值近似看作独立、均匀分布于 $[0, p)$ 的前提下,对 q 个不同字符串计算哈希,其中至少出现一对碰撞的概率近似为

$$1 - \exp\left(-\frac{q(q-1)}{2p}\right).$$

这里描述的是 q 个哈希值中“任意一对”发生碰撞的概率,不是达到 $\sqrt{p}$ 次后每次比较都有一半概率冲突。单独比较一对不同字符串时,误判概率近似为 $1/p$;上述整体碰撞概率约为 $1/2$ 时, $q \approx \sqrt{2p \ln 2} \approx 1.177\sqrt{p}$ 。

例如,对于大小约为 2^{32} 的哈希空间,生日界的数量级为 $\sqrt{2^{32}} = 2^{16} = 65536$;此时出现任意碰撞的概率约为 $1 - e^{-1/2} \approx 39.3\%$,而不是 50% 。实际使用 `int` 模数时应将具体的 p 代入公式。若选取接近 `long long` 上界的模数,两个余数相乘可能超过 `long long`,需要使用 `__int128` 或其他安全的模乘方法。

24.2 自然溢出:

自然溢出是指使用 `unsigned long long` 进行运算,结果会自动对 2^{64} 取模。若仍采用独立均匀的随机模型, $q = 2^{32} = 4294967296$ 个哈希值中出现任意碰撞的概率约为 $1 - e^{-1/2} \approx 39.3\%$;概率达到约 50% 时需要 $q \approx \sqrt{2 \ln 2} \cdot 2^{32}$ 。这种写法不需要 `__int128`,并且运算速度较快。

但是由于模数的特殊性,非常容易构造哈希冲突。

24.3 双哈:

将字符串 s 映射到两个模数下的哈希值组成的二元组。

若两个哈希使用相同的底数和字符映射,先在整数上定义同一个多项式 $H(s)$,

再分别计算 $H(s) \bmod p_1$ 和 $H(s) \bmod p_2$, 并且 p_1, p_2 互质, 则根据中国剩余定理, 这一对余数与 $H(s) \bmod (p_1 p_2)$ 一一对应。此时双哈碰撞等价于两个字符串的整数多项式之差能被 $p_1 p_2 = \text{lcm}(p_1, p_2)$ 整除。

若两个哈希使用不同底数或不同字符映射, 它们不再是同一个整数多项式在不同模数下的余数, 因此不能使用上述CRT 等价关系。只有在把两个哈希近似看作相互独立且均匀分布时, 才能估计一对不同字符串同时碰撞的概率约为 $1/(p_1 p_2)$, 而 q 个哈希值中出现任意双哈碰撞的概率近似为

$$1 - \exp\left(-\frac{q(q-1)}{2p_1 p_2}\right).$$

双哈只能显著降低随机数据下的碰撞概率, 不能保证完全无碰撞; 面对可能刻意构造的数据, 还需要随机化底数、模数或字符映射。

最简单的随机化方式是为每个字符随机映射一个权值, 而不是固定使用 s_i ; 也可以在哈希开始前随机选取合适的底数和模数。

```
const int Base[4] = {3333, 13331, 131, 13331},
    mod[4] = {1610612741, 1061067769, (int)1e9 + 7, (int)1e9 + 9};
int m1 = 0, m2 = 1;
void init(int n) {
    base[0][0] = base[0][1] = 1;
    for (int i = 1; i <= n; i++) {
        base[i][0] = 111 * base[i - 1][0] * Base[m1] % mod[m1];
        base[i][1] = 111 * base[i - 1][1] * Base[m2] % mod[m2];
    }
}
void init(int a[], int n) {
    for (int i = 1; i <= n; i++) {
        sum[i][0] = (111 * sum[i - 1][0] * Base[m1] + a[i]) % mod[m1];
        sum[i][1] = (111 * sum[i - 1][1] * Base[m2] + a[i]) % mod[m2];
    }
}
u64 sum_hash(int l, int r) {
    int now = (sum[r][0] - 111 * sum[l - 1][0] * base[r - l + 1][0]) % mod[m1];
    int cur = (sum[r][1] - 111 * sum[l - 1][1] * base[r - l + 1][1]) % mod[m2];
    if (now < 0) // 注意负数不能 u64
        now += mod[m1];
    if (cur < 0)
        cur += mod[m2];
    return (1ull * now) << 32 | cur;
}
```

注: 因为 `pair<int,int>` 是两个 `int`, 若要使用 `map` 储存字符串的哈希值, 将 `pair<int,int>` 压成 `long long` 后比较哈希值的常数会更小。

25 Manacher

可以用回文中心加回文半径的方式描述一个字符串的回文子串。

定义 $len_{1,i}$ 表示以 s_i 为中心的且回文串长度为奇数最长回文半径, $len_{0,i}$ 表示以 s_i 为中心且回文串长度为偶数的最长回文半径。

Manacher 算法的过程就是求出这两个数组的过程,为了简化问题,可以在原字符串的两个相邻字符之间插入一个相同的特殊字符(除首尾),然后只需要维护回文串长度为奇数的最长回文半径即可。下方代码使用 \$、#、^ 分别作为左哨兵、分隔符和右哨兵;这三个字符必须互不相同,并且都不能出现在输入字符串中。

和 Z 函数类似,维护一个右端点最靠右的回文区间 $[l, r]$, $mid = \lfloor \frac{l+r}{2} \rfloor$ 。

- 若 $i < r$, 先令 $len_i = \min(len_{mid-(i-mid)}, r-i)$ 。其中 $mid-(i-mid)$ 是 i 关于 mid 的镜像位置,取最小值是为了保证初始回文半径不越过当前右边界 r 。
- 若 $i \geq r$, 令 $len_i = 1$ 。
- 然后暴力扩展 len_i , 更新 $[l, r]$ 。

时间复杂度分析同 Z 函数, r 指针只变化了 $O(n)$ 次,时间复杂度: $O(n)$ 。

```
vector<vector<int>> manacher(string &s) {
    int n = s.size();

    vector<char> c(2 * n + 3);
    c[0] = '$', c[1] = '#';
    for (int i = 1; i <= n; i++) {
        c[1 + 2 * i - 1] = s[i - 1];
        c[1 + 2 * i] = '#';
    }
    c[2 * n + 2] = '^';

    int m = 2 * n + 2;
    int idx0 = 0, idx1 = 0;
    vector<int> A(m + 1);
    vector<int> len0(n + 1), len1(n + 1); // 这里为了保持长度一致, len0 实际上多了一个末尾的 0, len0
    的有效元素应为 len0[1] 到 len0[n - 1]
    int r = 0, mid;
    for (int i = 1; i < m; i++) {
        if (i < r)
            A[i] = min(A[2 * mid - i], r - i);
        else
            A[i] = 1;
        while (c[i - A[i]] == c[i + A[i]])
            A[i]++;
        if (i + A[i] > r)
            r = i + A[i] - 1, mid = i;
        if (!(i & 1))
            len1[++idx0] = A[i] >> 1;
        if ((i & 1) && i > 1)
            len0[++idx1] = A[i] >> 1;
    }
    return {len0, len1};
}
```


26 Trie

字典树,用于维护一个字符串集合,查询字符串。

字典树上一条边表示一个字符,把一个字符串按顺序按边插入树上。

Trie 上一个节点表示一个前缀,所以也可以称作前缀树。

特别地,若字符集只含 01,为 01-Trie,一般用于解决 01 串的字典序或计数问题。

```
struct Trie {
    int root, son[N][26], idx;
    bool cnt[N];
    void init() {
        for (int i = 0; i <= idx; i++)
            memset(son[i], 0, sizeof son[i]);
        for (int i = 0; i <= idx; i++)
            cnt[i] = 0;
        root = idx = 0;
    }
    void insert(int &p, string &s, int x) {
        if (!p)
            p = ++idx;
        if (x == s.size()) {
            cnt[p] = 1;
            return;
        }
        insert(son[p][s[x] - 'a'], s, x + 1);
    }
    int query(int p, string &s, int x) {
        if (x == s.size())
            return p;
        return query(son[p][s[x] - 'a'], s, x + 1);
    }
};
```


27 ACAM

即 AC 自动机。

ACAM 是 Kmp 和 Trie 的结合,用于解决一个在文本串匹配若干个模式串(字典)的问题。

简单来说就是在树上维护 Border。

Trie 上一个节点表示一个前缀, $border_i$ 对应 Trie 上一个节点,表示 $border_i$ 对应的前缀是 i 对应的前缀的最长后缀(注意此处的 Border 和 Kmp 单串 Border 的区别,此处的 Border 不要求是 i 对应前缀的前缀)。

求法也和单串的 Border 类似, $border_i = border_{fa_i}$ 往后扩展 s_i ,其中 fa_i 是 i 在 Trie 上的父亲。若 $border_{fa_i}$ 后不存在 s_i ,则继续迭代更小的 Border。

因为 $border_i$ 要求祖先 Border 已求出,所以要使用 BFS 求 Border。

时间复杂度分析同 KMP,指针的增量为 Trie 节点数。

时间复杂度: $O(\sum |T|)$ 。

匹配时,也和 Kmp 类似,在 Trie 上迭代 S Border 即可。

时间复杂度: $O(|S|)$ 。

区别是,对于文本串 S 在 Trie 上的当前位置 x ,其和字典 T 的匹配串集合是 x 到 fail 树根这条路径上的所有字符串。

注: ACAM 因为要跳 Border 的特殊性,一般将 ACAM 中 Trie 的根设为 0。

```
struct Trie {
    /*
     * 其它部分的 Trie 略
     */
    void insert( int &p, string &s, int x) {
        if (!p && x) // p = 0 为根
            p = ++idx;
    }
};

struct ACAM {
    Trie trie;
    int border[N];
    void init() {
        for (int i = 0; i <= trie.idx; i++)
            border[i] = 0;
        trie.init();
    }
    void build() {
        queue<int> q;
        for (int i = 0; i < 26; i++)
            if (trie.son[0][i])
                q.push(trie.son[0][i]);
        while (q.size()) {
            auto now = q.front();
            q.pop();
            for (int i = 0; i < 26; i++) {
                if (trie.son[now][i]) {
                    int cur = border[now];

```

```

        while (cur && !trie.son[cur][i]) {
            cur = border[cur];
        }
        border[trie.son[now][i]] = trie.son[cur][i];
        q.push(trie.son[now][i]);
    }
}
}
}
void solve(string &s){
    int now = 0;
    for (auto u : s) {
        while (now && !trie.son[now][u - 'a']) {
            now = border[now];
        }
        now = trie.son[now][u - 'a'];
        if (now)
            vis[now]++;
    }
}
};

```

27.1 Trie 图优化

在 AC 自动机 Trie 上维护 Border 时,需要不断跳 Border 直到匹配成功,注意到一个点的 border 是唯一的,那么一个点匹配一个字符所跳Border 的过程也是唯一的,那么可以将这一个过程进行路径压缩,压到 $trans[x][u]$ 中去,在后续可以快速得到一个节点匹配成功的Border,可用于优化 dp。

(其实严格意义上,只有补全了才能叫“自动机”)

若预处理 $trans[x][u]$ 表示 Trie 上 x 这个节点,接下去匹配 u 这个字符时会跳到的最终节点(匹配成功)。

- 如果 x 的 u 后继边有节点,则 $trans[x][u] =$ 对应的后继节点。
- 否则, $trans[x][u] = trans[Border_x][u]$ 。

在实际应用中, $trans$ 数组可以和 Trie 的 son 共用,因为 son 非空时, $trans = son$; son 为空时,可用 $trans$ 填补 son ,后续将 son 视作 $trans$ 即可。(用节点上的标记数组可以处理原字典串)

此时,找到失配后匹配成功的最大 Border 只需要一次了。

但是因为 $trans$ 需要预处理出所有节点的所有字符的失配后成功匹配的节点,所以时空复杂度均为 $O(\sum |T||c|)$,其中 $|c|$ 表示字符集大小。

```

// 补全 Trie 图
for (int i = 0; i < 26; i++) {
    auto &u = trie.son[now][i];
    if (!u) {
        u = trie.son[border[now]][i];
    } else {
        border[u] = trie.son[border[now]][i];
        q.push(u);
    }
}
// 匹配

```



```
int now = 0;
for(auto u : s){
    now = trie.son[now][u - 'a'];
}
```


28 PAM

又称作:回文自动机、回文树。

与 Manacher 不同, PAM 用最长回文后缀的方式描述一个字符串的回文子串。

PAM 是一个树形结构, PAM 上一个一个节点描述一个回文子串。令当前节点为 x , 其到根路径上经过的边, 按从根到 x 的顺序构成的字符串为 s , 则 x 描述的回文串为 $s's$, 其中 s' 表示 s 翻转后的结果。

当在 PAM 中加入 s_i 时, 以 s_i 结尾的最长回文后缀一定是以 s_{i-1} 结尾的回文后缀扩展一位得到的。而以 s_{i-1} 结尾的回文后缀, 很巧妙地, 是以 s_{i-1} 结尾的最长回文后缀的 Border。所以求 s_i 的最长回文后缀的过程和 ACAM 上跳 Border 的过程类似。

和 Manacher 类似的, 偶数长度的回文子串需要和奇数长度的回文子串分类讨论, 不区分的话, PAM 上同一个 s 不能区分是奇数长度的回文子串还是偶数长度的回文子串。

和 ACAM 类似地, 跳 Border 匹配时, 可以使用 Trie 图优化, 时间复杂度和 ACAM 分析相同。

28.1 实现约定

下方代码使用两个特殊根节点统一处理奇回文和偶回文:

- 节点 0 是偶根, 满足 $\text{len}[0] = 0$, 表示空回文串。
- 节点 1 是奇根, 满足 $\text{len}[1] = -1$, 是用于统一转移的虚拟节点, 不表示真实回文串。
- 代码中的 `border` 即通常所说的 fail 指针。对于普通节点 $x \geq 2$, $\text{border}[x]$ 指向 x 所表示回文串的最长真回文后缀; 两个根采用特殊约定 $\text{border}[0] = 1$, $\text{border}[1] = 0$ 。

这两个根之间的指向只服务于插入过程, 不能视为普通 fail 链反复遍历。单独沿 fail 链枚举回文后缀时, 应在到达根节点后停止。getborder 能够终止, 是因为跳到奇根后 $\text{len}[1] = -1$, 比较位置变为 $s[i]$ 与其自身。

字符串采用从 1 开始的下标: $s[1]$ 到 $s[n]$ 是实际输入, $s[0]$ 必须预先放置一个不会出现在输入中的哨兵字符, 例如 '#', 因此可以使用 `string s = "#" + input` 构造传入字符串。代码中的 $n = s.size() - 1$ 会排除该哨兵; 若直接传入普通的

0-based 字符串,就会漏处理字符并破坏边界匹配。

当前转移数组为 `son[][26]`, 字符编号使用 `c = s[i] - 'a'`, 所以实际输入只能包含 'a' 到 'z', 并分别映射到 0 到 25。若字符集不同,必须先离散化,并相应调整转移数组大小。

朴素跳 Border, 时间复杂度: $O(|S|)$ 。

```
struct PAM {
    int son[N][26], border[N], len[N];
    int num[N], cnt[N];
    int idx;
    void init() {
        for (int i = 0; i <= idx; i++)
            for (int j = 0; j < 26; j++)
                son[i][j] = 0;
        for (int i = 0; i <= idx; i++)
            border[i] = len[i] = 0;
        len[1] = -1;
        idx = 1;
        border[0] = 1;
    }

    void build(string &s) {
        int n = s.size() - 1;
        int cur = 0;
        for (int i = 1; i <= n; i++) {
            auto getborder = [&](int x) -> int {
                while (s[i - 1 - len[x]] != s[i])
                    x = border[x];
                return x;
            };
            int c = s[i] - 'a';
            cur = getborder(cur);
            if (!son[cur][c]) {
                idx++;
                len[idx] = len[cur] + 2;
                border[idx] = son[getborder(border[cur])][c];
                son[cur][c] = idx;
            }
            cur = son[cur][c];
        }
    };
};
```

补全 Trie 图, 时间复杂度: $O(|S||c|)$ 。

```
int c = s[i] - 'a';
int pos = s[(i - 1) - len[cur]] == s[i] ? cur : trans[cur][c];
if (son[pos][c])
    cur = son[pos][c];
else {
    son[pos][c] = ++idx;
    cur = idx;
    border[cur] = pos == 1 ? 0 : son[trans[pos][c]][c];
    len[cur] = len[pos] + 2;
    num[cur] = num[border[cur]] + 1;
    for (int j = 0; j < 26; j++) {
        if (len[cur] == 1)
            trans[cur][j] = j == c ? 0 : 1;
        else
            trans[cur][j] = s[i - len[border[cur]]] - 'a' == j
                ? border[cur]
                : trans[border[cur]][j];
    }
}
```

28.2 本质不同回文子串：

字符串的所有回文子串都能在 PAM 上找到,而 PAM 的节点数最多只有 $O(n)$ 个,所以字符串的本质不同回文子串只有 $O(n)$ 个。

因为这个性质,所以事实上在不强制在线增量维护回文子串时,可以使用回文中心 + 回文半径的方式暴力求出所有回文子串。

具体而言,枚举回文中心,从最长回文半径开始向内收缩,若当前收缩到的位置是已经出现的,则结束收缩。在这个过程中,收缩的次数等于本质不同回文子串的数量,而判断回文子串是否出现过可以使用 Hash 判断,对于 Hash 映射后的整数,若使用 map 维护,整个过程的时间复杂度是 $O(n \log n)$,若使用哈希表维护,整个过程的时间复杂度是 $O(n)$ 。但是常数不小。

在题目不严格卡 $O(n \log n)$ 做法,不强制在线增量,且题目只和本质不同的回文子串相关,而和回文子串的 endpos 无关时,可以平替(不过实际上貌似并不会更好写,只是不用学 PAM 也可以做)。

28.3 Palindrome Series:

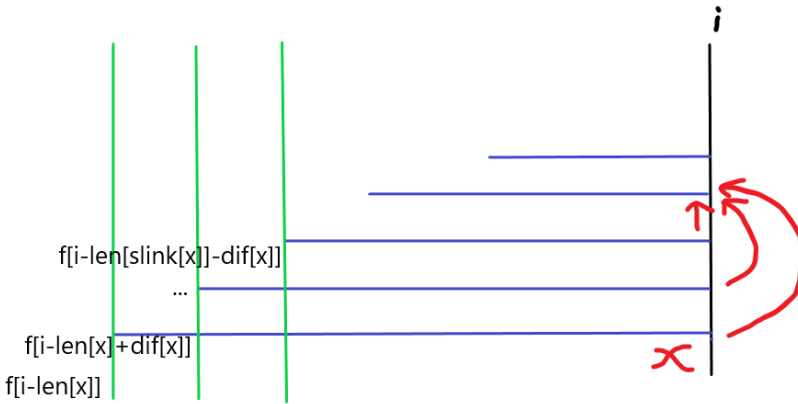
用于优化枚举回文后缀转移的 dp。朴素枚举的时间复杂度是 $O(n^2)$ 的,因为以 s_i 结尾的所有回文后缀是以 s_i 结尾的所有 Border,所以可以利用 Border 的性质优化到 $O(n \log n)$ 。

通常而言,此题 dp 问题的 dp 式子形如: $dp(i) \leftarrow dp(j), s[j+1, i]$ 是回文串。

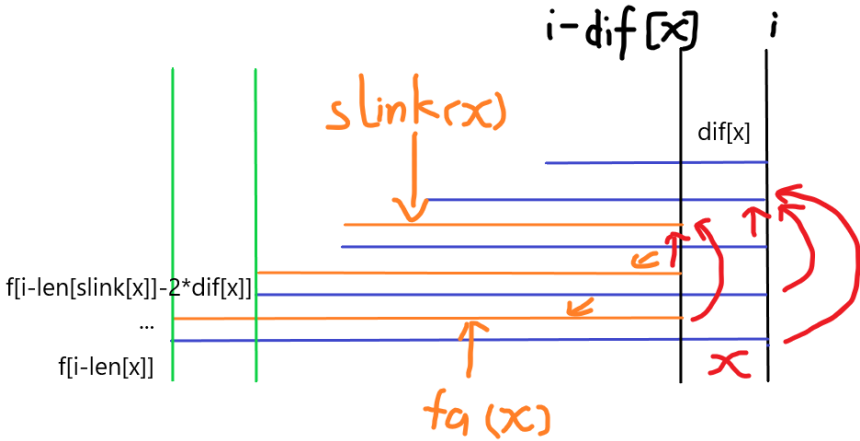
具体而言:

对于 $s[1, i]$ 的回文后缀,令 $dif(x)$ 表示 PAM 上 x 节点和 x 的最长回文后缀的长度之差(也就是和 fail 树上父节点的 len 之差)。对于 $s[1, i]$ 的连续回文后缀,将连续的一段相同的 $dif(x)$ 的节点(也就是 fail 树上到根的 fail 链上 dif 连续相等的一段)看成一个整体一起计算,令 x 到根的第一段连续 dif 相等的 dp 值之和为 $g(x)$ 。假设这个已经被处理好了,可以证明所有节点到根的 g 的数量之和是 $O(n \log n)$ 级别的。

关于 g 的计算:



令 $slink(x)$ 表示 x 到根路径上下一个 g 的开始节点。如上图,绿色所对应状态是 $g(x)$ 所需要的状态。



因为回文后缀的特殊性质,对于一个回文串的最大回文后缀同时也是这个回文的Border,所以 $i - dif(x)$ 所对应的节点到根的路径上,与 i 相比只多了一个 $dp(i - len(slink(x)) - dif(x))$ 。所以 $g(i)$ 只比 $g(i - dif(x))$ 多一个 $dp(i - len(slink(x)) - dif(x))$ 即可 $O(1)$ 转移。

注意特判 $i - dif(x)$ 是 $slink(x)$ 的情况, $slink(x)$ 算作下一个点,所以不计算到答案。

```

void build(string &s) {
    int cur = 0, pos;
    int n = s.size() - 1;
    for (int i = 1; i <= n; i++) {
        int c = s[i] - 'a';
        pos = s[(i - 1) - len[cur]] == s[i] ? cur : trans[cur][c];
        if (son[pos][c])
            cur = son[pos][c];
        else {
            son[pos][c] = ++idx;
            cur = idx;
            border[cur] = pos == 1 ? 0 : son[trans[pos][c]][c];
            int fa = border[cur];
            len[cur] = len[pos] + 2;
            /*
                diff[cur] = len[cur] - len[fa];
                slink[cur] = diff[fa] == diff[cur] ? slink[fa] : fa;
            */
            for (int j = 0; j < 26; j++) {
                if (len[cur] == 1)
                    trans[cur][j] = j == c ? 0 : 1;
                else
                    trans[cur][j] = s[i - len[border[cur]]] - 'a' == j
                        ? border[cur]
                        : trans[border[cur]][j];
            }
        }
    }
}

```

对 dp 式: $f[i] = \sum_j f[j-1], s[j, i] \text{ is Palindrome}, j \equiv i \pmod{2}$, 的优化转移:

```

f[0] = 1;
for (int i = 1, j; i <= n; i++) {
    for (j = a[i]; j > 1; j = slink[j]) {
        int dlt = len[slink[j]] + diff[j];
        g[j] = f[i - dlt];
        if (border[j] != slink[j])
            g[j] = (g[j] + g[border[j]]) % mod;
        if (i % 2 == 0)
            f[i] = (f[i] + g[j]) % mod;
    }
}

```


29 SAM

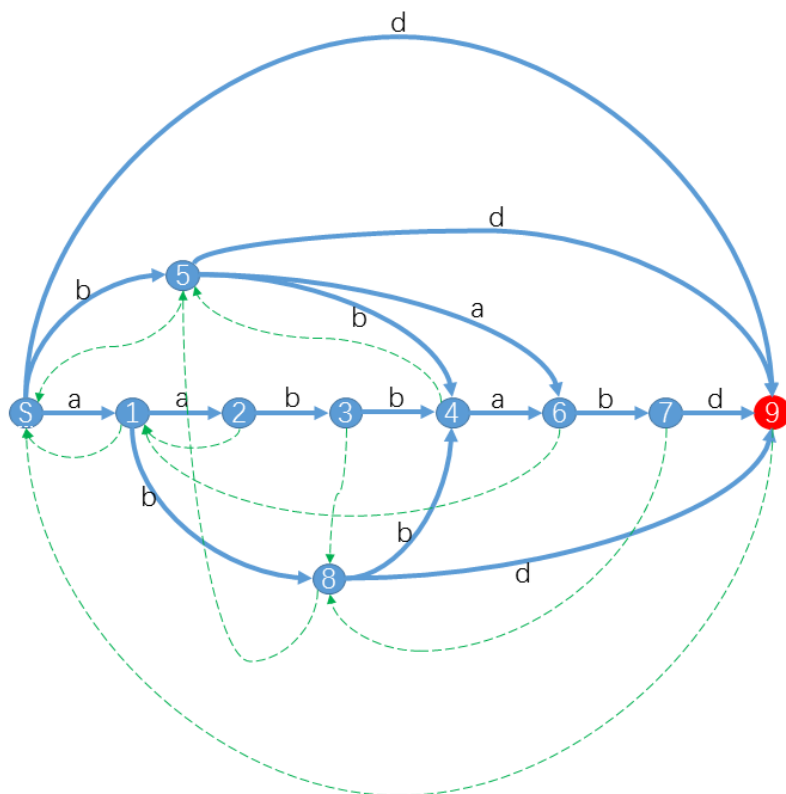
即后缀自动机。

SAM 主要用于表示一个字符串的所有子串,并支持子串存在性等询问。

“SAM 是最小 DFA”的标准表述是:对于固定字符串 S , SAM 是识别 S 的所有后缀的最小不完整确定有限状态自动机。根节点是初始状态,完整字符串对应的 **last** 状态及其 suffix link 祖先是接受状态;若把空后缀也计入语言,则根节点也是接受状态。不存在的转移表示拒绝,若要求完整 DFA,还需额外加入一个拒绝状态。

从根出发沿字符转移能够读出的字符串恰好是 S 的子串,因此判断一个字符串是否为子串时,只需检查对应路径是否存在。若把所有可达状态都设为接受状态,这张转移图也能识别所有子串,但此时不能直接沿用“最小 DFA”的结论。讨论 DFA 时必须明确所识别的语言和接受状态。

比如对于字符串 $S = aabbabd$,它的 SAM 的是:



29.1 Trie 图:

简单来说, SAM 上有两类边。第一类是字符转移边(上图的蓝边),它们形成一个 DAG,根节点表示空串。从根开始的每条路径所拼出的字符串都是 S 的子串;由于转移是确定的,每个子串对应唯一的一条路径,但多个不同子串可以结束在同一个状态。

对于子串 t , 定义 $endpos(t)$ 为 t 在 S 中所有出现位置的右端点集合。两个子串的 $endpos$ 集合完全相同时,它们属于同一个 $endpos$ 等价类; SAM 的每个状态恰好对应一个 $endpos$ 等价类。

转移 $son[x][c] = y$ 表示:当前路径对应的字符串末尾追加字符 c 后,自动机到达状态 y 。它不表示状态 x 的字符串集合是状态 y 的子集,也不能直接推出两个状态的 $endpos$ 包含关系; $endpos$ 的包含关系由 suffix link 描述。

空间复杂度: $O(|S||c|)$, 其中 $|c|$ 表示字符集大小。

29.2 fail 树:

有的地方也称作: slink 树、parent 树、后缀链接树。

第二类边是 suffix link, 代码中记为 `fail`。对于根以外的状态 v , `fail[v]` 指向 v 的最长代表串的最长真后缀中, $endpos$ 等价类与 v 不同的那个状态。因此有严格包含关系

$$endpos(v) \subsetneq endpos(fail(v)).$$

如上图, 状态 4 表示的等价类为 $\{aabb, abb, bb\}$, 它的 suffix link 指向表示 $\{b\}$ 的状态 5。每个非根状态只有一条 suffix link, 且最终都会到达根, 因此反向连接这些边会形成 fail 树。

令 $maxlen(v)$ 表示状态 v 中最长字符串的长度, 则该状态中的所有字符串是某个最长代表串的后缀, 其长度恰好构成区间

$$[minlen(v), maxlen(v)].$$

区间左端点由 suffix link 唯一确定:

$$minlen(v) = maxlen(fail(v)) + 1.$$

因此构造时只需显式维护代码中的 `maxlen`。沿某个状态的 suffix link 向根移

动,可以依次得到 endpos 集合逐渐扩大的后缀等价类;结合每个状态的长度区间,才能还原其代表的所有连续后缀。

SAM 是增量构造,可以对一个字符串在线地逐步加入字符构造SAM。至于构造部分原理,和做题无关,略。

时间复杂度: $O(n)$ 。

```
struct SAM {
    int maxlen[N], fail[N], son[N][26];
    int idx = 1;
    void extend(string &s) {
        int cur = 1, p, np;
        for (auto u : s) {
            int c = u - 'a';
            p = cur;
            cur = ++idx;
            np = cur;
            maxlen[np] = maxlen[p] + 1;
            for (; p && !son[p][c]; p = fail[p])
                son[p][c] = np;
            if (!p)
                fail[np] = 1;
            else {
                int q = son[p][c];
                if (maxlen[q] == maxlen[p] + 1)
                    fail[np] = q;
                else {
                    int nq = ++idx;
                    maxlen[nq] = maxlen[q], fail[nq] = fail[q];
                    memcpy(son[nq], son[q], sizeof son[nq]);
                    maxlen[nq] = maxlen[p] + 1;
                    fail[q] = fail[np] = nq;
                    for (; p && son[p][c] == q; p = fail[p])
                        son[p][c] = nq;
                }
            }
        }
    }
};
```

29.2.1 endpos 与出现次数:

fail 树中,后代状态的 endpos 集合包含于祖先状态,但这些集合彼此可能重叠,不能把子树内各状态的 endpos 集合大小直接相加。正确的集合表述是:对于每个前缀 $S[1, i]$,把右端点 i 记录在该前缀对应的状态 last_i 上,则 $\text{endpos}(v)$ 恰好由 fail 子树 v 内所有这些右端点组成。

统计出现次数时,可以在加入第 i 个字符所创建的非 clone 状态上令 $\text{occ}[\text{np}] = 1$, clone 状态令 $\text{occ}[\text{nq}] = 0$ 。构造完成后按 maxlen 从大到小遍历状态,并执行 $\text{occ}[\text{fail}[v]] += \text{occ}[v]$,最终 $\text{occ}[v]$ 就是 $|\text{endpos}(v)|$,也就是状态 v 中每个字符串的出现次数。

29.3 广义 SAM

SAM 通常针对单个字符串构造,广义 SAM 则表示多个字符串的子串集合。此时应把 `endpos` 定义为二元组“字符串编号、串内结束位置”,避免混淆不同字符串中的相同位置编号。构造时还必须阻止跨越两个字符串边界的子串进入自动机。

常见构造方式有两种:

- **逐串构造**:处理每个新字符串前把 `last` 重置为根,再逐字符扩展。所用扩展函数必须能够正确处理自动机中已经存在的转移,不能未经说明就把只证明过单串构造的模板重复调用。
- **Trie 上构造**:先把所有字符串插入 Trie,再按 Trie 的父子转移构造 SAM。相同前缀只在 Trie 中保存一次,适合字典规模的多串构造。

上方 `extend` 代码只说明从空 SAM 构造一个字符串的情形;若要实现广义 SAM,应根据所选方案另行给出初始化、扩展顺序和出现次数统计规则。

30 SA

即:后缀数组。

SA 和 SAM 相似,用于识别子串。

SA 将子串表示成某个后缀的前缀, SAM 则是将子串表示成某个前缀的后缀。

后缀排序是指将所有后缀 $suf[i]$ 看作独立的串,放在一起按照字典序进行升序排序。

后缀排名: $rk[i]$ 表示后缀 $suf[i]$ 在后缀排序中的排名。

后缀数组: $sa[i]$ 表示排名第 i 小的后缀。

即: $rk[sa[i]] = i$ 。

对于后缀排序的实现,略。最优可以做到 $O(n)$, 通常使用倍增 $O(n \log n)$ 。

但是空间复杂度: $O(n)$ 。

30.1 height 数组:

求出 SA 后,求 $suf[sa[i]]$ 和 $suf[sa[j]]$ 的 LCP, 等价于求:

$$\min\{Lcp(suf[sa[i]], suf[sa[i] + 1], \dots, suf[sa[j] - 1], suf[sa[j]])\}$$

令 $h[i] = Lcp(suf[sa[i - 1]], suf[sa[i]])$, 特别地 $h_1 = 0$ 。求 $suf[sa[i]]$ 和 $suf[sa[j]]$ 的 LCP 则等价于求:

$$\min\{h_{i+1}, \dots, h_j\}$$

至于 height 数组的求法,也略。是 $O(n)$ 的。

```
struct SA {
    int sa[N], h[N], rk[N], oldrk[N << 1], id[N], key1[N], cnt[N];
    int n;
    char s[N];
    bool cmp(int x, int y, int w) {
        return oldrk[x] == oldrk[y] && oldrk[x + w] == oldrk[y + w];
    }
    void init(int n, string &c) {
        for (int i = 1; i <= this->n; i++)
            s[i] = sa[i] = h[i] = 0;
        this->n = n;
        for (int i = 1; i <= n; i++)
            s[i] = c[i - 1];
        for (int i = 1; i <= 26; i++)
            cnt[i] = 0;
```

```

}
void get_sa() {
    int i, m = 26, p, w;
    for (int i = 1; i <= n; i++)
        ++cnt[rk[i]] = (s[i] - 'a' + 1);
    for (int i = 1; i <= m; i++)
        cnt[i] += cnt[i - 1];
    for (int i = n; i >= 1; i--)
        sa[cnt[rk[i]]--] = i;
    for (w = 1; w <= 1, m = p) {
        p = 0;
        for (int i = n; i >= n - w + 1; i--)
            id[++p] = i;
        for (int i = 1; i <= n; i++)
            if (sa[i] > w)
                id[++p] = sa[i] - w;
        for (int i = 1; i <= m; i++)
            cnt[i] = 0;
        for (int i = 1; i <= n; i++)
            ++cnt[key1[i] = rk[id[i]]];
        for (int i = 1; i <= m; i++)
            cnt[i] += cnt[i - 1];
        for (int i = n; i >= 1; i--)
            sa[cnt[key1[i]]--] = id[i];
        memcpy(olldrk + 1, rk + 1, n * sizeof(int));
        p = 0;
        for (int i = 1; i <= n; i++)
            rk[sa[i]] = cmp(sa[i], sa[i - 1], w) ? p : ++p;
        if (p == n)
            break;
    }
}
void get_h() {
    int k = 0;
    for (int i = 1; i <= n; i++) {
        if (rk[i] == 1) {
            h[1] = 0;
            k = 0;
            continue;
        }
        if (k)
            --k;
        while (s[i + k] == s[sa[rk[i] - 1] + k])
            ++k;
        h[rk[i]] = k;
    }
}
};

```

第三部分

数学

数论

本章组收录“数学”中的数论专题。

31 质数

即：素数。

31.1 哥德巴赫猜想

任意一个大于 2 的偶数都可以拆分成两个质数之和。

31.2 波利尼亚克猜想

对任意正整数 k , 存在无穷多对素数 $(p, p + 2k)$, 使得：

$$p \text{ is prime } \wedge p + 2k \text{ is prime}$$

31.3 质数距离：

$[1, n]$ 中的质数数量 $\pi(n) \sim \frac{n}{\ln n}$, 即相邻质数距离约为 $O(\ln n)$ 。

31.4 素性测试

即：判断一个数是否是质数。

31.4.1 试除法：

$a \mid n \rightarrow p_a \mid n$, 其中 p_a 是 a 的质因子。

那么只需要考虑 x 的质因子, 质数筛预处理 $O(\sqrt{n})$ 范围内的质数, 时间复杂度 $O(\frac{\sqrt{n}}{\ln n})$ 。

31.4.2 Miller-Rabin

Miller-Rabin 原本是概率性素性测试, 随机选取 k 个底数时, 时间复杂度为 $O(k \log n)$ 次模乘。对于有界整数, 可以使用经过证明的固定底数集合, 将其变成该值域内的确定性测试; 固定底数必须原样使用, 不能再根据 n 映射成其他底数。

```

int mul(int x, int y,
        int mod) { // 当前模板处理 32 位 int; 若改为 Long Long, 需要使用 __int128
    return 1LL * x * y % mod;
}

int qz(int x, int y, int mod) {
    int res = 1;
    for (; y; y >>= 1) {
        if (y & 1)
            res = mul(res, x, mod);
        x = mul(x, x, mod);
    }
    return res;
}

bool is_prime(int n) {
    if (n < 4) {
        return n == 2 || n == 3;
    }
    if (n % 2 == 0) {
        return 0;
    }
    int s = 0;
    int d = n - 1;
    while (d % 2 == 0) {
        d /= 2;
        s++;
    }
    vector<int> p = {2, 7, 61}; // int32
    // int64
    // vector<int> p = {2, 325, 9375, 28178, 450775, 9780504, 1795265022};
    // int128 直接随 k 次吧
    for (auto u : p) {
        int a = u % n;
        if (a == 0)
            continue;
        int x = qz(a, d, n);
        if (x == 1 || x == n - 1) {
            continue;
        }
        for (int i = 1; i < s; i++) {
            x = mul(x, x, n);
            if (x == n - 1) {
                break;
            }
        }
        if (x != n - 1) {
            return 0;
        }
    }
    return 1;
}

```

31.5 质因数分解

根据唯一分解定理： $n = \prod p_i^{\alpha_i}$ 。完整质因数分解需要同时保留每个素因子 p_i 的指数 α_i 。下方 `divide` 返回排序后的素因子序列，每个 p_i 重复出现 α_i 次；例如 12 返回 $\{2, 2, 3\}$ 。

31.5.1 试除法

$\geq \sqrt{n}$ 的质因数最多只有一个。

枚举 $< \sqrt{n}$ 的所有质数, 试除, 时间复杂度: $O(\frac{\sqrt{n}}{\ln n} + \log n)$ 。

31.5.2 Pollard-Rho

概率性质因数分解, 需要结合 Miller-Rabin。若待分解数的最小素因子为 p , 找到非平凡因子通常需要期望 $O(\sqrt{p})$ 次迭代, 因此常简写为随机期望 $O(n^{\frac{1}{4}})$; 这不是确定性的最坏时间复杂度。

```
struct Pollard_Rho {
    // Miller-Rabin

    int abs(int x) { // int128 不能调用 abs 库函数
        if (x < 0)
            return -x;
        return x;
    }

    int pollard_rho(int x) {
        int s = 0, t = 0;
        int c = rand() % (x - 1) + 1;
        int goal = 1, val = 1, temp;
        for (goal = 1; goal * 2, s = t, val = 1) {
            for (int step = 1; step <= goal; step++) {
                t = (mul(t, t, x) + 1LL * c) % x;
                val = mul(val, abs(t - s), x);
                if (step % 127 == 0) {
                    temp = __gcd(val, x);
                    if (temp == x)
                        return 0;
                    if (temp > 1)
                        return temp;
                }
            }
            temp = __gcd(val, x);
            if (temp == x)
                return 0;
            if (temp > 1)
                return temp;
        }
    }

    vector<int> res;
    void fac(int x) {
        if (x < 2)
            return;
        if (is_prime(x)) {
            res.push_back(x);
            return;
        }
        int p = 0;
        while (p <= 1 || p == x)
            p = pollard_rho(x);
        fac(p);
        fac(x / p);
    }

    vector<int> divide(int x) {
        res.clear();
        fac(x);
        sort(res.begin(), res.end());
        return res;
    }
};
```

31.6 质数筛：

31.6.1 埃氏筛

从 2 开始,不断用 p 标记 $i \times p$, $\sum \frac{1}{p_i} \sim \ln \ln n$, 时间复杂度: $O(n \ln \ln n)$ 。

但是有很多常数优化: - 只标记奇数, 具体而言, 只标记 $(2i+1)p$ - 只标记 $\geq p^2$

- 使用 `bitset` 替代标记数组

```
void ai(int n) {
    not_prime[0] = not_prime[1] = 1;
    for (int i = 4; i <= n; i += 2)
        not_prime[i] = 1;
    for (int i = 3; i <= n / i; i++) {
        if (not_prime[i])
            continue;
        for (int j = i * i; j <= n; j += 2 * i)
            not_prime[j] = 1;
    }
}
```

实现较好的常数较小的埃氏筛会比欧拉筛效率更高, 实测【洛谷】评测环境下, 10^8 的埃氏筛 300ms, 欧拉筛 900ms。

31.6.2 欧拉筛

仅用 x 的最小质因子标记 x , 时间复杂度: $O(n)$, 但是由于常数问题, 在大数据范围下效率可能不如埃氏筛。

```
void euler(int n) {
    fill(is_prime + 2, is_prime + n + 1, 1);
    for (int i = 2; i <= n; ++i) {
        if (is_prime[i]) {
            prime.push_back(i);
        }
        for (auto u : prime) {
            if (i > n / u)
                break;
            is_prime[i * u] = 0;
            if (i % u == 0) {
                break;
            }
        }
    }
}
```

31.6.3 min_25 筛

使用 `min_25` 筛求解 $[1, n]$ 中质数数量, 时间复杂度: $O(\frac{n^{\frac{3}{4}}}{\log n})$ 。

```

int count_pi(int n) {
    int len = 0, lim, lenp = 0;
    auto get_index = [&](int x) -> int {
        if (x <= lim)
            return x;
        else
            return len - n / x + 1;
    };
    lim = sqrt(n);
    vector<int> a(lim << 2), g(lim << 2);
    for (int i = 1; i <= n; i = a[len] + 1) {
        a[++len] = n / (n / i);
        g[len] = a[len] - 1;
    }
    for (int i = 2; i <= lim; ++i) {
        if (g[i] != g[i - 1]) {
            ++lenp;
            for (int j = len; a[j] >= 11l * i * i; --j)
                g[j] = g[j] - g[get_index(a[j] / i)] + lenp - 1;
        }
    }
    return g[len];
}

```

31.6.4 Meissel-Lehmer

Meissel-Lehmer 是专门用于计算 $[1, n]$ 中质数数量的特殊筛法。

这里仅记录模板, 仅供参考, 时间复杂度: $O(\frac{n^{\frac{2}{3}}}{\log^2 n})$ 。

```

i64 count_pi(i64 N) {
    if (N <= 1)
        return 0;
    int v = sqrt(N + 0.5);
    int n_4 = sqrt(v + 0.5);
    int T = min((int)sqrt(n_4) * 2, n_4);
    int K = pow(N, 0.625) / log(N) * 2;
    K = max(K, v);
    K = min<i64>(K, N);
    int B = N / K;
    B = N / (N / B);
    B = min<i64>(N / (N / B), K);

    vector<i64> l(v + 1);
    vector<int> s(K + 1);
    vector<bool> e(K + 1);
    vector<int> w(K + 1);
    for (int i = 1; i <= v; ++i)
        l[i] = N / i - 1;
    for (int i = 1; i <= v; ++i)
        s[i] = i - 1;

    const auto div = [](i64 n, int d) -> int { return double(n) / d; };
    int p;
    for (p = 2; p <= T; ++p)
        if (s[p] != s[p - 1]) {
            i64 M = N / p;
            int t = v / p, t0 = s[p - 1];
            for (int i = 1; i <= t; ++i)
                l[i] -= l[i * p] - t0;
            for (int i = t + 1; i <= v; ++i)
                l[i] -= s[div(M, i)] - t0;
            for (int i = v, j = t; j >= p; --j)
                for (int l = j * p; i >= l; --i)
                    s[i] -= s[j] - t0;
            for (int i = p * p; i <= K; i += p)
                e[i] = 1;
        }
}

```

```

e[1] = 1;
int cnt = 1;
vector<int> roughs(B + 1);
for (int i = 1; i <= B; ++i)
    if (!e[i])
        roughs[cnt++] = i;
roughs[cnt] = 0x7fffffff;
for (int i = 1; i <= K; ++i)
    w[i] = e[i] + w[i - 1];
for (int i = 1; i <= K; ++i)
    s[i] = w[i] - w[i - (i & -i)];

const auto query = [&](int x) -> int {
    int sum = x;
    while (x)
        sum -= s[x], x ^= x & -x;
    return sum;
};
const auto add = [&](int x) -> void {
    e[x] = 1;
    while (x <= K)
        ++s[x], x += x & -x;
};
cnt = 1;
for (; p <= n_4; ++p)
    if (!e[p]) {
        i64 q = i64(p) * p, M = N / p;
        while (cnt < q)
            w[cnt] = query(cnt), cnt++;
        int t1 = B / p, t2 = min<i64>(B, M / q), t0 = query(p - 1);
        int id = 1, i = 1;
        for (; i <= t1; i = roughs[++id])
            l[i] -= l[i * p] - t0;
        for (; i <= t2; i = roughs[++id])
            l[i] -= query(div(M, i)) - t0;
        for (; i <= B; i = roughs[++id])
            l[i] -= w[div(M, i)] - t0;
        for (int i = q; i <= K; i += p)
            if (!e[i])
                add(i);
    }
while (cnt <= v)
    w[cnt] = query(cnt), cnt++;

vector<int> primes;
primes.push_back(1);
for (int i = 2; i <= v; ++i)
    if (!e[i])
        primes.push_back(i);
l[1] += i64(w[v] + w[n_4] - 1) * (w[v] - w[n_4]) / 2;
for (int i = w[n_4] + 1; i <= w[B]; ++i)
    l[1] -= l[primes[i]];
for (int i = w[B] + 1; i <= w[v]; ++i)
    l[1] -= query(N / primes[i]);
for (int i = w[n_4] + 1; i <= w[v]; ++i) {
    int q = primes[i];
    i64 M = N / q;
    int e = w[M / q];
    if (e <= i)
        break;
    l[1] += e - i;
    i64 t = 0;
    int m = w[sqrt(M + 0.5)];
    for (int k = i + 1; k <= m; ++k)
        t += w[div(M, primes[k])];
    l[1] += 2 * t - (i + m) * (m - i);
}
return l[1];
}

```


32 初等数论函数

32.1 积性函数

若 $f(1) = 1, f(xy) = f(x)f(y), x \perp y$, 则 $f(x)$ 是积性函数

若 $f(1) = 1, f(xy) = f(x)f(y)$, 则 $f(x)$ 是完全积性函数

若 $f(x)$ 和 $g(x)$ 均为积性函数, 则以下函数也为积性函数:

$$h(x) = f(x^p)h(x) = f^p(x)h(x) = f(x)g(x)h(x) = \sum_{d|x} f(d)g\left(\frac{x}{d}\right)$$

- 若 $f(x)$ 是积性函数, 则 $f(x) = \prod f(p_i^{k_i})$
- 若 $f(x)$ 是完全积性函数, 则 $f(x) = \prod f(p_i^{k_i}) = \prod f(p_i)^{k_i}$

32.2 常见积性函数

- 单位函数: $\epsilon(n) = [n = 1]$
- 恒等函数: $\text{id}_k(n) = n^k$, $\text{id}_1(n)$ 通常简记作 $\text{id}(n)$
- 常数函数: $1(n) = 1$
- 除数函数: $\sigma_k(n) = \sum_{d|n} d^k$
- 欧拉函数: $\varphi(n) = \sum_{i=1}^n [(i, n) = 1]$
- 莫比乌斯函数:

$$\mu(n) = \begin{cases} 1 & n = 1 \\ 0 & \exists d > 1, d^2 \mid n \\ (-1)^{\omega(n)}, \text{ 其中 } \omega(n) \text{ 表示 } n \text{ 的质因子数量} \end{cases}$$

- 约数个数函数: $\tau(n) = d(n) = \sigma_0(n)$
- 约数和函数: $\sigma_1(n) = \sum_{d|n} d$, 也常简记为 $\sigma(n)$

32.3 欧拉筛求积性函数

$$f(x) = f(p_1^{k_1}) \times \prod f(p_i^{k_i}), i > 1$$

$$\text{令 } y = \prod p_i^{k_i}, \text{ 那么 } f(x) = f(p_1^{k_1}) \times f(y)$$

欧拉筛用 x 的最小质因子标记 x , 在欧拉筛的过程中, y 已被标记, 那么计算 $f(x)$ 时, 欧拉筛可以保证 $f(y)$ 已被计算, 若 $f(p_1^{k_1})$ 可以快速计算, 即可快速得到 $f(x)$ 的值。

综上, 可在 $O(nT(p^k))$ 时间内计算 $f(1) \dots f(n)$, 其中 $T(p^k)$ 为计算 $f(p^k)$ 的时间复杂度。

使用埃氏筛筛出最小质因子后同样可求。

32.4 欧拉函数

32.4.1 公式

$$\varphi(n) = n \prod (1 - \frac{1}{p_i})$$

32.4.2 欧拉定理

$$a^{\varphi(m)} \equiv 1 \pmod{m}, a \perp m$$

32.4.3 费马小定理

$$a^{p-1} \equiv 1 \pmod{p}, a \nmid p, p \in \mathbb{P}$$

不难发现, 费马小定理是欧拉定理的特殊情况。

32.4.4 扩展欧拉定理

$$a^b \equiv \begin{cases} a^{b \bmod \varphi(m)} & \gcd(a, m) = 1 \\ a^b & \gcd(a, m) \neq 1, b < \varphi(m) \\ a^{b \bmod \varphi(m) + \varphi(m)} & \gcd(a, m) \neq 1, b \geq \varphi(m) \end{cases} \pmod{m}$$

用于大指数取模。

32.4.5 欧拉降幂

求 $a^{a^{a^{\dots}}}$ mod m , 反复使用欧拉, 对指数不断 mod $\varphi(m)$ 。

结论是一个数自取 $O(\log n)$ 次 $\varphi(n)$ 就会降到 1。

简单证明: $\varphi(x), x > 2$ 是偶数 - $\varphi(x) \leq \frac{x}{2}, x \bmod 2 = 0$

所以至少是两次缩小一半规模。

33 Gcd

下文,若不做特殊说明,均令 $x < y$ 。

33.1 一般的 gcd 求法

33.1.1 辗转相除法(欧几里得算法)

$$\gcd(x, y) = \gcd(y \bmod x, x)。$$

单次询问时间复杂度: $O(\log n)$ 。

记忆化,预处理时空复杂度: $O(n^2)$, 单次询问时间复杂度: $O(1)$ 。

33.1.2 更相减损术 + Stein

- $x \equiv y \equiv 0 \pmod{2} \rightarrow \gcd(x, y) = 2 \times \gcd(\frac{x}{2}, \frac{y}{2})。$
- $x \equiv y \equiv 1 \pmod{2} \rightarrow \gcd(x, y) = \gcd(y - x, x)$, 此时 $y - x \not\equiv x \pmod{2}。$
- $x \bmod 2 = 0, y \bmod 2 = 1 \rightarrow \gcd(x, y) = \gcd(\frac{x}{2}, y)$
- $x \bmod 2 = 1, y \bmod 2 = 0 \rightarrow \gcd(x, y) = \gcd(x, \frac{y}{2})$

时间复杂度: $O(\log n)$ 。

优势是只涉及减和除以 2 两种操作,在二进制意义下,除以 2 可以视作右移一位。

对于有符号整数,应先用更宽的有符号类型取得绝对值,再转换为无符号整数执行减法和移位。下面的实现返回 `uint64_t`, 因为 $\gcd(\text{INT_MIN}, 0) = 2^{31}$, 该结果无法用 32 位 `int` 表示。

在高精度意义下,时间复杂度为 $O(\log^2 n)$, 优于辗转相除法,且容易实现(高精度取模是 $O(\log^2 n)$ 的,所以高精度意义下的辗转相除法是 $O(\log^3 n)$ 的)。

注:但是由于其和二进制相关的特性,使用二进制相关操作实现时,常数非常小,实测效果优于后文中实现不好的“根号分治”gcd。

实现如下:

```

uint64_t gcd(int a, int b) {
    auto magnitude = [](int x) -> uint64_t {
        return x < 0 ? uint64_t(-int64_t(x)) : uint64_t(x);
    };
    uint64_t x = magnitude(a), y = magnitude(b);
    if (!x || !y)
        return x | y;
    int az = __builtin_ctzll(x);
    int bz = __builtin_ctzll(y);
    int z = min(az, bz);
    x >>= az, y >>= bz;
    while (x != y) {
        if (x > y)
            swap(x, y);
        y -= x;
        y >>= __builtin_ctzll(y);
    }
    return x << z;
}

```

33.2 基于值域的 gcd 求法

令 $x, y \in [1, v]$ 。

33.2.1 分解质因数

令 $\omega(n)$ 表示 n 的不同素因子个数, $\Omega(n)$ 表示计入重数后的素因子个数。例如 $12 = 2^2 \times 3$, 则 $\omega(12) = 2$, $\Omega(12) = 3$ 。

线性筛预处理 $[1, v]$ 质数, 枚举倍数预处理 $[1, v]$ 所有质因子及其指数, 并存储其每一个质因子幂次的值。

求 $\gcd(x, y)$ 时, 遍历 x, y 质因子, 对指数取 $\min$, 其质因子幂次可以直接获取值。

当前方法对每个不同素因子只保存一组“素因子、指数”, 因此预处理时空复杂度为 $O(\sum_{i=1}^v \omega(i))$ 。一次询问合并 x, y 的两组分解, 时间复杂度为 $O(\omega(x) + \omega(y))$; 最坏情况应写成 $O(\max_{t \leq v} \omega(t))$, 而不是 $O(\omega(v))$ 。

33.2.2 “根号分治”

引理: $x = abc$, $a \leq b \leq c \leq \sqrt{n}$ 或 $a \leq b \leq \sqrt{n}, c > \sqrt{n} \wedge v \in \mathbb{P}$ 。

求出 $[1, v]$ 所有数的 a, b, c 分解。

具体求解过程为: $-x = 1, a = b = c = 1 - x \in \mathbb{P}, a = b = 1, c = x - x > 1 \wedge x \notin \mathbb{P}, \{a, b, c\} = \{pa_{\frac{x}{p}}, b_{\frac{x}{p}}, c_{\frac{x}{p}}\}$, 其中 $p \mid x \wedge p \in [2, x]$, 一般选取 x 的最小质因子即可。

使用线性筛可以 $O(n)$ 预处理。

求 $\gcd(x, y)$ 时,依次令

$$g_1 = \gcd(a_x, y), \quad g_2 = \gcd\left(b_x, \frac{y}{g_1}\right), \quad g_3 = \gcd\left(c_x, \frac{y}{g_1 g_2}\right),$$

则 $\gcd(x, y) = g_1 g_2 g_3$ 。计算后一项前必须同时从 y 中除去前面已经统计的因子,第三项的分母不能漏掉 g_1 。

对于后三式,容易分讨: - $y \leq \sqrt{n}$, 通过预处理 $[1, \sqrt{v}]$ 内的 $\gcd$, 调用即可。
- $y > \sqrt{n} \wedge x \leq \sqrt{n}$, $\gcd(x, y) = \gcd(x, y \bmod x)$, 此时 $x \leq \sqrt{n}, y \bmod x \leq \sqrt{n}$, 调用即可。
- $y > \sqrt{n} \wedge x > \sqrt{n}$, 可得 $x \in \mathbb{P} \vee y \in \mathbb{P} - x \mid y \rightarrow \gcd(x, y) = x - x \nmid y \rightarrow \gcd(x, y) = 1$

综上,预处理时空复杂度: $O(v)$, 单次询问时间复杂度: $O(1)$ 。

```
void primes(int n) {
    is_prime.set();
    is_prime[0] = is_prime[1] = 0;
    for (int i = 4; i <= n; i += 2) {
        is_prime[i] = 0;
        minn[i] = 2;
    }
    for (int i = 3; i <= n / i; i++) {
        if (!is_prime[i])
            continue;
        for (int j = i * i; j <= n; j += 2 * i) {
            if (is_prime[j])
                minn[j] = i;
            is_prime[j] = 0;
        }
    }
    for (int i = 2; i <= n; i++)
        if (is_prime[i])
            minn[i] = i;

    x1[1] = x2[1] = x3[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (is_prime[i]) {
            x1[i] = x2[i] = 1;
            x3[i] = i;
        } else {
            int now = i / minn[i];
            tmp[0] = x1[now] * minn[i];
            tmp[1] = x2[now];
            tmp[2] = x3[now];
            sort(tmp, tmp + 3);
            x1[i] = tmp[0], x2[i] = tmp[1], x3[i] = tmp[2];
        }
    }

    for (int i = 1; i <= sqrt; i++) {
        res[i][0] = res[0][i] = i;
        for (int j = 1; j <= i; j++) {
            res[i][j] = res[j][i] = res[j][i % j];
        }
    }
}

int calc(int x, int y) {
    while (1) {
        if (x > y)
            swap(x, y);
        else if (y <= sqrt)
            return res[x][y];
    }
}
```

```

        else if (x <= sqrt) {
            int z = y % x;
            y = x, x = z;
        } else if (y % x == 0)
            return x;
        else
            return 1;
    }
}

int gcd(int x, int y) {
    int a = calc(x1[x], y);
    int b = calc(x2[x], y / a);
    int c = calc(x3[x], y / a / b);
    return a * b * c;
}

```

但是事实上, 容易发现, 这里把 x 分解成了三个整数 a, b, c , 还需要 if-else 分支。

实际上常数和“质因数分解”求法大概差两倍, 更大的优势是空间严格线性。(虽然就算空间, 在 $v \leq 10^6$ 时, $O(\sum \omega(i))$ 和 $O(v)$ 也只差了大概两倍)

33.3 gcd 的势能均摊

33.3.1 连续求 gcd 的均摊

欧几里得算法的一次迭代未必让新的较大数减半, 但连续两次迭代后, 较大数一定至少减半, 因此单次 gcd 的迭代次数为 $O(\log V)$, 其中 V 是两个参数的最大值。

连续计算多个数的 gcd 时, 可以使用势能分析。令当前累计答案依次为 $G_0, G_1, \dots, G_k$, 计算 $G_i = \gcd(G_{i-1}, a_i)$ 时, 第一次取模计 $O(1)$, 后续迭代次数可由 G_{i-1} 到 G_i 的缩小量控制。所有缩小量在整段过程中望远镜相消, 因此总时间为 $O(k + \log V)$, 而不是对每次操作都独立估计为 $O(\log V)$ 。

在通常把定长整数 gcd 视为常数时间操作的模型下, 标准线段树 build 只访问 $O(n)$ 个节点并在每个内部节点合并一次, 因此建树时间和空间均为 $O(n)$, 区间询问为 $O(\log n)$ 。静态 ST 表则需要 $O(n \log n)$ 的预处理时间和空间, 但区间 gcd 询问为 $O(1)$ 。

33.3.2 区间 gcd 的均摊

因为 gcd 相当于对质因子幂指数取 min, 所以每次减少最少 $\times \frac{1}{2}$, 所以 $[i, i], \dots, [i, n]$ 这 $O(n)$ 个区间的 gcd 只会有 $O(\log n)$ 种取值, 且是连续的。

从后向前, i 从 $i+1$ 继承答案, 那么就可以 $O(n \log n)$ 地处理出所有 $O(n \log n)$ 段本质不同区间 gcd。

34 取模运算

由于“同余”的篇幅过大,模意义的一些基础概念单开一页。

34.1 阶:

若正整数 m, a , 满足 $\gcd(a, m) = 1$, 则使得 $a^n \equiv 1 \pmod{m}$ 的最小正整数 n 称为 a 模 m 的阶, 记作 $\delta_m(a)$ 。

34.1.1 性质

- $a^0, a^1, \dots, a^{\delta-1}$ 在模 m 意义下两两不同
- $a^\gamma \equiv a^{\gamma'} \pmod{m} \Leftrightarrow \gamma \equiv \gamma' \pmod{\delta}$
- $\delta \mid \varphi(m)$

34.2 逆元:

若正整数 m, a, b , 满足 $ab \equiv 1 \pmod{m}$, 则称 a, b 互为逆元。

34.2.1 求解

$$\gcd(a, m) = 1 \Leftrightarrow \exists b, ab \equiv 1 \pmod{m}$$

求解逆元等价于解同余方程 $ax \equiv 1 \pmod{m}$, 使用 `exgcd` 求解即可, 逆元存在定理也等价于裴蜀引理。

时间复杂度: $O(\log n)$ 。

34.2.2 预处理 n 个数的逆元

34.2.2.1 预处理 $1 \sim n$ 模质数 p 的逆元:

$$i^{-1} \equiv (p - \lfloor \frac{p}{i} \rfloor) \times (p \bmod i)^{-1} \pmod{p}$$

线性递推即可, $O(n)$ 。

注: 若 p 不为质数, $1 \sim n$ 中可能会存在 i 不存在 i^{-1} , 那么无法递推。

34.2.2.2 预处理任意 n 个数模质数 p 的逆元：

令 $S \equiv \prod_{i=1}^n a_i \pmod{p}$, 使用exgcd/费马小定理结合快速幂 求解 S^{-1} 。

$(\prod_{i=1}^m a_i)^{-1} = (\prod_{i=1}^{m+1} a_i)^{-1} \times a_{m+1}$, 反向递推可得每一个前缀积的逆元。

$$a_x^{-1} = \left(\prod_{i=1}^{x-1} a_i\right) \times \left(\prod_{i=1}^x a_i\right)^{-1}$$

时间复杂度： $O(n + \log p)$ 。

通常使用这种方式预处理阶乘和阶乘的逆元 $O(1)$ 询问组合数。

34.2.3 $O(1)$ 在线逆元

本节统一使用 p 表示逆元所在的模数。下方 Barrett 结构体内部的成员 `m` 以及 `init`、`inv` 的参数 `mod` 均取值为 p , 只是局部命名不同, 并不表示其他模数。调用顺序必须是先执行 `barret.init(p)`, 再执行逆元表的 `init(p)`。

该方法只处理质数模数 p , 查询参数必须先规范化到 $1 \leq x < p$; $x \equiv 0 \pmod{p}$ 时逆元不存在。理论上取 $B = \lceil p^{1/3} \rceil$, 预处理时间和空间为 $O(p/B + B^2) = O(p^{2/3})$, 之后可以 $O(1)$ 查询逆元。

大致做法是利用 $x \cdot x^{-1} \equiv 1 \pmod{p}$ 。若能选择一个非零的 u , 使 $v \equiv xu \pmod{p}$ 可以用绝对值较小的整数表示, 就可以预处理 v^{-1} , 并通过

$$x^{-1} \equiv u v^{-1} \pmod{p}$$

恢复 x 的逆元。

具体地, 将 x 写成 $x = aB + b$, 其中 $0 \leq b < B$, 再寻找满足 $1 \leq |u| \leq B$ 的 u , 使 $aBu \bmod p$ 落在靠近 0 或 p 、长度为 $O(p/B)$ 的区间内。当 $B^3 \geq p$ 时, 有 $p/B \leq B^2$, 同时 $|bu| < B^2$, 所以 $v = xu \bmod p$ 可以选取满足 $|v| < 2B^2$ 的整数代表。预处理这些小范围整数的逆元后即可常数时间回答询问。

当前固定 `B = 300` 的实现必须满足以下前提：

- p 是质数, 并且 $B^3 \geq p$ 。因此 `B = 300` 只能直接覆盖 $p \leq 27\,000\,000$, 不能覆盖常见的 10^9 级模数。
- `res1`、`res2` 至少能访问到下标 $\lfloor p/B \rfloor$, `iv`、`_iv` 至少能访问到下标 $2B^2$, 因此数组容量必须严格大于 $\max(\lfloor p/B \rfloor, 2B^2)$ 。
- 当前 `iv` 的线性递推只适用于 $1 \leq i < p$ 。由于代码预处理到 $2B^2$, 还需保证 $2B^2 < p$; 若不满足, 应对该模数改用普通线性预处理、快速幂或扩展欧几里得算法。`B = 300` 时, 这一条件是 $p > 180\,000$ 。

- 所有送入快速取模的乘积都必须先在 u64 中计算,不能先以 32 位 int 相乘再隐式转换。当前代码涉及的中间量上界约为 $p(p+B)$;它必须能放入 u64,与预计算倒数相乘后的结果必须能放入 u128。若项目中的 int 是普通 32 位类型,应在每个乘法前显式转换为 u64。

因此,在不增加其他回退分支时,固定 $B = 300$ 的这一模板实际上只适用于满足 $180\,000 < p \leq 27\,000\,000$ 的质数模数,并要求上述数组容量和整数位宽条件同时成立。

下方 Barrett 实现使用 u32 m 保存模数、u64 A 保存待约简整数、u128 保存预计算倒数及乘积。它不是任意范围的取模:当前没有额外的商修正步骤,因此调用 $\text{div}(A)$ 时应保证 $0 \leq A < 2^{96}/m$,同时保证商 $\lfloor A/m \rfloor$ 能放入 u32,并保证 $A \times \text{ivm}$ 不会溢出 u128。在本节的固定参数范围内,送入 calc 的中间量满足 $A < p(p+B)$,因而满足这些条件。

模数 $m = 0$ 会导致初始化除零,必须禁止;模数 $m = 1$ 没有逆元问题。当前模板已注明不处理 $m = 2$,因此质模 $p = 2$ 应直接特判:唯一非零剩余类 1 的逆元仍为 1。其他不满足范围条件的小质数也应走普通逆元算法,而不是进入本模板。

```
const int N = (1 << 21) | 1;
const int B = 300; // 需要满足  $B^3 \geq \text{mod}$ 
int res1[N], res2[N], iv[N], _iv[N];
struct Barrett {
    enum { s = 96 };
    static constexpr u128 s2 = u128(1) << s;
    u32 m;
    u128 ivm;
    void init(u32 m_) { m = (m_), ivm = ((s2 - 1) / m + 1); }
    u32 div(u64 a) const { return a * ivm >> s; }
    u32 calc(u64 a) const { return a - u64(div(a)) * m; }
} barret; // 顺带实现了 barrett 快速取模,不能处理模数为 2 的情况
void init(int mod) {
    int lim = mod / B;
    int _lim = -lim + mod;
    for (int u = 1; u <= B; u++) {
        int d = u * B;
        int a = 0;
        for (int i = 0; i <= lim; i++) {
            if (a <= lim) {
                res1[i] = u;
            } else if (a >= _lim) {
                res1[i] = -u;
            } else {
                int r = (_lim - a - 1) / d;
                a += r * d;
                i += r;
            }
            a += d;
            if (a >= mod)
                a -= mod;
        }
    }

    for (int a = 0; a <= lim; a++)
        res2[a] = barret.calc((a * B) * (res1[a] + mod));

    iv[1] = 1;

    for (int i = 2; i < 2 * B * B + 1; i++)
        iv[i] = barret.calc(iv[mod % i] * (mod - mod / i));
```

```
    for (int i = 1; i < 2 * B * B + 1; i++)
        _iv[i] = mod - iv[i];
}
int inv(int x, int mod) {
    if (mod < B)
        return iv[x];
    int a = x / B, b = x % B;
    int u = res1[a];
    int v = (res2[a] + b * u);
    if (v < 0)
        return barret.calc((u + mod) * _iv[-v]);
    return barret.calc((u + mod) * iv[v]);
}
barret.init(p);
init(p);
```

35 同余

35.1 扩展欧几里得算法

用于求解 $ax + by = \gcd(a, b)$ 的一组可行解。

$$ax_1 + by_1 = \gcd(a, b) = \gcd(b \bmod a, a) = (b \bmod a)x_2 + ay_2$$

将上式展开：

$$ax_1 + by_1 = (b - \lfloor \frac{b}{a} \rfloor \times a)x_2 + ay_2$$

因为只要求找到一组合法解,所以可以构造：

$$\begin{cases} y_1 = x_2 \\ x_1 = y_2 - \lfloor \frac{b}{a} \rfloor \times x_2 \end{cases}$$

所以,在辗转相除法求解 $\gcd(a, b)$ 的时候,递归迭代 x, y 即可得到一组构造解。

递归边界是 $\gcd(0, a)$, 此时 $0 \times x + a \times y = \gcd(0, a) = a$ 。

解得: $y = 1, x \in \mathbb{Z}$, 但是根据不同的 x 的取值, 迭代结果的值域也是不同的, 可以证明当 $x = 0$ 时, 迭代结果的值的绝对值是不超过 $\max(a, b)$ 的。

感性地理解, 每次 $x_1 = y_2 - \lfloor \frac{b}{a} \rfloor \times x_2$ 累乘了 $\lfloor \frac{b}{a} \rfloor$, 那么最终就会不超过 $\max(a, b)$ 因为是一路乘上去的, $y_2 - \lfloor \frac{b}{a} \rfloor \times x_2$ 不会使绝对值变大。

那么, $x = z \neq 0$, 结果值域可以近似认为扩大了 z 倍。

所以取 $x = 0, y = 1$ 反代即可。

上述过程只是求出了 $ax + by = \gcd(a, b)$ 的一组特解。

```
void exgcd(int a, int b, int &x, int &y) {
    int t;
    if (!b) {
        x = 1, y = 0;
    } else {
        exgcd(b, a % b, x, y);
        t = x;
        x = y;
        y = t - (a / b) * y;
    }
}
```

考虑通解：

因为 x 确定时, y 的解是唯一的, 所以我们考虑 x 的通解, y 同理。

已知 $ax' + by' = \gcd(a, b)$, 令 $x = x' + \delta \rightarrow a(x' + \delta) + by' = \gcd(a, b)$

δ 合法, 当且仅当 y' 有解, 可得: $by' = \gcd(a, b) - a(x' + \delta)$, 那么 $b \mid \gcd(a, b) - a(x' + \delta)$ 。

用 $ax' + by'$ 替换 $\gcd(a, b)$ 整理式子可得: $b \mid a\delta$ 。

所以 $b \mid a\delta$ 等价于 $\frac{b}{\gcd(a, b)} \mid \delta$, 即 δ 必须是 $\frac{b}{\gcd(a, b)}$ 的整数倍。

综上, 原方程关于 x 的通解为: $x \equiv x' \pmod{\frac{b}{\gcd(a, b)}}$ 。

y 同理, 可得 $y \equiv y' \pmod{\frac{a}{\gcd(a, b)}}$ 。

35.1.1 求解二元一次方程整数解

裴蜀引理: $ax + by = c$ 有整数解当且仅当 $\gcd(a, b) \mid c$ 。

广义裴蜀引理: $\sum_{i=1}^n a_i x_i = d$ 有整数解当且仅当 $\gcd(a_1, \dots, a_n) \mid d$

根据「裴蜀引理」, 二元一次方程 $ax + by = c$ 若有解, 则可以表示成:

$$ax + by = \frac{c}{\gcd(a, b)} \times \gcd(a, b)$$

使用 `exgcd` 求出 $ax + by = \gcd(a, b)$ 的一组特解, 乘以 $\frac{c}{\gcd(a, b)}$ 得到 $ax + by = c$ 的一组特解: x', y' 。

$$a(x' + d) + b(y' - e) = c \Rightarrow ad = be \Rightarrow \frac{a}{\gcd(a, b)}d = \frac{b}{\gcd(a, b)}e \Rightarrow d = t \times \frac{b}{\gcd(a, b)}, e = t \times \frac{a}{\gcd(a, b)}$$

代入原方程, 可得:

原方程的通解为:

$$\begin{cases} x = x' + t \times \frac{b}{\gcd(a, b)} \\ y = y' - t \times \frac{a}{\gcd(a, b)} \end{cases}$$

35.1.2 求解线性同余方程

对于线性同余方程 $ax \equiv b \pmod{c}$, 可以改写成 $ax + cy = b$ 的形式, 看作二元一次方程求整数解即可。

```

int tyfc(int a, int b, int c) {
    b %= c;
    int x, y;
    if (b % gcd(a, c))
        return -1;
    exgcd(a, c, x, y);
    int g = gcd(a, c);
    x *= b / g;
    c /= g;
    x = (x % c + c) % c;
    return x;
}

```

35.2 欧拉定理

$$a^{\varphi(b)} \equiv 1 \pmod{b}, \gcd(a, b) = 1$$

费马小定理: $a^{p-1} \equiv 1 \pmod{p}$, 其中 $p \in \mathbb{P}$ 且 $p \nmid a$ 。

费马小定理是欧拉定理在 $b \in \mathbb{P}$ 时的特殊情况。

35.3 扩展欧拉定理

$$a^b \equiv \begin{cases} a^{b \bmod \varphi(m)} & \gcd(a, m) = 1 \\ a^b & \gcd(a, m) \neq 1, b < \varphi(m) \\ a^{(b \bmod \varphi(m)) + \varphi(m)} & \gcd(a, m) \neq 1, b \geq \varphi(m) \end{cases} \pmod{m}$$

用于大指数取模。

35.3.1 欧拉降幂

求 $a^{a^{\dots}} \bmod m$ 时, 需要随模数递归计算指数。只有在 $\gcd(a, m) = 1$ 时, 才能直接将指数对 $\varphi(m)$ 取模; 若不互质, 则必须使用上方扩展欧拉定理, 并额外判断原指数是否已经达到 $\varphi(m)$, 不能无条件只保留指数模 $\varphi(m)$ 的结果。

结论是一个数自取 $O(\log n)$ 次 $\varphi(n)$ 就会降到 1。

简单证明: $-\varphi(x), x > 2$ 是偶数 $-\varphi(x) \leq \frac{x}{2}, x \bmod 2 = 0$

所以至少是两次缩小一半规模。

35.4 卢卡斯定理

对于质模数 p , 有 $\binom{n}{m} \equiv \binom{\lfloor \frac{n}{p} \rfloor}{\lfloor \frac{m}{p} \rfloor} \times \binom{n \bmod p}{m \bmod p} \pmod{p}$, 规定 $n < m$, $\binom{n}{m} = 0$ 。

Lucas 定理揭示了, 组合数 $\binom{n}{m}$ 对质数取模的结果等价于其 n, m 在 p 进制意义下每一位的 $\binom{n_i}{m_i} \bmod p$ 之积。

即: $\prod_{i=1}^{\infty} \binom{n_i}{m_i} \bmod p$ 。

对于 $\binom{x}{y}$, $x, y < p$, 预处理阶乘及其逆元, 即可 $O(1)$ 询问。

所以时间复杂度: $O(\log_p n + p)$ 。

```
int C(int x, int y, int mod) {
    auto calc = [](int x, int y, int mod) -> int {
        if (x < y)
            return 0;
        if (x == 0 || y == 0)
            return 1;
        return 1LL * fac[x] * inv[y] % mod * inv[x - y] % mod;
    };
    if (x < y)
        return 0;
    int res = 1;
    while (x || y) {
        int X = x % mod, Y = y % mod;
        res = 1LL * res * calc(X, Y, mod) % mod;
        x /= mod;
        y /= mod;
    }
    return res;
}
```

35.5 扩展卢卡斯定理

若模数不为质数, 令 $m = \prod_{i=1}^s p_i^{a_i}$ 。

$$\text{根据 crt, 等价于求解} \begin{cases} \binom{n}{k} \equiv r_1 \pmod{p_1^{a_1}} \\ \binom{n}{k} \equiv r_2 \pmod{p_2^{a_2}} \\ \vdots \\ \binom{n}{k} \equiv r_s \pmod{p_s^{a_s}} \end{cases}.$$

考虑求解 $\binom{n}{k} \bmod p^a$:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

令 $v_p(n)$ 表示 n 的质因数分解中质因子 p 的指数, $w_p(n)$ 表示 n 除尽 p 后的值。

$$\text{则原式可以写成 } p^{v_p(n!) - v_p(k!) - v_p((n-k)!)} \times \frac{w_p(n!)}{w_p(k!)w_p((n-k)!)}.$$

此时, 问题转换成求 $v_p(n) \bmod p^a$ 和 $w_p(n) \bmod p^a$ 。

所以 exlucas 本质是解决了阶乘对质数幂取模的问题。

Wilson 定理: $(p-1)! \equiv -1 \pmod{p}, p \in \mathbb{P}$ 。

推广: $\prod_{1 \leq i \leq m, i \perp m} i \equiv \pm 1 \pmod{m}$ 。

推论: $\prod_{1 \leq i \leq p^a, i \perp p^a} i \equiv \begin{cases} 1 & p = 2 \wedge a \geq 3 \\ -1 & \text{otherwise} \end{cases} \pmod{p^a}$ 。特别地, 模 4 时

完整单位块积为 -1 。

对于 p :

$$w_p(n!) \equiv (-1)^{\lfloor \frac{n}{p} \rfloor} \times (n \bmod p)! \times w_p(\lfloor \frac{n}{p} \rfloor!) \pmod{p}$$

可以发现, 实际上 lucas 就是模数为 p 的特殊情况。

对于 p^a :

令 $U_{p^a} = \prod_{1 \leq i \leq p^a, i \perp p^a} i$ 表示一个完整单位块的乘积, 则

$$w_p(n!) \equiv U_{p^a}^{\lfloor \frac{n}{p^a} \rfloor} \times \prod_{1 \leq i \leq n \bmod p^a, i \perp p^a} i \times w_p(\lfloor n/p \rfloor!) \pmod{p^a}.$$

不能把 U_{p^a} 无条件写成 -1 : 当 $p = 2, a \geq 3$ 时它等于 1, 只有模 4 等情形才为 -1 。

预处理 $\prod_{1 \leq i \leq (n \bmod p^a), i \perp p^a} i$, 递归求解, 容易得到时间复杂度: 预处理 $O(p^a)$, 单

次询问 $O(\log_p n)$ 。

下方模板在每次调用 `C` 时重置全局计数 `nn`, 并由 `init` 覆盖当前质数幂范围内的 `mem`。数组容量必须大于最大的 p^a ; 代码中的模乘使用 `1LL` 作为中间量, 若把模数类型扩展到 `long long`, 则应相应改用 `__int128`。

```
int phi(int x, int y) { return x / y * (y - 1); }
int nn, a[N], b[N];
int mod[N], mem[N];
int intchina() {
    int i, prod = 1, s = 0;
    for (i = 1; i <= nn; i++)
        prod = 1LL * prod * a[i];
    for (i = 1; i <= nn; i++) {
        int t = qz(prod / a[i], phi(a[i], mod[i]) - 1, a[i]);
        s = (s + 1LL * b[i] * (prod / a[i] % prod * t) % prod;
    }
    return s;
}
int n, m, p;

void init(int p, int mod) {
    int i;
    mem[0] = 1;
    for (i = 1; i <= mod; i++) {
        mem[i] = mem[i - 1];
        if (i % p)
            mem[i] = 1LL * mem[i] * i % mod;
    }
}

int l(int x, int p) {
    int res = p, ans = 0;
    while (res <= x) {
        ans += x / res;
        if (res > x / p)
            break;
        res *= p;
    }
    return ans;
}

int g(int x, int mod) {
    int res = qz(mem[mod], x / mod, mod);
    return 1LL * res * mem[x % mod] % mod;
}

int f(int x, int p, int mod) {
    if (x <= 1)
        return 1;
    return 1LL * f(x / p, p, mod) * g(x, mod) % mod;
}

int C(int n, int m, int p) {
    if (n < m)
        return 0;
    nn = 0;
    for (int i = 2; i <= p / i; i++) {
        int res = 1;
        while (p % i == 0) {
            res *= i;
            p /= i;
        }
        if (res > 1) {
            a[++nn] = res;
            mod[nn] = i;
        }
    }
    if (p > 1) {
        a[++nn] = p;
        mod[nn] = p;
    }
    for (int i = 1; i <= nn; i++) {
        int base = l(n, mod[i]) - l(m, mod[i]) - l(n - m, mod[i]);
```



```

int aa = qz(mod[i], base, a[i]);
int res1, res2;
init(mod[i], a[i]);
res1 = f(n, mod[i], a[i]),
res2 = 1LL * f(m, mod[i], a[i]) * f(n - m, mod[i], a[i]) % a[i];
aa = 1LL * aa * res1 % a[i];
aa = 1LL * aa * qz(res2, (a[i] - a[i] / mod[i]) - 1, a[i]) % a[i];
b[i] = aa;
}
return intchina();
}

```

35.6 BSGS

bsgs 用于求解指数同余方程,即:

$$a^x \equiv b \pmod{c}, \gcd(a, c) = 1$$

其基本原理是任意一个 $[0, n]$ 的整数都可以表示成 $ak + b$, 其中 $a \in [0, \lfloor \frac{n-1}{k} \rfloor], b \in [0, k)$ 。

枚举 a, b 的待选项只有 k 个, 预处理 b , 那么求解的时间复杂度即为 $O(\lfloor \frac{n-1}{k} \rfloor + k)$, $k = \sqrt{n}$ 时, 取到近似最优解, 时间复杂度: $O(\sqrt{n})$ 。

具体到原问题中, 首先确定 n , 根据鸽巢原理, 易得 $\exists x < c, a^x \equiv a^c \pmod{c}$, 所以, 若 $a^x \equiv b \pmod{c}$ 在 $[0, c)$ 中无解, 那么就无解。

$$a^{A\sqrt{c}+B} \equiv b \pmod{c} \rightarrow a^{A\sqrt{c}} \times a^B \equiv b \pmod{c}。$$

当 $\gcd(a, c) = 1$, 即 $a^{A\sqrt{c}} \pmod{c}$ 存在逆元时, $a^B \equiv (a^{A\sqrt{c}})^{-1} \times b \pmod{c}$ 。

预处理 $a^0, \dots, a^{\sqrt{n}-1}$, 查询求解 B 即可。

所以可以发现, bsgs 建立在 $\gcd(a, m) = 1$ 的前提下。

有求逆元和 `map` 询问的存在, 时间复杂度: $O(\sqrt{n} \log n)$ 。

小优化:

考虑将 $a^{A\sqrt{c}+B}$ 写成 $a^{A\sqrt{c}+\sqrt{c}-B'}$ 的形式, 那么可以将原式写成 $a^{A\sqrt{c}-B'} \equiv b \pmod{c} \rightarrow a^{A\sqrt{c}} \equiv b \times a^{B'} \pmod{c}, A \in [1, \lfloor \frac{n-1}{k} \rfloor + 1]$ 。

预处理 $b \times a^{B'}$, 若将 `hashmap` 的时间复杂度视作 $O(1)$, 则时间复杂度为: $O(\sqrt{n})$ 。

省去了求逆元的时间复杂度。(但是上式仍建立在逆元存在的基础上)

或者实际上本质上是求 $O(\sqrt{n})$ 个元素的逆元, 朴素也可以做到 $O(\sqrt{n} + \log n)$ 求解。

```

struct BSGS {
    unordered_map<int, int> mp;
    int bsgs(int a, int b, int p) {
        if (b == 1)
            return 0;
        b %= p;
        int sqt = ceil(sqrt(p));
        int A = b;
        mp.clear();
        for (int i = 0; i < sqt; i++) {
            mp[A] = i;
            A = 1LL * A * a % p;
        }
        int base = qz(a, sqt, p);
        A = 1;
        for (int i = 1; i <= sqt; i++) {
            A = 1LL * A * base % p;
            if (mp.count(A))
                return i * sqt - mp[A];
        }
        return -1;
    }
};

```

35.7 扩展 BSGS

用于求解：

$$a^x \equiv b \pmod{c}, \gcd(a, c) \neq 1$$

因为 $\gcd(a, c) \neq 1$ 逆元不存在, 不能使用 bsgs 求解。

若 $\gcd(a, c) \neq 1$, 根据 exgcd 原同余方程可以写作 $\frac{a}{\gcd(a, c)} \times a^{x-1} \equiv \frac{b}{\gcd(a, c)} \pmod{\frac{c}{\gcd(a, c)}}$, 其中 $\gcd(\frac{a}{\gcd(a, c)}, \frac{c}{\gcd(a, c)}) = 1$, 所以逆元存在, 可以将问题转换成 $a^{x-1} \equiv b' \pmod{c'}$ 的子问题, 依次类推, 直到 $\gcd(a, c') = 1$ 为止。

此时, 原式为: $\frac{a^y}{A} \times a^{x-y} \equiv \frac{b}{A} \pmod{\frac{c}{A}}$ 。

采用 bsgs 求解后, $x + y$ 即为原同余方程的解, 注意此时解的范围是 $[y, +\infty)$, 所以要特殊判断 $[0, y)$ 是否存在解。

时间复杂度: $O(\sqrt{n} + \log n)$ 与 bsgs 相同。

```

struct ExBSGS {
    int exbsgs(int a, int b, int c) {
        int sqt, base, i, A = 1;
        hashmap.clear();
        for (int i = 0; i < 32; i++) {
            if (A % c == b)
                return i;
            A = 1LL * A * a % c;
        }
        int d = 1, cnt = 0;
        while (gcd(a, c) != 1) {
            int g = gcd(a, c);
            if (b % g != 0)
                return -1;
            b /= g;
            c /= g;
            d = 1LL * d * (a / g) % c;
        }
    }
};

```

```

        cnt++;
    }
    sqt = ceil(sqrt(c));
    A = b;
    for (int i = 0; i < sqt; i++) {
        hashmap[A] = i;
        A = 1LL * A * a % c;
    }
    base = qz(a, sqt, c);
    A = 1LL * d * base % c;
    for (int i = 1; i <= sqt; i++) {
        auto it = hashmap.find(A);
        if (it != hashmap.end())
            return i * sqt - it->second + cnt;
        A = 1LL * A * base % c;
    }
    return -1;
}
int calc(int b, int n, int p) {
    b %= p, n %= p;
    if (n == 1 || p == 1)
        return 0;
    if (b == 0 && n != 0)
        return -1;
    return exbsgs(b, n, p);
}
};

```

35.7.1 离散对数

对于模数 m , 若 m 存在原根, 设 g 是模 m 的一个原根, 则 g 的幂只枚举模 m 的单位群:

$$x \in [0, \varphi(m)) \longleftrightarrow g^x \bmod m \in (\mathbb{Z}/m\mathbb{Z})^\times.$$

因此, 只有满足 $\gcd(b, m) = 1$ 的剩余类 b 才存在以 g 为底的离散对数; 0 和其他非单位元不在原根幂的枚举范围内。满足 $g^x \equiv b \pmod{m}$ 的 $x \bmod \varphi(m)$ 称为 b 关于 g 的离散对数。

存在原根的正整数模数恰为 $1, 2, 4, p^k, 2p^k$, 其中 p 是奇质数。特别地, 模 4 存在原根, 而 2^k 仅在 $k \geq 3$ 时不存在原根。

记作 $x = \text{ind}_g b$, 形如对数形式。

所以求解底数是原根时的特殊指数同余方程, 可以使用 bsgs 解决。

35.7.2 模数为质数的 N 次剩余

对于 $x^a \equiv b \pmod{p}, p \in \mathbb{P}$ 。

当 $b = 0$ 且 $a > 0$ 时应直接特判, 此时 $x = 0$ 。当 $b \neq 0$ 时, 解 x 也必为单位, 可以取原根 g 并令 $x = g^c$, 原式化为 $(g^a)^c \equiv b \pmod{p}$, 再使用 BSGS 求解相应的离散对数。

35.7.3 一般意义下的 N 次剩余

对于合数模数,可以先分解为 $m = \prod p_i^{a_i}$,分别求解模各个质数幂的同余式,再使用中国剩余定理合并。奇质数幂存在原根,模 2 和模 4 也存在原根;但原根方法仍只直接处理单位元。若 x 或右端点不是单位元,必须先单独处理其 p 进赋值,不能直接对非单位元取离散对数。

特别地, 2^k 在 $k \geq 3$ 时没有原根,其单位群也不是循环群,不能直接套用单个原根下的 BSGS 转换,需要使用针对 2 的幂模数的结构或其他算法。

35.8 中国剩余定理

$$\text{用于求解线性同余方程组} \begin{cases} x \equiv r_1 \pmod{m_1} \\ x \equiv r_2 \pmod{m_2} \\ \vdots \\ x \equiv r_k \pmod{m_k} \end{cases}, \text{其中 } m_i \text{ 两两互质。}$$

$$\text{令 } M = \prod_{i=1}^k m_i, \frac{M}{m_i} \times \frac{M}{m_i}^{-1} \equiv 1 \pmod{m_i}, c_i = \frac{M}{m_i} \times \left(\frac{M}{m_i}\right)^{-1} \pmod{M}.$$

$$\text{则原同余方程组的解为 } x \equiv \sum_{i=1}^k r_i \times c_i \pmod{M}.$$

可以证明,线性同余方程组在模意义下解唯一,所以上述解即为原方程组的唯一解。

时间复杂度: $O(k \log n)$ 。

注:若不保证模数为质数,仅保证两两互质,那么可以使用欧拉定理预处理欧拉函数求逆元。或直接使用 `exgcd` 求解逆元。

实现时还必须保证 $M = \prod m_i$ 能放入所选整数类型,并使用更宽类型计算 M 和 $r_i \frac{M}{m_i} c_i$ 。下方模板用 `__int128` 检查中间结果;若最终模数超过当前 `int` 的范围则返回 -2。若把求逆步骤改成费马小定理 a^{m_i-2} ,还必须额外保证对应的 m_i 是质数;仅有两两互质并不够。

```
struct CRT {
    int intchina(vector<int> &r, vector<int> &mod, int n) {
        int M = 1;
        for (int i = 1; i <= n; i++) {
            __int128 nxt = (__int128)M * mod[i];
            if (nxt > numeric_limits<int>::max())
                return -2;
            M = (int)nxt;
        }
        int ans = 0;
        for (int i = 1; i <= n; i++) {
            if (mod[i] == 1)
                continue;
            int now = M / mod[i];
            int inv = qz(now % mod[i], phi[mod[i]] - 1, mod[i]);
```

```

        ans = (ans + (__int128)r[i] * now % M * inv) % M;
    }
    return ans;
}
};

```

35.8.1 garner 算法

若 a 满足如下线性方程组, 且 $a < \prod_{i=1}^k p_i$

$$\begin{cases} a \equiv a_1 \pmod{p_1} \\ a \equiv a_2 \pmod{p_2} \\ \vdots \\ a \equiv a_k \pmod{p_k} \end{cases}$$

我们可以用以下形式的式子(称作 a 的混合基数表示)表示 a :

$$a = x_1 + x_2 p_1 + x_3 p_1 p_2 + \dots + x_k p_1 \dots p_{k-1}$$

garner 算法用来计算系数 $x_1, \dots, x_k$ 。

令 $r_{i,j}$ 为 p_i 在模 p_j 意义下的逆:

$$p_i \times r_{i,j} \equiv 1 \pmod{p_j}$$

把 a 代入我们得到的第一个方程:

$$a_1 \equiv x_1 \pmod{p_1}$$

代入第二个方程得出:

$$a_2 \equiv x_1 + x_2 p_1 \pmod{p_2}$$

方程两边减 x_1 , 除 p_1 后得:

$$a_2 - x_1 \equiv x_2 p_1 \pmod{p_2}$$

$$(a_2 - x_1) r_{1,2} \equiv x_2 \pmod{p_2}$$

$$x_2 \equiv (a_2 - x_1) r_{1,2} \pmod{p_2}$$

类似地,我们可以得到:

$$x_k = (\dots((a_k - x_1)r_{1,k} - x_2)r_{2,k} - \dots)r_{k-1,k} \bmod p_k$$

时间复杂度: $O(k^2 \log p)$ 。

下方代码用 p_j-2 次幂计算逆元,因此除了模数两两互质外,还要求每个 `mod[j]` 都是质数。若允许合数模数,应改用 `exgcd` 求逆。混合基数的累积乘积和最终答案同样可能溢出;模板返回 `-2` 表示结果已经超出当前 `int` 的可表示范围。

三模数 NTT 实现的任意模数 NTT 就是这个原理。

```
int Garner(vector<int> &r, vector<int> &mod, int n) {
    vector<int> x(n + 1);
    vector<vector<int>> iv(n + 1, vector<int>(n + 1));
    for (int i = 1; i <= n; i++) {
        for (int j = i + 1; j <= n; j++) {
            iv[i][j] = qz(mod[i], mod[j] - 2, mod[j]);
        }
    }
    for (int i = 1; i <= n; i++) {
        x[i] = r[i];
        for (int j = 1; j < i; j++) {
            x[i] = (__int128)iv[j][i] * (x[i] - x[j]) % mod[i];
            if (x[i] < 0)
                x[i] += mod[i];
        }
    }
    int res = x[1];
    int now = 1;
    for (int i = 2; i <= n; i++) {
        __int128 nxt = (__int128)now * mod[i - 1];
        if (nxt > numeric_limits<int>::max())
            return -2;
        now = (int)nxt;
        nxt = (__int128)res + (__int128)now * x[i];
        if (nxt > numeric_limits<int>::max())
            return -2;
        res = (int)nxt;
    }
    return res;
}
```

35.9 扩展中国剩余定理

$$\text{用于求解线性同余方程组} \begin{cases} x \equiv r_1 \pmod{m_1} \\ x \equiv r_2 \pmod{m_2} \\ \vdots \\ x \equiv r_k \pmod{m_k} \end{cases}, \text{其中 } m_i \text{ 不一定满足两两互}$$

质。

对于两个同余方程 $x \equiv r_1 \pmod{m_1}, x \equiv r_2 \pmod{m_2}$ 。

写成二元一次方程的形式:

$$x + b_1 m_1 = r_1, x + b_2 m_2 = r_2 \rightarrow r_1 - b_1 m_1 = r_2 - b_2 m_2$$

b_1, b_2 需要求解, r_1, r_2, m_1, m_2 均为常数, 所以原式可以写成 $m_1 b_1 - m_2 b_2 = r_1 - r_2$ 的二元一次方程。

使用 `exgcd` 求解即可。

注意会出现 $\gcd(m_1, m_2) \nmid (r_1 - r_2)$ 时无解的情况。

将解出的 b_1 或 b_2 代回原式, 可得:

$$x' = r_1 - b_1 m_1 \equiv r_1 \pmod{m_1} \equiv r_2 \pmod{m_2} \rightarrow x \equiv x' \pmod{\text{lcm}(m_1, m_2)}$$

至此, 成功将两个同余方程合并成了一个同余方程。

依次类推, 合并 $k-1$ 次, 即可得到最后的同余式。

实际上此处 `crt` 和 `exCRT` 讨论的都是 x 系数为 1 的同余式, 其在模意义下已经是确定的常数了。

对于一般的同余方程 $ax \equiv b \pmod{c}$, 可以使用 `exgcd` 先求出 x 的同余式通解, 再使用上述规则求解即可。

时间复杂度: $O(k \log n)$ 。

合并后的模数会逐步变成最小公倍数, 也可能超过整数范围。下方模板使用 `__int128` 计算新的模数和余数; 无解返回 -1, 结果模数超出当前 `int` 范围则返回 -2。若连最终最小公倍数都超过 64 位, 仅扩大中间乘法仍不够, 需要使用大整数或改变输出约定。

```
int eyyc(int a, int b, int c) {
    int g = gcd(a, b);
    if (c % g)
        return -1;
    int x, y;
    exgcd(a, b, x, y);
    b /= g;
    x = ((__int128)x * (c / g) % b + b) % b;
    return x;
}

struct ExCRT {
    int intchina(vector<int> &r, vector<int> &mod, int n) {
        int R = r[1], M = mod[1];
        for (int i = 2; i <= n; i++) {
            int rhs = ((__int128)R - r[i]) % mod[i];
            int x = eyyc(M, mod[i], rhs); // b 取正值, 保证求解得到的一定是非负值
            if (x == -1)
                return -1;
            __int128 nxtM = (__int128)M / gcd(M, mod[i]) * mod[i];
            if (nxtM > numeric_limits<int>::max())
                return -2;
            __int128 nxtR = (__int128)R - (__int128)x * M;
            nxtR %= nxtM;
            if (nxtR < 0)
                nxtR += nxtM;
            R = (int)nxtR;
        }
    }
};
```

```

        M = (int)nxtM;
    }
    return R;
}
};

```

35.10 同余最短路：

同余最短路用于求解模意义下从 x 到 y 的最短路。

例如给定正整数 $a_1, \dots, a_n$, 研究 $\sum_{i=1}^n a_i x_i$ ($x_i \geq 0$) 在每个剩余类中能够表示的最小值。若不限定剩余类或正下界, “能表示的最小非负整数” 恒为 0, 因为可以令所有 $x_i = 0$, 因而不是一个有意义的问题。

若选中任一 a_i , 则最终的数都可以通过剩下的 $n - 1$ 个数和调整 a_i 的系数得到。

即可以关于 a_i 划分成 a_i 的同余类。

在每个同余类中求出最小可达值 $\text{dist}[r]$ 后, 可以判断给定整数是否可表示, 也可以求给定下界 $L > 0$ 之后的最小可达值。对于模数 $m = a_i$ 的剩余类 r , 不小于 L 的最小候选值是

$$\text{dist}[r] + \max\left(0, \left\lceil \frac{L - \text{dist}[r]}{m} \right\rceil\right) m,$$

再对所有可达剩余类取最小值即可。

在模意义下, 多一个 a_j 相当于在 x 和 $(x + a_j) \bmod a_i$ 之间建了一条权值为 a_j 的边, 从 0 出发到 $x \in [0, a_i)$ 的最短路即为同余类中能表示的最小值。

35.10.1 转圈法求解同余最短路问题：

本质是: $xa \equiv c \pmod{m}, \gcd(a, m) \mid c$ 。

所以对于不同的 x 可以划分成 $\gcd(a, m)$ 和同余类, 在每个同余类内部正向转移跑完全背包, 因为是模意义下, 所以要跑两轮。

转移 $dp(x) + a \rightarrow dp((x + a) \bmod m)$ 。从 x 开始走 $k = \frac{m}{\gcd(a, m)}$ 步可以更新完 x 能更新的点, 那么从 i 开始走 $2k$ 步, 也就是“多走一圈”, 就相当于从 i 所在的剩余类中的每个点都走了 k 步, 也就可以更新完这个剩余类中的每个点。

时间复杂度: $O(nm)$ 。

36 Dirichlet 前缀和

对于 a_i , 求解 $b_i = \sum_{k|i} a_k$

$k|i$, 相当于 k 的质因子的指数是 i 的质因子的指数的子集偏序。

仿照埃氏筛, 时间复杂度: $O(n \ln \ln n)$, 且常数很小。

```
bitset<N> not_prime;
not_prime[1] = 1;
for (int i = 2; i <= n; i++) {
    if (not_prime[i])
        continue;
    for (int j = i; j <= n; j += i) {
        a[j] += a[j / i];
        not_prime[j] = 1;
    }
}
```

Dirichlet 后缀和:

对于 a_i , 求解 $b_i = \sum_{i|k} a_k$

稍微变形一下。对于同一个质数 i , 必须按倍数从大到小更新, 使较高次幂和较大倍数的贡献先进入 $a[j]$, 再继续传递到 $a[j / i]$; 若从小到大更新, 后加入 $a[j]$ 的贡献将无法继续向下传递。

```
bitset<N> not_prime;
not_prime[1] = 1;
for (int i = 2; i <= n; i++) {
    if (not_prime[i])
        continue;
    for (int j = n / i * i; j >= i; j -= i) {
        a[j / i] += a[j];
        not_prime[j] = 1;
    }
}
```


37 筛法

37.1 整除分块

又叫数论分块。

$\lfloor \frac{n}{i} \rfloor$ 只有 $O(\sqrt{n})$ 种数值,不同数值之间连续,且容易计算: $[l_i, r_i] \rightarrow \lfloor \frac{n}{l_i} \rfloor, r_i = \lfloor \frac{n}{\lfloor \frac{n}{l_i} \rfloor} \rfloor, l_i = r_{i-1} + 1$

对于求和式: $S(n) = \sum_{i=1}^n f(i)g(\lfloor \frac{n}{i} \rfloor)$, 根据整除分块可得: $S(n) = \sum_{i=1}^m g(\lfloor \frac{n}{l_i} \rfloor) \sum_{j=l_i}^{r_i} f(j)$, 其中 $m = O(\sqrt{n})$, 总时间复杂度: $O(\sqrt{n}T(n))$, 其中 $T(n)$ 为计算 $f(n)$ 前缀和的时间复杂度。

```
for (int i = 2, j; i <= n; i = j + 1) {
    j = n / (n / i);
    // 求 [i, j] 的贡献
}
```

37.1.1 多维整除分块

形如:

$$\sum_{i=1}^n \sum_{j=1}^m F_j(\lfloor \frac{a_j}{i} \rfloor)$$

此时,分块区间 $l_i = \min\{\lfloor \frac{a_j}{i} \rfloor\}$ 。

容易发现每个 a_j 产生 $O(\sqrt{a_j})$ 个区间,总时间复杂度为: $O(m\sqrt{n})$,但是区间有相同部分,常数较小。

```
for (int i = 1; i <= m; i++)
    maxn = max(maxn, a[i]);
for (int i = 2, j; i <= maxn; i = j + 1) {
    j = maxn;
    for (int k = 1; k <= m; k++) {
        j = min(j, a[k] / (a[k] / i));
    }
    // 求 [i, j] 的贡献
}
```

37.1.2 二次整除分块

形如:

$$\sum_{i=1}^n \lfloor \frac{n}{i^2} \rfloor$$

时间复杂度: $O(\sqrt[3]{n})$

```
for (int i = 1, j; i <= m; i = j + 1) {
    auto now = n / (i * i);
    j = sqrtl(n / now);
    // 求 [i, j] 的贡献
}
```

37.2 杜教筛

杜教筛用于求解一些积性函数前缀和: $S(n) = \sum_{i=1}^n f(i)$ 。

构造积性函数 $g(n)$ 。

$$\sum_{i=1}^n (f * g)(n) = \sum_{i=1}^n \sum_{d|i} f(d)g(\frac{i}{d}) = \sum_{i=1}^n g(d)S(\lfloor \frac{n}{d} \rfloor)$$

特别地:

$$g(1)S(n) = \sum_{i=1}^n (f * g)(n) - \sum_{i=2}^n g(d)S(\lfloor \frac{n}{d} \rfloor)$$

令 $T(n)$ 为求解 $S(n)$ 的时间复杂度, 结合整除分块, 那么:

$$T(n) = H(n) + \sum_{d|n} G(d)T(d)$$

预处理 $S(1) \dots S(m)$, 时间复杂度: $O(m + \frac{n}{\sqrt{m}})$, $m = n^{\frac{2}{3}}$ 时最优, 为: $O(n^{\frac{2}{3}})$ 。

递归过程中状态需要记忆化, 通常使用哈希表维护。

特别地, 若求一次 $S(n)$, 容易发现状态只有 $O(\sqrt{n})$ 个, 分为 $[1, \sqrt{n}]$ 和 $(\sqrt{n}, n]$ 分别储存即可。

```
int sum_euler(int n) {
    if (n <= 2e6)
        return euler[n];
    if (mem1.count(n))
        return mem1[n];
    int res = (1 + n) * n / 2;
    for (int i = 2, j; i <= n; i = j + 1) {
        j = n / (n / i);
        res -= (j - i + 1) * sum1(n / i);
    }
    return mem1[n] = res;
}
int sum_mu(int n) {
    if (n <= 2e6)
        return mu[n];
}
```

```
    if (mem2.count(n))  
        return mem2[n];  
    int res = 1;  
    for (int i = 2, j; i <= n; i) {  
        j = n / (n / i);  
        res -= (j - i + 1) * sum2(n / i);  
        i = j + 1;  
    }  
    return mem2[n] = res;  
}
```

37.3 min_25 筛

组合数学

本章组收录“数学”中的组合数学专题。

38 组合数列

38.1 组合数

$$\binom{n}{m} = \binom{n-1}{m} + \binom{n-1}{m-1}$$

$$\binom{n}{m} = \binom{n}{n-m}$$

$$\sum_{i=0}^n \binom{n}{i} = 2^n$$

$$\sum_{i=1}^n i \binom{n}{i} = n2^{n-1}$$

$$\sum_{i=0}^k \binom{n+i}{i} = \binom{n+k+1}{k}$$

$$\binom{n+1}{k+1} = \sum_{i=0}^n \binom{i}{k}$$

$$\sum_{i=0}^k \binom{n}{i} \binom{m}{k-i} = \binom{n+m}{k}$$

38.2 卡特兰数

$$H_n = \frac{\binom{2n}{n}}{n+1}$$

$$H_n = \begin{cases} \sum_{i=1}^n H_{i-1} H_{n-i} & n \geq 2 \\ 1 & n = 0, 1 \end{cases}$$

$$H_n = \frac{4n-2}{n+1} H_{n-1}$$

$$H_n = \binom{2n}{n} - \binom{2n}{n-1}$$

38.3 斯特林数

38.3.1 第一类斯特林数

存在 k 个置换环的大小为 n 的置换:

$$\begin{bmatrix} n \\ k \end{bmatrix}$$

约定 $\begin{bmatrix} 0 \\ 0 \end{bmatrix} = 1$; 当 $n > 0$ 时 $\begin{bmatrix} n \\ 0 \end{bmatrix} = 0$, 当 $k < 0$ 或 $k > n$ 时取 0。

38.3.1.1 递推公式

$$\begin{bmatrix} n \\ k \end{bmatrix} = \begin{bmatrix} n-1 \\ k-1 \end{bmatrix} + (n-1) \times \begin{bmatrix} n-1 \\ k \end{bmatrix}$$

38.3.1.2 性质式

$$n! = \sum_{i=0}^n \begin{bmatrix} n \\ i \end{bmatrix}$$

$$x^n = \sum_{i=0}^n \begin{bmatrix} n \\ i \end{bmatrix} (-1)^{n-i} x^i$$

38.3.2 第二类斯特林数

n 个有标号的球分配到 k 个无标号的盒子的方案数:

$$\left\{ \begin{matrix} n \\ k \end{matrix} \right\}$$

约定 $\left\{ \begin{matrix} 0 \\ 0 \end{matrix} \right\} = 1$; 当 $n > 0$ 时 $\left\{ \begin{matrix} n \\ 0 \end{matrix} \right\} = 0$, 当 $k < 0$ 或 $k > n$ 时取 0。

38.3.2.1 递推公式

$$\begin{Bmatrix} n \\ k \end{Bmatrix} = \begin{Bmatrix} n-1 \\ k-1 \end{Bmatrix} + k \times \begin{Bmatrix} n-1 \\ k \end{Bmatrix}$$

38.3.2.2 性质式

$$x^n = \sum_{i=0}^n \begin{Bmatrix} n \\ i \end{Bmatrix} x^i$$

特别地：

$$\sum_{i=0}^n i^k = \sum_{i=0}^k \begin{Bmatrix} k \\ i \end{Bmatrix} \frac{(n+1)^{i+1}}{i+1}$$

38.4 斐波那契数列

定义

$$f_i = \begin{cases} 0 & i = 0 \\ 1 & i = 1 \\ f_{i-1} + f_{i-2} & i \geq 2 \end{cases}$$

的数列为斐波那契数列。

38.4.1 通项公式：

$$f_n = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}}$$

若 5 在给定模数模意义下存在二次剩余, 结合二次剩余和逆元可在 $O(\log n)$ 时间求解 f_n 。

注: 常见模数 $10^9 + 7$ 和 998244353, 对 5 不存在二次剩余。 $10^9 + 9$ 对 5 存在二次剩余, 解为: 383008016, 616991993。

38.4.2 倍增:

$$f_{2k} = f_k(2f_{k+1} - f_k), f_{2k+1} = f_{k+1}^2 + f_k^2$$

根据奇偶性递推 f_n 即可。

时间复杂度: $O(\log n)$, 常数较小。

38.4.3 矩阵递推:

$$[f_{n-1}, f_n] = [f_{n-2}, f_{n-1}] \times \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$$

$$\text{令 } p = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}, [f_n, f_{n+1}] = [f_0, f_1] \times p^n$$

利用矩阵快速幂, 可在 $O(2^3 \times \log n)$ 的时间计算 f_n 。

预处理 2^i 矩阵, 可将询问转换成向量乘矩阵, 矩阵满足结合律, 时间复杂度: $O(2^2 \times \log n)$ 。

38.4.4 性质

- $f_{n-1}f_{n+1} - f_n^2 = (-1)^n$
- $f_{n+k} = f_k f_{n+1} + f_{k-1} f_n$, 特别地 $f_{2n} = f_n(f_{n+1} + f_{n-1})$
- $\gcd(f_a, f_b) = f_{\gcd(a, b)}$, 其中 $a, b \geq 1$
- $n \geq 1, k \geq 0 \rightarrow f_n \mid f_{nk}$
- 由 gcd 恒等式可得, 当 $a \geq 3, b \geq 1$ 时, $f_a \mid f_b \Leftrightarrow a \mid b$; $a = 1, 2$ 时 $f_a = 1$, 会整除任意 f_b , 上述等价关系不再普遍成立

38.4.5 模意义下的周期性

记斐波那契数列模 m 的最小正周期为 $\pi(m)$ 。正整数 T 是它的一个周期, 当且仅当 $(f_T, f_{T+1}) \equiv (0, 1) \pmod{m}$ 。

令模数为 m , 斐波那契数列的最小正周期不超过 $6m$ 。

模数较小时暴力枚举求周期即可。

斐波那契数列模 2 的最小正周期是 3, 模 5 的最小正周期是 20。

对于奇素数 $p \equiv 1, 4 \pmod{5}$, $p-1$ 是斐波那契数列模 p 的周期。

对于奇素数 $p \equiv 2, 3 \pmod{5}$, $2p+2$ 是斐波那契数列模 p 的周期。

上面给出的是候选周期,不一定是最小周期。可以分解候选周期,并依次尝试约去质因子;使用 $(f_T, f_{T+1}) \equiv (0, 1) \pmod{p}$ 检验,得到 $\pi(p)$ 。

对于素数幂,若 $k \geq 2$ 且 $M = \pi(p^{k-1})$,则 pM 是模 p^k 的一个候选周期,且

$$M \mid \pi(p^k) \mid pM.$$

因此 $\pi(p^k)$ 只能是 M 或 pM 。需要先检验 M 在模 p^k 下是否仍为周期,不能无条件认为最小周期每层都恰好乘 p 。

令 $m = \prod p_i^{a_i}$,逐层求出每个最小周期 $\pi(p_i^{a_i})$ 后,根据中国剩余定理有

$$\pi(m) = \operatorname{lcm}_i \pi(p_i^{a_i}).$$

38.5 错位排列

定义 D_n 表示满足 $p_i \neq i$ 的 n 元排列数量,并约定 $D_0 = 1$ 。

$$D_n = n! \sum_{i=0}^n \frac{(-1)^i}{i!}$$

$$D_n = nD_{n-1} + (-1)^n \quad (n \geq 1)$$

38.5.1 指数生成函数

$$D(x) = \sum_{n \geq 0} D_n \frac{x^n}{n!} = \exp(-\ln(1-x) - x)$$

38.6 贝尔数

定义 B_n 表示把大小为 n 的集合划分成若干非空子集的方案数,并约定空集只有一种划分,即 $B_0 = 1$ 。

$$B_{n+1} = \sum_{i=0}^n \binom{n}{i} B_i \quad (n \geq 0)$$

$$B_n = \sum_{i=0}^n \left\{ \begin{matrix} n \\ i \end{matrix} \right\} \quad (n \geq 0)$$

38.6.1 贝尔三角形

定义 $a_{0,0} = 1$ 。对于 $n \geq 1$, 先令 $a_{n,0} = a_{n-1,n-1}$, 再令

$$a_{n,m} = a_{n,m-1} + a_{n-1,m-1} \quad (1 \leq m \leq n),$$

则 $B_n = a_{n,0}$ 。

38.6.2 指数生成函数

$$B(x) = \sum_{n \geq 0} B_n \frac{x^n}{n!} = \exp(e^x - 1)$$

38.7 欧拉数

定义 $E(n, m)$ 表示 n 元排列中恰好有 m 个上升位置的排列数量, 即恰好有 m 个 $i \in [1, n-1]$ 满足 $p_i < p_{i+1}$ 。

38.7.1 递推式:

$$E(n, m) = \begin{cases} 1 & n = 0 \wedge m = 0 \\ 0 & m < 0 \vee (n = 0 \wedge m \neq 0) \\ 0 & n \geq 1 \wedge m \geq n \\ (n-m)E(n-1, m-1) + (m+1)E(n-1, m) & n \geq 1, 0 \leq m < n \end{cases}$$

38.8 分拆数

定义 $p(n, k)$ 表示 $n = \sum_{i=1}^k r_i$ 且 $r_1 \geq r_2 \geq \dots \geq r_k \geq 1$ 的方案数。约定 $p(0, 0) = 1$, 下标越界时取 0, 则 $p_n = \sum_{k=0}^n p(n, k)$, 特别地 $p_0 = 1$ 。

分拆数增长率相对不大: $-p_n \leq 2 \times 10^5, n \leq 50 - p_n \leq 10^6, n \leq 60 - p_n \leq 5 \times 10^6, n \leq 70$

普通生成函数:

$$\begin{aligned}
P(x) &= \sum_{n=0}^{\infty} p_n x^n = \prod_{i=1}^{\infty} (1 - x^i)^{-1} \\
&= \exp \left(- \sum_{i=1}^{\infty} \ln(1 - x^i) \right) = \exp \left(\sum_{i=1}^{\infty} \sum_{j=1}^{\infty} \frac{x^{ij}}{j} \right) \\
&= \exp \left(\sum_{n=1}^{\infty} \frac{\sigma_1(n)}{n} x^n \right).
\end{aligned}$$

其中 $\sigma_1(n) = \sum_{d|n} d$ 表示 n 的正因数之和。

38.8.1 性质:

设 $p_n(k)$ 是最大部分为 k 的 n 的分拆数量, $q_n(k)$ 是恰好 k 个部分的分拆数量。

$$p_n(k) = q_n(k)$$

根据此性质, 求 $p(n)$ 时可以根据拆分出 r_i 的大小根号分治, 时间复杂度: $O(n\sqrt{n})$ 。

设 p_n^o 是把 n 分拆成若干奇数部分的方案数, p_n^d 是把 n 分拆成若干互不相同部分的方案数。

$$p_n^o = p_n^d$$

38.8.2 五边形数定理:

Euler 五边形数定理给出

$$\Phi(x) = \prod_{i=1}^{\infty} (1 - x^i) = 1 + \sum_{k=1}^{\infty} (-1)^k (x^{k(3k-1)/2} + x^{k(3k+1)/2}).$$

令

$$g_k^- = \frac{k(3k-1)}{2}, \quad g_k^+ = \frac{k(3k+1)}{2} \quad (k \geq 1),$$

二者称为广义五边形数。其指数依次为 $1, 2, 5, 7, 12, 15, \dots$, 在 $\Phi(x)$ 中的符号按 $-, -, +, +, \dots$ 成对出现。

由于 $P(x) = \Phi(x)^{-1}$, 比较 $P(x)\Phi(x) = 1$ 的 x^n 系数, 并约定 $p_0 = 1$ 、 $p_n = 0$ ($n < 0$), 可得

$$p_n = \sum_{k=1}^{\infty} (-1)^{k-1} (p_{n-g_k^-} + p_{n-g_k^+}).$$

递推中的符号按 $+, +, -, -, \dots$ 成对出现。因为 $g_k^{\pm} = \Theta(k^2)$, 对每个 n 只有 $O(\sqrt{n})$ 个有效项, 计算 $p_0, \dots, p_n$ 的时间复杂度为 $O(n\sqrt{n})$ 。

39 反演

反演主要用于替换组合式求和,重点在于推式子,原理暂略。

39.1 二项式反演

形式 0:

$$g(n) = \sum_{i=0}^n (-1)^i \binom{n}{i} f(i) \Leftrightarrow f(n) = \sum_{i=0}^n (-1)^i \binom{n}{i} g(i)$$

形式 1:

$$g(n) = \sum_{i=0}^n \binom{n}{i} f(i) \Leftrightarrow f(n) = \sum_{i=0}^n (-1)^{n-i} \binom{n}{i} g(i)$$

形式 2:

$$g(n) = \sum_{i=n}^{\infty} \binom{i}{n} f(i) \Leftrightarrow f(n) = \sum_{i=n}^{\infty} (-1)^{i-n} \binom{i}{n} g(i)$$

这里默认数列具有有限支撑,即存在 N 使得 $i > N$ 时 $f(i) = 0$,从而两个看似无限的和实际都是有限和。令

$$F(x) = \sum_{i=0}^N f(i)x^i, \quad G(x) = \sum_{i=0}^N g(i)x^i,$$

则正变换等价于 $G(x) = F(1+x)$,代入 $x-1$ 即得到逆变换。若用于真正的无限数列,必须另外保证相关级数收敛且允许交换求和次序;仅写成普通形式幂级数并不会自动使每个系数中的无限和有定义,还需要局部分有限性或相应的拓扑收敛条件。

39.2 欧拉反演

Euler 函数恒等式为

$$n = \sum_{d|n} \varphi(d).$$

对其使用 Möbius 反演, 可得对应的反演式

$$\varphi(n) = \sum_{d|n} \mu(d) \frac{n}{d} = n \sum_{d|n} \frac{\mu(d)}{d}.$$

因此这一节是 Möbius 反演在 Euler 函数上的特殊应用, 而不是另一种独立的通用反演。

39.3 莫比乌斯反演

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g\left(\frac{n}{d}\right)$$

特殊地, $\sum_{d|n} \mu(d) = [n = 1]$

更特殊地, $\sum_{d|\gcd(i,j)} \mu(d) = [\gcd(i,j) = 1]$

39.4 子集反演

以下 S, T 均为同一个有限全集 U 的子集。对子集方向求和时, 有

$$g(S) = \sum_{T \subseteq S} f(T) \Leftrightarrow f(S) = \sum_{T \subseteq S} (-1)^{|S|-|T|} g(T).$$

另一组是对超集方向求和:

$$g(S) = \sum_{S \subseteq T \subseteq U} f(T) \Leftrightarrow f(S) = \sum_{S \subseteq T \subseteq U} (-1)^{|T|-|S|} g(T).$$

两组公式方向不同, 第二组不能仅由第一组的前提推出。

39.5 单位根反演

设 $m \geq 1$, 并在域 K 上运算。要求 m 在 K 中可逆, 且 K 中存在一个 m 次本原单位根 ω_m 。在复数域中可以取 $\omega_m = e^{2\pi i/m}$; 在素域 $\mathbb{F}_p$ 中使用时, 常见条件是 $m \mid (p-1)$, 此时本原单位根和 m^{-1} 都存在。

$$\frac{1}{m} \sum_{i=0}^{m-1} (\omega_m)^{i \times d} = [m \mid d]$$

扩展到 $d \equiv k \pmod{m}$ 的情况:

$$\frac{1}{m} \sum_{i=0}^{m-1} \omega_m^{i \times (d-k)} = [d \equiv k \pmod{m}]$$

把这个形式放到多项式上: 对于多项式 $f(x) = \sum_{i=0}^n a_i x^i$, 求所有下标模 m 同余于 k 的系数之和。

$$\begin{aligned} \sum_{i=0}^n a_i [i \equiv k \pmod{m}] &= \sum_{i=0}^n a_i \frac{1}{m} \sum_{j=0}^{m-1} \omega_m^{j(i-k)} \\ &= \frac{1}{m} \sum_{j=0}^{m-1} \omega_m^{-jk} \sum_{i=0}^n a_i (\omega_m^j)^i \\ &= \frac{1}{m} \sum_{j=0}^{m-1} \omega_m^{-jk} f(\omega_m^j). \end{aligned}$$

时间复杂度: $O(mT(n))$, 其中 $T(n)$ 表示多项式单点求值的时间复杂度。

39.6 斯特林反演

$$f(n) = \sum_{i=0}^n \left\{ \begin{matrix} n \\ i \end{matrix} \right\} g(i) \Leftrightarrow g(n) = \sum_{i=0}^n (-1)^{n-i} \left[\begin{matrix} n \\ i \end{matrix} \right] f(i)$$

第四部分

图论

40 最短路

40.1 Dijkstra

Dijkstra 只适用于边权非负的图。距离和中间加法应使用 64 位整数；下方取 $INF=2^{62}$ ，并假设单条边权与所有需要输出的有限最短路都落在 $(-INF, INF)$ 内。优先队列必须按距离从小到大弹出。

$O(m \log m)$ 实现：

```
vector<long long> dijkstra(int s, int n) {
    const long long INF = 1LL << 62;
    vector<long long> dis(n + 1, INF);
    using pli = pair<long long, int>;
    priority_queue<pli, vector<pli>, greater<pli>> q;
    dis[s] = 0;
    q.push({0, s});
    while (q.size()) {
        auto [cur, now] = q.top();
        q.pop();
        if (cur != dis[now])
            continue;
        for (auto [v, w] : p[now]) {
            if (w > INF - dis[now])
                continue;
            long long nxt = dis[now] + w;
            if (dis[v] > nxt) {
                dis[v] = nxt;
                q.push({nxt, v});
            }
        }
    }
    return dis;
}
```

稠密图上， $O(n^2)$ 实现优于 $O(m \log m)$ 实现：

```
vector<long long> dijkstra(int s, int n) {
    const long long INF = 1LL << 62;
    vector<long long> dis(n + 1, INF);
    vector<bool> vis(n + 1);
    dis[s] = 0;
    for (int _ = 1; _ <= n; _++) {
        int mini = 0;
        for (int i = 1; i <= n; i++) {
            if (!vis[i] && (!mini || dis[i] < dis[mini])) {
                mini = i;
            }
        }
        if (!mini || dis[mini] == INF)
            break;
        vis[mini] = 1;
        for (auto [v, w] : p[mini]) {
            if (vis[v] || w > INF - dis[mini])
                continue;
            long long nxt = dis[mini] + w;
            if (dis[v] > nxt) {
                dis[v] = nxt;
            }
        }
    }
    return dis;
}
```

```

}

```

40.2 Bellman-Ford

Bellman-Ford 基于松弛次数,特殊地,可以用来求解经过边数不超过某个数量的最短路。

原地松弛可以用于普通Bellman-Ford,并可能在一轮内传播多条边;但若限制至多经过 k 条边,第 i 轮必须只从第 $i - 1$ 轮的距离转移。下方模板每轮备份一次数组,时间复杂度为 $O(k(n + m))$ 。

```

vector<long long> bellman_ford(int s, int n, int k) {
    const long long INF = 1LL << 62;
    vector<long long> dis(n + 1, INF);
    dis[s] = 0;
    for (int _ = 1; _ <= k; _++) {
        auto pre = dis;
        bool flag = 0;
        for (int u = 1; u <= n; u++) {
            if (pre[u] == INF)
                continue;
            for (auto [v, w] : p[u]) {
                if (dis[v] > pre[u] + w) {
                    dis[v] = pre[u] + w;
                    flag = 1;
                }
            }
        }
        if (!flag)
            break;
    }
    return dis;
}

```

40.2.1 spfa

SPFA 使用队列只处理本轮距离被更新的顶点,可以看作 Bellman-Ford 的队列优化。普通最短路模板假设从源点不可达负环;若存在可达负环,必须使用后文的检测版本及时终止。

它在部分数据上运行较快,但最坏时间复杂度仍为 $O(nm)$,各种队列顺序优化也不改变这一最坏界。

标准实现使用 inq 标记顶点是否已在队列中,避免同一个顶点被重复加入队列。

```

vector<long long> spfa(int s, int n) {
    const long long INF = 1LL << 62;
    vector<long long> dis(n + 1, INF);
    vector<bool> inq(n + 1);
    dis[s] = 0;
    queue<int> q;
    q.push(s);
    inq[s] = 1;
    while (q.size()) {
        auto now = q.front();

```



```

    q.pop();
    inq[now] = 0;
    for (auto [v, w] : p[now]) {
        if (dis[v] > dis[now] + w) {
            dis[v] = dis[now] + w;
            if (!inq[v]) {
                q.push(v);
                inq[v] = 1;
            }
        }
    }
}
return dis;
}

```

40.2.2 判负环

从指定源点 s 出发, 只能发现从 s 可达的负环。若要判断图中任意位置是否存在负环, 应增加超级源点并向每个顶点连一条 0 权边; 实现上也可以等价地令所有顶点初始距离为 0 并全部入队。

Bellman-Ford 版:

```

bool bellman_ford(int s, int n) {
    const long long INF = 1LL << 62;
    vector<long long> dis(n + 1, INF);
    dis[s] = 0;
    for (int _ = 1; _ <= n; _++) {
        for (int i = 1; i <= n; i++) {
            for (auto [v, w] : p[i]) {
                if (dis[i] == INF)
                    continue;
                if (dis[v] > dis[i] + w) {
                    dis[v] = dis[i] + w;
                    if (_ == n)
                        return 1;
                }
            }
        }
    }
    return 0;
}

```

spfa 版:

```

bool spfa(int n) {
    vector<long long> dis(n + 1);
    vector<int> cnt(n + 1);
    vector<bool> inq(n + 1);
    queue<int> q;
    for (int i = 1; i <= n; i++) {
        q.push(i);
        inq[i] = 1;
    }
    while (q.size()) {
        auto now = q.front();
        q.pop();
        inq[now] = 0;
        for (auto [v, w] : p[now]) {
            if (dis[v] > dis[now] + w) {
                dis[v] = dis[now] + w;
                cnt[v] = cnt[now] + 1;
                if (cnt[v] >= n)
                    return 1;
            }
            if (!inq[v]) {
                q.push(v);
            }
        }
    }
}

```

```

        inq[v] = 1;
    }
}
}
return 0;
}

```

40.3 Floyd

应先把距离矩阵初始化为 INF , 再一次性令 $\text{dis}[i][i]=0$, 之后读入边并取最小值。不能在三重循环中反复把对角线改回 0, 否则已经得到的负对角值会被覆盖, 负环信息也会丢失。

```

const long long INF = 1LL << 62;
vector<vector<long long>> dis(n + 1, vector<long long>(n + 1, INF));
for (int i = 1; i <= n; i++)
    dis[i][i] = 0;
for (auto [u, v, w] : edge)
    dis[u][v] = min(dis[u][v], 1LL * w);
for (int k = 1; k <= n; k++) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (dis[i][k] == INF || dis[k][j] == INF)
                continue;
            dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
        }
    }
}

```

算法结束后, 若存在 i 满足 $\text{dis}[i][i] < 0$, 则图中存在负环。无向图读边时还需同时更新反向距离。

40.3.1 传递闭包

Floyd 可用于求解传递闭包, 即全源可达性。特别地, 使用 `bitset` 优化, 时间复杂度: $O(\frac{n^3}{w})$ 。

```

for (int k = 1; k <= n; k++) {
    for (int i = 1; i <= n; i++) {
        if (dis[i][k])
            dis[i] |= dis[k];
    }
}

```

注: Floyd 常数极小, $O(n^3)$ 的时间复杂度可以通过 $n = 1000$ 的数据。

40.3.2 最小环:

```

auto get_path = [&](auto self, int x, int y) -> void {
    if (!pos[x][y])
        return;
    auto now = pos[x][y];
    self(self, x, now);
    ans.push_back(now);
    self(self, now, y);
};
for (int k = 1; k <= n; k++) {
    for (int i = 1; i < k; i++) {
        for (int j = i + 1; j < k; j++) {
            if (dis[i][j] + Dis[j][k] < minn - Dis[k][i]) {
                minn = dis[i][j] + Dis[j][k] + Dis[k][i];
                ans.clear();
                ans.push_back(k);
                ans.push_back(i);
                get_path(get_path, i, j);
                ans.push_back(j);
            }
        }
    }
}
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        if (dis[i][j] > dis[i][k] + dis[k][j]) {
            dis[i][j] = dis[i][k] + dis[k][j];
            pos[i][j] = k;
        }
    }
}
}
}

```

时间复杂度: $O(n^3)$ 。

40.4 Johnson 全源最短路

Johnson 算法用于稀疏图的全源最短路,允许负边,但图中不能存在负环。

新增超级源点 0,向每个原顶点连接一条 0 权边。使用 Bellman-Ford 求出 $h_v = \text{dis}(0, v)$;若检测到负环,则原问题无解。随后将每条边重赋权为

$$w'(u, v) = w(u, v) + h_u - h_v.$$

由最短路三角不等式可知 $w'(u, v) \geq 0$,所以可以从每个顶点运行Dijkstra。设重赋权后的最短距离为 $d'(s, v)$,原距离为

$$d(s, v) = d'(s, v) - h_s + h_v.$$

下面用所有 $h_i = 0$ 初始化Bellman-Ford,这与显式添加超级源点的效果相同。模板返回 0 表示存在负环;所有边权、势能和重赋权结果均需落在 64 位与 INF 约定的范围内。

```

bool johnson(int n, vector<vector<long long>> &ans) {
    const long long INF = 1LL << 62;
    vector<long long> h(n + 1);
    for (int _ = 1; _ <= n; _++) {
        bool flag = 0;
        for (auto [u, v, w] : edge) {
            if (h[v] > h[u] + w) {
                h[v] = h[u] + w;
                flag = 1;
            }
        }
        if (!flag)
            break;
        if (_ == n)
            return 0;
    }

    vector<vector<pair<int, long long>>> g(n + 1);
    for (auto [u, v, w] : edge) {
        long long nw = (long long)((__int128)w + h[u] - h[v]);
        g[u].push_back({v, nw});
    }

    ans.assign(n + 1, vector<long long>(n + 1, INF));
    using pli = pair<long long, int>;
    for (int s = 1; s <= n; s++) {
        vector<long long> dis(n + 1, INF);
        priority_queue<pli, vector<pli>, greater<pli>> q;
        dis[s] = 0;
        q.push({0, s});
        while (q.size()) {
            auto [cur, u] = q.top();
            q.pop();
            if (cur != dis[u])
                continue;
            for (auto [v, w] : g[u]) {
                if (w > INF - dis[u])
                    continue;
                long long nxt = dis[u] + w;
                if (dis[v] > nxt) {
                    dis[v] = nxt;
                    q.push({nxt, v});
                }
            }
        }
        for (int v = 1; v <= n; v++) {
            if (dis[v] != INF)
                ans[s][v] = (long long)((__int128)dis[v] - h[s] + h[v]);
        }
    }
    return 1;
}

```

使用二叉堆时,时间复杂度为 $O(nm + nm \log m)$,通常写作 $O(nm \log m)$;若保存完整答案矩阵,空间复杂度为 $O(n^2 + m)$ 。

40.5 不同最短路算法的比较

最短路算法	Dijkstra	Bellman-Ford	Floyd	Johnson
图限制	非负权图	可含负边	可含负边	可含负边、无负环
解对象	单源最短路	单源最短路	全源最短路	全源最短路

最短路算法	Dijkstra	Bellman-Ford	Floyd	Johnson
判负环	不能	能	能	能
时间复杂度	$O(n^2)/O(m \log m)$	$O(nm)$	$O(n^3)$	$O(nm \log m)$

40.6 0-1 BFS

边权只有 0/1 的最短路问题,使用双端队列维护,若松弛时边权为 0,则将点加入队首,若松弛时边权为 1 则将点加入队尾。

使用邻接表时,每个顶点和每条边只会产生常数次有效处理,时间复杂度为 $O(n + m)$,空间复杂度为 $O(n + m)$ 。

40.7 差分约束

差分约束用于解决 n 元一次不等式组问题,且要求 n 元一次不等式是形如 $x_{a_i} \leq x_{b_i} + c_i$ 。

根据最短路三角不等式 $dis_v \leq dis_u + w$,约束 $x_{a_i} \leq x_{b_i} + c_i$ 应建立有向边 $b_i \rightarrow a_i$,边权为 c_i 。若误建成 $a_i \rightarrow b_i$,得到的是方向相反的约束。建图后求得的最短路数组给出一组可行解;图中存在负环当且仅当约束系统无解。

此外,最短需要一个源点,所以对于上述不等式组,需要做一些额外限制:

- 求解最短路需要源点和所有点连通,所以建图后每一个连通块的求解是独立的。求解一个不等式组,需要钦定一些变量的值,而对应在图中,这些需要被钦定的变量就是强连通分量缩点后入度为 0 的点。
- 对于原始问题: $x_{a_i} \leq x_{b_i} + c_i$ 的不等式,实际上是有无穷多组解的,因为给每一个元素加一个常数 d 后,不等式组仍成立。所以求一组解时,可以给每一个元素一个初值的限制也就是超级源点向每一个点连的边权,然后给超级源点钦定一个初值。

40.7.1 最小解

若问题要求每个 x_i 的最小解,那么 $dis_v \geq dis_u + w$, dis_v 合法的最小值为 $\max\{dis_u + w\}$ 才能使所有不等式成立。

使用最长路求解。

注:实际问题中还需要题目对每一个数有最小限制: $x_i \geq e_i$,否则可以无穷小。

40.7.2 最大解:

若问题要求每个 x_i 的最大解,那么 $dis_v \leq dis_u + w$, dis_v 合法的最大值为 $\min\{dis_u + w\}$ 才能使所有不等式成立。

使用最短路求解。

注:实际问题中还需要题目对每一个数有最大限制: $x_i \leq e_i$, 否则可以无穷大。

40.7.3 最后

所以实际上,我之前在洛谷板子 P5960 【模板】差分约束的代码实际上求的是 $x_i \leq 0$ 的最大解。

40.7.4 expand 1.

朴素的,由上可知,差分约束需要每一个不等式严格满足有两个变量 x_{a_i} 和 x_{b_i} ,这样才有三角不等式。

但是对于只有一个变量的不等式,可以转换成和超级源点的连边 $x_{a_i} \leq x_0 + c_i$ (和钦定初值同理)。

40.7.5 expand 2.

如果出现 $x_{a_i} = c_i$ 这种条件,可以转换成 $x_{a_i} \leq c_i$ 和 $x_{a_i} \geq c_i$ 从而转换成 $x_{a_i} \leq x_0 + c_i, x_0 \leq x_{a_i} - c_i$ 的两个不等式。

40.7.6 expand 3.

对于除法不等式形如: $\frac{x_{a_i}}{x_{b_i}} \leq c_i$, 可以通过取对数,将除法转换成减法。

$$\frac{x_{a_i}}{x_{b_i}} \leq c_i \rightarrow \log x_{a_i} - \log x_{b_i} \leq \log c_i。$$

40.8 最短路图/最短路树

只考虑从源点 s 可达且最短距离有限的顶点。对于原图中的边 (u, v, w) , 若

$$dis_v = dis_u + w,$$

则称它为紧边;由所有紧边组成的子图称为最短路图。从 s 到 t 的最短路与最短路图中 s 到 t 的路径对应。

最短路图不一定是 DAG。紧边环上的等式相加后,环的总边权必为 0;因此只在不存在总权为 0 的紧边环时,最短路图才是 DAG。对于非负权图,这种环只能由 0 权边组成。

若最短路图是 DAG,可以拓扑排序后进行路径计数等 dp。若存在 0 权紧边,即使没有形成环,紧边两端的距离也可能相等,优先队列对同距离顶点的弹出顺序不保证是拓扑序,所以直接在 Dijkstra 过程中同步计数仍可能漏算。只有当每条相关紧边都满足 $dis_u < dis_v$ (例如所有边权都严格为正)或另行保证了正确处理顺序时,在线 dp 才是安全的。

最短路树需要为每个可达的非源点选择一条紧入边。不存在紧边环时可以直接选择;存在 0 权紧环时则不能任取,否则可能选出环,应使用一次实际的搜索/松弛过程记录父边,保证最终得到以 s 为根的树。若允许反复经过 0 权环,则最短路“游走”的数量甚至可能无穷,需要先明确题目统计的是路径还是游走。

41 最小生成树

41.1 Kruskal

将边集升序后,使用并查集贪心合并顶点不在同一个连通块中的边即可。

排序必须按边权升序。并查集需要检查全部 m 条边,因此初始化和并查集操作的复杂度为 $O(n+m\alpha(n))$;总时间复杂度为 $O(m\log m+n+m\alpha(n))=O(m\log m+n)$,瓶颈通常是排序。

```
sort(edge.begin(), edge.end());
long long ans = 0;
int cnt = 0;
for (auto [u, v, w] : edge) {
    if (dsu.same(u, v))
        continue;
    dsu.merge(u, v);
    ans += w;
    cnt++;
}
if (cnt != n - 1) {
    // 图不连通,不存在生成树
    return;
}
```

上方代码默认 `edge` 的比较规则以边权为第一关键字。最终必须恰好选出 $n-1$ 条边;仅得到一个权值累加和并不能说明原图连通。

41.2 Prim

41.2.1 朴素维护

Prim 本质上求的是一棵根向叶子的有向树,用 dis_x 表示 x 向父节点的边权。

每次扩展一个 dis 最小的叶子,朴素维护时间复杂度: $O(n^2+m)$ 。

下面模板默认 `dis` 初始化为 `inf`、`vis` 初始化为 0。根节点也需要作为一次扩展被选中,因此共循环 n 次;若某次最小值仍为 `inf`,说明原图不连通。

```
long long ans = 0;
dis[1] = 0;
for (int _ = 1; _ <= n; _++) {
    int mini = 0;
    for (int i = 1; i <= n; i++) {
        if (!vis[i] && (!mini || dis[i] < dis[mini]))
            mini = i;
    }
    if (!mini || dis[mini] == inf) {
        // 图不连通,不存在生成树
        return;
    }
    vis[mini] = 1;
    ans += dis[mini];
}
```

```

for (auto [v, w] : p[mini]) {
    if (!vis[v])
        dis[v] = min(dis[v], w);
}
}

```

41.2.2 堆优化

时间复杂度同 Dijkstra, 使用二叉堆维护 $O((n + m) \log m)$ 。

下面的 q 为按候选边权从小到大取出的优先队列。根节点同样计入 n 次扩展；队列提前为空表示剩余顶点不可达, 必须报告无生成树。

```

long long ans = 0;
dis[1] = 0;
q.push({1, 0});
for (int _ = 1; _ <= n; _++) {
    while (q.size() && vis[q.top().v])
        q.pop();
    if (q.empty()) {
        // 图不连通, 不存在生成树
        return;
    }
    auto [now, __] = q.top();
    q.pop();
    vis[now] = 1;
    ans += dis[now];
    for (auto [v, w] : p[now]) {
        if (!vis[v]) {
            if (dis[v] > w) {
                q.push({v, w});
                dis[v] = w;
            }
        }
    }
}
}

```

41.2.3 其他数据结构优化

容易发现, 堆优化 Prim 只是用二叉堆优化了取最小值的过程。

那么同样地, 完全可以使用任意其他可以维护最值的数据结构来优化 Prim, 如线段树、set 等。

同时, 根据其他数据结构的功能, 还可以扩展出其他存图方式, 本质上只要能维护 dis 数组即可。

41.3 Boruvka

每轮先根据轮开始时的并查集, 为每个当前连通块记录一条最轻出边, 再统一尝试合并。若原图连通, 每个尚未完成的连通块都有出边, 合并后连通块数量至少减半, 因此最多进行 $O(\log n)$ 轮。计入并查集操作后, 时间复杂度为 $O((n + m)\alpha(n) \log n)$, 通常简写为 $O(m \log n)$ 。

```

int cnt = n - 1;
long long ans = 0;
dsu.init(n);
while (cnt) {
    vector<int> minn(n + 1, inf);
    vector<pii> mini(n + 1, {0, 0});
    for (auto [u, v, w] : edge) {
        int x = dsu.find(u), y = dsu.find(v);
        if (x == y)
            continue;
        if (minn[x] > w) {
            minn[x] = w;
            mini[x] = {u, v};
        }
        if (minn[y] > w) {
            minn[y] = w;
            mini[y] = {u, v};
        }
    }
    bool flag = 0;
    for (int i = 1; i <= n; i++) {
        auto [u, v] = mini[i];
        if (!u || dsu.same(u, v))
            continue;
        dsu.merge(u, v);
        ans += minn[i];
        cnt--;
        flag = 1;
    }
    if (!flag) {
        // 图不连通,不存在生成树
        return;
    }
}

```

Boruvka 本质上只要找到当前每个连通块向外的最小边即可,如果条件允许,也不见得一定要遍历所有边。

41.4 次小生成树

固定一棵最小生成树后,至少存在一棵最优的候选次小生成树,可以通过加入一条非树边,再从形成的环上删除一条树边得到。

设最小生成树权值为 W 。对于非树边 (u, v, w) ,加入它会与树上 u, v 路径形成一个环;若删除路径上的树边权值 x ,新树权值为

$$W + w - x.$$

若只要求得到一棵不同的生成树,并允许其权值仍等于 W ,删除路径上的最大边即可。若要求严格次小生成树,即答案必须严格大于 W ,则必须删除路径上权值严格小于 w 的最大边:

- 当路径最大边权 $mx_1 < w$ 时,删除 mx_1 ;
- 当 $mx_1 = w$ 时,删除路径上第二大的不同边权 mx_2 ;若不存在 mx_2 ,这条非树边不能产生严格候选。

因此倍增查询通常维护路径上前两大的不同边权,而不是只维护一个最大值。若允许负边权,空值必须初始化为 $-\text{inf}$,不能用 0 充当不存在的边权。

41.5 瓶颈路

x, y 最小瓶颈路指 x, y 的所有路径中最大边权最小的路径。

可以证明,最小生成树上 x, y 的路径是一条最小瓶颈路。

反之,最大瓶颈路指 x, y 的所有路径中最小边权最大的路径,最大生成树上 x, y 的路径是一条最大瓶颈路。

41.6 Kruskal 重构树

在 Kruskal 运行过程中,对于树边 (u, v, w) ,令合并时 u 的根节点为 x , v 的根节点为 y ,那么新建节点 t ,令 $a_t = w$,连接 $(t, x), (t, y)$,再把 x, y 所在集合的根更新为 t ,形成的树即为 Kruskal 重构树。

原图顶点 u, v 的最小瓶颈值等于 Kruskal 重构树上 u, v 的 LCA 的权值。若原图不连通,得到的是重构森林,只能查询同一连通块内的点对。

41.7 最小树形图

本文讨论以 r 为根的最小外向树形图:根 r 能沿有向边到达所有顶点,每个非根点恰好有一条入边,目标是使总边权最小。若要求所有点都能到达根的内向树形图,应反转所有边后再使用同一算法。

41.7.1 朱刘算法

```
bool zl(int n, int r, long long &res) {
    res = 0;
    while (1) {
        vector<node> pre(n + 1);
        for (int i = 1; i <= n; i++)
            pre[i].w = inf;
        for (auto u : edge) {
            if (u.v == r)
                continue;
            if (u.w <= pre[u.v].w) {
                pre[u.v] = {u.u, u.w};
            }
        }
        for (int i = 1; i <= n; i++) {
            if (i == r)
                continue;
            if (pre[i].w == inf)
                return 0;
        }
    }
}
```

```

        res += pre[i].w;
    }
    stack<int> q;
    vector<bool> vis(n + 1);
    vector<int> num(n + 1), col(n + 1);
    int id = 0;
    for (int i = 1; i <= n; i++) {
        int now = i;
        if (col[now] || i == r)
            continue;
        while (now && !vis[now] && !col[now]) {
            q.push(now);
            vis[now] = 1;
            now = pre[now].v;
        }
        if (vis[now]) {
            id++;
            while (q.top() != now) {
                int cur = q.top();
                q.pop();
                num[cur] = id;
                vis[cur] = 0;
                col[cur] = 2;
            }
            q.pop();
            num[now] = id;
            vis[now] = 0;
            col[now] = 2;
        }
        while (q.size()) {
            int cur = q.top();
            q.pop();
            num[cur] = ++id;
            vis[cur] = 0;
            col[cur] = 1;
        }
    }
    bool flag = 0;
    for (int i = 1; i <= n; i++)
        if (col[i] == 2) {
            flag = 1;
            break;
        }
    if (!flag)
        break;
    vector<Edge> tmp;
    for (auto u : edge) {
        if (num[u.u] == num[u.v])
            continue;
        tmp.push_back({num[u.u], num[u.v], u.w - pre[u.v].w});
    }
    edge = tmp;
    r = num[r];
    n = id;
}
return 1;
}

```

每轮必须先确认所有非根点都存在入边；若某点没有入边，则根无法到达该点，树形图不存在。上方接口返回是否有解，权值通过 `res` 输出。

时间复杂度： $O(nm)$ 。

41.7.2 左偏树优化。

时间复杂度： $O(m + n \log n)$ 。

42 二分图

二分图是一种特殊的图结构。

如果图 G 可以划分成两个点集 V_1, V_2 使得两个点集内部不存在边, 则图 G 是一个二分图。

下文记顶点数为 $n = |V|$, 边数为 $m = |E|$ 。

42.1 判定

42.1.1 图 G 是 2- 着色的 $\Leftrightarrow$ 图 G 是二分图。

容易证明, 将点属于的集合 V_i 作为点的颜色, 那么二分图相邻点一定是不同的。

dfs 朴素染色即可, 若出现相邻节点颜色相同则图不是二分图。每个点只会被染色一次, 每条边只会被邻接表扫描常数次数, 所以时间复杂度为 $O(n + m)$ 。

```
bool dfs(int x, int c) {
    col[x] = c;
    for (auto u : p[x]) {
        if (!col[u]) {
            if (!dfs(u, 3 - c))
                return 0;
        } else if (col[u] == c)
            return 0;
    }
    return 1;
}
for (int i = 1; i <= n; i++) {
    if (!col[i]) {
        if (!dfs(i, 1)) {
            cout << "No\n";
            return;
        }
    }
}
```

42.1.2 图 G 中不存在奇环 $\Leftrightarrow$ 图 G 是二分图。

奇环无法二染色。对每个连通块建立一棵 dfs 生成树; 删去所有非树边后, 每个连通块是一棵树, 整个图剩下的是生成森林, 而不一定是一棵树。

对于一条非树边 (u, v) , 它与生成树上 u 到 v 的路径组成一个基环。树上路径长度的奇偶性等于 $dep_u + dep_v$ 的奇偶性, 因此当 dep_u, dep_v 同奇偶时, 这条非树边会形成奇环。

反过来, 若所有边的两个端点深度奇偶性都不同, 那么直接按深度奇偶二染色

即可。因此“不存在奇环”和“可以二染色”是等价的。

建立生成森林并扫描所有边,时间复杂度为 $O(n + m)$ 。

```
// 维护 dfs 生成森林
for (int i = 1; i <= n; i++) {
    for (auto u : p[i]) {
        if (u == fa[i] || fa[u] == i)
            continue;
        if ((dep[u] & 1) == (dep[i] & 1)) {
            // 存在奇环
            return;
        }
    }
}
```

42.1.3 扩展域并查集判二分图

从二分图定义出发,点集需要能被划分成两个集合。

那么对于一条边 (u, v) 就表示了 u 和 v 不能在同一个集合中。

使用扩展域并查集维护即可。

初始化 $2n$ 个元素需要 $O(n)$, 处理 m 条边的并查集操作需要 $O(m\alpha(n))$, 总时间复杂度为 $O(n + m\alpha(n))$ 。

```
for (auto [u, v] : edge) {
    if (dsu.same(u, v) || dsu.same(u + n, v + n)) {
        // 不能划分到两个集合中
        return;
    }
    dsu.merge(u, v + n);
    dsu.merge(u + n, v);
}
```

使用扩展域并查集可以在线判断二分图。

42.2 应用

基于二分图的特殊性质,一些在一般图上比较困难的问题会变得相对容易。

42.2.1 1. 二分图最大匹配

匹配指一组边,图中每个点只出现在其中一条边中。

最大匹配指要求选出最多边。

42.2.1.1 求解:

一般情况下,使用匈牙利算法即可求二分图的最大匹配,时间复杂度: $O(nm)$ 。

注:极端情况下,二分图使用网络流算法 Dinic 求最大流,时间复杂度为 $O(m\sqrt{n})$ 。

42.2.1.2 构造:

匈牙利算法可得最大匹配的一种可行方案。

42.2.2 2. 二分图最小点覆盖

最小点覆盖问题是指,在一张无向图中选择最少的顶点,满足每条边至少有一个端点被选。

42.2.2.1 求解:

一般图的最小点覆盖问题是 NP-hard 的。Kőnig 定理说明,在二分图上,最小点覆盖的大小等于最大匹配的大小,即 $|C_{\min}| = |M_{\max}|$ 。

42.2.2.2 构造:

先求出一组最大匹配 M ,再从左部点集 L 中所有未匹配点出发搜索交错路:从 L 到 R 只经过非匹配边,从 R 到 L 只经过匹配边。设搜索到的左右部点集分别为 Z_L, Z_R ,则

$$C_{\min} = (L \setminus Z_L) \cup Z_R.$$

不能从每条最大匹配边中任取一个端点;任意选择虽然恰好选出 $|M|$ 个点,却不保证覆盖所有非匹配边。

42.2.3 3.二分图最大独立集

最大独立集问题是指,在一张无向图中选择最多的顶点,满足两两之间互不相邻。

42.2.3.1 求解:

最大独立集问题对于一般图是 NP-hard 的,在二分图上,最大独立集 = 顶点数 - 最小点覆盖。

42.2.3.2 构造：

取最小点覆盖补集。

42.2.4 4.有向无环图最小路径覆盖

最小路径覆盖问题是指,在一张有向图中,选择最少数量的简单路径,使得所有顶点都恰好出现在一条路径中。

一般的有向图上的最小路径覆盖问题是 NP-hard 的。

42.2.4.1 求解：

DAG 的最小不相交路径覆盖 = 顶点数量 - 相应二分图 $G' = (V^{\text{out}}, V^{\text{in}}, E')$ 的最大匹配。具体地,对原图的每条边 $u \rightarrow v$,在 G' 中连接 u^{out} 与 v^{in} 。

42.2.4.2 构造：

42.2.5 5.有向无环图最小路径重复点覆盖

这里选择最少数量的路径覆盖所有点,但允许一个点出现在多条路径中。它与上一节“不重复经过顶点”的路径覆盖不是同一个问题。

42.2.5.1 求解：

记原图为 G ,求出传递闭包 G^+ :若 i 在 G 中可以到达 j ,就在 G^+ 中加入边 $i \rightarrow j$ 。在 G^+ 中,一条路径的相邻顶点只要求在原图中存在可达关系,因而可能跳过若干中间点。

对 G^+ 求不相交路径覆盖。若其拆点二分图的最大匹配大小为 c ,则重复点路径覆盖的最小路径数为 $n - c$ 。把 G^+ 中的每条可达边展开成 G 中的实际路径后,跳过的中间点可能出现在多条路径中,这正是“允许重复点”的来源。

如果题目允许直接把“可达”视为一步,可以输出 G^+ 中的路径;如果要求输出只由原图边组成的路径,则必须把每条可达边展开。若顶点不允许重复,则不能使用传递闭包替代原图,应使用上一节基于原边的构造。

使用 bitset 按逆拓扑序计算可达集合时,每条边可能触发一次长度为 n 的按位或,时间复杂度为

$$O\left(n + m + \frac{n^2 + mn}{w}\right),$$

其中 w 是机器字位数;稠密图上最坏为 $O(n^3/w)$,空间复杂度为 $O(n^2/w)$ 个机器字。显式建立传递闭包还可能产生 $O(n^2)$ 条边,之后仍需另计最大匹配的时间。

42.2.5.2 构造:

先根据最大匹配把 G^+ 中的顶点连接成若干条链,再按需要将每条可达边替换为原图中的一条实际路径。

42.2.6 6. 二分图最大权匹配

二分图的最大权匹配是指二分图中边权和最大的匹配。若允许顶点不匹配,则空匹配也是合法方案,答案至少为 0,负权边没有必要被选择。相应地,把所有实边权设为 1 时,最大权匹配就是最大匹配。

42.2.6.1 求解:

经典 KM 算法直接求解的是完全、等部二分图上的最大权完美匹配。设补全后两侧大小为 N ,时间复杂度为 $O(N^3)$ 。用于其他目标时需要先明确建模:

- 若要求原图的完美匹配,缺失边不能当成普通的 0 权边;应赋成不可选的负无穷,并在计算后验证答案没有使用缺失边。真实负权边可能被完美匹配强制选择。
- 若求允许不选边的任意大小最大权匹配,可以令 $N = \max(|L|, |R|)$,用虚点补齐较小的一侧,并用权值为 0 的人工边补成完全图。真实负权边也可按 0 权人工选择处理;KM 得到完美匹配后,只保留其中被选中的真实正权边,其余人工边表示对应顶点未匹配。
- 若要求“先最大化匹配边数,再最大化权值”,则不能直接使用上一种 0 权补边方式,需要先求最大匹配数,或给每条真实边增加足够大的统一权值以固定第一目标。

42.2.6.2 构造:

根据 KM 得到的匹配数组恢复配对;使用虚点或人工边建模时,需要删去这些不对应原图实边的配对。

43 Tarjan

先简单介绍一下 dfs 生成树。

dfs 生成树在于更好地描述一个图,以往对于图问题的理解都太抽象了,无法形成对一个图的形式化描述。

43.1 有向图 dfs 树

对一个有向图进行 dfs 遍历,遍历到边 (u, v) 时,若 v 未被遍历过,那么保留边 (u, v) ,令 u 是 v 的父节点。

那么形成一棵自祖先指向后代的有向树。

在遍历过程中,图中的边可以分成若干类: - 树边: dfs 生成树的边 - 反祖边(回边):指向祖先的边 - 横叉边:边的两点在 dfs 生成树中无祖先-后代关系 - 前向边:指向后代的边

43.2 无向图 dfs 树

对一个无向图进行 dfs 遍历,遍历到边 (u, v) 时,若 v 未被遍历过,那么保留边 (u, v) ,令 u 是 v 的父节点。

保留的边形成一棵无向树。

在遍历过程中,图中的边可以分成两类: - 树边: dfs 生成树中的边 - 非树边:连接祖先-后代的边

与有向图 dfs 生成树相比,无向图 dfs 生成树性质更强。

无向图 dfs 树没有横叉边、前向边,是因为如果边的另一端的点是已经遍历过的,那么就会成为树边。连接祖先-后代的边除外,因为 dfs 是深度优先,所以可以出现后代被遍历过。

43.2.1 环

容易发现,无向图环由若干条树边和非树边形成。

对于某一条非树边,其和所对应的树上的树链形成的环称为基环(笔者自己叫的),那么任意一个树上的环,都是由若干个连续基环形成的。

43.3 强连通分量

43.3.1 定义:

强连通:在有向图中,任意两个结点 u, v 都满足 u 可达 v 且 v 可达 u 。

强连通分量:极大的强连通子图。

43.3.2 Tarjan:

dfs 整个图,递归到 x 时,将 x 入栈。

维护两个数组: $- dfn_x$ 表示 dfs 过程中首次访问到的时间戳 $- low_x$ 表示 dfs 树中 x 的的子树中能够回溯到的还在栈中的最小 dfn。

当回溯到 $low_x = dfn_x$ 时,将栈中的元素弹出,形成一个强连通分量。

```
void dfs(int x) {
    low[x] = dfn[x] = ++idx;
    vis[x] = 1;
    q.push(x);
    for (auto u : p[x]) {
        if (!dfn[u]) {
            dfs(u);
            low[x] = min(low[x], low[u]);
        } else if (vis[u]) {
            low[x] = min(low[x], dfn[u]);
        }
    }
    if (dfn[x] == low[x]) {
        ++id;
        while (q.top() != x) {
            auto now = q.top();
            q.pop();
            vis[now] = 0;
            num[now] = id;
            scc[id].push_back(now);
        }
        q.pop();
        vis[x] = 0;
        num[x] = id;
        scc[id].push_back(x);
    }
}
// 求完 SCC 后 q 不一定为空
for (int i = 1; i <= n; i++) {
    if (!dfn[i])
        dfs(i);
}
```

43.3.3 性质:

- 一个点只属于一个 SCC。
- SCC 缩点后是 DAG。
- 一般 DAG 可以按 dfs 完成时间的逆序得到拓扑序,但不能直接使用首次访问时间 dfn 的逆序。本文代码在 SCC 弹栈时递增编号;若原图存在一条跨 SCC

的边 $u \rightarrow v$, 则 $num_u > num_v$ 。因此, SCC 编号的降序是缩点 DAG 的一个拓扑序, 升序是逆拓扑序。

43.3.4 杂谈:

强连通分量是环套环, 也就是强连通分量中的每一条边都被至少一个环经过。

强连通分量是极大强连通子图, 所以要求是“最大环”, 所以要求能回溯到的最小点。

$dfn_x = low_x$ 时说明环不能再扩大了, 所以形成了一个强连通分量。

43.4 边双连通分量

43.4.1 定义:

边双连通: 两个点 u 和 v , 如果无论删去哪一条边都不能使它们不连通。

边双连通分量: 极大边双连通分量。

43.4.2 Tarjan:

考虑这样一个结论: - 对于任意两个有公共点或公共边的环, 属于同一个边双连通分量(也就是边双连通分量的传递性)。

而, 对于强连通分量, 同样的, 对于任意两个有公共点或公共边的环, 属于同一个强连通分量。

边双连通分量和强连通分量一样是环套环, 所以求边双连通分量的过程实际上就是求强连通分量的过程。

直接套用 Tarjan 求强连通分量即可。

注意一点: x 不能走反边到 fa_x , 一方面, 这会和子树中回到到 fa_x 的 dfn 产生混淆; 另一方面, 这不符合 low 在 dfs 子树中的定义。

```
void dfs(int x, int y) {
    dfn[x] = low[x] = ++idx;
    vis[x] = 1;
    q.push(x);
    for (auto [u, id] : p[x]) {
        if ((id ^ y) == 1)
            continue;
        if (!dfn[u]) {
            dfs(u, id);
            low[x] = min(low[x], low[u]);
        } else if (vis[u]) {
            low[x] = min(low[x], dfn[u]);
        }
    }
}
```

```

    }
    if (dfn[x] == low[x]) {
        id++;
        while (q.top() != x) {
            auto now = q.top();
            q.pop();
            vis[now] = 0;
            ecc[id].push_back(now);
        }
        q.pop();
        vis[x] = 0;
        ecc[id].push_back(x);
    }
}
// 求完 ECC 后 q 不一定为空
for (int i = 1; i <= n; i++) {
    if (!dfn[i])
        dfs(i, 0);
}

```

43.4.3 性质:

边双连通分量缩完图后是一棵树,树上每一个节点是一个边双,每一条边是一条桥(反之亦然:每一个边双是缩完图后一个节点,每一条桥是缩完图后的每一条边)。

43.5 点双连通分量

43.5.1 定义:

点双连通:对于两个点 u 和 v ,如果无论删去哪个点(只能删去一个,且不能删 u 和 v 自己)都不能使它们不连通。

点双连通分量:一个无向图中的极大点双连通的子图。

43.5.2 Tarjan:

和强连通分量、边双连通分量不同,点双连通分量不具有传递性,一个点可能会属于多个点双连通分量。

但是只有割点会属于多个点双(容易证明,如果不是割点,那么删除这个点,这个点属于的多个点双会不连通(如果不会不连通那么这多个点双又可以合并成一个点双),那么就是割点)。

点双连通分量统计时,需要在从儿子回溯时统计,不能在回溯割点的时候统计。

对于 dfs 树边 (u, v) ,若 $low_v < dfn_u$,则 v 的子树可以绕过 u 到达 u 的真祖先;若 $low_v \geq dfn_u$,则 v 的子树无法绕过 u 到达 u 的父侧。此时将递归 v 后栈中

的点弹出,并加入 u , 形成一个点双连通分量; 当 u 不是 dfs 树根时, u 也是割点。注意 low_v 可能严格处于 dfn_u 与 dfn_v 之间, 不能只判断两个端点相等。

若 x 是 dfs 树的根需要特判, 因为此时删除 u 不会有父节点使得 dfs 树变成两部分。

具体而言: - 有两个以上子节点, 那么删除 u 后, 子节点会相互独立(无向图没有横叉边) - 只有一个子节点, 这个点双没有割点。 - 没有子节点, 这个点自身构成一个点双。

// $p[x]$ 存储 (to, eid) , 同一条无向边的两个方向编号互为 $eid \wedge 1$
 // y 表示进入 x 的边编号; dfs 树根没有父边, 传入 -1

```
void dfs(int x, int y, int root) {
    dfn[x] = low[x] = ++idx;
    if (x == root && p[x].empty()) {
        vcc[++id].push_back(x);
        return;
    }
    q.push(x);
    int son = 0;
    for (auto [u, eid] : p[x]) {
        if (y != -1 && eid == (y ^ 1))
            continue;
        if (!dfn[u]) {
            dfs(u, eid, root);
            low[x] = min(low[x], low[u]);
            if (low[u] >= dfn[x]) {
                if (x != root)
                    cut[x] = 1;
                id++;
                while (q.top() != u) {
                    auto now = q.top();
                    q.pop();
                    vcc[id].push_back(now);
                }
                q.pop();
                vcc[id].push_back(u);
                vcc[id].push_back(x);
            }
            if (x == root)
                son++;
        } else
            low[x] = min(low[x], dfn[u]);
    }
    if (son >= 2 && x == root)
        cut[x] = 1;
}
// 求完 VCC 后 q 不一定为空
for (int i = 1; i <= n; i++)
    if (!dfn[i])
        dfs(i, -1, i);
```

43.5.3 性质:

点双连通分量缩完图后是一棵树, 树上每一个点是割点或是点双, 每一条边表示连接原图中割点和其所在点双的边(反之亦然: 每一个点双是缩完图后一个节点, 每一个割点是缩完图后的一个点)。

43.6 2-sat

2-sat 是用于解决若干个或命题解的算法。

有 n 个布尔变量 a_i 和 m 个形如 $c_j = l_{x_j} \vee l_{y_j}$ 的约束, 其中 l 表示变量或其否定, 求使所有 c_j 都为真的解。

2-sat 建边的本质是 $a \rightarrow b$ 的蕴含式, 所以不局限于 $a \vee b$ 。

43.6.1 求解:

根据离散数学基础, 易得: $a \vee b = \neg a \rightarrow b \wedge \neg b \rightarrow a$ 。

用 i 表示 a_i , $i+n$ 表示 $\neg a_i$ 。对于约束 $l_x \vee l_y$, 分别建立 $\neg l_x \rightarrow l_y$ 和 $\neg l_y \rightarrow l_x$ 两条边。例如, $a_x \vee a_y$ 对应边 $(x+n, y)$ 和 $(y+n, x)$ 。

那么可以发现, 建出来的图中, 同一个强连通分量中的点取值相同: - 如果有一个点为 1, 易得走下去都是 1。- 反之就都是 0。

所以原问题无解, 当且仅当 i 和 $i+n$ 在同一个强连通分量中。

有解时, 根据拓扑序, $u \rightarrow v$, u 为 1 时, v 一定为 1, u 为 0 时 v 可以为 1/0。

若存在边 (a, b) 那么一定存在边 $(\neg b, \neg a)$

若存在路径 $a \rightarrow b$ 那么一定存在路径 $\neg b \rightarrow \neg a$

结合上述结论, 每个 SCC 与其中所有文字取反后得到的 SCC 成对出现。本文 Tarjan 代码在 SCC 弹栈时递增编号, 编号越小, 在缩点 DAG 的正向拓扑序中越靠后。因此可以构造

$$a_i = [num_i < num_{i+n}].$$

也就是说, 在 a_i 与 $\neg a_i$ 所属的两个 SCC 中, 把正向拓扑序更靠后的文字取为真。若采用按正向拓扑序递增的 SCC 编号, 以上不等号需要反向, 不能脱离编号约定记忆真值方向。

蕴含图有 $2n$ 个点和 $2m$ 条边, 使用 Tarjan 求强连通分量并构造解的时间复杂度为 $O(n+m)$ 。

43.7 圆方树

43.7.1 仙人掌

43.7.2 广义圆方树

44 环计数问题

无向图环计数, 扩展性较差。

主要有以下三类:

44.1 普通环计数

以下默认讨论无自环、无重边的简单无向图; 简单环的长度至少为 3。

在一般无向图中精确计数所有简单环是 $\#P$ -complete 的计数问题, 而不是 P -complete。判定图中是否存在环本身可以在线性时间完成; 困难的是精确计数。当前没有已知的多项式时间精确算法, 通常只能在 $FP \neq \#P$ 的复杂性假设下说明其不存在, 不能写成无条件结论。

和哈密顿路径类似, 考虑状压 dp。

为避免同一个环从不同顶点开始而被重复统计, 钦定环上编号最小的节点 r 为起止点。

令 $f_{S,v}$ 表示从 $r = \min S$ 出发, 恰好经过点集 S , 当前终点为 v 的简单路径数量。对每个单点集合初始化 $f_{\{r\},r} = 1$; 转移时只加入满足 $u > r$ 、 $u \notin S$ 且 $(v,u) \in E$ 的点:

$$f_{S \cup \{u\},u} += f_{S,v}.$$

当 $|S| \geq 3$ 且 $(v,r) \in E$ 时, 将 $f_{S,v}$ 加入闭环计数。这样每个简单环只会以其最小编号点为起点, 但仍会按顺、逆两个方向各统计一次。

时间复杂度: $O(2^nm)$ 或 $O(2^n n^2)$ 。

因此最终答案直接除以 2。简单无向图中沿同一条边 $u \rightarrow v \rightarrow u$ 往返使用了同一条边两次, 不是简单环, 不应先计入再减去。

若题目讨论允许重边的多重图, 并把两条不同的平行边视为长度为 2 的环, 则应根据平行边数量另外统计; 这与沿同一条无向边往返不同。环数可能很大, 实际实现还需按题意使用大整数或取模。

44.2 三元环计数

给边定向,规定从度数小的点指向度数大的点,度数相同的点,由编号小的指向编号大的。

所有边都按“度数、编号”这一全序定向。先标记 u 的所有出邻点,再枚举两条连续的出边 $u \rightarrow v$ 、 $v \rightarrow w$;若同时存在 $u \rightarrow w$,则 (u, v, w) 构成一个三元环。每个无向三元环只会按上述全序统计一次。

可以证明,时间复杂度: $O(m\sqrt{m})$ 。

```
for (int i = 1; i <= m; i++) {
    if (deg[u[i]] < deg[v[i]])
        add(u[i], v[i]);
    else if (deg[v[i]] < deg[u[i]])
        add(v[i], u[i]);
    else if (u[i] < v[i])
        add(u[i], v[i]);
    else
        add(v[i], u[i]);
}
long long ans = 0;
for (int i = 1; i <= n; i++) {
    for (auto v : p[i])
        vis[v] = 1;
    for (auto u : p[i])
        for (auto v : p[u])
            if (vis[v])
                ans++;
    for (auto v : p[i])
        vis[v] = 0;
}
```

44.3 四元环计数

四元环即: $a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$,注意到:枚举 a, c 时, b, d 就可以独立统计了。

对边排序,度数小的排在前面,度数大的排在后面。

枚举度数大的点作为 a ,再枚举度数小的点作为 c ,求 a, c 之间有多少点满足和 a, c 都有边即可。

因为 b, d 这里独立了,所以枚举复杂度本质和前文三元环计数相同,可以证明,时间复杂度: $O(m\sqrt{m})$ 。

四元环数量可能超过 32 位整数范围,答案使用 long long 保存;若题目规模仍可能超过 64 位,则需使用大整数或按题意取模。

```

vector<int> a(n + 1);

for (int i = 1; i <= n; i++)
    a[i] = i;

auto cmp = [&](int x, int y) { return deg[x] < deg[y]; };
sort(a.begin() + 1, a.end(), cmp);
vector<int> rk(n + 1);

for (int i = 1; i <= n; i++)
    rk[a[i]] = i;

for (int i = 1; i <= n; i++) {
    for (auto u : p[i]) {
        if (rk[u] < rk[i]) {
            edge[i].push_back(u);
        }
    }
}

long long ans = 0;
vector<int> cnt(n + 1);

for (int i = 1; i <= n; i++) {
    for (auto u : edge[a[i]]) {
        for (auto v : p[u]) {
            if (rk[v] >= rk[a[i]])
                continue;

            ans += cnt[v];
            cnt[v]++;
        }
    }

    for (auto u : edge[a[i]]) {
        for (auto v : p[u]) {
            cnt[v] = 0;
        }
    }
}

```

44.4 五元环计数

无向图上,起止点相同的长度为五的路径只有两种情况: - 五元环 - 三元环上插入一次沿某条边走出再返回的二步回走

长度为五的路径数,容易通过 dp 实现。

矩阵迹 $\text{tr}(A^5)$ 统计带起点和方向的长度为 5 的闭游走,每个五元环贡献 10 次。先除以 10 后,一个三元环 (x, y, z) 产生的二步回走贡献为 $(\deg_x - 1) + (\deg_y - 1) + (\deg_z - 1)$,所以再枚举每个三元环减去这部分即可。

时间复杂度: $O(n^3)$ 。

dp 中的闭游走数量和最终答案都可能超过 32 位整数范围,统一使用 long long;若结果可能超过 64 位,则仍需使用大整数或取模。

```

long long dp[6][N][N];
for (int i = 1; i <= m; i++) {
    int u, v;
    cin >> u >> v;
    dp[1][u][v] = dp[1][v][u] = 1;
    deg[u]++, deg[v]++;
}
for (int i = 2; i <= 5; i++) {
    for (int j = 1; j <= n; j++) {
        for (int k = 1; k <= n; k++) {
            for (int l = 1; l <= n; l++) {
                dp[i][k][l] += dp[i-1][k][j] * dp[1][j][l];
            }
        }
    }
}
long long ans = 0;
for (int i = 1; i <= n; i++)
    ans += dp[5][i][i];
ans /= 10;
for (int i = 1; i <= n; i++) {
    for (int j = i + 1; j <= n; j++) {
        for (int k = j + 1; k <= n; k++) {
            if (dp[1][i][j] && dp[1][j][k] && dp[1][k][i]) {
                ans -= 1LL * deg[i] + deg[j] + deg[k] - 3;
            }
        }
    }
}
}

```

44.5 固定长度 $X \geq 6$ 的环计数

本文暂未给出 $X \geq 6$ 的具体实现,这不表示不存在高效或专门算法。

当 X 是固定常数时,枚举本身已经给出关于 n 的多项式算法;还可以使用颜色编码、分治、矩阵乘法和其他代数方法改进不同图类上的检测或计数复杂度。当 X 也是输入的一部分,或要求统计所有长度的简单环时,问题的复杂度则不同,需要单独讨论。

44.6 有向图三元环计数

即: $u \rightarrow v \rightarrow w \rightarrow u$

对于 v, w 考虑使用 `bitset` 维护可达 v 的点和 w 可达的点集。

枚举边 $v \rightarrow w$,对 `bitset` 求交,时间复杂度: $O(\frac{n^3}{w})$ 。

45 欧拉路

欧拉迹:经过图中每条边恰好一次的迹,起点和终点可以相同,也可以不同。

欧拉开路:起点和终点不同的欧拉迹。本文后续所说的“欧拉路径”专指欧拉开路。

欧拉回路:经过图中每条边恰好一次的回路(起点和终点相同)。

注: - 欧拉路径和欧拉回路对不限制点的经过次数,可重复经过一个点。 - 孤立点不经过任何边,不影响欧拉路径/回路是否存在,无需从图中删除。只需在连通性判定时忽略零度点;当 $m > 0$ 时,构造起点必须选择非零度点。 - 当图中没有边时,是否把单个顶点上的长度为 0 的迹视为欧拉回路,应按题目约定处理。

45.1 判定:

- 有向图

- 欧拉路径:图中恰好存在 1 个点出度比入度多 1 (该点为起点),恰好存在 1 个点入度比出度多 1 (该点为终点),其余点入度等于出度。
- 欧拉回路:所有点的入度等于出度;当 $m > 0$ 时,可以任选一个非零度点作为起点。

- 无向图

- 欧拉路径:图中恰好有两个点的度数为奇数(这两个点为起点和终点,起点和终点可互换),其余点的度数为偶数。
- 欧拉回路:所有点的度数都为偶数;当 $m > 0$ 时,可以任选一个非零度点作为起点。

上述度数条件还必须配合连通性条件:无向图中所有非零度点必须位于同一个连通块;有向图中所有入度或出度非零的点,必须在忽略边方向后位于同一个连通块。孤立点可以属于其他连通块,不影响结论。

若题目把“欧拉路径”用作允许起终点相同的广义称呼,则满足对应的欧拉开路条件或欧拉回路条件之一即可。

45.2 求解：

45.2.1 有向图欧拉路径(开路)：

根据判定条件,找到唯一的那一个出度 = 入度 +1 的点作为起点, dfs 递归第一次经过某条边,递归前删除这条边。

45.2.2 有向图欧拉回路：

根据判定条件,当 $m > 0$ 时任选一个非零度点作为起点, dfs 递归第一次经过某条边,递归前删除这条边。

45.2.3 无向图欧拉路径(开路)：

根据判定条件,找到度数为奇数的一个点,作为起点, dfs 递归第一次经过某条边,递归前删除这条边。

45.2.4 无向图欧拉回路：

根据判定条件,当 $m > 0$ 时任选一个非零度点作为起点, dfs 递归第一次经过某条边,递归前删除这条边。

注:无向图需要注意标记把同一条边的反边也给删了。

注:上述四种情况的算法实现流程完全一致。

时间复杂度: $O(n + m)$ 。

Hierholzer 算法在回溯时后序写入答案,因此构造完成后必须反转。即使度数条件已经满足,也应检查是否使用了全部 m 条边:有向图的顶点序列应有 $m + 1$ 个点,无向图的边序列应有 m 条边;否则说明仍有非零度部分与起点不连通。下方模板中的 m 均指原图边数,调用前应清空 `del`,无向图还需清空 `vis`;包装函数会清空 `ans`。

有向图：

```
void dfs(int x) {
    for (unsigned i = del[x]; i < p[x].size(); i = del[x]) {
        del[x]++;
        dfs(p[x][i]);
    }
    ans.push_back(x);
}

bool euler(int st) {
    ans.clear();
    dfs(st);
    reverse(ans.begin(), ans.end());
    return (int)ans.size() == m + 1;
}
```


无向图：

```
void dfs(int x) {
    for (unsigned i = del[x]; i < p[x].size(); i = del[x]) {
        del[x]++;
        auto [u, id] = p[x][i];
        if (vis[abs(id)])
            continue;
        vis[abs(id)] = 1;
        dfs(u);
        ans.push_back(id);
    }
}

bool euler(int st) {
    ans.clear();
    dfs(st);
    reverse(ans.begin(), ans.end());
    return (int)ans.size() == m;
}
```

递归实现的调用深度最坏可达 $O(m)$, 大图上可能导致栈溢出。下面给出等价的迭代写法。无向图假设同一条边在两个方向分别使用 id 和 $-id$, 其中 $|id| \in [1, m]$ 。

有向图迭代实现：

```
bool euler(int st) {
    vector<int> stk;
    ans.clear();
    stk.push_back(st);
    while (!stk.empty()) {
        int x = stk.back();
        if (del[x] < p[x].size()) {
            stk.push_back(p[x][del[x]++]);
        } else {
            ans.push_back(x);
            stk.pop_back();
        }
    }
    reverse(ans.begin(), ans.end());
    return (int)ans.size() == m + 1;
}
```

无向图迭代实现：

```
bool euler(int st) {
    vector<pair<int, int>> stk;
    ans.clear();
    stk.push_back({st, 0});
    while (!stk.empty()) {
        int x = stk.back().first;
        while (del[x] < p[x].size() && vis[abs(p[x][del[x]].second)])
            del[x]++;
        if (del[x] == p[x].size()) {
            int id = stk.back().second;
            stk.pop_back();
            if (id)
                ans.push_back(id);
        } else {
            auto [u, id] = p[x][del[x]++];
            vis[abs(id)] = 1;
            stk.push_back({u, id});
        }
    }
    reverse(ans.begin(), ans.end());
    return (int)ans.size() == m;
}
```


第五部分

多项式

46 基础-多项式全家桶

仅含【洛谷】紫以下多项式。

46.1 FFT

设两多项式长度分别为 n, m , 所需变换长度为不小于 $n + m - 1$ 的最小的 2 的幂 L 。调用乘法前应先执行 `init(L)`; 根表和 `rev` 按 L 动态分配, 不需要固定开到某个模数所支持的最大长度。

```
const ldb pi = acos(-1.0);
struct Complex {
    ldb x, y;
    Complex operator+(const Complex &t) const { return {x + t.x, y + t.y}; }
    Complex operator-(const Complex &t) const { return {x - t.x, y - t.y}; }
    Complex operator*(const Complex &t) const {
        return {x * t.x - y * t.y, x * t.y + y * t.x};
    }
};
vector<int> rev;
vector<array<Complex, 2>> g;
void init(int n) { // n 为预先确定的最大变换长度
    assert(n > 0 && (n & (n - 1)) == 0);
    rev.resize(n), g.resize(n);
    for (int mid = 1; mid < n; mid <= 1) {
        g[mid][0] = Complex({cos(pi / mid), -sin(pi / mid)});
        g[mid][1] = Complex({cos(pi / mid), sin(pi / mid)});
    }
}
void fft(vector<Complex> &a, int sign, int tot) {
    assert((int)rev.size() >= tot && (int)g.size() >= tot);
    for (int i = 0; i < tot; i++)
        if (i < rev[i])
            swap(a[i], a[rev[i]]);

    for (int mid = 1; mid < tot; mid <= 1) {
        auto w1 = g[mid][((sign + 1) / 2)];
        for (int i = 0; i < tot; i += (mid <= 1)) {
            auto wk = Complex({1, 0});
            for (int j = 0; j < mid; j++, wk = wk * w1) {
                auto x = a[i + j], y = wk * a[i + j + mid];
                a[i + j] = x + y, a[i + j + mid] = x - y;
            }
        }
    }
}
```

46.2 NTT

常见 NTT 模数及原根: $-65537 = 2^{16} + 1, g = 3$; $-998244353 = 119 \times 2^{23} + 1, g = 3$; $-1004535809 = 479 \times 2^{21} + 1, g = 3$; $-4179340454199820289 = 29 \times 2^{57} + 1, g = 3$; $-167772161 = 5 \times 2^{25} + 1, g = 3$

NTT 的变换长度 L 除了满足 $L \geq n + m - 1$, 还必须满足 $L \mid (mod - 1)$ 。例如

998244353 最多支持 2^{23} 点NTT,但实际容量仍应按程序中可能出现的最大变换长度分配; Newton 迭代等内部调用也要计入。下面的 `int` 模板适用于 998244353 等 32 位模数;表中的 64 位模数需要使用 64 位存储和 `__int128` 或安全模乘,不能直接套用该模板。整数快速幂 `qz` 内的乘法也必须使用 64 位中间量。

```
vector<int> rev, g, inv_g;
void init(int n) { // n 为预先确定的最大变换长度
    assert(n > 0 && (n & (n - 1)) == 0 && (mod - 1) % n == 0);
    rev.resize(n), g.resize(n + 1), inv_g.resize(n + 1);
    for (int mid = 1; mid < n; mid <= 1) {
        g[mid << 1] = qz(G, (mod - 1) / (mid << 1));
        inv_g[mid << 1] = qz(inv_G, (mod - 1) / (mid << 1));
    }
}

void ntt(vector<int> &a, int sign, int tot) {
    assert((int)rev.size() >= tot && (int)g.size() > tot);
    for (int i = 0; i < tot; i++)
        if (i < rev[i])
            swap(a[i], a[rev[i]]);
    for (int mid = 1; mid < tot; mid <= 1) {
        int g1 = ~sign ? g[mid << 1] : inv_g[mid << 1];
        for (int i = 0; i < tot; i += (mid << 1)) {
            int gk = 1;
            for (int j = 0; j < mid; j++, gk = 1ll * gk * g1 % mod) {
                int x = a[i + j], y = 1ll * gk * a[i + j + mid] % mod;
                a[i + j] = (x + 1ll * y) % mod;
                a[i + j + mid] = (x - 1ll * y + mod) % mod;
            }
        }
    }
    if (sign == -1) {
        int inv = qz(tot, mod - 2);
        for (int i = 0; i < tot; i++)
            a[i] = 1ll * a[i] * inv % mod;
    }
}
```

46.3 多项式乘法

```
vector<int> mul(vector<int> a, vector<int> b) {
    int tot = 0, bit = 0;
    int n = a.size(), m = b.size();
    while ((1 << bit) <= n - 1 + m - 1)
        bit++;
    tot = 1 << bit;
    assert((int)rev.size() >= tot && (int)g.size() > tot);
    a.resize(tot), b.resize(tot);
    vector<int> c(tot);
    for (int i = 0; i < tot; i++)
        rev[i] = (rev[i >> 1] >> 1) | ((i & 1) * (tot >> 1));
    ntt(a, 1, tot), ntt(b, 1, tot);
    for (int i = 0; i < tot; i++)
        c[i] = 1ll * a[i] * b[i] % mod;
    ntt(c, -1, tot);
    c.resize(n + m - 1);
    return c;
}
```

FFT 处理多项式乘法时,特殊处理:

```

vector<int> mul(vector<int> a, vector<int> b) {
    vector<Complex> A(a.size()), B(b.size());
    for (int i = 0; i < A.size(); i++)
        A[i].x = a[i];
    for (int i = 0; i < B.size(); i++)
        B[i].x = b[i];

    int tot = 0, bit = 0;
    int n = A.size(), m = B.size();
    while ((1 << bit) <= n + m - 1)
        bit++;
    tot = 1 << bit;
    assert((int)rev.size() >= tot && (int)g.size() >= tot);
    A.resize(tot), B.resize(tot);
    for (int i = 0; i < tot; i++)
        rev[i] = (rev[i >> 1] >> 1) | ((i & 1) * (tot >> 1));
    fft(A, 1, tot), fft(B, 1, tot);
    for (int i = 0; i < tot; i++)
        A[i] = A[i] * B[i];
    fft(A, -1, tot);
    vector<int> c(n + m - 1);
    for (int i = 0; i < n + m - 1; i++)
        c[i] = (int)llround(A[i].x / tot);
    return c;
}

```

这里假设输入系数为整数,且真实卷积系数与浮点计算结果的绝对误差小于 0.5,才能用 `llround` 唯一还原。若输入系数绝对值分别不超过 A, B ,则单个卷积系数的绝对值至多为 $\min(n, m)AB$;上述返回类型还要求该值不超过 `int` 范围。长度或系数过大时应使用拆系数FFT、NTT/CRT,不能把浮点舍入视为无条件精确。

46.4 任意模数多项式乘法

设两多项式的长度分别为 n, m ,输入系数的绝对值不超过 V ,则整数卷积的单个系数绝对值至多为 $B = \min(n, m)V^2$ 。所以“任意模数多项式乘法”并不是真正脱离值域限制,而是先借助若干NTT 模数恢复整数卷积,再对目标模数取模。

一种朴素实现是三模数 NTT,需要 9 次 DFT,常数较大。

具体实现为选取三个两两互质的 NTT 模数,进行三次NTT,得到同余方程组,再用 CRT 或 Garner 算法合并。设组合模数为 M :若待恢复系数非负,需要 $M > B$;若允许负系数并采用中心剩余表示,则需要 $M > 2B$,将大于 $M/2$ 的结果解释为负数后再对目标模数取模。

三个约 10^9 的模数之积会超过 64 位整数范围,不能直接计算 $m1 * m2 * m3$ 。实现时应使用 `__int128`、安全模乘,或用 Garner 算法直接在目标模数下合并,并保证每一步乘法的中间类型足够宽。

这里仅提供使用的三个 NTT 模数。

```
const int mod[3] = {998244353, 469762049, 1004535809},
        inv_G[3] = {332748118, 156587350, 334845270}, G = 3;
```

拆系数 FFT, 需要 4 次 DFT, 常数较小。

46.5 多项式乘法逆

以下在素数模数 mod 下计算 $a(x)^{-1} \bmod x^n$, 要求 $a_0 \neq 0$ 。更一般地, 常数项必须在系数环中可逆; 代码中的 a_0^{mod-2} 只适用于素数模数。

时间复杂度: $O(n \log n)$ 。

```
vector<int> inv(vector<int> &a) {
    int n = a.size();
    vector<int> A, B, b = {qz(a[0], mod - 2)};
    for (int len = 1, tot; len < (n << 1); len <= 1) {
        tot = len << 1;
        A.resize(tot), B.resize(tot);
        for (int i = 0; i < len; i++) {
            A[i] = i < a.size() ? a[i] : 0, B[i] = i < b.size() ? b[i] : 0;
        }
        for (int i = 0; i < tot; i++)
            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) * (tot >> 1));
        ntt(A, 1, tot), ntt(B, 1, tot);
        b.resize(tot);
        for (int i = 0; i < tot; i++)
            b[i] = (2ll - 1ll * A[i] * B[i] % mod + mod) * B[i] % mod;
        ntt(b, -1, tot);
        b.resize(len);
    }
    b.resize(n);
    return b;
}
```

46.6 多项式开根

以下均在奇素数模数下计算 $b(x)^2 \equiv a(x) \pmod{x^n}$, 所有迭代结果都只保留前 n 项。先保证 $a_0 = 1$, 并选择 $b_0 = 1$ 这一支:

```
vector<int> sqrt(vector<int> &a) {
    int n = a.size();
    vector<int> A, B, b = {1};
    for (int len = 1, tot; len < (n << 1); len <= 1) {
        tot = len << 1;
        A.resize(tot);
        for (int i = 0; i < len; i++)
            A[i] = i < a.size() ? a[i] : 0;
        b.resize(len);
        B = inv(b);
        B.resize(tot);
        for (int i = 0; i < tot; i++)
            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) * (tot >> 1));
        ntt(A, 1, tot), ntt(B, 1, tot);
        for (int i = 0; i < tot; i++)
            A[i] = 1ll * A[i] * B[i] % mod;
        ntt(A, -1, tot);
        for (int i = 0; i < len; i++)
            b[i] = (1ll * b[i] + A[i]) * inv_2 % mod;
    }
    b.resize(n);
}
```



```

    return b;
}

```

若 $a_0 \neq 0$ 且是模意义下的二次剩余, 先用 Cipolla 等算法求一个根 r , 再把 Newton 迭代的初值改为 $b = \{r\}$; 选择 r 或 $\text{mod} - r$ 会得到常数项不同的两支平方根。若 $a_0 = 0$, 还需要先处理最低非零项的次数, 不能直接套用这段迭代。

```

int I_mul_I;
struct Complex {
    int real, imag;
    Complex(int real = 0, int imag = 0) : real(real), imag(imag) {}
    bool operator==(const Complex &x) const {
        return x.real == real && x.imag == imag;
    }
    Complex operator*(const Complex &x) const {
        return Complex((1ll * x.real * real + 1ll * I_mul_I * x.imag % mod * imag) % mod,
            (1ll * x.imag * real + 1ll * x.real * imag) % mod);
    }
};
Complex qz(Complex x, int y) {
    Complex res = 1;
    for (; y; y >>= 1) {
        if (y & 1)
            res = res * x;
        x = x * x;
    }
    return res;
}
bool check(int x) { return qz(x, mod - 1 >> 1) == 1; }
int solve(int n) { // 求解 n 的二次剩余
    if (!n)
        return 0;
    if (!check(n))
        return -1;
    int a = rand() % mod;
    while (!a || check((1ll * a * a + mod - n) % mod))
        a = rand() % mod;
    I_mul_I = (1ll * a * a + mod - n) % mod;
    return qz(Complex(a, 1), mod + 1 >> 1).real;
}

```

46.7 多项式除法

以下假设系数数组已经去除无意义的高位零, 且除式 $b(x)$ 非零; 反转后用于求逆的常数项就是 $b(x)$ 的最高次项, 因此它必须在模意义下可逆。当 $\deg a < \deg b$ 时, 商为零, 余数为 $a(x)$, 应直接返回。

```

pair<vector<int>, vector<int>> divide(vector<int> a, vector<int> b) {
    int n = a.size(), m = b.size();
    if (n < m)
        return {{0}, a};
    auto ar = a, br = b;
    reverse(ar.begin(), ar.end());
    reverse(br.begin(), br.end());
    ar.resize(n - m + 1), br.resize(n - m + 1);
    auto now = inv(br);
    auto cr = mul(ar, now);
    cr.resize(n - m + 1);
    reverse(cr.begin(), cr.end());
    auto cur = subtract(a, mul(b, cr));
    return {cr, cur};
}

```

46.8 多项式 $\ln$

以下按 $\ln a(x) = \int a'(x)/a(x) dx$ 计算 $\ln a(x) \bmod x^n$ 。保证 $a_0 = 1$ ；在素数模数下还需 $n \leq \text{mod}$ ，使积分中出现的 $1, 2, \dots, n-1$ 都可逆。

```
vector<int> ln(vector<int> &a) {
    int n = a.size();
    vector<int> A(n - 1);
    for (int i = 1; i < n; i++)
        A[i - 1] = 1ll * a[i] * i % mod;
    auto B = inv(a);

    int bit = 0;
    while ((1 << bit) < (n << 1))
        bit++;
    int tot = 1 << bit;
    A.resize(tot), B.resize(tot);
    for (int i = 0; i < tot; i++)
        rev[i] = (rev[i >> 1] >> 1) | ((i & 1) * (tot >> 1));

    ntt(A, 1, tot), ntt(B, 1, tot);
    for (int i = 0; i < tot; i++)
        A[i] = 1ll * A[i] * B[i] % mod;
    ntt(A, -1, tot);

    vector<int> b(n);
    for (int i = 1; i < n; i++)
        b[i] = 1ll * A[i - 1] * iv[i] % mod; // i 的逆元
    return b;
}
```

46.9 多项式 $\exp$

以下计算 $\exp(a(x)) \bmod x^n$ ，并选择常数项为 1 的形式幂级数解。保证 $a_0 = 0$ ；在素数模数下同样需要 $n \leq \text{mod}$ ，使相关阶乘和积分分母可逆。

```
vector<int> exp(vector<int> &a) {
    int n = a.size();
    vector<int> b = {1}, A, B;
    for (int len = 1, tot; len < (n << 1); len <= 1) {
        tot = len << 1;
        A.resize(tot);
        for (int i = 0; i < len; i++)
            A[i] = i < n ? a[i] : 0;
        b.resize(len);
        B = ln(b);
        b.resize(tot), B.resize(tot);
        for (int i = 0; i < tot; i++)
            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) * (tot >> 1));
        ntt(b, 1, tot), ntt(A, 1, tot), ntt(B, 1, tot);
        for (int i = 0; i < tot; i++)
            b[i] = (1ll - B[i] + A[i] + mod) * b[i] % mod;
        ntt(b, -1, tot);
        b.resize(len);
    }
    b.resize(n);
    return b;
}
```

46.10 多项式幂

以下约定 $0^0 = 1$, 指数 k 为非负整数, 所有结果均截断至模 x^n 。

先考虑 $a_0 = 1$ 。

在素数模数 mod 下, 若截断长度满足 $n \leq mod$, 可以使用

$$a(x)^k = \exp(k \ln a(x)) \pmod{x^n}.$$

此时传给 $\exp$ 的标量 k 可以对 mod 取模。这依赖于次数低于模数、积分分母均可逆, 不能作为任意特征和任意截断次数下的指数降模规则。

也可以直接快速幂, 时间复杂度为 $O(M(n) \log k) = O(n \log n \log k)$; 当 k 的位数为 $O(\log n)$ 时可写成 $O(n \log^2 n)$ 。

```
vector<int> qz(vector<int> &a, int k) {
    int n = a.size();
    auto x = a;
    vector<int> res = {1};
    int bit = 1;
    while (bit < (n << 1))
        bit <<= 1;
    x.resize(bit), res.resize(bit);
    int tot = bit;
    for (int i = 0; i < tot; i++)
        rev[i] = (rev[i >> 1] >> 1) | ((i & 1) * (tot >> 1));
    for (; k >>= 1; k >>= 1) {
        x.resize(tot), ntt(x, 1, tot);
        if (k & 1) {
            res.resize(tot), ntt(res, 1, tot);
            for (int i = 0; i < tot; i++) {
                res[i] = 1ll * res[i] * x[i] % mod;
            }
            ntt(res, -1, tot), res.resize(n);
        }
        for (int i = 0; i < tot; i++) {
            x[i] = 1ll * x[i] * x[i] % mod;
        }
        ntt(x, -1, tot), x.resize(n);
    }
    res.resize(n);
    return res;
}
```

$\ln + \exp$, 时间复杂度: $O(n \log n)$ 。

```
vector<int> qz(vector<int> &a, int k) {
    auto A = ln(a);
    int n = a.size();
    for (int i = 0; i < n; i++)
        A[i] = 1ll * A[i] * k % mod;
    return exp(A);
}
```

一般情况下, 设 t 是 $a(x)$ 最低非零项的次数、该项系数为 c , 则可以写成

$$a(x) = x^t c b(x), \quad b_0 = 1,$$

从而

$$a(x)^k = x^{tk} c^k b(x)^k \pmod{x^n}.$$

必须先用**原始指数**判断 tk : 若 $k > 0$ 且 $tk \geq n$, 结果为零; 否则只需求 $b(x)^k \pmod{x^{n-tk}}$, 乘上 c^k 后整体右移 tk 位。不能先对 k 取模再计算次数平移。若 $a(x)$ 是零多项式, 则 $k > 0$ 时结果为零, $k = 0$ 时按上述约定返回 1。

在素数模数 mod 下, $c \neq 0$ 时可用费马小定理将 c^k 的指数对 $mod - 1$ 取模; 而使用 `ln + exp` 计算 $b(x)^k$ 时, 只有在所需截断长度不超过 mod 的前提下, 标量 k 才可对 mod 取模。两处降模的对象和条件不同。对于合数模数, 只有 c 与模数互质时才能使用欧拉定理, 形式对数与指数也不能直接沿用上述素数域模板。

46.11 拉格朗日插值

n 个模意义下互不相同的点, 可以确定一个不超过 $n - 1$ 次的多项式。以下代码使用费马小定理求逆, 因此假设模数为素数, 且输入坐标已经规范化到 $[0, mod)$ 。

拉格朗日插值 $O(n^2)$ 根据 n 个坐标求 $f(k)$ 。

插值公式:

$$f(k) = \sum_{i=0}^{n-1} y_i \prod_{\substack{0 \leq j < n \\ j \neq i}} \frac{k - x_j}{x_i - x_j}$$

按照公式计算即可。

```
int f(vector<int> &x, vector<int> &y, int k) {
    int n = x.size();
    int ans = 0;
    for (int i = 0; i < n; i++) {
        int fz = y[i], fm = 1;
        for (int j = 0; j < n; j++) {
            if (i == j)
                continue;
            fz = 1ll * fz * (k - x[j] + mod) % mod;
            fm = 1ll * fm * (x[i] - x[j] + mod) % mod;
        }
        ans = (ans + 1ll * fz * qz(fm, mod - 2)) % mod;
    }
    if (ans < 0)
        ans += mod;
    return ans;
}
```

特别地, 若给定的坐标连续, 可以 $O(n)$ 求 $f(k)$ 。

具体而言, 若给定 $(i, f(i)), i \in [0, n]$, 这些 $n + 1$ 个采样点需要在模意义下互不相同, 因此要求 $n < mod$ 。令 $S = \prod_{j=0}^n (k - j)$, 当 k 不等于任一采样点时:

$$f(k) = \sum_{i=0}^n f(i) \frac{(-1)^{n-i} S}{(k-i)! (n-i)!}.$$

若 $k \equiv i \pmod{mod}$, 上式会出现形式上的 $0/0$, 应直接返回样本值 $f(i)$ 。也可以预处理

$$pre_i = \prod_{j=0}^{i-1} (k-j), \quad suf_i = \prod_{j=i}^n (k-j),$$

并统一使用

$$f(k) = \sum_{i=0}^n f(i) pre_i suf_{i+1} \frac{(-1)^{n-i}}{i!(n-i)!},$$

其中 $pre_0 = suf_{n+1} = 1$ 。预处理阶乘、逆阶乘以及前后缀积后即可 $O(n)$ 求解, 该写法在查询点恰好是采样点时也不会除以零。

第六部分

计算几何

47 计算几何全家桶

来源:牛客计算几何算法课程——陈俊杰

仅供参考,可能有需要注意的点。

```
#include <bits/stdc++.h>
using namespace std;

// using point_t=Long Long;
using point_t = long double; // 全局数据类型

constexpr point_t eps = 1e-8;
constexpr point_t INF = numeric_limits<point_t>::max();
constexpr long double PI = 3.14159265358979323841;

// 点与向量
template <typename T> struct point {
    T x, y;

    bool operator==(const point &a) const {
        return (abs(x - a.x) <= eps && abs(y - a.y) <= eps);
    }
    bool operator<(const point &a) const {
        if (abs(x - a.x) <= eps)
            return y < a.y - eps;
        return x < a.x - eps;
    }
    bool operator>(const point &a) const { return !(*this < a || *this == a); }
    point operator+(const point &a) const { return {x + a.x, y + a.y}; }
    point operator-(const point &a) const { return {x - a.x, y - a.y}; }
    point operator-() const { return {-x, -y}; }
    point operator*(const T k) const { return {k * x, k * y}; }
    point operator/(const T k) const { return {x / k, y / k}; }
    T operator*(const point &a) const { return x * a.x + y * a.y; } // 点积
    T operator^(const point &a) const {
        return x * a.y - y * a.x;
    } // 叉积,注意优先级
    int toleft(const point &a) const {
        const auto t = (*this) ^ a;
        return (t > eps) - (t < -eps);
    }
    // to-left 测试
    T len2() const { return (*this) * (*this); } // 向量长度的平方
    T dis2(const point &a) const {
        return (a - (*this)).len2();
    } // 两点距离的平方

    // 涉及浮点数
    long double len() const { return sqrtl(len2()); } // 向量长度
    long double dis(const point &a) const { return sqrtl(dis2(a)); } // 两点距离
    long double ang(const point &a) const {
        return acosl(max(-1.01, min(1.01, ((*this) * a) / (len() * a.len()))));
    } // 向量夹角
    point rot(const long double rad) const {
        return {x * cos(rad) - y * sin(rad), x * sin(rad) + y * cos(rad)};
    } // 逆时针旋转(给定角度)
    point rot(const long double cosr, const long double sinr) const {
        return {x * cosr - y * sinr, x * sinr + y * cosr};
    } // 逆时针旋转(给定角度的正弦与余弦)
};

using Point = point<point_t>;

// 极角排序
struct argcmp {
    bool operator()(const Point &a, const Point &b) const {
```

```

    const auto quad = [](const Point &a) {
        if (a.y < -eps)
            return 1;
        if (a.y > eps)
            return 4;
        if (a.x < -eps)
            return 5;
        if (a.x > eps)
            return 3;
        return 2;
    };
    const int qa = quad(a), qb = quad(b);
    if (qa != qb)
        return qa < qb;
    const auto t = a ^ b;
    // if (abs(t)<=eps) return a*a<b*b-eps; // 不同长度的向量需要分开
    return t > eps;
}

};

// 直线
template <typename T> struct line {
    point<T> p, v; // p 为直线上一点, v 为方向向量

    bool operator==(const line &a) const {
        return v.toleft(a.v) == 0 && v.toleft(p - a.p) == 0;
    }
    int toleft(const point<T> &a) const {
        return v.toleft(a - p);
    }
    // to-Left 测试
    bool operator<(const line &a) const // 半平面交算法定义的排序
    {
        if (abs(v ^ a.v) <= eps && v * a.v >= -eps)
            return toleft(a.p) == -1;
        return argcmp()(v, a.v);
    }
}

// 涉及浮点数
point<T> inter(const line &a) const {
    return p + v * ((a.v ^ (p - a.p)) / (v ^ a.v));
} // 直线交点
long double dis(const point<T> &a) const {
    return abs(v ^ (a - p)) / v.len();
} // 点到直线距离
point<T> proj(const point<T> &a) const {
    return p + v * ((v * (a - p)) / (v * v));
} // 点在直线上的投影
};

using Line = line<point_t>;

// 线段
template <typename T> struct segment {
    point<T> a, b;

    bool operator<(const segment &s) const {
        return make_pair(a, b) < make_pair(s.a, s.b);
    }

    // 判定性函数建议在整数域使用

    // 判断点是否在线段上
    // -1 点在线段端点 / 0 点不在线段上 / 1 点严格在线段上
    int is_on(const point<T> &p) const {
        if (p == a || p == b)
            return -1;
        return (p - a).toleft(p - b) == 0 && (p - a) * (p - b) < -eps;
    }

    // 判断线段直线是否相交
    // -1 直线经过线段端点 / 0 线段和直线不相交 / 1 线段和直线严格相交

```

```

int is_inter(const line<T> &l) const {
    if (l.toleft(a) == 0 || l.toleft(b) == 0)
        return -1;
    return l.toleft(a) != l.toleft(b);
}

// 判断两线段是否相交
// -1 在某一线段端点处相交 / 0 两线段不相交 / 1 两线段严格相交
int is_inter(const segment<T> &s) const {
    if (is_on(s.a) || is_on(s.b) || s.is_on(a) || s.is_on(b))
        return -1;
    const line<T> l{a, b - a}, ls{s.a, s.b - s.a};
    return l.toleft(s.a) * l.toleft(s.b) == -1 &&
        ls.toleft(a) * ls.toleft(b) == -1;
}

// 点到线段距离
long double dis(const point<T> &p) const {
    if ((p - a) * (b - a) < -eps || (p - b) * (a - b) < -eps)
        return min(p.dis(a), p.dis(b));
    const line<T> l{a, b - a};
    return l.dis(p);
}

// 两线段间距离
long double dis(const segment<T> &s) const {
    if (is_inter(s))
        return 0;
    return min({dis(s.a), dis(s.b), s.dis(a), s.dis(b)});
}
};

using Segment = segment<point_t>;

// 多边形
template <typename T> struct polygon {
    vector<point<T>> p; // 以逆时针顺序存储

    size_t nxt(const size_t i) const { return i == p.size() - 1 ? 0 : i + 1; }
    size_t pre(const size_t i) const { return i == 0 ? p.size() - 1 : i - 1; }

    // 回转数
    // 返回值第一项表示点是否在多边形边上
    // 对于狭义多边形,回转数为 0 表示点在多边形外,否则点在多边形内
    pair<bool, int> winding(const point<T> &a) const {
        int cnt = 0;
        for (size_t i = 0; i < p.size(); i++) {
            const point<T> u = p[i], v = p[nxt(i)];
            if (abs((a - u) ^ (a - v)) <= eps && (a - u) * (a - v) <= eps)
                return {true, 0};
            if (abs(u.y - v.y) <= eps)
                continue;
            const Line uv = {u, v - u};
            if (u.y < v.y - eps && uv.toleft(a) <= 0)
                continue;
            if (u.y > v.y + eps && uv.toleft(a) >= 0)
                continue;
            if (u.y < a.y - eps && v.y >= a.y - eps)
                cnt++;
            if (u.y >= a.y - eps && v.y < a.y - eps)
                cnt--;
        }
        return {false, cnt};
    }

    // 多边形面积的两倍
    // 可用于判断点的存储顺序是顺时针或逆时针
    T area() const {
        T sum = 0;
        for (size_t i = 0; i < p.size(); i++)
            sum += p[i] ^ p[nxt(i)];
    }
};

```

```

    return sum;
}

// 多边形的周长
long double circ() const {
    long double sum = 0;
    for (size_t i = 0; i < p.size(); i++)
        sum += p[i].dis(p[nxt(i)]);
    return sum;
}

};

using Polygon = polygon<point_t>;

// 凸多边形
template <typename T> struct convex : polygon<T> {
    // 闵可夫斯基和
    convex operator+(const convex &c) const {
        const auto &p = this->p;
        vector<Segment> e1(p.size()), e2(c.p.size()),
            edge(p.size() + c.p.size());
        vector<point<T>> res;
        res.reserve(p.size() + c.p.size());
        const auto cmp = [](const Segment &u, const Segment &v) {
            return argcmp()(u.b - u.a, v.b - v.a);
        };
        for (size_t i = 0; i < p.size(); i++)
            e1[i] = {p[i], p[this->nxt(i)]};
        for (size_t i = 0; i < c.p.size(); i++)
            e2[i] = {c.p[i], c.p[c.nxt(i)]};
        rotate(e1.begin(), min_element(e1.begin(), e1.end(), cmp), e1.end());
        rotate(e2.begin(), min_element(e2.begin(), e2.end(), cmp), e2.end());
        merge(e1.begin(), e1.end(), e2.begin(), e2.end(), edge.begin(), cmp);
        const auto check = [](const vector<point<T>> &res, const point<T> &u) {
            const auto back1 = res.back(), back2 = *prev(res.end(), 2);
            return (back1 - back2).topleft(u - back1) == 0 &&
                (back1 - back2) * (u - back1) >= -eps;
        };
        auto u = e1[0].a + e2[0].a;
        for (const auto &v : edge) {
            while (res.size() > 1 && check(res, u))
                res.pop_back();
            res.push_back(u);
            u = u + v.b - v.a;
        }
        if (res.size() > 1 && check(res, res[0]))
            res.pop_back();
        return {res};
    }

    // 旋转卡壳
    // 例: 凸多边形的直径的平方
    T rotcaliper() const {
        const auto &p = this->p;
        if (p.size() == 1)
            return 0;
        if (p.size() == 2)
            return p[0].dis2(p[1]);
        const auto area = [](const point<T> &u, const point<T> &v,
            const point<T> &w) { return (w - u) ^ (w - v); };

        T ans = 0;
        for (size_t i = 0, j = 1; i < p.size(); i++) {
            const auto nxti = this->nxt(i);
            ans = max({ans, p[j].dis2(p[i]), p[j].dis2(p[nxti])});
            while (area(p[this->nxt(j)], p[i], p[nxti]) >=
                area(p[j], p[i], p[nxti])) {
                j = this->nxt(j);
                ans = max({ans, p[j].dis2(p[i]), p[j].dis2(p[nxti])});
            }
        }
        return ans;
    }
};

```

```

}

// 判断点是否在凸多边形内
// 复杂度  $O(\log n)$ 
// -1 点在多边形边上 / 0 点在多边形外 / 1 点在多边形内
int is_in(const point<T> &a) const {
    const auto &p = this->p;
    if (p.size() == 1)
        return a == p[0] ? -1 : 0;
    if (p.size() == 2)
        return segment<T>{p[0], p[1]}.is_on(a) ? -1 : 0;
    if (a == p[0])
        return -1;
    if ((p[1] - p[0]).toleft(a - p[0]) == -1 ||
        (p.back() - p[0]).toleft(a - p[0]) == 1)
        return 0;
    const auto cmp = [&](const point<T> &u, const point<T> &v) {
        return (u - p[0]).toleft(v - p[0]) == 1;
    };
    const size_t i =
        lower_bound(p.begin() + 1, p.end(), a, cmp) - p.begin();
    if (i == 1)
        return segment<T>{p[0], p[i]}.is_on(a) ? -1 : 0;
    if (i == p.size() - 1 && segment<T>{p[0], p[i]}.is_on(a))
        return -1;
    if (segment<T>{p[i - 1], p[i]}.is_on(a))
        return -1;
    return (p[i] - p[i - 1]).toleft(a - p[i - 1]) > 0;
}

// 凸多边形关于某一方向的极值
// 复杂度  $O(\log n)$ 
// 参考资料: https://codeforces.com/blog/entry/48868
template <typename F> size_t extreme(const F &dir) const {
    const auto &p = this->p;
    const auto check = [&](const size_t i) {
        return dir(p[i]).toleft(p[this->nxt(i)] - p[i]) >= 0;
    };
    const auto dir0 = dir(p[0]);
    const auto check0 = check(0);
    if (!check0 && check(p.size() - 1))
        return 0;
    const auto cmp = [&](const point<T> &v) {
        const size_t vi = &v - p.data();
        if (vi == 0)
            return 1;
        const auto checkv = check(vi);
        const auto t = dir0.toleft(v - p[0]);
        if (vi == 1 && checkv == check0 && t == 0)
            return 1;
        return checkv ^ (checkv == check0 && t <= 0);
    };
    return partition_point(p.begin(), p.end(), cmp) - p.begin();
}

// 过凸多边形外一点求凸多边形的切线, 返回切点下标
// 复杂度  $O(\log n)$ 
// 必须保证点在多边形外
pair<size_t, size_t> tangent(const point<T> &a) const {
    const size_t i = extreme([&](const point<T> &u) { return u - a; });
    const size_t j = extreme([&](const point<T> &u) { return a - u; });
    return {i, j};
}

// 求平行于给定直线的凸多边形的切线, 返回切点下标
// 复杂度  $O(\log n)$ 
pair<size_t, size_t> tangent(const line<T> &a) const {
    const size_t i = extreme([&](...) { return a.v; });
    const size_t j = extreme([&](...) { return -a.v; });
    return {i, j};
}

```

```

};

using Convex = convex<point_t>;

// 圆
struct Circle {
    Point c;
    long double r;

    bool operator==(const Circle &a) const {
        return c == a.c && abs(r - a.r) <= eps;
    }
    long double circ() const { return 2 * PI * r; } // 周长
    long double area() const { return PI * r * r; } // 面积

    // 点与圆的关系
    // -1 圆上 / 0 圆外 / 1 圆内
    int is_in(const Point &p) const {
        const long double d = p.dis(c);
        return abs(d - r) <= eps ? -1 : d < r - eps;
    }

    // 直线与圆关系
    // 0 相离 / 1 相切 / 2 相交
    int relation(const Line &l) const {
        const long double d = l.dis(c);
        if (d > r + eps)
            return 0;
        if (abs(d - r) <= eps)
            return 1;
        return 2;
    }

    // 圆与圆关系
    // -1 相同 / 0 相离 / 1 外切 / 2 相交 / 3 内切 / 4 内含
    int relation(const Circle &a) const {
        if (*this == a)
            return -1;
        const long double d = c.dis(a.c);
        if (d > r + a.r + eps)
            return 0;
        if (abs(d - r - a.r) <= eps)
            return 1;
        if (abs(d - abs(r - a.r)) <= eps)
            return 3;
        if (d < abs(r - a.r) - eps)
            return 4;
        return 2;
    }

    // 直线与圆的交点
    vector<Point> inter(const Line &l) const {
        const long double d = l.dis(c);
        const Point p = l.proj(c);
        const int t = relation(l);
        if (t == 0)
            return vector<Point>();
        if (t == 1)
            return vector<Point>{p};
        const long double k = sqrt(r * r - d * d);
        return vector<Point>{p - (l.v / l.v.len()) * k,
                             p + (l.v / l.v.len()) * k};
    }

    // 圆与圆交点
    vector<Point> inter(const Circle &a) const {
        const long double d = c.dis(a.c);
        const int t = relation(a);
        if (t == -1 || t == 0 || t == 4)
            return vector<Point>();
        Point e = a.c - c;

```

```

    e = e / e.len() * r;
    if (t == 1 || t == 3) {
        if (r * r + d * d - a.r * a.r >= -eps)
            return vector<Point>{c + e};
        return vector<Point>{c - e};
    }
    const long double costh = (r * r + d * d - a.r * a.r) / (2 * r * d),
        sinh = sqrt(1 - costh * costh);
    return vector<Point>{c + e.rot(costh, -sinh), c + e.rot(costh, sinh)};
}

// 圆与圆交面积
long double inter_area(const Circle &a) const {
    const long double d = c.dis(a.c);
    const int t = relation(a);
    if (t == -1)
        return area();
    if (t < 2)
        return 0;
    if (t > 2)
        return min(area(), a.area());
    const long double costh1 = (r * r + d * d - a.r * a.r) / (2 * r * d),
        costh2 = (a.r * a.r + d * d - r * r) / (2 * a.r * d);
    const long double sinh1 = sqrt(1 - costh1 * costh1),
        sinh2 = sqrt(1 - costh2 * costh2);
    const long double th1 = acos(costh1), th2 = acos(costh2);
    return r * r * (th1 - costh1 * sinh1) +
        a.r * a.r * (th2 - costh2 * sinh2);
}

// 过圆外一点圆的切线
vector<Line> tangent(const Point &a) const {
    const int t = is_in(a);
    if (t == 1)
        return vector<Line>();
    if (t == -1) {
        const Point v = {-(a - c).y, (a - c).x};
        return vector<Line>{{a, v}};
    }
    Point e = a - c;
    e = e / e.len() * r;
    const long double costh = r / c.dis(a), sinh = sqrt(1 - costh * costh);
    const Point t1 = c + e.rot(costh, -sinh), t2 = c + e.rot(costh, sinh);
    return vector<Line>{{a, t1 - a}, {a, t2 - a}};
}

// 两圆的公切线
vector<Line> tangent(const Circle &a) const {
    const int t = relation(a);
    vector<Line> lines;
    if (t == -1 || t == 4)
        return lines;
    if (t == 1 || t == 3) {
        const Point p = inter(a)[0], v = {-(a.c - c).y, (a.c - c).x};
        lines.push_back({p, v});
    }
    const long double d = c.dis(a.c);
    const Point e = (a.c - c) / (a.c - c).len();
    if (t <= 2) {
        const long double costh = (r - a.r) / d,
            sinh = sqrt(1 - costh * costh);
        const Point d1 = e.rot(costh, -sinh), d2 = e.rot(costh, sinh);
        const Point u1 = c + d1 * r, u2 = c + d2 * r, v1 = a.c + d1 * a.r,
            v2 = a.c + d2 * a.r;
        lines.push_back({u1, v1 - u1});
        lines.push_back({u2, v2 - u2});
    }
    if (t == 0) {
        const long double costh = (r + a.r) / d,
            sinh = sqrt(1 - costh * costh);
        const Point d1 = e.rot(costh, -sinh), d2 = e.rot(costh, sinh);
        const Point u1 = c + d1 * r, u2 = c + d2 * r, v1 = a.c - d1 * a.r,

```

```

        v2 = a.c - d2 * a.r;
        lines.push_back({u1, v1 - u1});
        lines.push_back({u2, v2 - u2});
    }
    return lines;
}

// 圆的反演
tuple<int, Circle, Line> inverse(const Line &l) const {
    const Circle null_c = {{0.0, 0.0}, 0.0};
    const Line null_l = {{0.0, 0.0}, {0.0, 0.0}};
    if (l.toleft(c) == 0)
        return {2, null_c, l};
    const Point v =
        l.toleft(c) == 1 ? Point{1.v.y, -1.v.x} : Point{-1.v.y, 1.v.x};
    const long double d = r * r / l.dis(c);
    const Point p = c + v / v.len() * d;
    return {1, {(c + p) / 2, d / 2}, null_l};
}

tuple<int, Circle, Line> inverse(const Circle &a) const {
    const Circle null_c = {{0.0, 0.0}, 0.0};
    const Line null_l = {{0.0, 0.0}, {0.0, 0.0}};
    const Point v = a.c - c;
    if (a.is_in(c) == -1) {
        const long double d = r * r / (a.r + a.r);
        const Point p = c + v / v.len() * d;
        return {2, null_c, {p, {-v.y, v.x}}};
    }
    if (c == a.c)
        return {1, {c, r * r / a.r}, null_l};
    const long double d1 = r * r / (c.dis(a.c) - a.r),
        d2 = r * r / (c.dis(a.c) + a.r);
    const Point p = c + v / v.len() * d1, q = c + v / v.len() * d2;
    return {1, {(p + q) / 2, p.dis(q) / 2}, null_l};
}
};

```

// 圆与多边形面积交

```

long double area_inter(const Circle &circ, const Polygon &poly) {
    const auto cal = [](const Circle &circ, const Point &a, const Point &b) {
        if ((a - circ.c).toleft(b - circ.c) == 0)
            return 0.01;
        const auto ina = circ.is_in(a), inb = circ.is_in(b);
        const Line ab = {a, b - a};
        if (ina && inb)
            return ((a - circ.c) ^ (b - circ.c)) / 2;
        if (ina && !inb) {
            const auto t = circ.inter(ab);
            const Point p = t.size() == 1 ? t[0] : t[1];
            const long double ans = ((a - circ.c) ^ (p - circ.c)) / 2;
            const long double th = (p - circ.c).ang(b - circ.c);
            const long double d = circ.r * circ.r * th / 2;
            if ((a - circ.c).toleft(b - circ.c) == 1)
                return ans + d;
            return ans - d;
        }
        if (!ina && inb) {
            const Point p = circ.inter(ab)[0];
            const long double ans = ((p - circ.c) ^ (b - circ.c)) / 2;
            const long double th = (a - circ.c).ang(p - circ.c);
            const long double d = circ.r * circ.r * th / 2;
            if ((a - circ.c).toleft(b - circ.c) == 1)
                return ans + d;
            return ans - d;
        }
        const auto p = circ.inter(ab);
        if (p.size() == 2 && Segment{a, b}.dis(circ.c) <= circ.r + eps) {
            const long double ans = ((p[0] - circ.c) ^ (p[1] - circ.c)) / 2;
            const long double th1 = (a - circ.c).ang(p[0] - circ.c),
                th2 = (b - circ.c).ang(p[1] - circ.c);
            const long double d1 = circ.r * circ.r * th1 / 2,

```



```

        d2 = circ.r * circ.r * th2 / 2;
        if ((a - circ.c).topleft(b - circ.c) == 1)
            return ans + d1 + d2;
        return ans - d1 - d2;
    }
    const long double th = (a - circ.c).ang(b - circ.c);
    if ((a - circ.c).topleft(b - circ.c) == 1)
        return circ.r * circ.r * th / 2;
    return -circ.r * circ.r * th / 2;
};

long double ans = 0;
for (size_t i = 0; i < poly.p.size(); i++) {
    const Point a = poly.p[i], b = poly.p[poly.nxt(i)];
    ans += cal(circ, a, b);
}
return ans;
}

// 点集的凸包
// Andrew 算法, 复杂度  $O(n \log n)$ 
Convex convexhull(vector<Point> p) {
    vector<Point> st;
    if (p.empty())
        return Convex{st};
    sort(p.begin(), p.end());
    const auto check = [](const vector<Point> &st, const Point &u) {
        const auto back1 = st.back(), back2 = *prev(st.end(), 2);
        return (back1 - back2).topleft(u - back1) <= 0;
    };
    for (const Point &u : p) {
        while (st.size() > 1 && check(st, u))
            st.pop_back();
        st.push_back(u);
    }
    size_t k = st.size();
    p.pop_back();
    reverse(p.begin(), p.end());
    for (const Point &u : p) {
        while (st.size() > k && check(st, u))
            st.pop_back();
        st.push_back(u);
    }
    st.pop_back();
    return Convex{st};
}

// 半平面交
// 排序增量法, 复杂度  $O(n \log n)$ 
// 输入与返回值都是用直线表示的半平面集合
vector<Line> halfinter(vector<Line> l, const point_t lim = 1e9) {
    const auto check = [](const Line &a, const Line &b, const Line &c) {
        return a.topleft(b.inter(c)) < 0;
    };
    // 无精度误差的方法, 但注意取值范围会扩大到三次方
    /*const auto check = [](const Line &a, const Line &b, const Line &c) {
    {
        const Point
        p = a.v * (b.v ^ c.v), q = b.p * (b.v ^ c.v) + b.v * (c.v ^ (b.p - c.p)) - a.p * (b.v ^ c.v); return
        p.topleft(q) < 0;
    };*/
    l.push_back({{-lim, 0}, {0, -1}});
    l.push_back({{0, -lim}, {1, 0}});
    l.push_back({{lim, 0}, {0, 1}});
    l.push_back({{0, lim}, {-1, 0}});
    sort(l.begin(), l.end());
    deque<Line> q;
    for (size_t i = 0; i < l.size(); i++) {
        if (i > 0 && l[i - 1].v.topleft(l[i].v) == 0 &&
            l[i - 1].v * l[i].v > eps)
            continue;

```

```

    while (q.size() > 1 && check(l[i], q.back(), q[q.size() - 2]))
        q.pop_back();
    while (q.size() > 1 && check(l[i], q[0], q[1]))
        q.pop_front();
    if (!q.empty() && q.back().v.toleft(l[i].v) <= 0)
        return vector<Line>();
    q.push_back(l[i]);
}
while (q.size() > 1 && check(q[0], q.back(), q[q.size() - 2]))
    q.pop_back();
while (q.size() > 1 && check(q.back(), q[0], q[1]))
    q.pop_front();
return vector<Line>(q.begin(), q.end());
}

// 点集形成的最小最大三角形
// 极角序扫描线,复杂度  $O(n^2 \log n)$ 
// 最大三角形问题可以使用凸包与旋转卡壳做到  $O(n^2)$ 
pair<point_t, point_t> minmax_triangle(const vector<Point> &vec) {
    if (vec.size() <= 2)
        return {0, 0};
    vector<pair<int, int>> evt;
    evt.reserve(vec.size() * vec.size());
    point_t maxans = 0, minans = INF;
    for (size_t i = 0; i < vec.size(); i++) {
        for (size_t j = 0; j < vec.size(); j++) {
            if (i == j)
                continue;
            if (vec[i] == vec[j])
                minans = 0;
            else
                evt.push_back({i, j});
        }
    }
    sort(evt.begin(), evt.end(),
        [&](const pair<int, int> &u, const pair<int, int> &v) {
            const Point du = vec[u.second] - vec[u.first],
                dv = vec[v.second] - vec[v.first];
            return argcmp({du.y, -du.x}, {dv.y, -dv.x});
        });
    vector<size_t> vx(vec.size()), pos(vec.size());
    for (size_t i = 0; i < vec.size(); i++)
        vx[i] = i;
    sort(vx.begin(), vx.end(), [&](int x, int y) { return vec[x] < vec[y]; });
    for (size_t i = 0; i < vx.size(); i++)
        pos[vx[i]] = i;
    for (auto [u, v] : evt) {
        const size_t i = pos[u], j = pos[v];
        const size_t l = min(i, j), r = max(i, j);
        const Point vecu = vec[u], vecv = vec[v];
        if (l > 0)
            minans = min(
                minans, abs((vec[vx[l - 1]] - vecu) ^ (vec[vx[l - 1]] - vecv)));
        if (r < vx.size() - 1)
            minans = min(
                minans, abs((vec[vx[r + 1]] - vecu) ^ (vec[vx[r + 1]] - vecv)));
        maxans = max({maxans, abs((vec[vx[0]] - vecu) ^ (vec[vx[0]] - vecv)),
            abs((vec[vx.back()] - vecu) ^ (vec[vx.back()] - vecv))});
        if (i < j)
            swap(vx[i], vx[j]), pos[u] = j, pos[v] = i;
    }
    return {minans, maxans};
}

// 平面最近点对
// 扫描线,复杂度  $O(n \log n)$ 
point_t closest_pair(vector<Point> points) {
    sort(points.begin(), points.end());
    const auto cmpy = [](const Point &a, const Point &b) {
        if (abs(a.y - b.y) <= eps)
            return a.x < b.x - eps;
    }
}

```

```

    return a.y < b.y - eps;
};
multiset<Point, decltype(cmpy)> s{cmpy};
point_t ans = INF;
for (size_t i = 0, l = 0; i < points.size(); i++) {
    const point_t sqans = sqrtl(ans) + 1; // 整数情况
    // const point_t sqans=sqrtl(ans)+1; // 浮点数情况
    while (l < i && points[i].x - points[l].x >= sqans)
        s.erase(s.find(points[l++]));
    for (auto it = s.lower_bound(Point{-INF, points[i].y - sqans});
         it != s.end() && it->y - points[i].y <= sqans; it++) {
        ans = min(ans, points[i].dis2(*it));
    }
    s.insert(points[i]);
}
return ans;
}

// 判断多条线段是否有交点
// 扫描线,复杂度  $O(n\log n)$ 
bool segs_inter(const vector<Segment> &segs) {
    if (segs.empty())
        return false;
    using seq_t = tuple<point_t, int, Segment>;
    const auto seqcmp = [](const seq_t &u, const seq_t &v) {
        const auto [u0, u1, u2] = u;
        const auto [v0, v1, v2] = v;
        if (abs(u0 - v0) <= eps)
            return make_pair(u1, u2) < make_pair(v1, v2);
        return u0 < v0 - eps;
    };
    vector<seq_t> seq;
    for (auto seg : segs) {
        if (seg.a.x > seg.b.x + eps)
            swap(seg.a, seg.b);
        seq.push_back({seg.a.x, 0, seg});
        seq.push_back({seg.b.x, 1, seg});
    }
    sort(seq.begin(), seq.end(), seqcmp);
    point_t x_now;
    auto cmp = [&](const Segment &u, const Segment &v) {
        if (abs(u.a.x - u.b.x) <= eps || abs(v.a.x - v.b.x) <= eps)
            return u.a.y < v.a.y - eps;
        return ((x_now - u.a.x) * (u.b.y - u.a.y) + u.a.y * (u.b.x - u.a.x)) *
            (v.b.x - v.a.x) <
            ((x_now - v.a.x) * (v.b.y - v.a.y) + v.a.y * (v.b.x - v.a.x)) *
            (u.b.x - u.a.x) -
            eps;
    };
    multiset<Segment, decltype(cmp)> s{cmp};
    for (const auto [x, o, seg] : seq) {
        x_now = x;
        const auto it = s.lower_bound(seg);
        if (o == 0) {
            if (it != s.end() && seg.is_inter(*it))
                return true;
            if (it != s.begin() && seg.is_inter(*prev(it)))
                return true;
            s.insert(seg);
        } else {
            if (next(it) != s.end() && it != s.begin() &&
                (*prev(it)).is_inter(*next(it)))
                return true;
            s.erase(it);
        }
    }
    return false;
}

// 多边形面积并
// 轮廓积分,复杂度  $O(n^2\log n)$ ,  $n$ 为边数

```

```

// ans[i] 表示被至少覆盖了 i+1 次的区域的面积
vector<long double> area_union(const vector<Polygon> &polys) {
    const size_t siz = polys.size();
    vector<vector<pair<Point, Point>>> segs(siz);
    const auto check = [](const Point &u, const Segment &e) {
        return !((u < e.a && u < e.b) || (u > e.a && u > e.b));
    };

    auto cut_edge = [&](const Segment &e, const size_t i) {
        const Line le{e.a, e.b - e.a};
        vector<pair<Point, int>> evt;
        evt.push_back({e.a, 0});
        evt.push_back({e.b, 0});
        for (size_t j = 0; j < polys.size(); j++) {
            if (i == j)
                continue;
            const auto &pj = polys[j];
            for (size_t k = 0; k < pj.p.size(); k++) {
                const Segment s = {pj.p[k], pj.p[pj.nxt(k)]};
                if (le.toleft(s.a) == 0 && le.toleft(s.b) == 0) {
                    evt.push_back({s.a, 0});
                    evt.push_back({s.b, 0});
                } else if (s.is_inter(le)) {
                    const Line ls{s.a, s.b - s.a};
                    const Point u = le.inter(ls);
                    if (le.toleft(s.a) < 0 && le.toleft(s.b) >= 0)
                        evt.push_back({u, -1});
                    else if (le.toleft(s.a) >= 0 && le.toleft(s.b) < 0)
                        evt.push_back({u, 1});
                }
            }
        }
        sort(evt.begin(), evt.end());
        if (e.a > e.b)
            reverse(evt.begin(), evt.end());
        int sum = 0;
        for (size_t i = 0; i < evt.size(); i++) {
            sum += evt[i].second;
            const Point u = evt[i].first, v = evt[i + 1].first;
            if (!(u == v) && check(u, e) && check(v, e))
                segs[sum].push_back({u, v});
            if (v == e.b)
                break;
        }
    };

    for (size_t i = 0; i < polys.size(); i++) {
        const auto &pi = polys[i];
        for (size_t k = 0; k < pi.p.size(); k++) {
            const Segment ei = {pi.p[k], pi.p[pi.nxt(k)]};
            cut_edge(ei, i);
        }
    }
    vector<long double> ans(siz);
    for (size_t i = 0; i < siz; i++) {
        long double sum = 0;
        sort(segs[i].begin(), segs[i].end());
        int cnt = 0;
        for (size_t j = 0; j < segs[i].size(); j++) {
            if (j > 0 && segs[i][j] == segs[i][j - 1])
                segs[i + (++cnt)].push_back(segs[i][j]);
            else
                cnt = 0, sum += segs[i][j].first ^ segs[i][j].second;
        }
        ans[i] = sum / 2;
    }
    return ans;
}

// 圆面积并
// 轮廓积分, 复杂度  $O(n^2 \log n)$ 
// ans[i] 表示被至少覆盖了 i+1 次的区域的面积

```

```

vector<long double> area_union(const vector<Circle> &circs) {
    const size_t siz = circs.size();
    using arc_t = tuple<Point, long double, long double, long double>;
    vector<vector<arc_t>> arcs(siz);
    const auto eq = [](const arc_t &u, const arc_t &v) {
        const auto [u1, u2, u3, u4] = u;
        const auto [v1, v2, v3, v4] = v;
        return u1 == v1 && abs(u2 - v2) <= eps && abs(u3 - v3) <= eps &&
            abs(u4 - v4) <= eps;
    };

    auto cut_circ = [](const Circle &ci, const size_t i) {
        vector<pair<long double, int>> evt;
        evt.push_back({-PI, 0});
        evt.push_back({PI, 0});
        int init = 0;
        for (size_t j = 0; j < circs.size(); j++) {
            if (i == j)
                continue;
            const Circle &cj = circs[j];
            if (ci.r < cj.r - eps && ci.relation(cj) >= 3)
                init++;
            const auto inters = ci.inter(cj);
            if (inters.size() == 1)
                evt.push_back(
                    {atan2l((inters[0] - ci.c).y, (inters[0] - ci.c).x), 0});
            if (inters.size() == 2) {
                const Point dl = inters[0] - ci.c, dr = inters[1] - ci.c;
                long double argl = atan2l(dl.y, dl.x),
                    argr = atan2l(dr.y, dr.x);
                if (abs(argl + PI) <= eps)
                    argl = PI;
                if (abs(argr + PI) <= eps)
                    argr = PI;
                if (argl > argr + eps) {
                    evt.push_back({argl, 1});
                    evt.push_back({PI, -1});
                    evt.push_back({-PI, 1});
                    evt.push_back({argr, -1});
                } else {
                    evt.push_back({argl, 1});
                    evt.push_back({argr, -1});
                }
            }
        }
        sort(evt.begin(), evt.end());
        int sum = init;
        for (size_t i = 0; i < evt.size(); i++) {
            sum += evt[i].second;
            if (abs(evt[i].first - evt[i + 1].first) > eps)
                arcs[sum].push_back(
                    {ci.c, ci.r, evt[i].first, evt[i + 1].first});
            if (abs(evt[i + 1].first - PI) <= eps)
                break;
        }
    };

    const auto oint = [](const arc_t &arc) {
        const auto [cc, cr, l, r] = arc;
        if (abs(r - l - PI - PI) <= eps)
            return 2.0l * PI * cr * cr;
        return cr * cr * (r - l) + cc.x * cr * (sin(r) - sin(l)) -
            cc.y * cr * (cos(r) - cos(l));
    };

    for (size_t i = 0; i < circs.size(); i++) {
        const auto &ci = circs[i];
        cut_circ(ci, i);
    }
    vector<long double> ans(siz);
    for (size_t i = 0; i < siz; i++) {
        long double sum = 0;

```

```

    sort(arcs[i].begin(), arcs[i].end());
    int cnt = 0;
    for (size_t j = 0; j < arcs[i].size(); j++) {
        if (j > 0 && eq(arcs[i][j], arcs[i][j - 1]))
            arcs[i + (++cnt)].push_back(arcs[i][j]);
        else
            cnt = 0, sum += oint(arcs[i][j]);
    }
    ans[i] = sum / 2;
}
return ans;
}

```

47.1 补充

47.1.1 三维凸包

增量法, $O(n^2)$ 实现:

```

struct point {
    ldb x, y, z;
    point operator-(const point &t) const {
        return {x - t.x, y - t.y, z - t.z};
    }
    point operator^(const point &t) const {
        return {y * t.z - z * t.y, z * t.x - x * t.z, x * t.y - y * t.x};
    }
    ldb operator*(const point &t) const { return x * t.x + y * t.y + z * t.z; }
    ldb len() const { return sqrtl(x * x + y * y + z * z); }
};

struct face {
    int v[3];
    point normal(const vector<point> &a) const {
        return (a[v[1]] - a[v[0]]) ^ (a[v[2]] - a[v[0]]);
    }
    double area(const vector<point> &a) const {
        point n = normal(a);
        return n.len() * 0.5;
    }

    bool visible(const vector<point> &a, const point &p) const {
        point n = normal(a);
        return ((p - a[v[0]]) * n) > 0;
    }
};

mt19937_64 rng(
    chrono::high_resolution_clock::now().time_since_epoch().count());
uniform_real_distribution<ldb> unif(-0.5, 0.5);
for (int i = 0; i < n; i++) {
    ldb r = unif(rng) * eps;
    a[i].x += r;
    a[i].y += unif(rng) * eps;
    a[i].z += unif(rng) * eps;
}

vector<face> f, g;

f.reserve(8 * n);
f.push_back({0, 1, 2});
f.push_back({2, 1, 0});

for (int i = 3; i < n; i++) {
    g.clear();
    for (unsigned j = 0; j < f.size(); j++) {
        bool visf = f[j].visible(a, a[i]);
    }
}

```

```

    if (!visf) {
        g.push_back(f[j]);
    }
    for (int t = 0; t < 3; t++) {
        int u = f[j].v[t];
        int vtx = f[j].v[(t + 1) % 3];
        vis[u][vtx] = visf ? 1 : 0;
    }
}
for (unsigned j = 0; j < f.size(); j++) {
    for (int t = 0; t < 3; t++) {
        int u = f[j].v[t];
        int vtx = f[j].v[(t + 1) % 3];
        if (vis[u][vtx] && !vis[vtx][u]) {
            face nf;
            nf.v[0] = u;
            nf.v[1] = vtx;
            nf.v[2] = i;
            g.push_back(nf);
        }
    }
}
f.swap(g);
}

```

47.1.2 动态凸包

增量法, $O(n \log n)$:

```

struct point {
    int x, y;
    bool operator<(const point &t) const { // andrew 按 x 维护上下凸壳
        if (x != t.x)
            return x < t.x;
        return y < t.y;
    }
};

bool bad(auto it, set<point> &s, bool lower) {
    if (it == s.begin())
        return 0;
    auto itn = next(it);
    if (itn == s.end())
        return 0;
    auto itp = prev(it);

    auto cr = ((*it - *itp) ^ (*itn - *itp));
    return lower ? cr <= 0 : cr >= 0; // == 0 取决于共线点是否删除
}

void insert_chain(set<point> &s, point p, bool lower) {
    auto it = s.insert(p).first;
    if (bad(it, s, lower)) {
        s.erase(it);
        return;
    }
    while (1) {
        if (it == s.begin())
            break;
        auto itp = prev(it);
        if (bad(itp, s, lower))
            s.erase(itp);
        else
            break;
    }
    while (1) {
        auto itn = next(it);
        if (itn == s.end())
            break;
    }
}

```

```

        if (bad(itn, s, lower))
            s.erase(itn);
        else
            break;
    }
}

```

47.1.3 最小圆覆盖

$O(n)$:

```

point circle_center(point a, point b, point c) {
    ldb A = a.len2(), B = b.len2(), C = c.len2();
    ldb u1 = a.x - b.x, u2 = a.x - c.x;
    ldb v1 = a.y - b.y, v2 = a.y - c.y;
    point o;
    o.y = ((C - A) * u1 - (B - A) * u2) / (2 * v1 * u2 - 2 * v2 * u1);
    o.x = ((C - A) * v1 - (B - A) * v2) / (2 * u1 * v2 - 2 * u2 * v1);
    return o;
}

random_device rd;
mt19937 g(rd());
circle min_circle(vector<point> a) {
    shuffle(a.begin(), a.end(), g);
    point now = a[0];
    ldb r = 0;
    for (unsigned i = 0; i < a.size(); i++) {
        if (now.dis(a[i]) - r > eps) {
            now = a[i];
            r = 0;
            for (unsigned j = 0; j < i; j++) {
                if (now.dis(a[j]) - r > eps) {
                    now.x = (a[i].x + a[j].x) / 2.0;
                    now.y = (a[i].y + a[j].y) / 2.0;
                    r = now.dis(a[j]);
                    for (unsigned k = 0; k < j; k++) {
                        if (now.dis(a[k]) - r > eps) {
                            now = circle_center(a[i], a[j], a[k]);
                            r = now.dis(a[k]);
                        }
                    }
                }
            }
        }
    }
    return {now, r};
}

```

47.1.4 自适应辛普森积分

时间复杂度视作： $O(\log \epsilon T f(x))$, 其中 T 是迭代步长, $f(x)$ 是求解 $f(x)$ 的时间复杂度。

```

struct Simpson {
    ldb f(ldb x) { // 对 f 求积分
        // f(x) 的值
    }
    ldb simpson(ldb l, ldb r) {
        ldb mid = (l + r) / 2.0;
        return (f(l) + 4 * f(mid) + f(r)) * (r - l) / 6;
    }
    ldb calc(ldb l, ldb r, ldb eps, ldb ans, int step) {
        ldb mid = (l + r) / 2.0;
        ldb L = simpson(l, mid), R = simpson(mid, r);
    }
}

```



```
    if (fabs(L + R - ans) <= 15 * eps && step < 0)
        return L + R + (L + R - ans) / 15;
    return calc(l, mid, eps / 2.0, L, step - 1) +
           calc(mid, r, eps / 2.0, R, step - 1);
}
ldb calc(ldb l, ldb r, ldb eps) {
    return calc(l, r, eps, simpson(l, r), 8); // 8 是迭代的步长,可调整
}
};
```


48 计算几何基础概念

48.1 浮点数与精度问题

48.1.1 特殊值:

- $+0.0$ -0.0
- 在常见的 IEC 60559/IEEE 754 浮点实现中,非零数除以 ± 0.0 可能得到 $\pm \text{inf}$,对负数开平方等无效运算可能得到 NaN 。
- NaN 与任何数(包括自身)做有序比较或相等比较都返回 `false`,只有 `!=` 返回 `true`。

这些行为依赖实现和浮点环境,不能把主动除以零或制造 NaN 当作可移植的分支技巧。应在运算前检查分母、方向向量和定义域,必要时再使用 `std::isfinite`、`std::isinf`、`std::isnan` 检查结果。

1. 若问题能用整数解决则不用浮点数
2. 除非时限紧张,否则使用 `long double`
3. 减少数学库函数的调用
4. 进行浮点数比较时,加入容限(误差) `eps`

下文公式中的 $= 0$ 、 > 0 和 < 0 ,若使用浮点数实现,均应通过与数据尺度匹配的误差比较完成;若输入是整数,点积、叉积等判定可使用足够宽的整数类型精确计算,并注意中间乘法溢出。

48.2 点

(x, y) ,横坐标: x ,纵坐标: y 。

48.2.1 向量:

坐标系中一个点对应一个向量,通常使用向量表示一个点。

48.2.2 点积:

点积是一个值:

$$\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y$$

几何意义:

$$\vec{a} \cdot \vec{b} = |\vec{a}| \times |\vec{b}| \times \cos \theta$$

- 向量的长度: $|\vec{a}| = \sqrt{\vec{a} \cdot \vec{a}}$
- 向量的夹角: $\cos \theta = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| \times |\vec{b}|}$, 要求 $\vec{a}, \vec{b}$ 均为非零向量
- $\vec{a}$ 在 $\vec{b}$ 方向上的标量投影: $\frac{\vec{a} \cdot \vec{b}}{|\vec{b}|}$, 要求 $\vec{b} \neq \vec{0}$; 当 $\vec{a}, \vec{b}$ 均非零时, 它也等于 $|\vec{a}| \cos \theta$
- $\vec{a}$ 在 $\vec{b}$ 上的向量投影: $\text{proj}_{\vec{b}} \vec{a} = \frac{\vec{a} \cdot \vec{b}}{\vec{b} \cdot \vec{b}} \vec{b}$, 要求 $\vec{b} \neq \vec{0}$
- 向量垂直: $\vec{a} \cdot \vec{b} = 0$

48.2.3 叉积:

向量的叉积仅在 2, 3 维坐标系中存在, 更高维通常没有意义。

- 2 维是一个标量: $\vec{a} \times \vec{b} = a_x b_y - a_y b_x$
- 3 维是一个向量: $\vec{a} \times \vec{b} = (a_y b_z - a_z b_y, -a_x b_z + a_z b_x, a_x b_y - a_y b_x)$

几何意义:

$$\vec{a} \times \vec{b} = |\vec{a}| \times |\vec{b}| \times \sin \theta \hat{k}$$

- 平行四边形面积: $|\vec{a} \times \vec{b}| = |\vec{a}| \times |\vec{b}| \times |\sin \theta|$
- 向量平行: $\vec{a} \times \vec{b} = \vec{0}$
- to-left 测试

48.2.4 to-left 测试

判断点 P 在有向直线 AB 左侧/右侧上。

$$\begin{cases} \overrightarrow{AB} \times \overrightarrow{AP} > 0 & P \text{ 在有向直线 } AB \text{ 左侧} \\ \overrightarrow{AB} \times \overrightarrow{AP} < 0 & P \text{ 在有向直线 } AB \text{ 右侧} \\ \overrightarrow{AB} \times \overrightarrow{AP} = 0 & P \text{ 在有向直线 } AB \text{ 上} \end{cases}$$

48.2.5 向量逆时针旋转

$\vec{a}$ 逆时针旋转 θ :

$$(a_x, a_y) \rightarrow (\cos \theta a_x - \sin \theta a_y, \sin \theta a_x + \cos \theta a_y)$$

48.3 线段

$\vec{a}, \vec{b}$ 记录线段两 endpoints。

48.3.1 判断点是否在线段上:

- 点 P 在 AB 所在直线上: $\overrightarrow{PA} \times \overrightarrow{PB} = 0$
- 点 P 在 AB 之间:
 - 若线段包含端点, 则 $\overrightarrow{PA} \cdot \overrightarrow{PB} \leq 0$; 若只判断开线段内部, 才使用 < 0
 - $\min(A_x, B_x) \leq P_x \leq \max(A_x, B_x) \wedge \min(A_y, B_y) \leq P_y \leq \max(A_y, B_y)$

48.3.2 判断两条线段是否相交:

- 若只判断严格相交, 则要求点 A 和点 B 在直线 CD 的严格不同侧, 且点 C 和点 D 在直线 AB 的严格不同侧。
- 若把端点接触和共线重叠也视为相交, 还要分别判断四个端点是否落在另一条线段上; 点在线段上的判定使用上一节的闭区间条件。

48.4 直线

通常使用点向式表示: 直线上的一点 P 和方向向量 $\vec{v}$ 表示一条直线, 参数方程为 $\vec{X} = \vec{P} + t\vec{v}$ 。方向向量必须满足 $\vec{v} \neq \vec{0}$ 。

48.4.1 求点 A 到直线的距离:

$$d = \frac{|\vec{v} \times \overrightarrow{PA}|}{|\vec{v}|}$$

48.4.2 求点 A 在直线上的投影点B:

$$\vec{B} = \vec{P} + \frac{(\vec{A} - \vec{P}) \cdot \vec{v}}{\vec{v} \cdot \vec{v}} \vec{v}, \quad \vec{v} \neq \vec{0}.$$

48.4.3 两直线交点:

设两条直线分别为 $\vec{P}_1 + t\vec{v}_1$ 和 $\vec{P}_2 + s\vec{v}_2$, 且两个方向向量均非零。令

$$D = \vec{v}_1 \times \vec{v}_2.$$

当 $D \neq 0$ 时, 两直线有唯一交点, 参数和交点分别为

$$t = \frac{(\vec{P}_2 - \vec{P}_1) \times \vec{v}_2}{D} = \frac{\vec{v}_2 \times (\vec{P}_1 - \vec{P}_2)}{D},$$

$$\vec{Q} = \vec{P}_1 + t\vec{v}_1.$$

参数 t 可能为负, 不能对分子取绝对值。当 $D = 0$ 时: 若 $(\vec{P}_2 - \vec{P}_1) \times \vec{v}_1 \neq 0$, 两直线平行且不重合; 否则两直线重合, 没有唯一交点。浮点实现中可按向量长度缩放容限, 例如比较 $|D|$ 与 $\varepsilon|\vec{v}_1||\vec{v}_2|$, 而不是先执行除法。接口应分别返回“唯一交点、平行、重合、方向无效”等状态, 不能依赖除以零产生的 `inf` 或 `NaN` 区分。

48.5 多边形

由点集表示。

- 一般按逆时针顺序
- 不一定满足凸性
- 注意第一个点与最后一个点的处理

48.5.1 多边形的面积:

先定义有向二倍面积

$$S_2 = \sum_{i=0}^{n-1} \vec{P}_i \times \vec{P}_{(i+1) \bmod n}.$$

多边形的普通面积为

$$S = \frac{|S_2|}{2}.$$

特别地,三角形的面积: $\frac{|\vec{a} \times \vec{b}|}{2}$

48.5.2 多边形的方向:

在标准笛卡尔坐标系中,若顶点沿一个不自交多边形的边界依次给出,则:

- $S_2 > 0$: 逆时针;
- $S_2 < 0$: 顺时针;
- $S_2 = 0$: 退化,不能据此确定方向。

对自交多边形, $S_2/2$ 表示带符号的代数面积,不能直接用来描述整个图形的普通面积或统一方向。

48.5.3 判断点是否在多边形内

48.5.3.1 射线法

从查询点 P 向 x 轴正方向引出射线。首先使用“点在线段上”的闭区间判定检查 P 是否落在任意一条边上;边界点可以单独返回 **BOUNDARY**,也可以按题意统一归入内部或外部。

对非边界点,边 AB 仅在以下两种情况之一成立时贡献一次穿越:

- $A_y \leq P_y < B_y$ 且 $\overrightarrow{AB} \times \overrightarrow{AP} > 0$;
- $B_y \leq P_y < A_y$ 且 $\overrightarrow{AB} \times \overrightarrow{AP} < 0$ 。

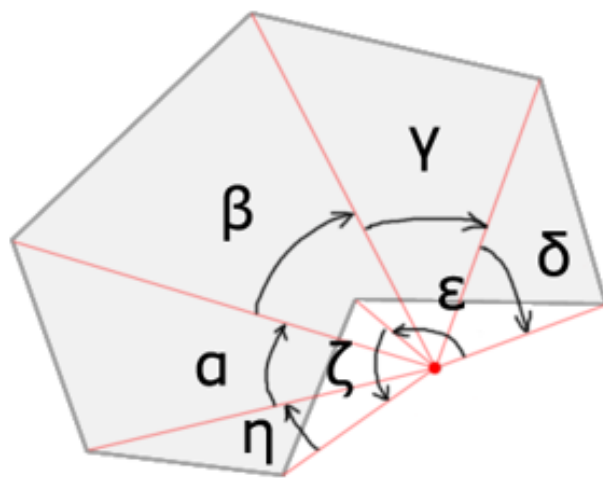
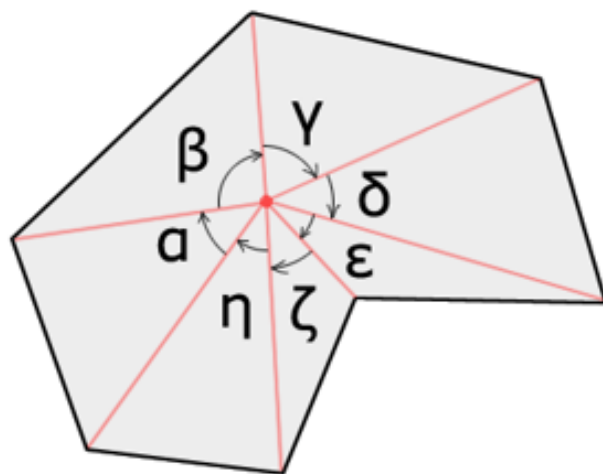
这相当于纵坐标区间包含较低端点、排除较高端点,水平边不计数。射线经过普通穿越顶点时贡献一次;经过局部极小点时两条边均贡献,经过局部极大点时两条边均不贡献,因此不会错误改变交点数的奇偶性。穿越次数为奇数时 P 在奇偶填充意义下位于内部,否则位于外部。

48.5.3.2 回转数法

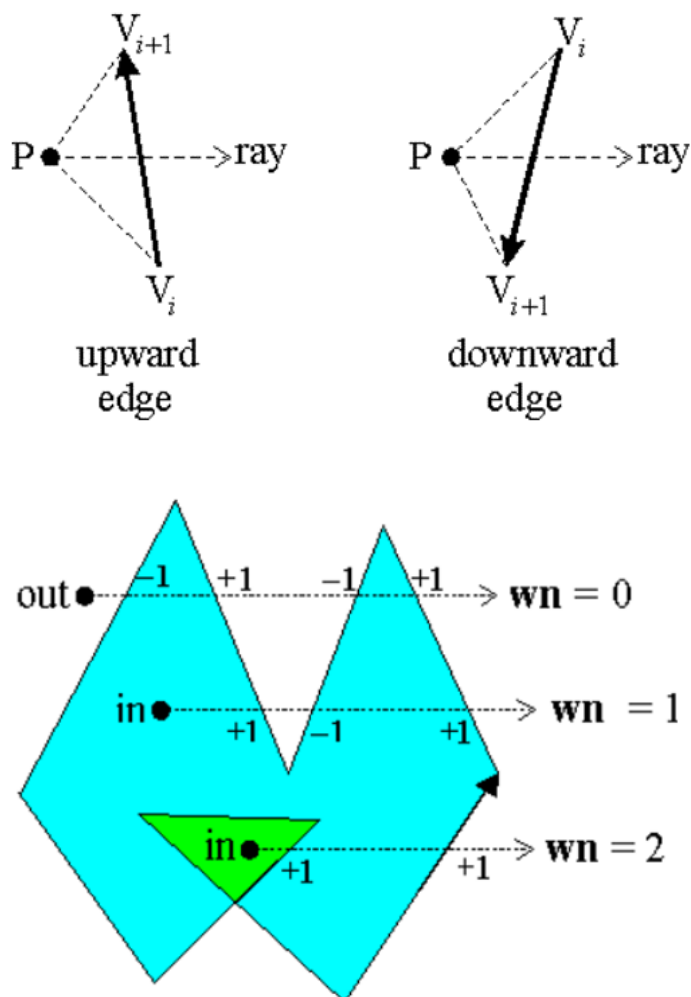
回转数: 闭合曲线绕查询点的有向总圈数。

遵循非零规则: 当回转次数为 0 时,点在曲线外部。

- 一种实现方法: 计算相邻两边夹角(有方向)的和。



- 另一种实现方法是沿用射线法的半开规则：向上穿越贡献 $+1$ ，向下穿越贡献 -1 。这种写法避免了反三角函数；只有在坐标为整数、又积使用足够宽的整数类型且中间运算不溢出时，方向判定才没有浮点误差。浮点坐标仍需使用稳健的误差比较。



对简单多边形, 奇偶规则与非零回转数规则给出相同的内外结果; 对自交多边形, 两者分别对应奇偶填充和非零填充, 结果可能不同。

48.5.3.3 判断点是否在凸多边形内

- n 次 to-left 测试 $O(n)$ 。
- 二分 $O(\log n)$ 。

上述方法要求多边形凸、非退化, 且顶点按同一方向沿边界给出; 二分实现还需单独约定点落在边或顶点上的返回结果。

49 极角序

49.1 极坐标系

- 极点: O
- 极轴: $\overrightarrow{OL}$
- 极径: r
- 极角: φ
- 极坐标: (r, φ)

对于非零向量 (x, y) , 可以在一个约定的、长度为 2π 的半开区间(如 $[0, 2\pi)$)内定义其极角。当 $x \neq 0$ 时有

$$\tan \varphi = \frac{y}{x}$$

但该等式不能唯一确定 φ : 正切函数以 π 为周期, 因而无法区分象限; 当 $x = 0$ 时, 右侧也没有定义。零向量 $(0, 0)$ 的极角本身没有定义, 排序时应将其排除, 或明确约定它的位置。

49.2 极角排序

49.2.1 排序范围小于 π

若所有非零向量的极角都位于同一个长度严格小于 π 的角区间内, 并且该区间不跨越所选的排序断点, 则可以用 to-left 测试, 即叉积的符号比较极角。若角度范围达到 π , 反向向量的叉积为 0, 不能仅靠叉积区分先后。

49.2.2 完整一周的排序

49.2.2.1 使用 `atan2`

对非零向量使用 `atan2(y, x)` 计算极角, 其值域为 $[-\pi, \pi]$ 。若要按 $[0, 2\pi)$ 排序, 可将负角加上 2π 后再排序。

`atan2` 的主要优势是同时利用 x, y 判断象限, 并且可以处理 $x = 0, y \neq 0$, 而不是简单地比 `atan(y/x)` “精度更高”。它仍是浮点运算, 会受到舍入误差影响; 但它不需要先计算 y/x , 也避免了这一步可能产生的溢出、下溢和额外舍入。

49.2.2.2 使用半平面和叉积

可以先划分半平面,再在同一半平面内比较叉积。例如约定从 x 轴正半轴开始逆时针排序,即依次为 x 轴正半轴、上半平面、 x 轴负半轴、下半平面。下面的整数实现同时约定零向量排在最前,同方向向量按模长从小到大排序:

```
struct Point {
    long long x, y;
};

using i128 = __int128_t;

bool zero(const Point &a) {
    return a.x == 0 && a.y == 0;
}

int half(const Point &a) {
    return a.y < 0 || (a.y == 0 && a.x < 0);
}

i128 cross(const Point &a, const Point &b) {
    return (i128)a.x * b.y - (i128)a.y * b.x;
}

i128 len2(const Point &a) {
    return (i128)a.x * a.x + (i128)a.y * a.y;
}

bool cmp(const Point &a, const Point &b) {
    if (zero(a) != zero(b))
        return zero(a);
    if (zero(a))
        return false;
    if (half(a) != half(b))
        return half(a) < half(b);
    i128 c = cross(a, b);
    if (c != 0)
        return c > 0;
    return len2(a) < len2(b);
}
```

在中间量不溢出的精确整数运算下,该比较器满足严格弱序。若不关心同方向向量的长度次序,也可以在叉积为 0 时直接返回 `false`,将它们视为等价;若后续算法需要确定顺序,则保留模长这一tie-break。对于浮点坐标,叉积同样会有舍入误差,并非“无精度损失”;也不应直接在排序比较器中用可能破坏传递性的 `eps` 判等。

时间复杂度: $O(n \log n)$ 。

50 凸包

50.1 凸包基础

50.1.1 定义

50.1.1.1 凸集与凸多边形

若集合 C 中任意两点之间的整条线段仍属于 C , 则称 C 为凸集。一个简单多边形连同其内部构成凸集时, 称它为凸多边形。若边界表示保留共线中间点, 某些内角可以等于 π ; 所有内角都在 $(0, \pi)$ 内描述的是所有顶点均为严格转向、且已删除边界共线中间点的规范表示。

50.1.1.2 凸包

点集 S 的凸包 $\text{conv}(S)$ 是包含 S 的最小凸集, 也就是 S 中有限个点的所有凸组合构成的集合。

有限点集的凸包允许退化: 空集的凸包按约定为空, 单点的凸包是该点, 两点或全体点共线时凸包是线段; 只有存在至少三个不共线点时, 凸包才是具有正面积的凸多边形。

50.1.1.3 极点

对于凸集 C 中的点 $p \in C$, 若 $p = \lambda x + (1 - \lambda)y$ ($x, y \in C, 0 < \lambda < 1$) 只能在 $x = y = p$ 时成立, 则称 p 为 C 的极点。有限点集凸包的极点正是凸包真正的顶点; 凸包边内部的共线点不是极点。

支持线使整个凸集位于它的一个闭半平面内。支持线与凸集的交集可能是一个极点, 也可能是一整条边; 只有交集为单点时, 该点才是这个方向暴露出的极点。因此, 仅知道“其余点位于同一个闭半平面”而允许其它点落在支持线上, 不能作为极点的充分判据; 若其它不同坐标的点都严格位于同一个开半平面, 则该支持线只接触 p , 足以证明 p 是极点。

50.1.1.4 极边

对于二维非退化凸包, 极边是支持线与凸包相交得到的完整一维面。支持线上的共线中间点可以位于极边上, 但极边的两个端点才是极点。

50.1.2 求凸包

50.1.2.1 极点法

枚举每个点,判断每个点是否是极点。

依据:先删除重复坐标。一个点不是极点,当且仅当它属于其它点的凸包。根据 Carathéodory 定理,平面中的该点一定属于至多三个其它点的凸包;这里必须检查闭三角形及其边界,并允许退化为线段,不能只检查严格的三角形内部。

时间复杂度: $O(n^4)$ 。

50.1.2.2 极边法

枚举两点确定一条直线,判断它是否是凸包的支持线。

依据:其它点在该直线的同一个闭半平面内。若支持线上还有其它共线点,应取这一维面的两个最远端点作为规范凸包的极边,不能把支持线上的任意点对都输出成边。

时间复杂度: $O(n^3)$ 。

50.1.2.3 增量法

每次用一个点去更新凸包。

- 若点在凸包内,舍弃
- 若点在凸包外,找凸包的切线

可推广至三维凸包、动态凸包。

若当前凸包有 h 个顶点,暴力判断点是否在凸包内需要 h 次 to-left 测试,即 $O(h)$ 。

找凸包的切线:切点的前驱和后继都在对应射线的同一侧,暴力需要 $O(h)$ 。

依次插入 n 个点、每次都暴力扫描当前凸包时,总时间复杂度为 $O(n^2)$ 。

对于已经按逆时针顺序给出并完成规范化的静态凸多边形,预处理 $O(h)$ 后,单次点内判定或严格外点的切点查询都可以做到 $O(\log h)$ 。若输入是 n 个无序点,另需 $O(n \log n)$ 构造凸包;这两部分复杂度不能混写成一次查询 $O(n \log n)$ 。

50.1.2.4 Gift wrapping

先删除重复坐标,从一个确定的极点出发;每一步遍历其它点,选择下一条极边。

若同一方向上有多个共线候选,只输出真正极点时必须选择离当前点最远者,否则会把边内部点当成下一顶点。空集和单点直接返回,全共线点集返回线段的两个端点。

时间复杂度为 $O(nh)$,最坏为 $O(n^2)$,其中 h 表示所选输出策略下的凸包顶点数。

起点可以取字典序最小点等确定的坐标极值端点。若某一坐标极值由一整条极边取得,应再用另一坐标选择该极边的一个端点,而不是任取边内部点。

50.1.2.5 Graham scan

基于极角排序的求凸包算法。下面默认返回按逆时针排列、首点不在末尾重复的真正极点,不保留极边内部的共线点。首先按坐标删除重复点;设去重后点数为 m ,则 $m = 0, 1, 2$ 时直接返回对应的退化凸包。

算法流程:

- 取纵坐标最小、再取横坐标最小的点作为基点;
- 将其它点按相对基点的极角排序,同极角时按到基点的距离递增;
- 依次处理排序后的点。当栈顶两个点与新点不构成严格逆时针转向,即叉积小于等于 0 时,弹出栈顶;
- 将新点入栈,最终得到只含真正极点的凸包。全共线输入最终只保留两个端点。

若题目要求保留所有边界共线点,不能只把弹栈条件机械改成“叉积小于 0”:还要正确处理同极角组,并将最大极角的最后一组按距离反向;全共线输入也应单独约定输出顺序。

时间复杂度: $O(n \log n)$ 。瓶颈为极角排序。

50.1.2.6 Andrew's algorithm

基于坐标字典序排序的求凸包算法。输出约定与 Graham scan 相同。

以最左最右两点为界,凸包可以分为上凸包和下凸包两部分。

算法流程:

- 对点集以横坐标为第一关键字、纵坐标为第二关键字排序,并删除重复坐标;
- 去重后点数为 0, 1, 2 时直接返回;
- 正序遍历各点维护下凸壳,倒序遍历各点维护上凸壳。若只保留真正极点,当末尾两点与新点的叉积小于等于 0 时弹出末点;
- 拼接上下凸壳时,各自删除一个重复端点,且最终不重复首点。全共线输入返

回字典序最小、最大的两个端点。

若要保留边界共线点,通常只在叉积小于 0 时弹出,但必须特判全共线输入,避免上下凸壳把中间点重复输出。

时间复杂度: $O(n \log n)$ 。瓶颈为排序。

50.2 凸包进阶

50.2.1 凸包二分

50.2.1.1 判断点是否在凸多边形中

以下假设凸多边形有 $h \geq 3$ 个顶点,已经删除冗余共线点并按逆时针顺序排列,且默认边界算作内部。退化情况应单独处理:空凸包不包含任何点,单点只包含自身,线段只包含位于该闭线段上的点。

按横坐标(x)二分: 确定最左、最右端点 L, R , 将边界分成横坐标单调的上凸壳和下凸壳。先排除横坐标位于 $[x_L, x_R]$ 外的查询点,再用 $\text{cross}(R - L, p - L)$ 的符号选择对应凸壳,在该链上二分找到查询点横坐标所在的相邻顶点区间,并通过方向测试判断是否越过边界。也可以分别二分两条链;最左、最右点不唯一、查询点落在弦 LR 上或存在竖直极边时,需要按既定边界语义处理。

单次查询的时间复杂度为 $O(\log h)$ 。

利用极角序二分: 固定一个顶点 v_0 ,先用射线 v_0v_1 和 v_0v_{h-1} 判断查询点是否位于整个扇形内,再按 $\text{cross}(v_i - v_0, p - v_0)$ 的符号二分找到相邻射线 v_0v_i, v_0v_{i+1} ,最后判断查询点是否位于闭三角形 $v_0v_iv_{i+1}$ 中。

已有规范凸多边形时,检查或建立上述索引需要 $O(h)$,单次查询为 $O(\log h)$, q 次查询为 $O(h + q \log h)$ 。若从 n 个无序点开始,还要另加一次 $O(n \log n)$ 的凸包构造,不能把它计入每次查询。

50.2.1.2 判断直线是否与凸多边形相交

设直线为 $l(t) = p + tv$,其中方向向量 $v \neq 0$,对凸多边形顶点定义 $f_i = \text{cross}(v, v_i - p)$ 。直线与闭凸多边形相交,当且仅当 $\min f_i \leq 0 \leq \max f_i$ 。利用凸多边形边向量的循环极角序,可以分别查询这个线性函数在两个相反方向上的支撑极值;极值形成相等平台时,返回平台任一端点即可。

规范化预处理后, 单次查询为 $O(\log h)$; 若相切也算相交, 等号必须保留。点或线段等退化凸包需单独处理。

50.2.1.3 过一点求凸多边形切线

本节讨论从严格位于凸多边形外部的点 p 引出的两条支撑切线。若 p 位于内部, 则不存在经过 p 的支持线; 若 p 位于边界上, 可能得到所在边的支持线, 或在顶点处得到一族支持线, 需要另行约定返回协议, 不能简单归入“没有切线”。

对于严格外点, 切点处相邻顶点位于射线 pv_i 的同一侧。沿规范的循环凸多边形考察相邻顶点相对射线的方向符号, 两个切点对应这一循环序列的两个转折位置, 可以通过循环二分分别找到。

该 $O(\log h)$ 查询要求顶点已经按逆时针顺序排列, 并且已经删除冗余共线点; 若允许切线与整条边重合, 还要规定返回边的哪个端点。方向为 0、查询点位于边界以及 $h < 3$ 时均需单独处理。规范化需要 $O(h)$, 之后每个严格外点的两切点查询为 $O(\log h)$ 。

50.2.2 动态凸包(不删)

只增动态凸包维护一个数据结构, 支持以下两个操作:

- 修改: 在当前点集中加入一个点
- 询问: 查询一点是否在当前点集的凸包内

分别使用按横坐标有序的结构维护上凸壳和下凸壳, 才能完成整个凸包的点内查询。相同横坐标下, 上凸壳只保留必要的最高候选, 下凸壳只保留必要的最低候选; 重复点和边界共线点采用与静态凸包一致的删除策略。

初始化时应单独处理空集、单点和线段。新点位于当前横坐标范围之外时, 某一侧可能没有前驱或后继, 应按凸壳端点插入处理, 不能假设两个邻居始终存在。

- 询问: 分别在上下凸壳中二分相邻点, 单次最坏时间为 $O(\log h)$;
- 修改: 二分找到新点横坐标附近的前驱、后继。若新点不可能成为该凸壳的顶点则舍弃; 否则插入, 并向左右连续删除所有破坏凸性的旧顶点。

设某次插入共删除 k 个旧顶点。使用邻接迭代器, 并采用类似 `std::set::erase(iterator)` 的摊还常数时间删除接口时, 本次代价为 $O(\log h + k)$, 单次最坏可能达到 $O(h)$ 。若数据结构的迭代器删除不是这一复杂度, 或每删除一个点都按键重新查找, 则还会多出相应的对数因子。

一个点一旦从不断扩大的凸包中被删除, 之后不会重新成为极点, 所以每个点在每条凸壳上至多被删除一次。连续插入 N 个点时, 总删除次数为 $O(N)$, 总时间为

$O(N \log N)$, 即每次插入摊还 $O(\log N)$, 而不是单次最坏 $O(\log N)$ 。若另有 q 次点内查询, 总复杂度为 $O((N + q) \log N)$ 。

51 闵可夫斯基和

对于任意两个平面点集 $A, B \subseteq \mathbb{R}^2$, 它们的闵可夫斯基和定义为

$$A + B = \{\vec{a} + \vec{b} \mid \vec{a} \in A, \vec{b} \in B\}$$

闵可夫斯基和并不限于凸多边形。下文只讨论非空凸多边形, 并将凸多边形视为包含内部和边界的闭凸集; 若 A, B 都是凸集, 则 $A + B$ 仍是凸集。

51.1 求两个凸多边形的闵可夫斯基和

两个凸多边形的边方向序列按极角归并后, 可以得到和多边形的边方向序列。这个对应关系不是“一条输入边保留为一条结果边”: 若两个当前边方向相同, 应将它们的边向量相加为一条结果边, 并同时向后移动; 重复点、零边和冗余的共线中间点也应删除。

线性归并要求两个输入已经是规范的凸包:

- 顶点均按逆时针顺序给出, 并且末尾不重复首点;
- 删除连续重复点和冗余的共线中间点;
- 分别旋转顶点序列, 使 (y, x) 字典序最小的顶点位于开头, 从而让两组非零边向量使用同一个极角断点。

设规范后的顶点序列分别为 $a_0, \dots, a_{n-1}$ 和 $b_0, \dots, b_{m-1}$, 从 $a_0 + b_0$ 开始, 按极角归并两组边向量并依次求前缀和即可。判断两条边是否应合并时, 不能只判断叉积为 0, 还要确认两者同向; 反向平行边的叉积同样为 0。全部边处理完后会再次得到起点, 返回顶点序列时应删除这个重复终点, 并在循环意义下清理冗余的共线中间点。

上述流程以两个至少有三个非共线顶点的二维凸多边形为主。单点与其它集合的和只是平移; 全体点共线时应先压缩为线段, 并对点、线段等退化情况单独处理, 不能继续依赖面积符号或“逆时针多边形”的约定。

若两个输入已经规范化, 归并的时间复杂度和结果空间复杂度都是 $O(n + m)$ 。若将所有边重新做一次比较排序, 时间复杂度是 $O((n + m) \log(n + m))$; 若输入只是无序点集, 还要先求凸包, 总时间复杂度为 $O(n \log n + m \log m)$ 。

51.2 求两个凸多边形的最大、最小距离

令两个非空凸多边形对应的点集分别为 A, B , 并定义

$$D = A - B = A + (-B) = \{\vec{a} - \vec{b} \mid \vec{a} \in A, \vec{b} \in B\}$$

构造 $-B$ 时, 将 B 的每个顶点取相反数即可; 这个变换保持逆时针方向, 但仍需按前述规则重新旋转起点, 才能与 A 的边序列线性归并。

则两者之间的最大、最小距离分别为

$$\max_{\vec{p} \in D} \|\vec{p}\|, \quad \min_{\vec{p} \in D} \|\vec{p}\|$$

这两个问题的扫描方式不同:

- 范数是凸函数, 最大值可以在 D 的某个顶点取得, 因此枚举 D 的顶点即可;
- 最近点可能位于边的内部。求最小值时, 应先判断原点是否位于 D 的内部或边界上, 若是则答案为 0; 否则枚举 D 的每条边, 计算原点到线段的最小距离。若 D 退化为点或线段, 则分别计算点距或点到线段的距离。

因此, 不能用“只枚举和多边形顶点”的方法同时求最小距离。输入已经规范化时, 构造 D 并完成上述扫描的总时间复杂度为 $O(n + m)$ 。

使用整数坐标时, 点的加减、边向量相加、叉积、点积和长度平方都应根据坐标上界选用足够宽的中间类型。在已经由坐标范围证明中间结果不会溢出的前提下, 竞赛中常见的 `long long` 坐标实现可使用 `__int128`; 运算前应先提升类型, 不能先在 `long long` 中溢出再转换。若推导出的范围仍超过 `__int128`, 则需要使用更宽的整数类型或多精度整数。

52 半平面交

52.1 概念

设有向直线由点 P 和非零方向向量 $\vec{v}$ 表示。本文把包含边界的左半平面定义为

$$H(P, \vec{v}) = \{X \mid \vec{v} \times (\vec{X} - \vec{P}) \geq 0\}.$$

浮点实现中的“在右侧”应按叉积小于负容限判断；容限应与参与运算的向量尺度匹配，或先对方向进行规范化。落在边界上的点仍属于该闭半平面。整数输入可以使用足够宽的中间类型进行精确方向判定。

半平面交：多个半平面的交集。

- 半平面交涉及的直线均使用点向式表示。

半平面交一定是凸集，但结果可能为空、无界、退化为线段或点，也可能是有正面积的有界凸多边形。

可以按题意加入一个外框，把问题改成“原半平面交与该外框的交”。只有在题目本来就要求裁剪到该范围，或能够证明外框包含所有可能的有效答案时，这一变换才不影响目标结果。任意选取固定“大数”可能把非空但远离原点的交集误判为空，也会把原本无界的交集改成有限面积的多边形，因此不能用它无条件代替无界性判断。

52.2 做法

52.2.1 法一：朴素算法

在已知交集是有界凸多边形，或已经合法加入外框的前提下，求出各直线的交点，判断各交点是否在所有半平面内。

对满足条件的各点求凸包。

时间复杂度： $O(n^3)$ 。

52.2.2 法二:增量法

从一个能够覆盖目标结果的初始凸多边形开始,每次用直线切割当前多边形。若没有经过证明的初始边界,该方法只能得到裁剪后的交集,不能表示一般无界半平面交。

时间复杂度: $O(n^2)$ 。

52.2.3 法三:排序增量法

求凸包算法中,对点进行排序可以有效降低复杂度。

利用凸多边形各边方向向量的有序性,同样可以对各直线进行排序后求半平面交。

对于方向向量同向的直线,只保留对应左半平面最严格的一条。具体地,对 $L_1 = (P_1, \vec{v})$ 与 $L_2 = (P_2, \vec{v})$:若

$$\vec{v} \times (\vec{P}_2 - \vec{P}_1) > 0,$$

则 $H(P_2, \vec{v}) \subset H(P_1, \vec{v})$, 应保留 L_2 ; 叉积为 0 时两条边界重合, 只需保留一条。“最左”只有结合方向和左半平面定义后才有意义, 不能作为独立的几何比较规则。

52.2.3.1 维护过程:

将直线按方向极角递增排序, 并先合并同向平行线。双端队列中只保存仍可能成为边界的直线, 其方向严格递增; 插入阶段维护的是一条尚未首尾闭合的候选边界链。闭合清理完成前, 不能把所有相邻直线交点或队列中半平面的交直接当作当前完整答案。

对新的半平面 L , 一种经典的“队列只存直线、从队尾加入”实现采用以下顺序。每次准备求端部两线交点前, 都必须先确认它们不平行; 若平行, 应转入下文的同向归并、反向相容或矛盾分支, 不能继续执行交点测试。

下面的相邻交点版流程以“目标结果是有界正面积多边形”或“已经合法裁剪到外框”为接口前提。若要原生返回无界、线段、射线、直线或点, 相容的反向平行线之间没有有限交点, 必须改用能够表示无穷边和低维集合的额外状态; 以下队列步骤本身不是处理这些结果的完整模板。

1. 当队列至少有两直线、队尾两线不平行, 且它们的交点严格位于 L 的右侧时, 持续弹出队尾;

2. 当队列至少有两直线、队首两线不平行,且它们的交点严格位于 L 的右侧时,持续弹出队首;
3. 检查 L 与当前端部是否出现平行关系,按对应分支处理后再决定是否将 L 加入队尾。

全部直线处理完后还要进行闭合清理。只有当队列至少有三条直线且待求交的端部两线不平行时,才能用队首半平面检查队尾两线的交点,或用队尾半平面检查队首两线的交点;仍不满足时继续从对应端弹出。清理结束后,也只有在相邻直线以及队尾、队首直线均不平行,并且结果已确认为有界多边形时,才能计算这些交点作为多边形顶点;其它情况应转入空、无界或退化结果分类。

这里“先弹队尾,再弹队首”是上述具体队列表示和入队方式的一部分,不是脱离实现的不变几何定理。在上述两个循环都以“队列至少有两直线”为条件时,不能机械交换顺序:队列恰有两条线且其交点被新半平面排除时,错误地先弹队首可能删掉本应保留的约束。若改为存交点、直接裁剪凸多边形,或相应调整循环条件和队列不变量,操作顺序可以不同。关键是维护极角有序的候选边界链,并在所有输入处理后完成首尾闭合检查。

每条直线至多入队、出队各一次,合并同向线和维护队列共 $O(n)$,加上极角排序后的总时间复杂度为 $O(n \log n)$ 。

52.2.3.2 平行线与空集判断

求相邻直线交点前必须先判断平行,不能除以接近 0 的叉积。同向平行线按前文规则只保留最严格者;反向平行线不能合并,也不能仅凭方向相差 180° 判空。

例如,有向直线 $(P_1 = (0, 0), \vec{v}_1 = (1, 0))$ 表示 $y \geq 0$, $(P_2 = (0, 1), \vec{v}_2 = (-1, 0))$ 表示 $y \leq 1$ 。两方向恰好相差 180° ,交集却是非空条带 $0 \leq y \leq 1$ 。

更一般地,若 $\vec{v}_2$ 是 $-\vec{v}_1$ 的正倍数,令

$$\Delta = \vec{v}_1 \times (\vec{P}_2 - \vec{P}_1),$$

则只考虑这两个闭半平面时: $\Delta > 0$ 得到非空条带, $\Delta = 0$ 退化为公共边界直线, $\Delta < 0$ 才互相矛盾。其它半平面仍可能继续缩小该交集。

方向角差本身不是一般的空集判据。没有合法外框时,队列不足三条也不表示交集为空,它还可能是整个平面、半平面、条带、直线或其它无界/退化凸集。若接口要区分 EMPTY、UNBOUNDED、退化非空集和有界多边形,必须进行相应的可行性与维数判断。

52.2.3.3 边界与退化结果

由于本文采用闭半平面,交点恰好落在新直线上时仍是可行点,不应按“右侧”弹出;浮点实现通常只在叉积小于负容限时弹出。这有助于保留零面积交集,但仅凭“不弹边界点”不能可靠地区分空集、线段和点。若需要支持多线共点等退化情况,还应去重交点并做额外的非空性和维数检查;许多只返回正面积有界多边形的模板会直接把这些情况排除在接口契约之外。

52.2.4 法四:分治法

时间复杂度: $O(n \log n)$ 。

$[l, r]$ 直线的半平面交是 $[l, mid]$ 半平面交和 $[mid + 1, r]$ 半平面交的交集。若各子问题都能按契约表示为有界凸多边形,核心在于实现 $O(n)$ 求两个凸多边形的交;无界或退化结果不能未经表示转换就直接套用这一合并。

52.2.4.1 实现一:

使用法三求两个凸多边形半平面交得到凸多边形的交。

因为半平面交求出的交的边是有序的,所以合并 $[l, mid]$ 和 $[mid + 1, r]$ 时,两个凸多边形都是有序的,那么使用法三维护的时间复杂度瓶颈即为维护双端队列的复杂度,为 $O(n)$ 。

52.2.4.2 实现二:

对两个凸多边形的顶点进行扫描线,对两相邻竖线之间的梯形求交。单次 $O(1)$,共 $O(n)$ 条竖线。

53 旋转卡壳

点集的直径是平面最远点对的距离。直径一定可以由其凸包上的两个顶点取得,因此原始点集应先求凸包。

53.1 输入约定

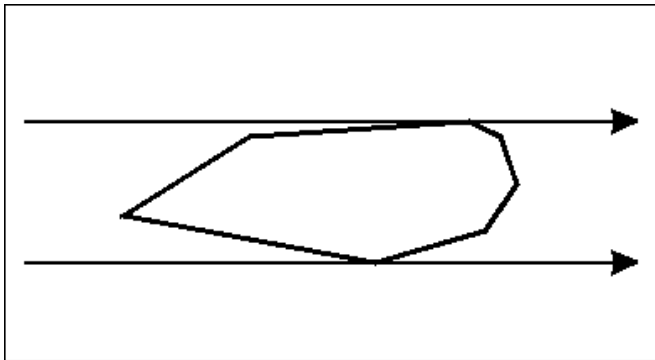
设凸包顶点为 $p_0, p_1, \dots, p_{n-1}$, 下标按模 n 计算。旋转卡壳的通用流程要求:

- 删除重复坐标、末尾重复的首点以及由此产生的零长度边;
- 顶点统一按逆时针顺序排列;
- 删除凸包边上的共线中间点。若业务需要保留这些点,可以在规范化副本上执行本文流程,再映射回原下标;不能直接套用下述严格推进规则;
- 空集的直径没有定义; $n = 1$ 时直径为 0, $n = 2$ 或全体点共线时直接计算线段两端的距离,不套用一般卡壳流程。

若输入是 N 个未排序点,求凸包通常需要 $O(N \log N)$;下文的线性复杂度只计算已有规范凸包后的卡壳过程。整数坐标实现还应根据坐标范围为叉积和距离平方选择足够宽的中间类型。

53.2 旋转卡壳求直径

用两条平行支撑线绕凸包旋转,它们与凸包的接触点或接触边构成对踵对。



对于边 $p_i p_{i+1}$, 定义二倍有向面积

$$A(i, j) = \text{cross}(p_{i+1} - p_i, p_j - p_i)$$

在逆时针凸包上, 固定 i 后, $A(i, j)$ 随 j 沿凸包循环呈单峰变化, 但最大值可能形成平台。边 $p_i p_{i+1}$ 逆时针移动时, 使面积最大的顶点下标 j 也只会沿逆时针方向移动, 因此可以使用双指针。

53.3 算法流程

1. 对第一条边从 $j = 1$ 开始, 仅在 $A(0, j+1) > A(0, j)$ 时推进, 使 j 停在最大值平台的第一个点。
2. 依次枚举边 $p_i p_{i+1}$ 。当 $A(i, j+1) > A(i, j)$ 时向后移动 j , 直到面积不再严格增大。
3. 使用 p_j 分别与当前边的两个端点 p_i, p_{i+1} 更新直径。
4. 若 $A(i, j+1) = A(i, j)$, 说明 p_j, p_{j+1} 位于同一个对踵平台, 还要使用 p_{j+1} 与当前边的两个端点更新答案。输入已经删除共线中间点时, 最大值平台至多由一条支撑边的这两个端点构成。
5. 继续枚举下一条边, 且不重置 j 。

也可以在面积相等时继续移动指针, 但必须在每次移动前后更新平台上的候选, 并给循环设置单圈或步数边界; 不能使用“相等就一直移动, 循环结束后只检查最后一个点”的流程。若不删除共线中间点, 则需要能越过非最大平坦段、单调维护整个平台并重新证明复杂度的广义实现。对于同一条边, 点到直线的距离只相差一个固定的边长分母, 所以可以直接比较 $A(i, j)$, 无需开方。文中的大小与相等比较默认使用精确几何谓词; 浮点实现需要采用统一且与数据尺度匹配的容差策略。

边指针枚举 n 条边; 对踵指针使用只增不减的展开下标, 访问顶点时再对 n 取模, 并设置明确的推进步数上界。初始化和完整枚举期间该指针总共只推进 $O(n)$ 次, 因此时间复杂度为 $O(n)$ 。

53.4 应用

53.4.1 两个凸多边形间的最大、最小距离

设两个非空凸多边形为 P, Q , 并将它们视为包含内部和边界的闭凸集。定义差多边形

$$D = P + (-Q) = \{p - q \mid p \in P, q \in Q\}$$

任意 $z \in D$ 都对应一对点之差, 所以

$$d_{\max}(P, Q) = \max_{z \in D} \|z\|, \quad d_{\min}(P, Q) = \min_{z \in D} \|z\|$$

- 最大距离可以在 D 的某个顶点取得, 构造 D 后扫描所有顶点即可;
- 求最小距离时, 若原点位于 D 的内部或边界上, 则 $P \cap Q \neq \emptyset$, 包括相交、相切或一个多边形包含另一个的情况, 答案为 0; 否则最近点可能位于 D 的边内部, 应扫描所有边并计算原点到线段的距离, 不能只扫描顶点。

构造 $-Q$ 后仍需将其按与 P 相同的极角断点重新规范化, 再按极角归并两组边向量; 方向相同的边向量应相加, 并同时推进两个指针。两个输入凸包已经去重、同向有序并规范起点时, 构造 D 及上述扫描可以在 $O(|P| + |Q|)$ 时间内完成; 点或线段等退化凸包需单独处理。

最小距离也可以直接对两个凸包做专门的线性卡壳: 必须先完整处理边相交、相切和包含, 不相交时在相应事件中计算点到线段或线段到线段的距离, 并处理平行边同时推进的情况。这不是求直径的“最大点到边面积”流程, 不能直接复用上面的更新式。

53.4.2 凸多边形的最小面积、最小周长外接矩形

最小面积或最小周长外接矩形一定存在一条支撑边与凸包的某条边共线。枚举每条凸包边作为基准方向时, 除当前边外, 还需维护三个单调移动的支撑点: 法向距离最大的点, 以及沿边方向投影最小、最大的两个点。由这些投影可以得到当前外接矩形的长和宽, 并更新面积或周长。

各支撑方向出现相等投影时同样需要规定平台的推进方式。规范凸包上的所有指针总共只前进 $O(n)$ 次, 因此卡壳部分的时间复杂度为 $O(n)$; 若对每条边重新扫描所有顶点, 则会退化为 $O(n^2)$ 。单点和线段的外接矩形需按题目是否允许退化矩形单独约定。

54 扫描线

54.1 扫描线

扫描线把问题拆成按某个坐标排序的事件,并用数据结构维护相邻事件之间的状态。使用扫描线前必须明确:

- 哪些位置会改变状态;
- 同一位置的多个事件如何成批处理;
- 数据结构中的顺序在扫描过程中为何仍然有效;
- 端点接触、重合和退化对象是否计入答案。

54.2 轴对齐矩形面积并

给出 n 个边平行于坐标轴的矩形。沿 x 轴扫描时,每个非退化矩形产生左边加入、右边删除两个事件,维护扫描线上被覆盖的 y 区间总长度。

将所有 y 端点离散化后,可以使用线段树维护区间覆盖次数和覆盖总长度。设上一个事件横坐标为 x_{pre} 、当前覆盖长度为 L ,到达新的横坐标 x 时先累加

$$L(x - x_{pre})$$

再统一处理该横坐标的所有加入、删除事件。同一横坐标必须分组,否则事件的任意先后容易污染状态语义。区间通常按半开形式 $[y_l, y_r)$ 维护;零宽或零高矩形对面积没有贡献。时间复杂度为 $O(n \log n)$ 。

54.3 判断线段集中是否存在交点

朴素做法需要 $O(n^2)$ 。扫描线算法从左向右处理线段端点,并维护所有活动线段在当前扫描位置的纵向次序。必须先约定交点语义:下文的通用版本将线段视为闭集,端点接触和共线重合也算相交。

54.3.1 一般位置下的简化算法

先假设所有线段均非竖直、端点横坐标互不相同,并且不存在共端点、端点落在其它线段上、共线重合或三线共点等退化情况。事件只有左、右端点:

- 遇到左端点时插入该线段,并检查它与活动集合中前驱、后继是否相交;
- 遇到右端点时,记录该线段原来的前驱、后继,删除后检查这两个新相邻的线段是否相交。

活动集合按线段在当前横坐标处的 y 值排序。虽然比较式依赖扫描横坐标,但在这个“只判断是否存在并在发现后立即返回”的算法中,可以证明它在返回前不会改变已有元素的相对次序:取横坐标最小的交点,在该交点左侧足够近的位置,两条相交线段必然相邻;否则夹在它们之间的线段会更早相交或经过同一个交点。它们成为相邻项时,只可能刚插入了其中一条线段,或刚删除了原本夹在中间的线段,因此已经进行过邻接检查。在发现第一个交点之前,任意两条活动线段的真实纵向次序都没有交换。

因此,依赖当前横坐标的 `set` 比较器不是无条件可用:必须保证上面的不变量。比较器还必须形成严格弱序;相同 y 时应按事件一侧的斜率和唯一编号继续区分,不能直接使用可能破坏传递性的 `eps` 判等。整数坐标可以通过有理数交叉相乘进行精确比较,并使用足够宽的中间类型。

在上述一般位置假设下,每条线段至多插入、删除一次,每次只检查常数个相邻项,时间复杂度为 $O(n \log n)$ 。如果需要枚举全部 K 个交点,则纵向次序确实会在交点处改变,必须将交点也加入事件; Bentley–Ottmann 一类算法的复杂度是输出敏感的 $O((n + K) \log n)$,不能继续沿用只含端点的 `set`。

54.3.2 同横坐标与退化情况

若要支持任意闭线段,不能只给同一横坐标的事件规定一个简单的全局先后顺序,而应成批处理。所有在该横坐标开始、结束或退化为竖线的线段在该位置都有效;必须先完成共端点、端点在线段上等接触检查,再按扫描线左侧的次序删除结束线段,并按右侧的次序插入开始线段。若只把严格内点相交计入答案,则事件批处理和线段相交谓词都要相应排除端点接触。

竖线段没有单一的 $y(x)$,不能作为普通元素插入上述活动集合。对 $x = x_0$ 、纵坐标范围为 $[y_l, y_r]$ 的竖线段,应在 x_0 的事件批次内查询所有当前有效非竖线段中是否存在 $y(x_0) \in [y_l, y_r]$ 的线段;这里“当前有效”同时包括在 x_0 结束和开始的线段。判断存在性时,找到第一个不低于 y_l 的元素并验证其不高于 y_r 即可。同一 x_0 上的多条竖线段还要将 y 区间排序,检测端点接触或区间重叠;零长度线段则按 $[y_l, y_r]$ 退化为一点的同批查询处理。

共线重合、零长度线段、一个端点落在另一条线段内部等情况必须使用完整的方向测试和包围盒判断,不能只检查两个严格叉积异号。事件应携带线段唯一编号,删除时应定位这条具体线段;仅按当前 y 做一次等价键查找,可能找到错误元素。支持这些退化情况的实现仍可做到 $O(n \log n)$ 的存在性判定,但需要上述批事件、区间

查询和严格的数据结构契约,不能由简化版直接推出。

54.4 三角形面积并

设输入为 n 个非退化闭三角形,共有 $3n$ 条边。令 K 为相交边对数;多条边经过同一点时仍按边对计数,共线重合的一对边也计入。于是 $K = O(n^2)$,且最坏可以达到 $\Theta(n^2)$ 。因此,三角形面积并不能无条件套用只有线性事件数的 $O(n \log n)$ 扫描线。

54.4.1 法一:按横坐标分条积分

事件横坐标至少包括所有三角形顶点、所有非重合边交点,以及共线重合段端点的横坐标。将不同事件横坐标排序后,在两个相邻事件 x_j, x_{j+1} 之间,没有边开始、结束或交换纵向次序。每个三角形与竖截线的交集为空或一个区间,这些区间端点都是 x 的一次函数;覆盖区间并的组合保持不变,因此总覆盖长度 $L(x)$ 也是一次函数。该竖条面积可以用梯形公式精确计算:

$$\frac{x_{j+1} - x_j}{2} (L(x_j^+) + L(x_{j+1}^-))$$

这里必须使用开竖条内活动边给出的单侧极限,并维护竖截区间的并,而不只是给边排序。同一横坐标的顶点和交点应统一处理;竖直边只位于事件位置,本身没有正宽度。共端点、多边共点、相切和共线重合边还需要统一的去重与覆盖计数规则。

一个容易实现但较慢的版本,可以在每个竖条中点求出所有三角形的截线区间,排序合并,并用同一组边函数计算竖条两端的覆盖长度。共有 $O(n + K)$ 个竖条时,这种实现的复杂度为 $O((n + K)n \log n)$,最坏可达 $O(n^3 \log n)$ 。若要达到 $O((n + K) \log n)$ 一类的输出敏感复杂度,还需要输出敏感地发现边交点,并动态维护活动边排列、覆盖计数以及 $L(x)$ 的一次函数系数;若预先枚举所有边对,前处理本身已经是 $O(n^2)$ 。

54.4.2 法二:构造并集轮廓

三角形并是一个多边形区域,但它可能包含多个不连通分量和孔洞,并且可能出现共线重合边。完整的轮廓法至少需要:

1. 将所有非退化三角形统一为逆时针方向;
2. 求出所有边交点,共线重合时还要加入重合段端点;

3. 按交点参数将每条输入边切成原子子线段;
4. 判断每条原子子线段两侧是否分别属于并集内部和外部,只保留分隔内外的线段,并将其定向为并集内部位于左侧;
5. 为重合边设置唯一归属规则,删除内部边和重复边;
6. 将暴露的有向子线段连接成若干闭环,并累加

$$\frac{1}{2} \text{cross}(p_i, p_{i+1})$$

在标准右手直角坐标系中,若所有边都定向为并集内部位于左侧,则外轮廓为逆时针、孔洞轮廓为顺时针,因此有向面积会自动扣除孔洞。仅检查子线段中点是否位于其它三角形内部,只在无重合的一般位置下才可能足够;一般输入需要检查线段两侧的局部覆盖状态并去重。

显式构造交点或完整轮廓时,复杂度必须包含 K 。若朴素地为每条切分后的子线段再次遍历所有三角形,最坏还会达到三次量级;要得到输出敏感界,必须真正构造线段排列、平面图或等价的多边形布尔并结构,不能把“找到轮廓”本身省略。整数端点产生的交点通常是有理数,事件排序、切边和端点合并应使用精确谓词及可靠的有理数表示;直接用含糊的 `eps` 合并事件可能改变平面图拓扑。

55 圆

55.1 圆基础

- 圆的表示: 圆心、半径 $r \geq 0$ 。除非特别说明, 下文的圆与圆关系、交点和公切线公式均假设半径严格为正; $r = 0$ 时图形退化为一个点, 需要单独处理
- 圆的周长: $C = 2\pi r$
- 圆的面积: $S = \pi r^2$

55.2 圆的关系问题

55.2.1 点与圆的关系

- 点在圆外
- 点在圆上
- 点在圆内

判定方法: 将点到圆心距离的平方与半径平方比较, 可以避免开平方。

平方比较只避免了开平方, 并不会自动消除误差。输入及中间运算采用精确数值表示且不溢出时, 比较结果才是精确的; 算法竞赛中最常见的情形是整数输入, 并使用足够宽的中间类型。例如输入为 `long long` 时, 中间乘积常需使用 `__int128`。若输入为浮点数, 平方比较仍有舍入误差, 需要使用与数据尺度匹配的容限。

55.2.2 直线与圆的关系

- 相离
- 相切
- 相交

设直线为 $\vec{X} = \vec{P} + t\vec{v}$, 其中 $\vec{v} \neq \vec{0}$, 圆心为 C 。可以比较

$$[\vec{v} \times (\vec{C} - \vec{P})]^2 \quad \text{与} \quad r^2(\vec{v} \cdot \vec{v})$$

来区分相离、相切和相交, 从而避免除法与开平方。该比较在数值及中间运算均精确且不溢出时可以得到精确结果; 浮点输入仍需容限。

55.2.3 圆与圆的关系

令两个正半径圆的半径分别为 r_1, r_2 , 圆心距离为 d 。按下列顺序判断可以得到互斥分类:

- 相同: $d = 0$ 且 $r_1 = r_2$;
- 同心但不同圆: $d = 0$ 且 $r_1 \neq r_2$, 较小圆严格内含于较大圆;
- 相离: $d > r_1 + r_2$;
- 外切: $d = r_1 + r_2$;
- 相交于两点: $|r_1 - r_2| < d < r_1 + r_2$;
- 内切: $d = |r_1 - r_2|$;
- 严格内含: $0 < d < |r_1 - r_2|$ 。

若讨论圆盘的覆盖关系, 则较小圆盘被较大圆盘完全覆盖当且仅当

$$d + \min(r_1, r_2) \leq \max(r_1, r_2).$$

浮点实现中的相等关系均应使用容限。若允许零半径, 应先把退化圆当作点处理, 否则同圆、内切和外切条件可能同时成立, 以上分类及后续切线数量表不再适用。

55.3 交集问题

55.3.1 直线与圆的交点

前提: 直线方向向量 $\vec{v} \neq \vec{0}$, 且直线与圆不相离。

直线: $\{P, \vec{v}\}$

圆: $\{C, r\}$

设圆心到直线的距离为 d , 投影点为 C' , 则交点为

$$C' \pm \frac{\sqrt{\max(0, r^2 - d^2)}}{|\vec{v}|} \vec{v}.$$

相切时根号部分为 0, 两个表达式表示同一个点。 $\max$ 用于吸收浮点误差造成的微小负数; 它不能代替前面的相交关系判断。

55.3.2 圆与圆交点

以下公式要求 $r_1, r_2 > 0$ 、 $d > 0$, 且

$$|r_1 - r_2| \leq d \leq r_1 + r_2.$$

重合圆有无穷多个交点;相离、严格内含或同心但半径不同的圆没有交点,这些情况均应在使用公式前返回。

两圆圆心连线与一圆心到交点连线夹角 α 。

根据余弦定理,先计算

$$c = \frac{r_1^2 + d^2 - r_2^2}{2r_1d},$$

浮点实现中将 c 限制到 $[-1, 1]$,再令

$$\cos \alpha = c, \quad \sin \alpha = \sqrt{\max(0, 1 - c^2)}.$$

严格相交时可得到两个交点;内切或外切时 $\sin \alpha = 0$,两个结果重合,只应返回一个交点。

将两圆心连线对应向量进行伸缩旋转。

55.3.3 圆与多边形面积交

按多边形的每条有向边 $P_i P_{i+1}$,计算有向三角形 $CP_i P_{i+1}$ 与圆盘的带符号面积贡献,全部求和后再取绝对值。对凹多边形不能把各三角形贡献分别取绝对值后直接相加,否则重叠或位于外部的部分无法正确抵消。

55.3.4 多个圆的面积并

这里求的是多个圆盘覆盖区域的面积。预处理时应先丢弃零半径圆,将完全相同的圆去重;若圆 i 满足

$$d_{ij} + r_i \leq r_j,$$

则圆盘 i 被圆盘 j 完全覆盖,不会贡献圆并边界。相等比较在浮点实现中需要容限,但不能把带 eps 的比较器直接用于排序。

对保留下来的每个圆,求出其边界被其它圆覆盖的极角区间。只有严格部分相交的两圆才需要生成覆盖角区间;若通过 acos 计算半角,应先将浮点余弦值限制到 $[-1, 1]$ 。跨越 2π 的区间应拆成两段;无覆盖区间时,整个圆周均为可见边界。扫描角度事件时,同一极角的所有事件必须分组后统一更新覆盖计数,不能依赖同角事件

的任意先后顺序。实际排序应使用严格比较,再在排序结果中按容限合并近似同角事件。

设当前圆心为 (x_0, y_0) 、半径为 r 。对按逆时针方向遍历、极角从 θ_l 到 θ_r 的一段可见圆弧,它对边界积分 $\oint (x dy - y dx)$ 的贡献为

$$2S_{\text{arc}} = x_0 r (\sin \theta_r - \sin \theta_l) - y_0 r (\cos \theta_r - \cos \theta_l) + r^2 (\theta_r - \theta_l).$$

将所有可见圆弧的贡献相加再除以 2,即可得到圆并面积。每个圆产生并排序 $O(n)$ 个事件时,总时间复杂度为 $O(n^2 \log n)$ 。

55.4 切线问题

55.4.1 过圆外一点圆的切线

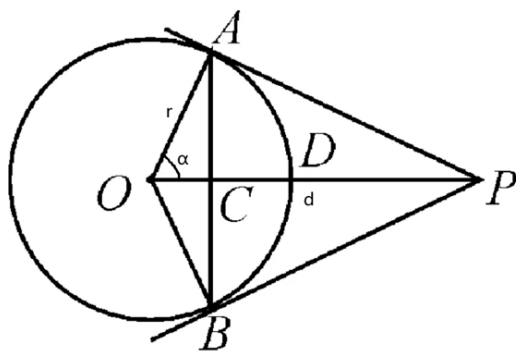
设点到圆心的距离为 d ,本节要求 $r > 0$ 且 $d > r$ 。令

$$c = \frac{r}{d},$$

浮点实现中先将 c 限制到 $[-1, 1]$,再使用 $\cos \alpha = c$ 。

- 向量伸缩、旋转

点在圆上时只有一条切线,点在圆内时没有实切线,应在套用上述构造前分别处理。



55.4.2 两圆的公切线

以下数量按不同的几何直线计数,并要求两个圆的半径均为正:

- 相同:无穷多条;

- 相离: 4 条;
- 外切: 3 条;
- 相交: 2 条;
- 内切: 1 条;
- 内含或同心但不同圆: 0 条。

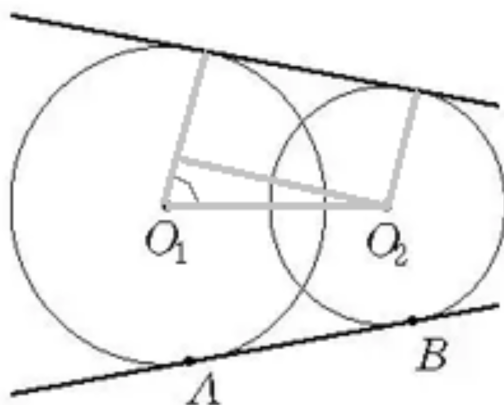
外切时两条重合的内公切线只计作一条,内切时两条重合的外公切线也只计作一条。零半径圆退化为点,不适用这张切线数量表。

55.4.2.1 外公切线

对两个不同且非同心的正半径圆,外公切线构造使用

$$c = \frac{r_1 - r_2}{d}.$$

仅当 $|r_1 - r_2| \leq d$ 时存在外公切线。浮点实现应将 c 限制到 $[-1, 1]$ 后再计算角度。

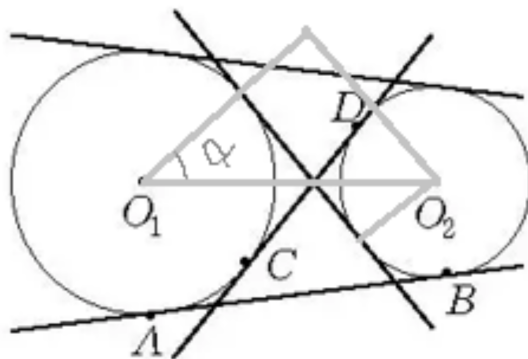


55.4.2.2 内公切线

内公切线构造使用

$$c = \frac{r_1 + r_2}{d}.$$

仅当 $r_1 + r_2 \leq d$ 时存在内公切线。同样应先将浮点计算得到的 c 限制到 $[-1, 1]$ 。



55.5 圆的反演

给定反演中心点 O 和反演半径 R , 若平面上点 P 和 P' 满足:

- 点 P' 在射线 OP 上
- $|\overrightarrow{OP}| \times |\overrightarrow{OP'}| = R^2$

则称点 P 和点 P' 互为反演点。

性质:

- 圆外的点的反演点在圆内, 反之亦然
- 圆上的点的反演点为其自身
- 不过点 O 的圆, 其反演图形也是不过点 O 的圆
- 过点 O 的圆, 其反演图形是不过点 O 的直线
- 两个图形相切, 则它们的反演图形也相切

55.6 最小圆覆盖

问题描述: 给定平面上 n 个点, 求一个半径最小的圆, 能覆盖所有的点。本节约定空点集返回空值, 因为最小半径虽可取 0, 但圆心不唯一; 若具体接口选择返回以原点为圆心的零圆, 也必须明确这是人为约定。

边界情况:

- 一个点的最小覆盖圆以该点为圆心, 半径为 0; 所有点重合时同理;
- 两个不同点的最小覆盖圆以它们的中点为圆心, 以两点距离的一半为半径;
- 三个不共线的点可以唯一确定外接圆;
- 三个点共线时不能直接套用外心公式, 覆盖它们的最小圆由最远点对作为直

径确定。

定理:如果点 p 不在点集 S 的最小圆覆盖圆内,那么它一定在 $\{p\} \cup S$ 的最小覆盖圆上,即最小覆盖圆一定经过点 p 。

将输入点按与原顺序独立的均匀随机排列打乱后,使用随机增量算法可以得到期望 $O(n)$ 的时间复杂度。常见三重循环实现的最坏时间复杂度仍为 $O(n^3)$;随机化影响的是期望运行时间,不影响算法返回结果的正确性。

56 基础三维几何

56.1 点

三维向量 (x, y, z)

56.1.1 点积

$$\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y + a_z b_z$$

当 $\vec{a}, \vec{b}$ 均非零时, 点积满足

$$\vec{a} \cdot \vec{b} = |\vec{a}| |\vec{b}| \cos \theta.$$

点积本身不是投影值。当 $\vec{b} \neq \vec{0}$ 时, $\vec{a}$ 在 $\vec{b}$ 方向上的标量投影与向量投影分别为

$$\text{comp}_{\vec{b}} \vec{a} = \frac{\vec{a} \cdot \vec{b}}{|\vec{b}|}, \quad \text{proj}_{\vec{b}} \vec{a} = \frac{\vec{a} \cdot \vec{b}}{\vec{b} \cdot \vec{b}} \vec{b}.$$

56.1.2 叉积

$$\vec{a} \times \vec{b} = (a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x)$$

几何意义: $|\vec{a} \times \vec{b}| = |\vec{a}| |\vec{b}| \sin \theta$

特别地, 当 $\vec{a}, \vec{b}$ 不平行时, $\vec{a} \times \vec{b}$ 非零, 可以作为 $\vec{a}, \vec{b}$ 所在平面的法向量。

在 $\vec{a}, \vec{b}$ 所在平面选定单位法向量 $\hat{n}$ 后, $(\vec{a} \times \vec{b}) \cdot \hat{n}$ 对应这两个向量在该有向平面内的二维叉积。

56.1.3 三重积

即 $(\vec{a} \times \vec{b}) \cdot \vec{c}$

在数值上,

$$(\vec{a} \times \vec{b}) \cdot \vec{c} = \begin{vmatrix} a_x & a_y & a_z \\ b_x & b_y & b_z \\ c_x & c_y & c_z \end{vmatrix}.$$

三重积是有符号量,其绝对值才是三个向量张成的平行六面体体积:

$$V = |(\vec{a} \times \vec{b}) \cdot \vec{c}|.$$

特别地,由 O, A, B, C 四点构成的四面体体积为

$$V_{OABC} = \frac{|(\overrightarrow{OA} \times \overrightarrow{OB}) \cdot \overrightarrow{OC}|}{6}.$$

56.2 线段

线段 AB 可以写成

$$\vec{X} = \vec{A} + t(\vec{B} - \vec{A}), \quad 0 \leq t \leq 1.$$

点在线段上的判定与二维类似,但两条空间线段即使投影相交也可能异面;判断相交时必须验证它们在三维空间中确实存在同一个交点。

56.3 直线

空间直线仍可使用点向式 $\vec{X} = \vec{P} + t\vec{v}$ 表示,其中 $\vec{v} \neq \vec{0}$ 。两条空间直线除相交、平行或重合外,还可能异面,因此不能直接套用二维直线的全部结论。

56.4 平面

56.4.1 三点式

三点 P_1, P_2, P_3 只有在不共线时才能唯一确定一个平面,即

$$(\vec{P}_2 - \vec{P}_1) \times (\vec{P}_3 - \vec{P}_1) \neq \vec{0}.$$

此时可以取法向量 $\vec{n} = (\vec{P}_2 - \vec{P}_1) \times (\vec{P}_3 - \vec{P}_1)$, 平面方程为

$$(\vec{X} - \vec{P}_1) \cdot \vec{n} = 0.$$

若三点共线,则有无穷多个平面经过它们,不能得到唯一平面。

56.4.2 点法式

平面上一点 P 与垂直于该平面的非零法向量 $\vec{n}$ 可以表示平面:

$$(\vec{X} - \vec{P}) \cdot \vec{n} = 0, \quad \vec{n} \neq \vec{0}.$$

56.5 球

球心 $O(x, y, z)$ 和半径 r 。下文涉及球面距离和球面三角形时均假设 $r > 0$ 。

体积公式: $V = \frac{4}{3}\pi r^3$

表面积公式: $S = 4\pi r^2$

56.5.1 球截面

设平面到球心的距离为 δ :

- $\delta = 0$ 时,截面是半径为 r 的大圆;
- $0 < \delta < r$ 时,截面是半径为 $\sqrt{r^2 - \delta^2}$ 的小圆;
- $\delta = r$ 时,平面与球面相切,交集退化为一个点;
- $\delta > r$ 时,平面与球面没有交点。

56.5.2 球面弧

设球面上两点为 P, Q , 令 $\vec{u} = \vec{P} - \vec{O}, \vec{v} = \vec{Q} - \vec{O}$, 则它们的中心角可以写成

$$\theta = \text{atan2}(|\vec{u} \times \vec{v}|, \vec{u} \cdot \vec{v}) \in [0, \pi],$$

也可以用 $\arccos((\vec{u} \cdot \vec{v})/r^2)$ 计算;浮点实现应先把余弦值限制在 $[-1, 1]$ 内。

球面距离为

$$d = r\theta.$$

当 P, Q 既不重合也不互为对跖点时, 连接它们的最短球面曲线唯一, 是两点所在大圆的劣弧。当 P, Q 为对跖点时, 任意经过它们的大圆半圆都是最短路, 长度均为 πr , 因此最短路不唯一。小圆弧一般不是球面的测地线, 也不是两点间的最短路。

对于非重合、非对跖的两点, 在固定的同一个大圆上有一条劣弧和一条优弧, 优弧只是其中较长的一条。若允许任意球面曲线通过绕行增加长度, 则不存在“所有曲线中最长”的连接方式。

56.5.3 球冠

球冠可视为球面上到极点的球面距离不超过 d 的区域, 其中 $0 \leq d \leq \pi r$ 。

- 球冠的高 h
- 球冠的底面半径: a

球冠曲面面积(不含底面圆): $S = 2\pi r h$

对应球缺的体积: $V = \frac{\pi h^2(3r-h)}{3}$

中心角(以球心为顶点, 弧对应的角): θ

$$\theta = \frac{d}{r}$$

$$h = r(1 - \cos \theta)$$

$$a = r \sin \theta$$

56.5.4 球面三角形

球面上三个点 A, B, C 两两不互为对跖点, 且不全位于同一个大圆上时, 可以用大圆劣弧围成非退化球面三角形。下文取标准的凸球面三角形, 设顶点 A, B, C 处的内角分别为 α, β, γ , 且角度均使用弧度制; 设 $a, b, c \in (0, \pi)$ 为对应边的中心角, 即

$$a = \angle BOC, \quad b = \angle COA, \quad c = \angle AOB.$$

三条边的实际弧长分别为 ra, rb, rc 。

球面三角形面积: $r^2(\alpha + \beta + \gamma - \pi)$ 。

球面余弦定理:

- $\cos a = \cos b \cos c + \sin b \sin c \cos \alpha$
- $\cos b = \cos c \cos a + \sin c \sin a \cos \beta$
- $\cos c = \cos a \cos b + \sin a \sin b \cos \gamma$

若使用实际弧长 s_a, s_b, s_c , 则代入三角函数前必须先换成中心角 $a = s_a/r, b = s_b/r, c = s_c/r$ 。

球面正弦定理:

- $\frac{\sin a}{\sin \alpha} = \frac{\sin b}{\sin \beta} = \frac{\sin c}{\sin \gamma}$

57 杂项

57.1 皮克定理

顶点坐标均是整数的简单多边形,其面积 A 和内部格点数目 i 和边上格点数目 b 的关系为 $A = i + \frac{b}{2} - 1$ 。

57.2 平面最近点对

问题描述:给出平面上 n 个点,求距离最近的点对。以下假设 $n \geq 2$;一旦发现两个点坐标完全相同,应立即返回 0。

引理:如果平面上 n 个点的最近点对距离为 $d > 0$,那么将这些点放进边长为 d 的半开网格中,每个格子内的点数不超过 4。

57.2.1 法一:经典分治算法

- 预处理:将所有点按横坐标排序;
- 分:按排序下标将点集均分为左右两个子集;
- 治:分别求出左右两个子集的最近点对距离 d_l, d_r ;
- 合:令 $d = \min(d_l, d_r)$, 求出跨越分界线且距离小于 d 的点对。

为了得到 $O(n \log n)$ 的复杂度,递归返回时还应将左右两部分按纵坐标线性归并。然后从这个有序序列中筛出横坐标到分界线距离小于 d 的点;条带已经按纵坐标有序,每个点只需检查其后纵坐标之差小于 d 的常数个点,标准结论为至多 7 个。

这样每一递归层的归并和条带扫描总计 $O(n)$,递推式为 $T(n) = 2T(n/2) + O(n)$,总时间复杂度为 $O(n \log n)$ 。若在每个递归结点重新将条带按纵坐标排序,则总时间复杂度会变为 $O(n \log^2 n)$ 。

57.2.2 法二:扫描线

对所有点按横坐标(x)排序,从左到右扫描,记录当前的最近点对距离 d 。

对于一个新的点,遍历之前的点中所有与其横坐标(x)之差小于 d 且纵坐标(y)之差小于 d 的所有点,更新最近点对。

用一个 `multiset` 存放与新点横坐标(x)之差小于 d 的点,内部按纵坐标(y)排序。

使用队列按 x 坐标顺序入队各点,当队首与新点横坐标(x)之差大于等于 d 时出队,并且将其从 `multiset` 中移除。

在 `multiset` 中二分找到纵坐标(y)之差小于 d 的分界点。

时间复杂度: $O(n \log n)$ 。

57.2.3 法三:随机化算法

先将点均匀随机打乱,用前两个点的距离初始化 d ;若两点重合则直接返回 0。令前 $i-1$ 个点的最近点对距离为 $d > 0$,作 $d \times d$ 的半开网格,使用哈希表记录每个格子内有哪些点。格子编号应使用 $(\lfloor x/d \rfloor, \lfloor y/d \rfloor)$;坐标为负数时,不能用向零截断的整数除法代替下取整。

对第 i 个点,找到其所在的格子,并找到以此格子为中心的九宫格内的所有点,更新最近点对。

如果最近点对被更新且 $d' > 0$,以新的距离 d' 重构网格。若发现重复点使 $d' = 0$,应在除法或重构网格之前立即返回 0。

在随机排列独立于输入、哈希表操作期望为 $O(1)$ 的前提下,该算法的期望时间复杂度为 $O(n)$ 。任意固定或对抗性的插入顺序不保证线性复杂度,最坏可以达到 $O(n^2)$ 。

57.3 辛普森积分

57.3.1 普通辛普森积分

辛普森积分简单来说就是使用二次函数去拟合待求曲线。

对于一个二次函数 $f(x) = ax^2 + bx + c$,有

$$\int_l^r f(x) dx = \frac{r-l}{6} \left(f(l) + 4f\left(\frac{l+r}{2}\right) + f(r) \right)$$

对于一个曲线 $g(x)$,若其积分区间为 $[l, r]$,将 $[l, r]$ 均分成 $2n$ 个区间,其中第 i 个区间为 $[x_i, x_{i+1}]$, $x_i = l + ih$, $h = \frac{r-l}{2n}$ 。

对于两个相邻的区间,三个点 $(x_i, g(x_i))$, $(x_{i+1}, g(x_{i+1}))$, $(x_{i+2}, g(x_{i+2}))$ 拟合成一段二次函数,套用二次函数积分公式即可。

57.3.2 自适应辛普森积分

对于普通辛普森积分,若 n 取太小,无法保证拟合精度;而若 n 取太大,无法保证时间。

所以可以采用自适应辛普森积分。

记 $S = S(l, r)$ 为当前区间的一次辛普森估计, $m = (l + r)/2$, 并令

$$S_2 = S(l, m) + S(m, r)$$

$S_2 - S$ 并不是真实误差。只有在被积函数足够光滑、误差已经进入相应渐近区间时, Richardson 外推才给出

$$I - S_2 \approx \frac{S_2 - S}{15}$$

因此,要求当前区间的绝对误差不超过 ε 时,通常以

$$|S_2 - S| \leq 15\varepsilon$$

作为接受条件,并返回校正后的结果

$$S_2 + \frac{S_2 - S}{15}$$

若不满足条件,则分别以 $\varepsilon/2$ 递归处理 $[l, m]$ 和 $[m, r]$ 。强制细分最少层数只能降低误差偶然抵消的风险,不能替代终止保护;实现还必须设置最大递归深度,并在浮点中点已经等于端点时停止。达到这些上限时,只能返回当前近似或报告未达到目标精度,不能继续承诺 ε 误差界。

已知不连续点应预先作为分界,将原区间拆成若干子区间分别积分;若函数在分界点无定义,还应使用单侧极限或避免在该点直接求值。端点奇异、无穷区间等瑕积分需要极限、截断或变量代换,不能仅依靠继续二分。算法没有只由 ε 决定的统一复杂度;若实际访问 K 个递归结点、单次函数求值代价为 T_f ,则可记为 $O(KT_f)$,其中 K 取决于函数性质和误差要求。

57.4 仿射变换

仿射变换将点映射为 $P \mapsto AP + b$,其中 A 是线性变换, b 是平移。讨论椭圆与圆、平行四边形与正方形之间的可逆变换时,需要假设 A 可逆。

57.4.1 应用

椭圆 $\xleftrightarrow{\text{可逆仿射变换}}$ 圆

在主轴方向已经对应平行的常见情形下, 同一个仿射变换能将多个椭圆分别变成圆, 当且仅当它们的长短轴比例相同, 即离心率相同。

平行四边形 $\xleftrightarrow{\text{可逆仿射变换}}$ 正方形

任意一个非退化平行四边形都能通过可逆仿射变换变成正方形。对于多个非退化平行四边形, 设第 i 个的一组相邻边向量为 u_i, v_i , 并记边矩阵

$$M_i = [u_i \ v_i]$$

同一个仿射变换能将它们分别变成正方形, 所得正方形不要求同大小、同位置或同朝向。当且仅当任选一个 M_1 后, 对每个 i 都存在 $\lambda_i > 0$ 和正交矩阵 R_i (即 $R_i^\top R_i = I$), 使

$$M_1^{-1} M_i = \lambda_i R_i$$

平移 b 不影响这个形状条件。特别地, 若所有平行四边形的两组对应边已经分别平行, 则还必须保证 $\|u_i\|/\|v_i\|$ 对所有 i 相同。仅有“底角相等、底边平行”并不充分: 单位正方形和边长为 2, 1 的矩形满足这两个条件, 却不可能被同一个可逆仿射变换同时变成正方形。

仿射变换将面积统一乘以 $|\det A|$ 。

例如, 当 $a, b > 0$ 时, $(x, y) \mapsto (x/a, y/b)$ 对应的面积变化为 $S \mapsto S/(ab)$ 。

第七部分

动态规划

58 背包 dp

其实感觉背包只是一种特别的、人为总结的 dp 问题,在 dp 水平足够之后,不需要专门地学习,也能得到其绝大多数的做法。

背包 dp 更多担任的是引导入门 dp 的角色,可以通过一系列的背包 dp 帮助新手了解 dp 思想,熟悉 dp。

以下最大价值类模板默认所有物品体积为正整数,并令 dp_j 表示“总体积不超过 j 时的最大价值”,因此可以把整个 dp 数组初始化为 0。若题目要求“总体积恰好为 j ”,则必须令 $dp[0] = 0$ 、其余状态初始化为 $-INF$,并只从可达状态转移;后文的可行性背包和方案数背包则默认讨论恰好达到对应体积。

58.1 01 背包

```
for (int i = 1; i <= n; i++)
    for (int j = V; j >= v[i]; j--)
        dp[j] = max(dp[j], dp[j - v[i]] + w[i]);
```

时间复杂度: $O(nV)$ 。

空间复杂度: $O(V)$ 。

58.2 完全背包

```
for (int i = 1; i <= n; i++)
    for (int j = v[i]; j <= V; j++)
        dp[j] = max(dp[j], dp[j - v[i]] + w[i]);
```

时间复杂度: $O(nV)$ 。

空间复杂度: $O(V)$ 。

58.3 多重背包

```
for (int i = 1; i <= n; i++)
    for (int j = V; j >= v[i]; j--)
        for (int k = 1; k <= min(s[i], j / v[i]); k++)
            dp[j] = max(dp[j], dp[j - k * v[i]] + k * w[i]);
```

时间复杂度: $O(\sum_i V \min(s_i, V/v_i))$, 粗略上界为 $O(nV^2)$ 。

空间复杂度: $O(V)$ 。

58.3.1 二进制优化

```

for (int i = 1; i <= n; i++) {
    int cnt = min(s[i], V / v[i]);
    int num = 1;
    while (cnt) {
        int take = min(num, cnt);
        cnt -= take;
        for (int k = V; k >= take * v[i]; k--)
            dp[k] = max(dp[k], dp[k - take * v[i]] + take * w[i]);
        if (num <= cnt)
            num <= 1;
    }
}

```

时间复杂度: $O(V \sum_i \log(\min(s_i, V/v_i) + 1))$ 。

空间复杂度: $O(V)$ 。

58.3.2 单调队列优化

```

for (int i = 1; i <= n; i++) {
    int now = (i & 1);
    int cur = (now ^ 1);
    int cnt = min(s[i], V / v[i]);
    dp[now] = dp[cur];
    for (int j = 0; j < v[i]; j++) {
        q.clear();
        for (int k = j; k <= V; k += v[i]) {
            int T = k / v[i];
            while (q.size() && q.front().first < k - cnt * v[i])
                q.pop_front();
            if (q.size())
                dp[now][k] = max(dp[now][k], q.front().second + T * w[i]);
            while (q.size() && q.back().second < dp[cur][k] - T * w[i])
                q.pop_back();
            q.push_back({k, dp[cur][k] - T * w[i]});
        }
    }
}

```

时间复杂度: $O(nV)$ 。

空间复杂度: $O(V)$ 。

注: - 实现上的一些细节,对于每个余数单独跑一遍,只需要开一个单调队列,常数更小。- 因为常数问题,实现不好的单调队列优化很可能跑不过二进制优化。

58.4 分组背包

```

for (int i = 1; i <= n; i++)
    for (int j = V; j >= 0; j--)
        for (int k = 1; k <= m[i]; k++)
            if (j >= v[i][k])
                dp[j] = max(dp[j], dp[j - v[i][k]] + w[i][k]);

```

时间复杂度: $O(V \sum_i m_i)$ 。

空间复杂度: $O(V)$ 。

58.5 多维背包

```
for (int i = 1; i <= n; i++)
    for (int j = V1; j >= v1[i]; j--)
        for (int k = V2; k >= v2[i]; k--)
            dp[j][k] = max(dp[j][k], dp[j - v1[i]][k - v2[i]] + w[i]);
```

时间复杂度: $O(n \prod V_i)$ 。

空间复杂度: $O(\prod V_i)$ 。

58.6 混合背包

这个没有固定形式,原则上可以把上述所有背包问题全部杂合在一起,每种物品都可以是上面任意一种类型。

58.7 可行性背包

```
dp[0] = 1;
for (int i = 1; i <= n; i++)
    for (int j = V; j >= v[i]; j--)
        dp[j] |= dp[j - v[i]];
```

时间复杂度: $O(nV)$ 。

空间复杂度: $O(V)$ 。

58.7.1 bitset 优化

```
dp[0] = 1;
for (int i = 1; i <= n; i++)
    dp |= (dp << v[i]);
```

时间复杂度: $O(\frac{nV}{w})$ 。

空间复杂度: $O(\frac{V}{w})$ 。

58.8 限和背包的二进制优化

可以发现,本质上,01 背包和完全背包都是多重背包。

对于 01 背包,可以把体积、价值相同的物品合并,视作多重背包,从而应用二进制优化。

结论:对于一个体积为 V 的可行性背包,若所有物品的体积之和不超
过 W ,则使用二进制优化的时间复杂度为 $O(V\sqrt{W})$,使用 `bitset` 还可以
进一步优化到 $O(\frac{V\sqrt{W}}{w})$ 。

证明:根号分治,对于价值 $> \sqrt{W}$ 的物品,不超过 $\sqrt{W}$ 个,那么这一部分直接跑 01 背包是 $O(V\sqrt{W})$ 的。

对于价值 $\leq \sqrt{W}$ 的物品,可得: $\sum_{i=1}^{\sqrt{W}} i \times a_i \leq W, a_i \geq 0$, 求 $\sum \log a_i$ 。

应用拉格朗日乘子法:

$$\text{令 } F = \sum_{i=1}^{\sqrt{W}} \log a_i - \lambda \left(\sum_{i=1}^{\sqrt{W}} i \times a_i - W \right), \quad \frac{\partial F}{\partial a_i} = \frac{1}{a_i} - \lambda i = 0 \Rightarrow a_i = \frac{1}{\lambda i}.$$

代入反解 $a_i = \frac{\sqrt{W}}{i}$ 。

$$\text{此时, } \sum_{i=1}^{\sqrt{W}} \log a_i = \sqrt{W} \log \sqrt{W} - \log(\sqrt{W}!).$$

根据斯特林近似: $\log(n!) = n \log n - n + \frac{1}{2} \log(2\pi n) + o(1)$ 。

代回,可得 $\sum_{i=1}^{\sqrt{W}} \log a_i \sim \sqrt{W}$

综上:对于价值 $\leq \sqrt{W}$ 的物品,这一部分时间复杂度仍为 $O(V\sqrt{W})$ 。

58.9 背包求方案数

以 01 背包为例:

```
dp[0] = 1;
for (int i = 1; i <= n; i++)
    for (int j = V; j >= v[i]; j--)
        dp[j] += dp[j - v[i]];
```

时间复杂度: $O(nV)$ 。

空间复杂度: $O(V)$ 。

58.9.1 卷积优化

每一个物品可以视作一个形式幂级数, n 个物品求方案数等价于 n 个形式幂级数卷积。

若物品的体积为 v , 物品的数量为 s , 则 $F(x) = \sum_{i \geq 0} x^{iv} = \frac{1 - x^{(s+1)v}}{1 - x^v}$ 。

$$G(x) = \prod_{i=1}^n \frac{1 - x^{(s_i+1)v_i}}{1 - x^{v_i}} =$$

$$\exp\left(\sum \ln F(x)\right)$$

$\ln F(x) = \ln(1 - x^{(s+1)v_i}) - \ln(1 - x^v)$, 按泰勒展开, 可得 $\sum_{n=1}^{\infty} \frac{(x^v)^n}{n} -$

$$\sum_{n=1}^{\infty} \frac{(x^{(s+1)v})^n}{n}.$$

按 s, v 算 x^i 的系数贡献即可。

时间复杂度: $O(V \log V)$ 。

空间复杂度: $O(V)$ 。

58.9.2 可撤销性

58.9.2.1 多项式角度

删除一个物品,相当于少乘一个形式幂级数,做一次多项式除法即可。

若是 01 背包,使用长除法,撤销单个物品就是 $O(V)$ 的。这里各物品对应多项式的常数项均为 1,所以递推不需要额外求常数项逆元;更一般的除式则要求常数项可逆。若多重背包采用二进制拆分,可以逐个撤销拆出的 01 物品,但这样需要 $O(V \log(s_i + 1))$ 。

58.9.2.2 dp 角度

删除一个物品,把这个物品的贡献减掉即可。

注意枚举顺序的影响:若是 01 背包, $dp_j - dp_{j-v_i}$ 需要保证 dp_{j-v_i} 已经把这个物品的贡献减掉了,所以要顺序枚举;完全背包则反之,因为可以无限制取,所以需要保证 dp_{j-v_i} 还没有把这个物品的贡献减掉,所以要逆序枚举。

01 背包:

```
for (int j = v[i]; j <= V; j++)
    dp[j] -= dp[j - v[i]];
```

完全背包:

```
for (int j = V; j >= v[i]; j--)
    dp[j] -= dp[j - v[i]];
```

多重背包的每个二进制拆分组都可以和 01 背包一样处理。

若要在 $O(V)$ 内撤销整个数量上限为 s_i 的物品类型,可以按体积同余类维护滑动区间和。设加入该类型前后的方案数分别为 Q_t, P_t ,在同一余数类内有

$$P_t = \sum_{r=0}^{s_i} Q_{t-r}.$$

因此按 t 从小到大,可以用已经恢复的 Q 递推当前 Q_t :

```

if (s[i]) {
    for (int j = 0; j < v[i]; j++) {
        int sum = dp[j];
        for (int k = j + v[i]; k <= V; k += v[i]) {
            dp[k] -= sum;
            if (k / v[i] >= s[i])
                sum -= dp[k - s[i] * v[i]];
            sum += dp[k];
        }
    }
}
}

```

代码执行到当前体积前, sum 始终是递推所需的最近 s_i 个 Q 之和; $s_i = 0$ 时没有贡献, 直接跳过。以上写法针对精确整数方案数; 若方案数对模数取模, 每次减法后还需要规范化到模意义下。

01 背包、完全背包以及上述直接滑动窗口写法, 撤销一次加入操作的时间复杂度均为 $O(V)$ 。

58.10 前后缀背包

令 $\text{pre}[i][j]$ 表示只使用前 i 个物品、容量为 j 的背包结果, $\text{suf}[i][j]$ 表示只使用第 i 至第 n 个物品的结果。

作用是, 将前缀 $i - 1$ 的背包将后缀 $i + 1$ 的背包拼起来, 就可以得到去掉第 i 个物品的结果, 前文中可撤销性基于求方案数, 做前后缀背包可以支持更一般的情况。

预处理时间复杂度: $O(nV)$ 。

```

vector<int> ans(V + 1, -INF);
for (int j = 0; j <= V; j++)
    for (int k = 0; k <= j; k++)
        if (pre[i - 1][j - k] != -INF && suf[i + 1][k] != -INF)
            ans[j] = max(ans[j], pre[i - 1][j - k] + suf[i + 1][k]);
dp = ans;

```

拼接时必须始终从固定的前缀表和后缀表读取, 不能一边写 dp 一边再把它作为前缀状态读取, 否则会重复使用后缀物品。求出完整结果数组的单次拼接是 max-plus 卷积, 时间复杂度为 $O(V^2)$; 若只询问容量 V 的答案, 则只需枚举一次分界, 时间复杂度为 $O(V)$ 。

58.11 树上背包

下面的模板令 $\text{dp}[x][j]$ 表示在 x 的子树中恰好选择 j 个点的最大价值, 每个点在这一维上占用一个单位。不可达状态必须初始化为 $-\text{INF}$ 。

```

sz[x] = 1;
fill(dp[x], dp[x] + V + 1, -INF);
dp[x][1] = w[x];
for (auto u : p[x]) {
    dfs(u);
    for (int i = min(sz[x], V); i >= 1; i--) {
        for (int j = min(sz[u], V - i); j >= 1; j--) {
            if (dp[x][i] != -INF && dp[u][j] != -INF)
                dp[x][i + j] = max(dp[x][i + j], dp[x][i] + dp[u][j]);
        }
    }
    sz[x] += sz[u];
}

```

对于上述“单位体积、按选点数计”的模板,利用子树大小限制枚举上下界后,总时间复杂度可以分析为 $O(nV)$ 。若每个点具有一般体积,直接对每条父子关系做容量上的朴素max-plus 合并,通常只能保证 $O(nV^2)$;没有额外结构或优化时,不能无条件沿用 $O(nV)$ 的结论。

空间复杂度: $O(nV)$ 。

通过链剖分结合 NTT 的科技,可以做到 $O(n \log^3 n)$ 甚至 $O(n \log^2 n)$,但是看起来太 useless 了。

58.11.1 有依赖的背包

有依赖的背包是一种特殊的树上背包,注意祖先节点一定要选。

58.12 可行性完全背包的最少物品数

对于一个可行性完全背包,求最少拿几个物品可以恰好得到体积 V 。直接把状态改为最少物品数即可:

```

fill(dp, dp + V + 1, INF);
dp[0] = 0;
for (int i = 1; i <= n; i++)
    for (int j = v[i]; j <= V; j++)
        if (dp[j - v[i]] != INF)
            dp[j] = min(dp[j], dp[j - v[i]] + 1);

```

时间复杂度为 $O(nV)$,空间复杂度为 $O(V)$;若 $dp[V] == INF$,则体积 V 不可达。

58.12.1 倍增 NTT

令 $F(x) = \sum x^{v_i}$,则 $[x^V]F(x)^t \neq 0$ 表示恰好使用 t 个物品可以得到 V ,它对 t 并不单调。例如只有体积为 2 的物品且 $V = 2$ 时, $t = 1$ 可达, $t = 2$ 反而不可达,因此不能直接对 $F(x)^t$ 二分或倍增。

若要恢复单调性,应令

$$H(x) = 1 + \sum_{v \in S} x^v,$$

其中 S 是去重后的物品体积集合, 并把系数只解释为可达或不可达。此时 $[x^V]H(x)^t \neq 0$ 等价于“使用不超过 t 个物品可以得到 V ”, 该条件才随 t 单调。每次卷积后都要截断至 V 次, 并把非零系数重新置为 1, 即在布尔半环上做乘法。

预处理 $H(x)^{2^0}, H(x)^{2^1}, \dots$ 。由于物品体积为正, 若有解则最少物品数不超过 V ; 应先检查 $H(x)^V$ 是否可达, 仍不可达就直接判定无解。随后从高位到低位倍增, 寻找最后一个仍不可达的 t 。整个过程只需 $O(\log V)$ 次卷积, 总复杂度为 $O(M(V) \log V) = O(V \log^2 V)$ 。

若用单模 NTT 实现布尔卷积, 需要选择支持所需变换长度且满足 $\text{mod} > V+1$ 的模数。因为两个布尔多项式单次卷积的每个系数至多为 $V+1$, 该条件可以避免“实际非零却恰好模 mod 为零”; 若不在每次卷积后布尔化, 则不能无条件用单模系数是否为零判断可达性。

第八部分

杂项

59 区间变换

除“区间求并”一节另有说明外,本文均讨论闭区间 $[l, r]$, 并假设 $l \leq r$ 。闭区间包含端点, 因此两个区间在端点相接也算相交: 判断新区间与已选区间不相交时使用 $l > r$, 判断某个已结束的分组能否复用时使用 $r < l$ 。若改用半开区间 $[l, r)$, 相应边界应改为允许取等。

59.0.1 选最少的点覆盖所有区间

```
int calc(vector<pii> a) {
    sort(a.begin(), a.end(), [&](pii x, pii y) -> bool {
        return x.second == y.second ? x.first < y.first : x.second < y.second;
    });
    int res = 0, r = 0;
    bool has = false;
    for (auto u : a) {
        if (!has || u.first > r) {
            res++;
            r = u.second;
            has = true;
        }
    }
    return res;
}
```

59.0.2 选最多的区间互不相交

同【选最少的点覆盖所有区间】。

59.0.3 区间分成组内区间无交的最少组数

```
int calc(vector<pii> a) {
    sort(a.begin(), a.end());
    priority_queue<int, vector<int>, greater<int>> q;
    for (auto [l, r] : a) {
        if (q.size() && q.top() < l)
            q.pop();
        q.push(r);
    }
    return q.size();
}
```

59.0.4 选最少的区间覆盖整段区间

这里覆盖的是从 s 向右延伸至 t 的连续线段, 并约定 $s \geq t$ 时目标已经完成, 答案为 0。每次应在所有满足 $l \leq cur$ 的候选区间中选择右端点最大的一个; 若最远右端点不能超过 cur , 说明无法继续覆盖。

```

int calc(int s, int t, vector<pii> a) {
    if (s >= t)
        return 0;
    sort(a.begin(), a.end());
    int res = 0, cur = s, i = 0;
    while (cur < t) {
        int r = cur;
        while (i < a.size() && a[i].first <= cur) {
            r = max(r, a[i].second);
            i++;
        }
        if (r <= cur)
            return -1;
        cur = r;
        res++;
    }
    return res;
}

```

59.0.5 区间求并

本节单独约定输入为闭整数区间,返回其并集中整数点的数量,因此每个合并后区间 $[l, r]$ 的贡献是 $r - l + 1$ 。下面选择把 $[l, r]$ 与 $[r + 1, r']$ 这样的相邻整数区间也合并;这不会改变整数点总数,但能得到更紧凑的合并结果。若求实数轴上的总长度,则贡献应改为 $r - l$,且只有新区间左端点不大于当前右端点时才合并。

```

long long calc(vector<pii> a) {
    if (a.empty())
        return 0;
    sort(a.begin(), a.end());
    long long res = 0, l = a[0].first, r = a[0].second;
    for (int i = 1; i < a.size(); i++) {
        auto u = a[i];
        if (u.first <= r + 1) {
            r = max(r, 1ll * u.second);
        } else {
            res += r - l + 1;
            l = u.first;
            r = u.second;
        }
    }
    res += r - l + 1;
    return res;
}

```


60 树同构

用于判断有根树与无根树同构。

60.1 有根树同构

对于两棵有根树 $T_1(V_1, E_1, r_1)$ 和 $T_2(V_2, E_2, r_2)$, 如果存在一个双射 $\varphi: V_1 \rightarrow V_2$, 使得 $\forall u, v \in V_1, (u, v) \in E_1 \Leftrightarrow (\varphi(u), \varphi(v)) \in E_2$ 且 $\varphi(r_1) = r_2$ 则称 T_1, T_2 同构。

60.2 无根树同构

和有根树同构类似, 没有 $\varphi(r_1) = r_2$ 的限制。

简单地, 对于无根树如果能通过对其中一棵树的节点进行重新标号使得和另一棵树完全相同, 则称这两棵无根树同构。

无根树同构可以转换成有根树同构, 具体而言: - 若两棵无根树的重心数量不同, 则不同构。- 若都只有一个重心, 以各自唯一重心为根后判断有根树同构。- 若都有两个重心, 不能在两棵树中各自任取一个重心作为根, 因为同构映射可能交换两个重心。可以分别计算以两个重心为根的规范编码, 再比较排序后的两个编码; 也可以删去两重心之间的边, 在该边中插入一个虚根并连接两个重心, 然后统一以虚根为根。

所以用于解决有根树同构的算法也可以解决无根树同构。

60.3 AHU:

一段合法的括号序和一棵**有序**有根树唯一对应, 而且一棵树的括号序由各棵子树的括号序按子节点顺序拼接而成。

括号序用 **0** 表示递归该节点, **1** 表示从该节点回溯。

本文讨论的普通树没有固定的子节点顺序。为了得到无序有根树的唯一规范编码, 需要先递归求出所有子树编码, 将它们排序后再拼接; 任意调整拼接顺序只会得到同一无序树的不同有序表示, 不能直接作为唯一编码。

所以判断两棵无序有根树同构, 可以比较排序子树编码后得到的规范括号序。

60.3.1 朴素做法:

朴素地用 `vector` 存下每一棵子树的括号序,按从小到大升序拼接子树的括号序作为当前节点的括号序。

最坏情况下,若树是“毛毛虫”,那么拼接括号序的过程是 $O(n^2)$ 的。

```
string q[N];
void dfs1(int x, int y) {
    vector<string> tmp;
    q[x] = "0";
    for (auto u : p[x]) {
        if (u == y)
            continue;
        dfs1(u, x);
        tmp.push_back(q[u]);
    }
    sort(tmp.begin(), tmp.end());
    for (auto u : tmp)
        q[x] += u;
    q[x] += "1";
}
```

60.3.2 优化:

一个节点的无序子树类型由“所有儿子子树类型编号组成的有序序列”唯一确定。因此可以把排序后的儿子编号序列离散化成一个整数,代替完整括号序。关键不是节点处于同一深度,而是所有待比较的树必须共享同一份“规范序列 $\rightarrow$ 类型编号”映射;比较两棵树时,不能在处理第二棵树前清空 `mp` 或重置编号。

后序遍历保证计算当前节点时,所有儿子的类型编号已经确定。构造所有 `tmp` 的总元素数为 $O(n)$,但单个序列的排序、复制以及 `map<vector<int>, int>` 的字典序比较都不是 $O(1)$ 。

设 n 为所有待比较树的总节点数,当前节点度数为 d_x 。对子编号排序的总代价为 $O(\sum_x d_x \log d_x)$; `map` 每次操作需要 $O(\log n)$ 次比较,每次向量比较最坏为 $O(d_x + 1)$ 。利用 $\sum_x d_x = O(n)$,总时间复杂度仍可界为 $O(n \log n)$,空间复杂度为 $O(n)$ 。

```
int q[N], idx;
map<vector<int>, int> mp; // 待比较的所有树共享,处理中途不能清空
void dfs1(int x, int y) { // 以重心为根 dfs
    vector<int> tmp;
    for (auto u : p[x]) {
        if (u == y)
            continue;
        dfs1(u, x);
        tmp.push_back(q[u]);
    }
    sort(tmp.begin(), tmp.end());
    auto it = mp.find(tmp);
    if (it != mp.end()) {
        q[x] = it->second;
    } else {
        q[x] = ++idx;
        mp[tmp] = idx;
    }
}
```

注:若按后序分层收集所有整数序列,并使用适合整数键的基数排序统一编号,可以把确定性AHU 规范化做到 $O(n)$;仅把 map 换成普通比较排序并不能自动得到该复杂度。

60.4 树哈希:

定义哈希函数 $h(x)$, $f(x) = (c + \sum_{y \in \text{son}(x)} h(f(y))) \bmod M$ 。对子树贡献求和使结果与儿子顺序无关,但也把整个儿子多重集压缩成了一个模意义下的和,因此不同多重集仍可能碰撞。

判断有根树 T_1, T_2 同构时,比较 $f(r_1), f(r_2)$ 即可。

下面给出单个 64 位自然溢出哈希的示例。u64 加法等价于模 2^{64} ,随机 mask 只是在不同运行间改变映射,不能提供确定正确性:

```
const u64 mask = std::chrono::steady_clock::now().time_since_epoch().count();
u64 shift(u64 x) {
    x ^= mask;
    x ^= x << 13;
    x ^= x >> 7;
    x ^= x << 17;
    x ^= mask;
    return x;
}
u64 q[N];
void dfs1(int x, int y) { // 以重心为根 dfs
    q[x] = 1;
    for (auto u : p[x]) {
        if (u == y) continue;
        dfs1(u, x);
        q[x] += shift(q[u]);
    }
}
```

自然溢出仍然只是一路 64 位概率哈希,并不等价于双哈希。若需要降低随机碰撞概率,可以使用两个独立种子或模数的哈希;但可交换求和本身仍可能产生多重集结构碰撞,双哈也只能降低概率,不能给出数学上的零碰撞保证。面对可构造数据或要求确定正确的场景,应优先使用上面的AHU 规范编码。

时间复杂度: $O(n)$ 。

与 AHU 算法相比: - AHU 是确定性规范编码,不存在概率哈希碰撞。- 树哈希更容易实现、常数通常更小,但结论带有碰撞概率。- 树哈希可以通过换根 dp, $O(n)$ 求出以所有节点为根时的哈希值。

61 哈希

哈希通过某种规则把原数据映射为较短的摘要。由于值域通常小于原数据空间,哈希一般不是单射:不同对象可能得到相同的哈希值,这种情况称为哈希冲突。

因此,哈希值不同可以推出原对象不同;哈希值相同通常只能作为原对象相同的必要条件,还需要接受一定的误判概率,或在哈希相同时继续比较原对象。

61.1 哈希表:

开放寻址和拉链法只能处理多个键落到同一桶的情况,并不能消除哈希冲突。哈希函数分布较均匀且负载因子受到控制时,插入、删除、查询的期望复杂度为 $O(1)$;最坏情况下仍可能退化到 $O(n)$ 。

61.1.1 开放寻址法:

开放寻址法使用定长数组直接保存键值对。

取一个表长 M ,假设插入的数是 x ,初始位置为 $f(x) \bmod M$ 。若该位置已经有元素,则依次检查下一个位置,直到找到可用位置。探测次数至多为 M ;若整张表都没有可用位置,插入必须报告失败,不能继续循环。

删除元素时,不能直接把位置标为空,否则会截断其它键的探测链。通常使用“墓碑”标记:查询越过墓碑继续探测,插入则可以复用遇到的第一个墓碑。负载因子过高或墓碑过多时,应扩容并重新散列。

61.1.2 拉链法:

拉链法为每个桶维护一个链表、动态数组或其它容器。

取一个模数 M ,假设插入的数是 x ,则 $f(x) = x \bmod M$,将 x 插入 $f(x)$ 位置所在的数据结构。

事实上,开放寻址法和拉链法可通过修改哈希函数,改善键的分布。

开放寻址法还可以修改步长。拉链法也可修改位置上的数据结构。

基于开放寻址法的哈希表实现:

```

mt19937_64 rnd(chrono::steady_clock::now().time_since_epoch().count());
struct Hash_Map {
    enum State : unsigned char { EMPTY, OCCUPIED, DELETED };
    State state[N]{};
    u64 a[N];
    int b[N];
    stack<int> q;
    void clear() {
        while (q.size()) {
            auto now = q.top();
            q.pop();
            state[now] = EMPTY;
        }
    }
    const u64 s1 = rnd(), s2 = rnd() | 1, s3 = rnd() | 1;
    u64 f(u64 x) {
        x += s1;
        x = (x ^ (x >> 33)) * s2;
        x = (x ^ (x >> 30)) * s3;
        return x;
    }
    bool find(u64 x) {
        int now = f(x) % N;
        for (int cnt = 0; cnt < N; cnt++) {
            if (state[now] == EMPTY)
                return false;
            if (state[now] == OCCUPIED && a[now] == x)
                return true;
            now = (now + 1) % N;
        }
        return false;
    }
    int get(u64 x) {
        int now = f(x) % N;
        int del = -1;
        for (int cnt = 0; cnt < N; cnt++) {
            if (state[now] == OCCUPIED && a[now] == x)
                return now;
            if (state[now] == DELETED && del == -1)
                del = now;
            if (state[now] == EMPTY) {
                if (del != -1) {
                    now = del;
                } else {
                    q.push(now);
                }
                state[now] = OCCUPIED, a[now] = x, b[now] = 0;
                return now;
            }
            now = (now + 1) % N;
        }
        if (del != -1) {
            state[del] = OCCUPIED, a[del] = x, b[del] = 0;
            return del;
        }
        return -1;
    }
    bool erase(u64 x) {
        int now = f(x) % N;
        for (int cnt = 0; cnt < N; cnt++) {
            if (state[now] == EMPTY)
                return false;
            if (state[now] == OCCUPIED && a[now] == x) {
                state[now] = DELETED;
                return true;
            }
            now = (now + 1) % N;
        }
        return false;
    }
    int &operator[](const u64 x) {
        int now = get(x);

```

```

        if (now == -1)
            throw overflow_error("Hash_Map is full");
        return b[now];
    }
};

```

其中, `state` 的初值均为 `EMPTY`, 键数组使用 `u64`, 不会截断传入的键。新键对应的值会初始化为 0。两个随机乘数强制取奇数, 使乘法在模 2^{64} 意义下是一个排列, 不会仅因乘数为偶数而系统性丢失部分桶。`get` 最多探测 N 个位置, 表满且没有墓碑可复用时返回 -1; `operator[]` 在这种情况下显式报错。模板不自动扩容, 使用时应让 N 明显大于同时保存的键数, 并在墓碑过多时重建哈希表。

61.2 多项式模数哈希:

$$h(a) = \sum_{i=1}^n a_i \cdot base^{i-1} \bmod p$$

61.2.1 优点:

- 可以将字符串映射到 $[0, p)$ 中, 方便存储。
- 模运算具有优秀的性质, 可以支持很多其它操作(比如修改字符串中的某一个字符, 只需要加减取模即可)。

61.2.2 缺点:

- 映射范围较小容易产生哈希冲突。

61.3 自然溢出:

使用 `unsigned long long` 进行运算, 不使用取模, 任凭数据在底层自动“溢出” 64 位, 仅保留低 64 位的结果。

61.3.1 优点:

- 运算速度快。

61.3.2 缺点:

- 本质是对 2^{64} 取模, 模数过于特殊, 十分容易卡。

61.4 值域与哈希冲突

若哈希函数把较大的输入域映射到 M 个值中,则由鸽巢原理可知冲突必然存在。冲突多少取决于哈希函数在输入集合上的实际分布,不能由一个没有对应映射关系的 gcd 直接推出。

最大公约数只在某些特定映射中能给出精确结论。例如对完整剩余系 $\mathbb{Z}_M$ 上的仿射映射

$$h(x) = (Ax + B) \bmod M,$$

有

$$h(x_1) = h(x_2) \iff A(x_1 - x_2) \equiv 0 \pmod{M}.$$

令 $d = \gcd(A, M)$, 则该映射的像集大小为 M/d , 每个可达值恰有 d 个原像; 仅当 $d = 1$ 时, 它才是 $\mathbb{Z}_M$ 上的一个排列。这个结论不能直接套到字符串的多项式哈希上。选择较大的质数作为模数便于在有限域中分析和运算, 但并不会消除冲突。

61.5 多项式哈希冲突概率:

设哈希值域大小为 M , 并假设 q 个对象的哈希值相互独立且在 $[0, M-1]$ 中均匀分布, 则出现至少一次碰撞的概率为

$$P_{\text{coll}} = 1 - \prod_{i=0}^{q-1} \left(1 - \frac{i}{M}\right) \approx 1 - \exp\left(-\frac{q(q-1)}{2M}\right).$$

当 $q \approx \sqrt{2M \ln 2} \approx 1.177\sqrt{M}$ 时, 该概率约为 $1/2$ 。若希望碰撞概率至多为 ε , 则近似需要

$$M \geq \frac{q(q-1)}{-2 \ln(1-\varepsilon)},$$

当 ε 很小时即要求 M 远大于 q^2 , 而不是要求 $M > \sqrt{q}$ 。一次碰撞会让对应的一对不同对象产生假阳性, 并不意味着其它比较全部失效。

上述估计依赖独立、均匀的随机模型。固定模数、固定底数的多项式哈希可能被针对性构造; 随机选择并隐藏底数或种子只能降低被构造的风险, 不能提供确定性。

61.6 双模数多项式哈希：

使用两个模数分别计算多项式哈希，只有两路结果均相同时，才把两个字符串视为可能相同。

若两路计算的是同一个整数多项式 H ，即使用相同的 `base` 和字符映射，且 $\gcd(M_1, M_2) = 1$ ，则由中国剩余定理可得

$$H(x_1) \equiv H(x_2) \pmod{M_1}, \quad H(x_1) \equiv H(x_2) \pmod{M_2}$$

等价于

$$H(x_1) \equiv H(x_2) \pmod{M_1 M_2}.$$

若模数不互质，等效模数只能写成 $\text{lcm}(M_1, M_2)$ ；若两路使用不同的底数，则它们不是同一个整数多项式，也不能用中国剩余定理合并。

只有在两路哈希可近似看作均匀且相互独立时，单次比较的碰撞概率才可近似相乘，生日界也才可按有效空间 $M_1 M_2$ 估计。推广到更多模数时同样需要互质性或独立性等相应前提，不能无条件把各路概率直接相乘。

61.7 集合哈希：

判断两个序列排序后是否相同，本质上是判断两个多重集是否相同。

61.7.1 哈希做法：

两个多重集相同一定会得到相同摘要，反过来通常不成立。因此，构造确定性摘要可以视作增加用于排除“不相同”的必要条件。

例如此处， n 个数平方和相等，是这 n 个数一一相同的必要条件。

n 个数相同 $\rightarrow n$ 个数平方和相等。

但是也会出现 $1^2 + 1^2 + 1^2 + 1^2 + 8^2 = 2^2 + 4^2 + 4^2 + 4^2 + 4^2 = 68$ 的情况。

可以继续增加立方和、四次方和等条件来排除一部分冲突，但只维护固定的少量幂和，尤其是 $k < n$ 时，通常仍可能被构造出冲突。在没有随机模型时，这里只有“能排除更多已知反例”的结论，不能自动表述为碰撞概率变小。

若多重集大小为 n ，则在特征为 0，或有限域特征大于 n 等能合法使用 Newton 恒等式的条件下，前 n 个幂和可以确定各阶基本对称多项式，进而确定该多重集；只维护少量幂和时没有这一保证。

若维护前 k 个幂和,预处理或逐个插入的总复杂度为 $O(kn)$,单点修改和一次区间摘要比较均为 $O(k)$;只有把 k 视为常数时才能简写为每次 $O(1)$ 。

61.7.2 随机化

一种更容易分析的随机化方法是:选取大质数 p ,为每个不同的值 v 分配一个相互独立且在 $\mathbb{F}_p$ 中均匀的随机权值 $R(v)$,相同的 v 始终使用相同的权值,并维护

$$F(A) = \sum_{a \in A} R(a) \pmod{p}.$$

对两个预先固定的不同多重集,若至少一个元素的出现次数之差不被 p 整除,则在理想随机模型下 $F(A) = F(B)$ 的概率为 $1/p$ 。通常还要求随机种子不被对手预知;多路独立权值可以进一步降低误判概率。若出现次数之差可能是 p 的倍数,或输入能根据种子自适应构造,则上述概率界不再成立。

61.7.3 多项式哈希做法:

update on 2025.4.20

2024 郑州 G 中使用了多项式哈希维护多重集摘要。设值域为 $[0, D]$, c_v 表示值 v 的出现次数,可以定义频次多项式

$$F_A(X) = \sum_{v=0}^D c_v X^v.$$

选取质数 p 和随机点 $x \in \mathbb{F}_p$,维护 $F_A(x) \bmod p$ 。若两个多重集不同、频次之差不全为 p 的倍数且差多项式的次数不超过 $D < p$,则由非零多项式的根数界,单个随机点发生碰撞的概率至多为 D/p 。使用多组模数或随机点时,只有各组随机选择相互独立,碰撞概率才能相乘。没有这些值域、模数和随机性前提,幂和摘要本身没有自动的小碰撞概率。

61.8 sum hash

上文的做法就是一种 Sum Hash。

Sum Hash 可以作为判断集合或多重集是否相同的概率摘要。

对于多重集 S ,可以维护 $f(S) = \sum_{a \in S} h(a)$;求和时需要计入重复元素。若 $f(S_1) \neq f(S_2)$,则两者一定不同;若摘要相同,只能认为两者可能相同。

其中, $h(a)$ 的计算方式有很多,上文的 x^a 就是一种。

更常见的随机化写法是直接令 $h(a) = R(a)$, 其中 R 是由隐藏种子确定的稳定随机映射。若值域可能包含 0, 则不能无条件使用 $h(a) = a \times R(a)$: 当 $a = 0$ 时, 该式恒为 0; 在模 2^{64} 的自然溢出下, 偶数 a 还会损失随机位。若在质数域中且 $a \neq 0$, 乘以 a 仍会保持均匀性, 因此问题不在乘法本身, 而在使用条件没有写明。直接使用 $R(a)$ 更通用。若要使用上文的 $1/p$ 误判界, 求和还需满足对应的出现次数条件。

update on 2025.4.20:

2024 郑州 G 题引申了只取少量幂和的 sum hash 并不充分: 当 $k = O(\log n)$ 时, 仍能构造两组不同的多重集, 使得 $\forall i \leq k, \sum_{j=1}^n a_j^i$ 相同。

隐藏种子后再做随机映射, 可以阻止针对固定公开参数预先构造的部分数据, 但这是一种概率保证, 不能把有限个幂和变成确定充分条件。若需要维护区间加等操作, 还必须证明所选摘要能够由区间信息正确更新; 静态随机映射的 Sum Hash 一般不能仅由原摘要推出整体加值后的新摘要。

61.9 xor hash

xor 运算的一个性质: $a \oplus a = 0, a \oplus b \neq 0 (b \neq a)$ 。

所以使用 xor 可以很好地维护数字出现次数的奇偶性。

若某一个多重集内所有数字出现次数均为偶数次, 那么 $\bigoplus_{a \in S} a = 0$ 。

但是, $\bigoplus_{a \in S} a = 0$ 只是“所有数字均出现偶数次”的必要条件, 并不充分。例如 $2 \oplus 4 \oplus 6 = 0$, 但这三个数都只出现了一次。

可以为每个不同的值 v 分配独立均匀的 w 位随机串 $R(v)$, 再维护这些随机串的异或和。对于一个预先固定、且确实存在奇数次数元素的多重集, 其摘要误为 0 的概率为 2^{-w} 。该结论依赖独立随机映射和固定输入; 公开种子下的自适应构造不受此界保护。

61.9.1 xor hash 应用:

排列, 是一个特殊的序列, 其中每个数在 $[1, n]$ 中不重不漏的出现一次。

同一个长度的所有排列的异或和相同, 所以可以预处理 $1 \oplus 2 \oplus \dots \oplus n$ 。但是, 直接比较原数值的异或和只是必要条件, 即使序列长度为 n 且所有值都在 $[1, n]$ 内也不充分。例如

$$1 \oplus 1 \oplus 1 \oplus 2 \oplus 2 = 1 = 1 \oplus 2 \oplus 3 \oplus 4 \oplus 5.$$

概率判定时, 必须先确认序列长度为 n , 并检查每个值都在 $[1, n]$ 内, 再比较

$$\bigoplus_{i=1}^n R(a_i) \quad \text{与} \quad \bigoplus_{i=1}^n R(i),$$

其中 $R(i)$ 是相互独立的 w 位随机标签。在这些前提下,任意固定的非排列至少有一个值的出现次数奇偶性与目标不同,误判概率为 2^{-w} 。使用同一组标签完成 Q 次固定查询时,总误判概率至多为 $Q/2^w$,不能把各次错误事件当作相互独立。若要求确定正确,应直接统计每个值的出现次数或使用 `vis` 数组判重。

61.9.2 xor hash 再扩展:

能用 xor hash 出现次数为偶数的情况,是基于 $a \oplus a = 0$,也即是出现次数是 2 的倍数。那如果现在需要判断出现次数是 k 的倍数呢?

可以把异或推广为各位独立的模 k 加法,使同一个标签累加 k 次后得到 0。若标签有 d 个 k 进制位,一次运算需要 $O(d)$,通常可写成 $O(\log_k U)$,其中 U 是标签空间大小。

“所有出现次数都是 k 的倍数”一定会使摘要为 0,反过来仍可能冲突。若 k 为质数,并把每个值独立均匀地映射到 $\mathbb{F}_k^d$,则对任意固定的非零次数余量向量,误判为 0 的概率为 k^{-d} ; k 为合数时,非零系数未必可逆,不能直接沿用这个概率界。随机映射只能给出相应模型下的概率保证,不能避免冲突。

62 优秀的编码习惯(卡常)

用时间复杂度和空间复杂度衡量一个算法的时间效率和空间效率。

描述时空复杂度时,通常使用渐进符号表示,因此一般只考虑时空复杂度函数中的最高阶项。

简便地,通常只使用 O 描述时空复杂度。

所以,描述时空复杂度时,通常会忽略常数,例如 $100n$ 和 n 均为 $O(n)$ 。

但是程序的实际执行效率却会受常数影响,而在部分情况下,这一部分的影响并不能忽略。

与一般“卡常”文章相比,本文着重于相同做法下的不同实现导致的常数差异,侧重于讨论小常数代码习惯,而非通常的“硬件优化”。

在特定数据范围、实现和运行环境下,常数较小但渐进复杂度稍劣的代码可能反而更快。例如重链剖分配合数据结构与LCT的实际速度经常受到操作比例、实现质量和测试数据影响,不能只根据 $\log$ 的个数判断。

本文表格保留的是一次历史测试记录,不是严格、可复现的benchmark:原记录没有完整保存处理器型号、编译器具体版本与全部参数、测试源码、输入与随机种子、输出校验和以及多次运行的统计分布,也有测试对象被编译器消除的情况。因此这些数据只能作为当时环境下的现象记录,不能据此推出一般性的STL或语言特性常数结论。

若要重新比较,应固定并公开硬件、编译器及参数和测试源码;让各实现完成语义相同的工作;预热后独立运行多次并报告中位数、离散程度;使用校验和或等价的防优化措施消费结果,同时检查输出正确性。以下各表均应在这一限制下阅读。

62.1 STL 常数

STL 容器提供了通用接口、异常安全和较完整的语义保证,其实际开销依赖具体实现与使用方式;手写实现也不天然更快。下面保留一组旧测试数据作为观察样本。

原记录注明的环境为: 2025.9.11 洛谷、C++20、开启 O2,插入元素为 $[1, n]$ 。由于最终只保留了单次值,没有完整的多次统计和测试源码,所以不能复现实验或用这些数值比较细小差异。

注:不同平台、编译环境会导致实际情况不同,以下测试数据仅供参考。

容器	1e6 次操作 int	5e6 次操作 int	空的 5e6 个 不使用	5e6 个,每个一 个元素
vector	push_back 17ms 5MB	push_back 20ms 33MB	测试对象被优 化掉,结果无 效	push_back 300ms 267MB
vector	reserve 7ms 4MB	reserve 12ms 19MB	/	/
deque	push_back 7ms 4MB	push_back 20ms 20MB	>251ms >512MB	/
list	push_back 46ms 31MB	push_back 208ms 153MB	48ms 115MB	push_back 254ms 268MB
priority_queue	push 33ms 5MB	push 172ms 32 MB	60ms 153MB	push 272ms 306MB
set	insert 241ms 46MB	insert 1.53s 229 MB	89ms 229MB	insert 348ms 459MB
unordered_set	insert 78ms 42MB	insert 356ms 198MB	106ms 267MB	insert >312ms >512MB

注: - queue 和 stack 默认以 deque 为底层容器,也可以改用满足接口要求的其它容器;适配器与底层容器的操作路径相关,但不能直接断言耗时完全一致。- priority_queue 默认以 vector 存储元素,并额外维护堆性质,因此 push、pop 与普通 vector::push_back 不是语义或复杂度相同的操作。- set、map 及其 multi 版本通常采用相近的平衡树结构,但元素大小、比较器和操作分布不同,标准也不保证某一种具体实现; unordered 容器同样不能据此视为时空效率完全一致。

这些容器有部分重叠的操作,但迭代器、访问方式、稳定性和复杂度保证并不相同。下面仅按原测试所比较的操作分成两组保留数据。

注:因为 list 插入 5e7 个元素内存过大,所以 list 只插入 1e7 个元素。

容器	插入 5e7 个元 素	遍历 5e7 个元 素	插入 5e7 个元 素并 rand()	随机访问 5e7 次
数组/array	赋值 5ms	auto 75ms	769ms	1.97s
vector	push_back 180ms	auto 200ms	920ms	2.44s

容器	插入 5e7 个元素	遍历 5e7 个元素	插入 5e7 个元素并 rand()	随机访问 5e7 次
deque	push_back 160ms	auto 190ms	880ms	3.95s
list	push_back 420ms	auto 450ms	560ms	不支持随机访问

容器	插入 1e6 个元素并删除	插入 5e6 个元素并删除
set	310ms	1.82s
priority_queue	80ms	430ms

这些旧数据至多说明容器选择可能影响实际常数。实际使用时应先按语义和复杂度选择合适容器,再针对目标平台和真实负载进行可靠测量。

62.2 数组下标访问

多维数组在内存中是按行优先连续存储的。

如果遍历时没有按照最右维向左的顺序,那么每次访问内存时不连续,会极大地降低CPU 缓存命中率。

应用场景主要是多维 dp 时,考虑 dp 状态维度的设计,使得最内层的循环位于数组的最右侧。

一个经典的场景是 ST 表预处理,以洛谷 P3865 为例 $N = 10^5$: - 按行遍历:最慢点 300ms - 按列遍历:最慢点 440ms

62.3 快速读入

这里需要比较的只有 关流 cin 和 fread 快读,默认 $|a_i| \leq n$

读入方式	$n = 10^5$	$n = 10^6$	$n = 10^7$	$n = 10^8$
scanf	10ms	72ms	700ms	>2.7s
cin 关流	9ms	58ms	565ms	>2.7s
getchar	6ms	30ms	290ms	>2.7s
fread	5ms	16ms	133ms	1.39s

同时,因为部分平台问题,实际情况会有差异。

下面的 `gc()` 必须返回 `int`,才能把所有字节值与 EOF 区分开; `read(x)` 和 `read_char(c)` 以 `bool` 表示是否成功读到一个值,调用方可以据此在文件结束时停止。该 `read(int&)` 假设输入整数位于 `int` 范围内;若输入范围更大,还应改用更宽类型并检查累积过程是否溢出。

```
namespace IO
{
    char buf[1 << 21], *p1 = buf, *p2 = buf;
    static inline int gc()
    {
        if (p1 == p2)
        {
            p2 = (p1 = buf) + fread(buf, 1, 1 << 21, stdin);
            if (p1 == p2)
                return EOF;
        }
        return static_cast<unsigned char>(*p1++);
    }
    static inline bool read(int &x)
    {
        int c = gc(), f = 1;
        while (c != EOF && c != '-' && (c < '0' || c > '9'))
            c = gc();
        if (c == EOF)
            return false;
        if (c == '-')
        {
            f = -1;
            c = gc();
        }
        if (c == EOF || c < '0' || c > '9')
            return false;
        x = 0;
        while (c >= '0' && c <= '9')
        {
            x = x * 10 + c - '0';
            c = gc();
        }
        x *= f;
        return true;
    }
    static inline bool read_char(char &x)
    {
        int c = gc();
        while (c != EOF && (c == ' ' || c == '\n' || c == '\r' || c == '\t'))
            c = gc();
        if (c == EOF)
            return false;
        x = static_cast<char>(c);
        return true;
    }

    char out_buf[1 << 21];
    int out_pos = 0;
    static inline void flush()
    {
        fwrite(out_buf, 1, out_pos, stdout);
        out_pos = 0;
    }
    struct Flusher
    {
        ~Flusher() { flush(); }
    };
    static Flusher flusher;
    static inline void pc(char c)
    {
        if (out_pos == (1 << 21))

```



```
        flush();
        out_buf[out_pos++] = c;
    }

    static inline void write(long long x)
    {
        static char stk[25];
        int top = 0;
        unsigned long long y;

        if (x < 0)
        {
            pc('-');
            y = 0 - static_cast<unsigned long long>(x);
        }
        else
            y = static_cast<unsigned long long>(x);

        if (y == 0)
            pc('0');

        while (y)
        {
            stk[++top] = y % 10 + '0';
            y /= 10;
        }
        while (top)
            pc(stk[top--]);
        pc('\n');
    }
}

using IO::flush;
using IO::read;
using IO::read_char;
using IO::write;
```

write 先把有符号整数转换为无符号幅值,因此也能安全输出 LLONG_MIN。静态 Flusher 会在程序正常结束时调用 flush();若需要立即看到输出仍可手动刷新,异常终止则不保证缓冲区被写出。

62.4 快速取模

取模方式	P3373 线段树 2	5e8 次 rand×rand 求和
不取模	78ms	5.75s
直接取模	460ms	7.17s
const	103ms	6.21s
Barrett	119ms	6.44s

在这组旧数据对应的实现中,编译期常量模数已经得到较好的优化;模数运行期读入时是否需要Barrett 等方法,仍应根据目标编译器、数据范围和可靠 benchmark 决定。

62.5 小常数数据结构

原测试中观察到:树状数组 < 线段树 < 平衡树。该顺序只针对表中的题目、实现和操作分布,不是普遍性能定理。

数据结构	P3372 线段树 1
树状数组	31ms
zkw线段树	41ms
递归式线段树	72ms
平衡树	154ms

62.6 枚举变量的上下界

部分情况下,枚举变量时可以避免一些冗余的情况。

例如枚举无序且互异的二元组 $(i, j), i, j \in [1, n]$ 时,可以钦定 $i < j$,从而避免把 (i, j) 和 (j, i) 各枚举一次。

枚举两个二维点组成的无序点对时,只能按一个固定的全序去重,例如限制 $\text{id1} < \text{id2}$,或按 (x, y) 的字典序限制第二个点更大。不能同时要求 $x_2 \geq x_1$ 且 $y_2 \geq y_1$:点 $(0, 1)$ 与 $(1, 0)$ 无论交换为哪一种顺序都不满足这两个条件,会被完全漏掉。只有题目本身要求坐标支配关系时,才能使用这样的偏序筛选。

dfs 搜索时,可以在递归的过程中顺带计算信息,而非在递归边界从头算一遍信息。

62.7 位运算

调用优秀实现的位运算库函数,把 $O(w)$ 的计算优化到 $O(1)$ 。

62.8 局部变量

缩小变量作用域通常有利于可读性,也可能帮助编译器进行别名分析和寄存器分配,但局部变量并不保证比全局变量更快。

例如局部的 `res` 没有逃逸且寄存器压力较小时,开启优化的编译器往往能把它长期保存在寄存器中;但局部变量也可能被溢出到栈上,编译器能够证明没有外部修改的全局变量也可能得到相同优化。是否更快取决于生成代码和运行环境,应通

过可靠benchmark 验证。

62.9 lambda 表达式

lambda 的调用运算符通常在使用处可见,因此经常能够被内联;定义在同一翻译单元内的普通函数同样可能被内联,是否成功由编译器、函数大小、调用方式和优化选项决定。捕获对象以及用 `std::function` 做类型擦除还可能引入额外开销,所以不能把“lambda 更容易内联”当作无条件结论。

63 范数

距离	范数符号	公式
曼哈顿距离	L^1	$ x_1 - x_2 + y_1 - y_2 $
欧几里得距离	L^2	$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$
切比雪夫距离	L^∞	$\max(x_1 - x_2 , y_1 - y_2)$

将点 (x, y) 变换为 $(u, v) = (x + y, x - y)$ 后,原坐标系中的曼哈顿距离等于新坐标系中的切比雪夫距离:

$$|\Delta x| + |\Delta y| = \max(|\Delta u|, |\Delta v|).$$

反过来,将点 (x, y) 变换为

$$(u, v) = \left(\frac{x + y}{2}, \frac{x - y}{2} \right)$$

后,原坐标系中的切比雪夫距离等于新坐标系中的曼哈顿距离:

$$\max(|\Delta x|, |\Delta y|) = |\Delta u| + |\Delta v|.$$

上述第二个变换在实数坐标中始终成立;若要求变换后的坐标仍为整数,则必须满足 $x \equiv y \pmod{2}$ 。等价地,从 (u, v) 逆变换回整数点时,需要 $u \equiv v \pmod{2}$ 。若不满足同奇偶条件,可以直接保存倍增后的坐标 $(x + y, x - y)$,此时

$$|\Delta u| + |\Delta v| = 2 \max(|\Delta x|, |\Delta y|),$$

从而避免使用半整数。

64 一些有用的库函数

64.1 GCC、Clang 等编译器提供的非 `cpp` 标准函数

函数	功能
<code>__builtin_popcount</code>	统计 <code>unsigned int x</code> 的二进制中 1 的数量
<code>__builtin_ctz</code>	统计 <code>unsigned int x</code> 的二进制中末尾 0 的个数
<code>__builtin_clz</code>	统计 <code>unsigned int x</code> 的二进制中开头 0 的个数
<code>__builtin_parity</code>	统计 <code>unsigned int x</code> 的二进制中 1 的数量的奇偶性

`__builtin_ctz`/`__builtin_clz` 中传入 0 是未定义行为。

以上函数时间复杂度均为 $O(1)$, 效率较高。

以上函数均有在末尾加 1 表示参数 `unsigned long` 和在末尾加 11 表示参数 `unsigned long long` 的变形。

需要正确传入参数, 否则会触发未定义行为。

64.2 C++20 引入了`<bit>` 头文件

函数	功能
<code>popcount</code>	统计 x 的二进制中 1 的数量
<code>countr_zero</code>	统计 x 的二进制中末尾 0 的个数
<code>countl_zero</code>	统计 x 的二进制中开头 0 的个数
<code>countr_one</code>	统计 x 的二进制中末尾 1 的个数
<code>countl_one</code>	统计 x 的二进制中开头 1 的个数
<code>bit_width</code>	返回表示 x 所需的位数, 即满足 $2^n > x$ 的最小非负整数 n ; $x = 0$ 时返回 0
<code>bit_floor</code>	返回不大于 x 的最大 2 的幂; $x = 0$ 时返回 0
<code>bit_ceil</code>	返回不小于 x 的最小 2 的幂; $x = 0$ 时返回 1

`countr_zero` 和 `countl_zero` 可以传入 0, 返回对应类型的总位数。例如 `bit_width(13u) == 4`、`bit_floor(13u) == 8`、`bit_ceil(13u) == 16`。

以上函数模板只接受无符号整数类型, 不应把有符号整数直接传入。
`bit_ceil(x)` 还要求结果能够由参数类型表示, 否则行为未定义。

以上函数时间复杂度均为 $O(1)$, 效率通常与对应的编译器内建函数相当。

64.3 mt19937

```
mt19937 rng(time(nullptr));

int rnd(int l, int r) {
    uniform_int_distribution<int> dist(l, r);
    int num = dist(rng);
    return num;
}

vector<int> a;
shuffle(a.begin(), a.end(), rng);
```

64.4 nth-element

```
// 默认升序规则(less<>)
nth_element(first, nth, last);
// 自定义比较规则
nth_element(first, nth, last, cmp);
// 第 k 小, k 从 1 开始
nth_element(nums.begin(), nums.begin() + k - 1, nums.end());
// 自定义降序比较规则
nth_element(nums.begin(), nums.begin() + k - 1, nums.end(),
    [](int a, int b) { return a > b; }); // 第 k 大
```

执行后, `nth` 指向的元素与完整排序后该位置的元素相同; `[first, nth)` 中的元素按比较规则不会排在 `nth` 之后, `(nth, last)` 中的元素不会排在它之前, 但两侧各自都不保证有序。

`nth_element` 平均使用 $O(n)$ 次比较即可找到顺序统计量。默认升序规则下 `nums[k-1]` 是第 k 小; 使用上面的降序比较器时则是第 k 大。